

企业级业务通识架构与大厂商化场景建模项目

○. 写在前面的话

都有就等于没有，都学就等于没学，学习要有重点，尤其是短时间学习，不要求全，不要用学校的思维学习，最快速学习迭代（小步快跑，快速迭代）书以杂读，业以精钻。

- ☆表示了解即可，忘了无所谓，所以饭前甜点
- ☆☆表示尽可能理解记住，能掌握最好，忘了知道怎么查
- ☆☆☆表示需要理解，需要掌握，吃饭的家伙

首先送给大家三句话：

- 种一棵树最好时间是十年前，其次是现在
- 知行合一，知道了不去做，等于不知道
- 每个人都有觉醒期，但觉醒的早晚决定个人的命运

一、业务指标体系基础

1. 运营指标

1.1 用户获取

用户获取是运营的起始，用户获取接近线性思维，或者说是一个固定的流程：**用户接触-用户认知-用户兴趣-用户行动 / 下载**。每一个流程都涉及多个数据指标。

1.1.1 渠道到达量

俗称曝光量，即产品推广页中有多少用户浏览。它可以在应用商店，可以在朋友圈，可以在搜索引擎，只要有流量的地方，都会有渠道曝光。

曝光量是一个蛮虚荣的数字，想一想现代人，每天要接触多少信息？其中蕴含了多少推广，最后能有几个吸引到用户？更多时候，渠道到达量和营销推广费挂钩，却和效果相差甚远。

广告和营销还会考虑推广带来的品牌价值。用户虽没有点击或和产品交互，但是用户知道有这么一个东西，它会潜移默化地影响用户未来的决策。然而品牌价值很难量化，在广告计算中，系统只会将用户的行为归因到最近一次的广告曝

光。

广告点击率称为 **CTR**，广告点击量/广告浏览量，除了广告，它也应用在各类推荐系统的评价中。

1.1.2 渠道转化率

既然广告已经曝光，那么用户应该行动起来，转化率是应用最广阔的指标。业界将转化率和成本结合，衍生出 **CPC**，**CPM**，**CPT**，**CPS**，**CPA** 等。

【CPC（按点击付费）】 **CPC**—英文全称 Cost Per Click。CPC 是一种点击付费广告，根据广告被点击的次数收费，指每用户点击成本，按点击计价，对广告主来说，这个比 **CPM** 的土豪作派理性多了。也有很多人会认为，CPC 不公平，用户虽然没有点击，但是曝光带来了品牌隐形价值，这对广告位供应方是损失。如关键词广告一般采用这种定价模式，比较典型的有百度联盟的百度竞价广告以及淘宝的直通车广告。

【CPM（按展示付费）】 **CPM**—英文全称 Cost Per Mille 或者是 Cost Per Thousand Impression，它指每千人成本，它按多少人看到广告计费，传统媒介比较倾向采用。CPM 是一种展示付费广告，只要展示了广告主的广告内容，广告主就为此付费。这种广告的效果不是很好，但是却能给有一定流量的网站、博客带来稳定的收入。CPM 推广效果取决于印象，用户可能浏览也可能忽略，所以它适合在各类门户或者大流量平台采用 banner 形式展现品牌性。比如说一个内置广告横幅的单价是 1 元/cpm 的话，意味着每一千个人次看到这个 banner 的话就收 1 元，如此类推，10000 人次访问的主页就是 10 元。

【CPT（按时长付费）】 **CPT**—英文全称 Cost Per Time。CPT 是一种以时间来计费的广告，国内很多的网站都是按照“一个月多少钱”这种固定收费模式来收费的，这种广告形式很粗糙，无法保障客户的利益。但是 CPT 的确是一种很省心的广告，能给你的网站、博客带来稳定的收入。阿里妈妈的按周计费广告和门户网站的包月广告都属于这种 CPT 广告。

【CPS（按销售付费）】 **CPS**—英文全称 Cost Per Sales。CPS 是一种以实际销售产品数量来计算广告费用的广告，这种广告更多的适合购物类、导购类、网址导航类的网站，需要精准的流量才能带来转化。凡客的网站联盟是这种广告形式的典型代表。

【CPA（按行为付费）】 **CPA**—Cost Per Action，指每行动成本，按用户行为计价，行为能是下载也能是订单购买。CPA 收益高于前两者，风险也大得多，它对需求方有利对供应方不利。CPA 计价方式是指按广告投放实际效果，即按回应的有效问卷或定单来计费，而不限广告投放量。CPA 的计价方式对于网站而言有一定的风险，但若广告投放成功，其收益也比 CPM 的计价方式要大得多。比如 7

天酒店在投放网络广告的时候，有一部分就是这么干的，按照每注册一个会员，给网站付 20 元。

以上五种是常见的推广方式。渠道推广是依赖技术的行业，用户画像越精准，内容与用户越匹配，则越容易产生收益。

还有一种指标 **eCPM (effective cost per mille)**，每一千次展示可获得收入，这是广告主预估自身收益的指标。

相比而言，**CPM 和 CPT 对网站有利**，而 **CPC、CPA、CPS 则对广告主有利**。目前比较流行的计价方式是 **CPM 和 CPC**，最为流行的则为 **CPM**。

1.1.3 渠道 ROI

ROI 是一个广泛适用的指标，即投资回报比。

市场营销、运营活动，都是企业获利为出发点，通过利润/投资量化目标。利润的计算涉及财务，很多时候用更简单的收入作分子。当运营活动的 ROI 大于 1，说明这个活动是成功的，能赚钱。

除了收入，ROI 也能推广到其他指标，有些产品商业模式并不清晰，赚不到钱，那么收入会用其他量化指标代替。譬如注册用户量，这也就是获客成本了。

1.1.4 日应用下载量

App 需要下载，这是一个中间态，如果不注意该环节也会流失不少用户。应用商店的产品介绍，推广文案都会影响。有些动辄几百 M 的产品，常将部分安装留在初次启动应用时以补丁形式完成，如各类游戏，就是怕漫长的下载时间造成玩家流失。

第三方平台下载到用户注册 App，这步骤数据容易出错，主要是用户对不上。技术上通过唯一设备 ID 匹配。

1.1.5 日新增用户数

新增用户数是用户获取的核心指标。

新增用户可以进一步分为自然增长和推广增长，自然增长可以是用户邀请，用户搜索等带来的用户，而推广是运营人员强控制下增长的用户量。前者是一种细火慢炖的优化，后者是烹炸爆炒的营销。

1.1.6 用户获客成本

用户获取必然涉及成本，而这是运营新手最容易忽略的。获取新增用户，运营都应该知道的事。获客成本应该直接和新增用户的财务挂钩，比如地推费用，新用户礼品。但是整个产品的运营环节成本不应该计算入内。直面上的获客成本

成本，微信粉丝在 10~20 元，产品根据不同业务形态价格差异极大。金融理财类的产品，一个有效用户成本超过四位数，非常夸张。而行业的整体获客成本仍旧在上升。

1.1.7 一次会话用户数

一次会话用户，指新用户下载完 App，仅打开过产品一次，且该次使用时长在 2 分钟以内。这类用户，很大可能是黑产或者机器人，连羊毛党都算不上。

这是产品推广的灰色地带，通过各种技术刷量，获取虚假的点击量谋取收益。该指标属于风控指标，用于监管。

1.2 用户活跃(☆☆重点理解)

用户活跃是运营的核心阶段，不论移动端、网页端或者微信端，都有相关指标。另外一方面，现在数据分析也越来越注重用户行为，这是精细化的趋势。

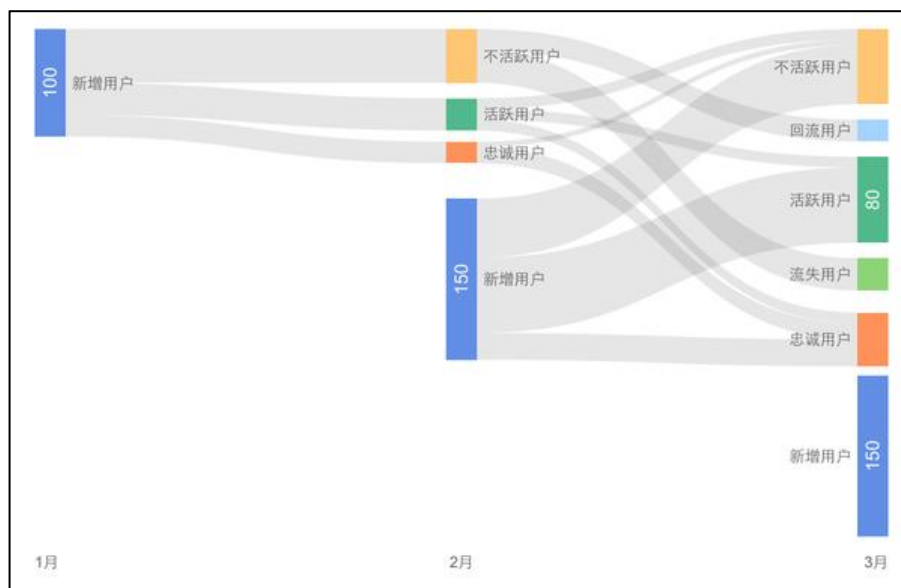
1.2.1 日活跃用户/月活跃用户

行业默认的活跃标准是用户用过产品，广义上，网页浏览内容算「用」，在公众号下单算「用」，不限于打开 APP。

活跃指标是用户运营的基础，可以进一步计算**活跃率：某一段时间内活跃用户在总用户量的占比**。按时间维度，则有日活跃率 DAU、周活跃率 WAU 和月活跃率 MAU。活跃用户数，衡量的是产品的市场体量；活跃率，看的则是产品的健康。

可仅仅打开产品，能否作为产品健康的度量？答案是否定的。成熟的运营体系，会将活跃用户再细分出新用户、活跃用户、忠诚用户、不活跃用户、流失用户、回流用户等。流失用户是长期不活跃，忠诚用户是长期活跃，回流用户是曾经不活跃或流失，后来又再次打开产品的活跃用户。

通过不同的活跃状态，将产品使用者划分出几个群体，不同群体构成了产品的总用户量。健康的产品，流失用户占比不应该过多，且新增用户量要大于流失用户量。



1.2.2 PV 和 UV

PV 是互联网早期 Web 站点时代的指标，也可以理解为网页版活跃。**PV (PageView)** 是页面浏览量，用户在网页的一次访问请求可以看作一个 PV，用户看了十个网页，则 PV 为 10。

UV (Unique Visitor) 是一定时间内访问网页的人数，正式名称独立访客数。在同一天内，不管用户访问了多少网页，他都只算一个独立访客。怎么确认用户是不是同一个人呢？技术上通过网页缓存 cookie 或者 IP 判断。如果这两者改变了，则用户算作全新的访客。

PV 和 UV 是很老的概念，但是数据分析绕不开他们，除了产品上各页面的浏览，在第三方平台如微信，各类营销活动都只能通过 Web 页实现，PV 和 UV 便需要发光发热了。

有一点需要注意的是，微信浏览器不会长期保留 cookie，手机端的 IP 也一直变动，基于此统计的 UV 会有误差（不是大问题，只是 uv 中的新访客误差较大）。这里可以通过微信提供的 openid 取代 cookie 作为 uv 基准，需要额外的技术支持。

1.2.3 用户会话次数

用户会话也叫 session，是用户在时间窗口内的所有行为集合。用户打开 App，搜索商品，浏览商品，下单并且支付，最后退出，整个流程算作一次会话。

会话的时间窗口没有硬性标准，网页端是约定俗成的 30 分钟内，在 30 分钟内用户不管做什么都属于一次会话。而超过 30 分钟，不如出去吃个饭回来再操

作，或者重现打开，都属于第二次会话了。

用户会话	操作	时间
1	打开APP	2017-05-20 13:00:00
1	浏览内容	2017-05-20 13:10:00
1	浏览内容	2017-05-20 13:30:00
2	浏览内容	2017-05-20 14:10:00

移动端的时间窗口默认为 5 分钟。

用户会话次数和活跃用户数结合，能够判断用户的粘性。如果日活跃用户数为 100，日会话次数为 120，说明大部分用户都只访问了产品一次，产品并没有粘性。

用户会话依赖埋点采集，不记录用户的操作，是无法得知用户行为从哪里开始和结束的。另外一方面，用户会话是用户行为分析的基础。

1.2.4 用户访问时长

顾名思义，用户访问时长是一次会话持续的时间。不同产品类型的访问时长不等，社交肯定长于工具类产品，内容平台肯定长于金融理财，如果分析师发现做内容的产品大部分用户访问时长只有几十秒，那么最好分析一下原因。

1.2.5 功能使用率

除了关注活跃，运营和数据分析师也应该关注产品上的重要功能。如收藏，点赞，评论等，这些功能关系产品的发展以及用户使用深度，没有会喜欢一个每天打开产品却不再做什么的用户。

功能使用率也是一个很宽泛的范围，譬如用户浏览了一篇文章，那么浏览中有多少用户评论了，有多少用户点赞了，便能用点赞率和评论率这两个指标，然后看不同文章点赞率和评论率有没有差异，点赞率和评论率对内容运营有没有帮助，这些都属于功能使用率。又譬如视频网站，核心的功能使用率就是视频播放量和视频播放时长。

微信公众号指标即可以单独说，也能把它作为产品的功能延伸看待。图文送达率，转化分享率，二次转化分享率，关注者增量等和本文其他指标一脉相承。只是第三方数据多有不便，更多分析依赖假设。

1.3 用户留存(☆☆☆重点掌握)

如果说活跃数和活跃率是产品的市场大小和健康程度的话，那么用户留存就是产品能够可持续发展。

1.3.1 留存率

用户在某段时间使用产品，过了一段时间后，仍旧继续使用的用户，被称为留存用户。 $\text{留存率} = \text{仍旧使用的用户} / \text{当初的总用户量}$ 。

在今天的互联网行业，留存是比新增和活跃提到次数更多的指标，因为移动的人口红利没有了，用户越来越难获取，竞争也越来越激烈，如何留住用户比获得用户更重要。

【新增用户留存】：新用户留存以用户新增作为留存起始条件进行计算。假设产品某天新增用户 1000 个，第二天仍旧活跃的用户有 350 个，那么称次日留存率有 35%，如果第七天仍旧活跃的用户有 100 个，那么称七日留存率为 10%。
新用户留存是最为常用的留存计算方式。在大多数情况下，如果没有特殊说明，留存=新用户留存。



Facebook 有一个著名的 40-20-10 法则，即新用户次日留存率为 40%，七日留存率为 20%，三十日留存率为 10%，有此表现的产品属于数据比较好的。

上面的案例都是围绕新用户展开，还有一种留存率是活跃用户留存率，或者老用户活跃率，即某时间活跃的用户在之后仍旧活跃的比率。它更多用周留存和月留存的维度。

【活跃用户留存】：活跃用户留存以用户活跃作为留存起始条件进行计算。
示例：如果昨日活跃的 100 个用户，其中 40 人在今日再次回到产品，则昨日的活跃用户次日留存率为 40%。

新增留存率和活跃率是不同的，新增留存率关注于产品的新手引导，各类福利，而活跃留存率和产品氛围、运营策略、营销方式等有关，更看重产品和运营的水平。

用户留存怎么算？留存率的时间粒度与计算公式：

时间粒度：时间粒度可以分为日、周、月、年。

用户在新增或使用产品后当日回到产品的比率，计为当日留存率。

用户在新增或使用产品后第 2 天回到产品的比率，计为次日留存率。

用户在新增或使用产品后第 n 天回到产品的比率，计为 n 日留存率。

用户在新增或使用产品后第 8~14 天回到产品的比率，计为次周留存率。

用户在新增或使用产品后第 n 周内回到产品的比率，计为 n 周留存率。（注：这里的周是指 7 天，而非自然周）

计算公式：最常见的用户留存率是以日的时间粒度做计算的，公式为：第 1 天新增的用户，在第 n 天后依然有使用产品的用户数 / 第 1 天的新增用户数。其中 N 代表的就是 N 日留存率的日期，例如 7 日留存率就是指，第 1 天使用产品的用户，在 7 天之后依然有使用的比例。

常见误区：7 日留存率 vs 7 日内留存率

7 日内留存率指用户在往后一周内任意一天回到产品的比例。例如第一天新增 100 个用户，第二天回到产品的 40 个用户，第三天回到产品的 20 个用户，……依次算第 8 天回到产品的用户数为 15 人。将七天内的用户数相加去重后得到 50 个用户。

7 日内留存率为 $50/100 \times 100\% = 50\%$

7 日留存率则为 $15/100 \times 100 = 15\%$

【各种时间粒度留存率的适用场景】：产品经理需要检测一个产品的健康程度，可以通过日留存率和周留存率指标来进行观察。通过新用户的日留存率可以了解用户的产品体验是否良好、产品是否能够满足用户需求等，有多少用户愿意留下来。通过周留存率可以了解到用户是否对产品上瘾，是否持续回访使用。月留存率或者年留存率适合检测订阅会员付费类的产品。 N 个用户付费了 VIP 会员，下一个月或者下一年持续付费的比例。

1.3.2 用户流失率

用户流失率和留存率恰好相反。如果某产品新用户的次日留存为 30%，那么反过来说明有 70% 的用户流失了。

流失率在一定程度上能预测产品的发展，如果产品某阶段有用户 10 万，月流失率为 20%，简单推测，5 个月后产品将失去所有的用户。这个模型虽然简陋，用户回流和新增等都没有考虑，但是它确实反应了产品未来的生命周期不容乐观。

这里可以引出一个公式，生命周期 = $(1/\text{流失率}) \times \text{流失率的时间维度}$ 。它是经验公式，不一定有效。

产品的流失率过高有问题么？未必，这取决于产品的背景形态，某产品主打婚礼管理工具，它的留存率肯定低，大多数用户结婚后就不用。但这类产品一定有生存下去的逻辑。旅游类的应用也是，用户一年也打开不了几次，但依旧能发展。

1.3.3 退出率

退出率是网页端的一个指标。网页端追求访问深度，用户在一次会话中浏览多少页面，当用户关闭网页时，可认为用户没有「留存」住。退出率公式：从该页退出的页面访问数/进入该页的页面访问数，某商品页进入 PV1000，该页直接关闭的访问数有 300，则退出率 30%。

跳出率是退出率的特殊形式，有且仅浏览一个页面就退出的次数/访问次数，仅浏览一个页面意味着这是用户进入网站的第一个页面，俗称落地页 LandingPage。

退出率用于网页结构优化，内容优化。跳出率常用于推广和运营活动的分析，两者容易混淆。

1.4 营销

营销也有自己的数据体系，互联网的数据体系就是脱胎于此才发展出 AARRR 框架。产品的发展模式有两种，如果一款产品能够在短时间获得百万用户，AARRR 框架更适合它；如果一款产品从第一个用户起即有明确的商业模式，也能尝试套用市场营销的概念。

1.4.1 用户生命周期

用户生命周期来源于市场营销理论，旧称客户生命周期。

它有两种含义，一种是针对用户个体/群体的营销生存窗口。用户会随时间推移发生变化，这种变化带来无数营销机会，对市场和企业是机遇。如怀胎十月，它就是一个生命周期为十月的营销窗口，企业会围绕这时期的用户建立特定营销。搬家，大学毕业，买房等都具有典型的周期特征。

另外一种是用户关系管理层面的生命周期，它对运营人员更重要。产品和用户的业务关系会随着时间推移改变。在传统营销中，分为潜在用户，兴趣用户，新客户，老/熟客户，流失客户。这几个层层递进的阶段和用户活跃很像。

对于一款母婴产品，我既要知道营销的生存窗口，即怀孕了几个月，因为孕早期和孕晚期的营销侧重点不一样，刚怀孕肯定是最合适的。也要知道用户本身和产品对应的关系，这位妈妈是新客户，还是曾经用过 App 但流失了。

营销数据分析中，最关键的环节就是新客户—流失客户这个阶段，一位用户能和产品互动多久，将决定产品的生命力。听起来和留存挺像的，上文提过的生命周期计算公式，就是脱胎于市场营销。

1.4.2 用户生命周期价值

生命周期价值是用户在生命周期内能为企业提供多少收益，它需要涉及财务定义。互联网行业更多提到生命周期，而不是生命周期价值，因为互联网的商业模式没有传统营销的买和卖那么简单明确。

举个例子，微信用户的生命周期价值能否计算？并不能，不论是游戏或者微信理财，都推导不出一个泛化的模型。但是部分产品，如金融和电商，生命周期价值是可计算的。

以互联网金融举例，某 App 提供理财和现金贷款两种业务，公司从这两个业务中获得收入通常是一个较稳固的比率，而成本支出平摊每个用户头上也是固定常数。所以利润就变成了用户理财和贷款的金额大小，以及生命周期的长短。这两者都是可估算的。

生命周期价值比生命周期重要，因为公司要活下去，就得赚更多的钱，而不是用户使用时间的长短。

1.4.3 客户/用户忠诚指数

忠诚指数是对活跃留存的再量化。活跃仅是产品的使用与否，A 用户和 B 用户都是天天打开 App，但是 B 产生了消费，那么 B 比 A 更忠诚。数据往往需要更商业的指标描述用户，消费与否就是一个好维度。

我们可以用一个简化模型表示：

$$L = \sum_{t=1}^n \frac{s}{s+1}$$

t 是一个时间窗口，s 代表消费次数，代表的距今某段时间内的消费次数。若时间窗口选择月，那么 t=1 是距今第 1 个月内的消费次数，t=2 是距今第 2 个月内的消费次数，列举数据如下。

用户	第1个月消费次数	第2个月消费次数	第3个月消费次数	第4个月消费次数	第5个月消费次数	第6个月消费次数
A	0	0	0	3	0	0
B	2	4	0	0	2	1
C	0	1	0	1	0	0

将消费次数代入 $s/(s+1)$ ，对数据进行转换，它的目的是收敛。以忠诚角度看，消费 10 次和消费 100 次的差异并不大，都属于很高且难以流失的用户，10/11 和 100/101 的关系，并且有效规避极值。对于消费 0 次，1 次，2 次的用户，则对应 0，0.5 和 0.66，在业务上也具备可解释性。

用户	第1个月消费次数	第2个月消费次数	第3个月消费次数	第4个月消费次数	第5个月消费次数	第6个月消费次数	忠诚指数
A	0	0	0	3	0	0	0.75
B	2	4	0	0	2	1	2.63
C	0	1	0	1	0	0	1

各月份求和得出的指数能反应用户在消费方面的忠诚。图例只是解释，实际应用过程中需要归一化，并且考虑时间权重：越近的消费肯定越忠诚。上述的模型在于简单，适合各类商业模式的早期分析，如金融投资，便可以计算用户每个季度的投资次数。

1.4.4 客户/用户流失指数

流失指数是对流失的再量化，它是忠诚指数的反面。流失率衡量的是全体用户，而为了区分不同用户的精细差异，需要流失指数。在早期，流失指数=1-忠诚指数。

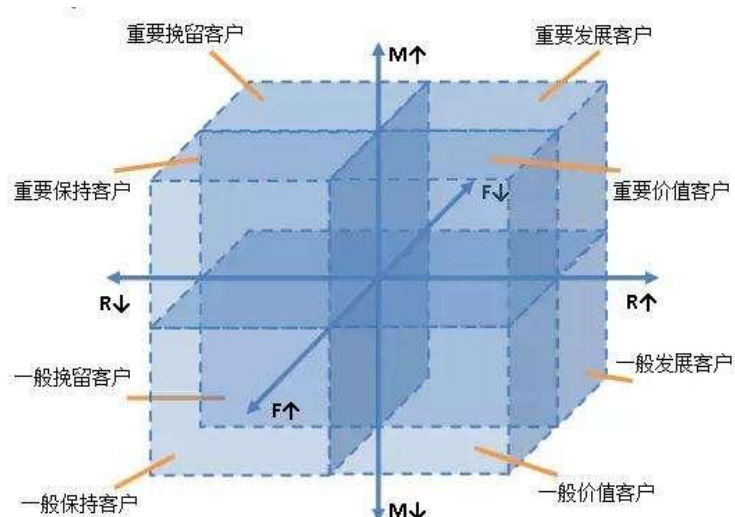
流失指数和忠诚指数的具体定义能根据业务需要调整，比如忠诚按是否消费，流失按是否打开活跃，只要解释能站住脚。

在拥有足够的行为数据后，可以用回归预测流失的概率，输出[0,1]之间的数值，此时流失的概率便是流失指数。

1.4.5 客户/用户价值指数(☆☆重点记忆)

用户价值指数是衡量历史到当前用户贡献的收益（生命周期价值是整个周期，包括未来），它是精细化运营的前提，不同价值的用户采取不同策略以最大化效果。

用户价值指数的主流计算方式有两种，一种是 **RMF 模型**，利用 **R 最近一次消费时间**，**M 总消费金额**，**F 消费频次**，将用户划分成多个群体。不同群体即代表了不同的价值指数。



第二种是主成分分析 PCA, 把多个指标转化为少数几个综合指标(即主成分), 其中每个主成分都能够反映原始变量的大部分信息, 且所含信息互不重复。假设有一个旅游攻略网站, 怎么界定优质的内容贡献者? 用户的文章发布量? 文章被点赞数? 用户被关注数? 文章好评数? 文章更新频次? 每个指标都挺重要的, 主成分分析能囊括上述所有指标, 将其加工成两到三个指标(通常是线性相关指标被合并)。这时再加工成价值指数则不难了。

上述各类指数, 都是针对用户营销的明细数据。如何应用呢? 最经典的是矩阵法, 将指标划分出多个象限, 如用户价值指数和用户流失指数。

用户流失指数	低, 高	高, 高
	低, 低	高, 低
		用户价值指数

对于用户价值高且流失指数高的用户, 应该采取积极的唤回策略, 对于用户价值低且流失指数高, 那么考虑成本的平衡适当运营即可...这就是精细化运营的一个案例, 也是市场营销多年来总结出的有效方法。

1.5 传播/活动

把传播和活动放到一起讲, 它们是一体两面。

1.5.1K 因子

国外用得广泛的概念: 每位用户平均向多少用户发出邀请, 发出的邀请又有

多少有效的转化率，即每一个用户能够带来几个新用户，当 K 因子大于一时，每位用户能至少能带来一个新用户，用户量会像滚雪球般变大，最终达成自传播。当 K 因子足够大时，就是快口相传的病毒营销。

国内的邀请传播，主体自然是微信朋友圈。微信分享功能和网页都是能增加参数统计的，不难量化。

1.5.2 病毒传播周期

活动、广告、营销等任何能称之为传播的形式都会有传播周期。病毒性营销强则强矣，除非有后续，它的波峰往往只持续两三天。这也是拉新的黄金周期。

另外一种传播周期是围绕产品的邀请机制，它指种子用户经过一定周期所能邀请的用户。因为大部分用户在邀请完后均会失去再邀请的动力，那么传播周期能大大简化成如下：假设 1000 位种子用户在 10 天邀请了 1500 位用户，那么传播周期为 10 天，K 因子为 1.5，这 1500 位用户在未来的 10 天内将再邀请 2250 位用户。

理论上，通过 K 因子和传播周期，能预测依赖传播带来的用户量，可实际的操作意义不大，它们更多用于各类活动和运营报告的解读分析。

1.5.3 用户分享率

现在产品都会内嵌分享功能，对内容型平台或者依赖传播的产品，分享率是较为重要的指标，它又可以细分为微信好友/群，微信朋友圈，微博等渠道。

有一点值得注意，数据只能知道用户转发与否，转发给谁是无法跟踪的。所以产品用物质激励用户分享要当心被薅羊毛。

1.5.4 活动曝光量/浏览量

传播和线上活动是息息相关的，这两者的差异不大。想要做好一个活动，单纯知道活动的浏览量是不够的，好的活动一定是数据分析出来的。以朋友圈最常见的红包营销举例。它的分析通过网页参数，如下：

aaa.com/activity/bigsales/?source=weixin&content=h9j76g&inviter=00001×tamp=1495286598

问号后面的是网页参数，source=weixin 说明网页是分享到微信的。content=h9j76g 是页面具体内容，这里则是营销红包的类型。inviter=00001 说明是哪个用户分享出去的，timestamp 则是分享的具体时间戳。不同用户的分享页面有不同参数，按此作区分。

当这些页面被用户分享到朋友圈时，数据采集系统会记录所有页面的打开浏览。而页面参数则是活动精细化分析的前提。通过 source=weixin，数据分析师知

道了红包活动在微信的浏览量，相对应的还有 QQ 和微博。content 则能看出用户喜欢哪个类型的红包，哪种红包被领取得多，成本又是多少。inviter 则能看出平均每个分享者的分享页能带来多少浏览量。

参数越多，分析的维度就能越细，活动可优化的空间也越大。如果大家有心的话，可以看朋友圈（包括网页）各种活动的网页参数，观察其他产品的分析维度，它山之石可以攻玉，这是一个好习惯。

1.5.5 活动参与率

活动参与率衡量活动的整体情况，可以套用用户活跃的分析指标。

这个活动的参与人数（活跃数）多少？有多少老用户参与了这个活动？有多少新增用户因为这个活动来，传播类的活动分享数据怎么样？活动中的各个流程转化如何？活动带来多少新订单。其实，运营活动可以看作一个短生命周期的产品，产品的一切指标都能应用于其中。

好的活动应该机制化，把它融入到产品的功能机制中，比如滴滴打车的红包，美图饿了么的红包，都是从活动逐渐变成一种打法和抓手。更早期的各类网游，也是通过活动的推成出新成为了现在常态化的游戏功能。

活动的机制化，意味着数据要分析活动指标，发现优点以改进，之后同样常态化成报表：今天使用了多少红包，今天有多少用户因为活动新增，等等。

1.6 营收(☆☆重点记忆)

产品，运营或者市场人员，从来不是为活跃、留存负责，而是商业，是企业的根本财务。数据分析也不是为了提高活跃和留存，而是像一个巨头的漏斗，最终将业务驱动于此，即回归商业的本质。

1.6.1 活跃交易用户数

从产品曝光到用户下载，用打开活跃到产生收入，产品的指标在一步步往商业靠拢，活跃交易用户则是核心指标。整个流程呈现漏斗状。

这里的交易，即是买方的消费，也包含卖方的供应。若平台包含 B 端和 C 端，则两端同等重要，均需要纳入数据体系。

和活跃用户一样，活跃交易用户也可以区分成首单用户（第一次消费），忠诚消费用户，流失消费用户等。细分交易数据和指标，关系到产品商业化的进展，所以是有必要的。其实到这个环节，各类指标已经更倾向用户画像，而非报表统计了。

活跃用户交易比，统计交易用户在活跃用户中的占比。当产品活跃用户足够

多，但是交易用户少，此时的商业化是有问题的，俗称的变现困难，很多公司都倒在这一步。

1.6.2 GMV

成交总金额，只要用户下单，生成订单号，便可以算在 GMV 里，不管用户是否真的购买了。互联网电商更偏好这个指标。

成交金额对应的是实际流水，是用户购买后的消费金额。销售收入则是成交金额减去退款。至于利润、净利率，涉及到财务成本，数据分析挺难拿到这类数据，所以不太用到。

把上述的三个指标看作用户支付的动态环节，则能再产生两个新指标，这也是数据分析的思维之一。成交金额与 GMV 的比率，实际能换算成订单支付率；销售收入和成交金额，也涉及到了退款率，当分析陷入卡顿时，不妨观察下这两个指标，或许有帮助。

1.6.3 客单价

传统行业，客单价是一位消费者每一次到场消费的平均金额。在互联网中，则是每一笔用户订单的收入，总收入/订单数。

很多游戏或直播平台，并不关注客单价，因为行业的特性它们更关注一位用户带来的直接价值。超市购物，用户购买是长周期性的，客单价可以用于调整超市的经营策略，而游戏这类行业，用户流失率极高，运营人员更关注用户平均付费，这便是 ARPU 指标，总收入/用户数。

ARPU 可以再一步细分，当普通用户占比太多，往往还会采用每付费用户平均收入 ARPPU，总收入/收费用户数。

1.6.4 复购率

若把复购率说成营收届的留存率，你就会知道它有多重要了。和新增用户一样，获得一个新付费用户的成本已经高于维护熟客的成本。

在不少分析场景中，会将首单用户单独拎出来作为一个标签，将两次消费以上的用户作为老客，之所以这样做，是从一到二的意义远不止加一那么简单。

用户第一次消费，可能是体验产品，可能是优惠，可能也是运营极大力地推动，各类因素促成了首单。而他们的第二次消费占比会有断崖式下跌（对应次日留存率的下跌），因为这时候的消费逐渐取决于用户对产品的信任，模式的喜欢或者习惯开始养成。

很多时候，用户决策越长往往意味着客单价越高，如投资，旅游。此时首单复购率越是一个需要关注的指标，它意味着更多的利润。

复购率更多用在整体的重复购买次数统计：**单位时间内，消费两次以上的用户数占购买总用户数。**

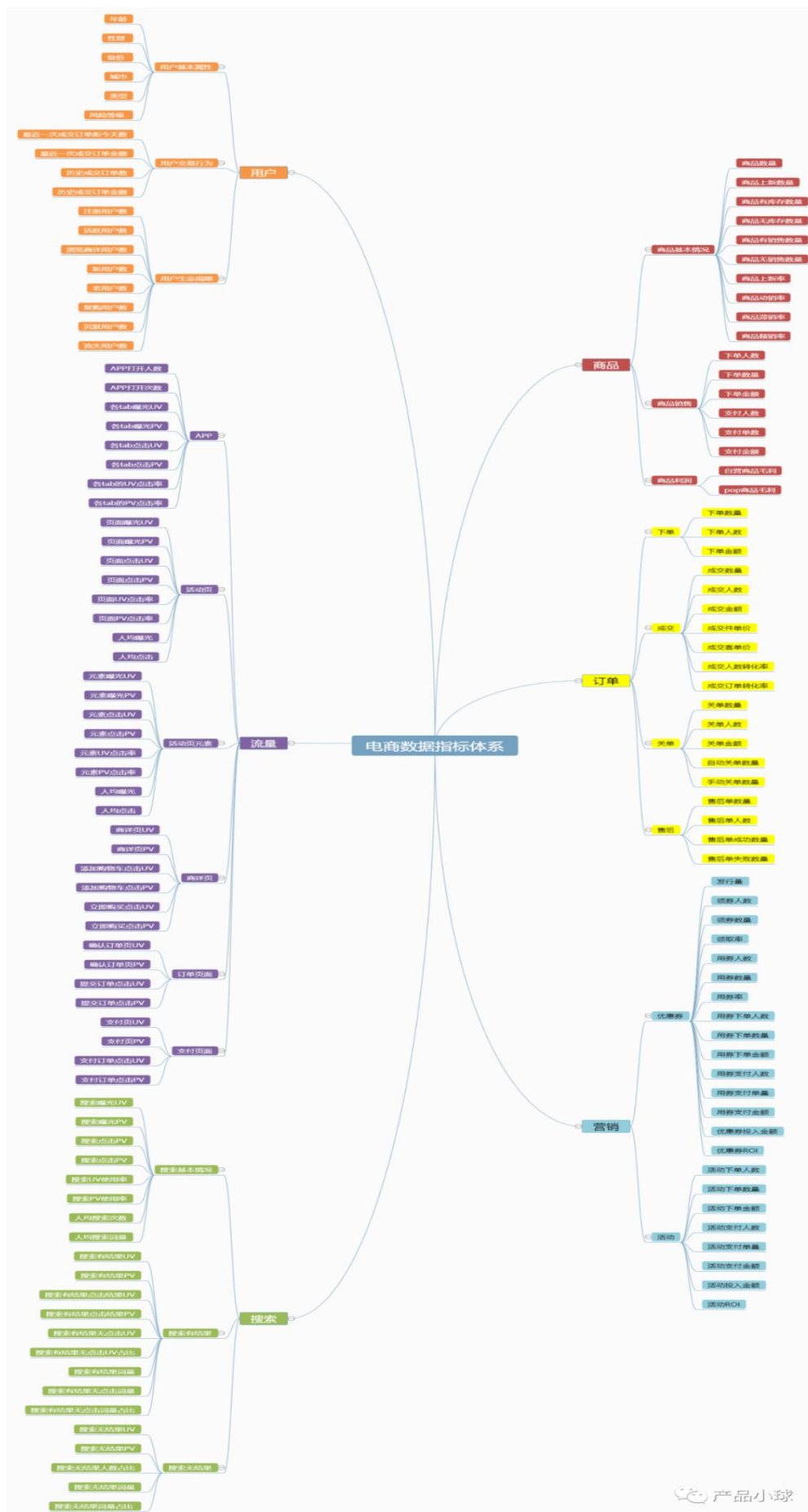
回购率是另外一个指标，指的是上一个时间窗口内的交易用户，在下一个时间窗口内仍旧消费的比率。例如某电商 4 月的消费用户数 1000，其中 600 位在 5 月继续消费，则回购率为 60%。600 位中有 300 位消费了两次以上，则复购率是 50%。

1.6.5 退货率

退货率是一个风险指标，越低的退货率一定越好，它不仅直接反应财务水平的好坏，也关系用户体验和用户关系的维护。

2. 电商数据指标(☆☆☆重点掌握)

指标体系具体如下：



2.1 用户

用户是电商“人—货—场”体系的根基，没有人来，再优质的货，再豪华的场，也没有任何意义。

把用户运营好，才有可能形成转化。

1) 用户基本属性

年龄；

性别；

省份；

城市；

类型：例如用户的当前身份，为学生、白领等；

风险等级：在很多涉及金融的分期电商，用户的风险等级是一个重要属性，决定该用户是否可以获取额度并使用分期购物。

以上用户的基本属性，主要是帮助平台建立用户画像，分析用户的电商消费喜好。

2) 用户交易行为

最近一次成交订单距今天数；

最近一次成交订单金额；

历史成交订单数；

历史成交订单金额。

通过前两个指标，可以判断该用户当前的活跃状态，是否需要对齐进行激活或者召回。

通过后两个指标，则可以清楚计算得到用户的客单价，也就是该用户的 ARPU 值，是衡量用户价值的关键。

3) 用户生命周期

注册用户数：注册电商平台用户数；

活跃用户数：登录 APP 用户数；

浏览商详用户数：浏览商品详情页用户数；

新用户数：历史成交订单数为 0 的用户数；

老用户数：历史成交订单数大于 0 的用户数；

复购用户数：历史成交订单数大于 1 的用户数；

沉默用户数：距离上次登录 APP 大于 30 天，小于 90 天的用户数；

流失用户数：距离上次登录 APP 大于等于 90 天的用户数。

根据用户的生命周期，对用户进行区分，有利于对用户进行分层精细化运营，针对不同阶段的用户，采取不同的运营策略，例如新用户可通过新人优惠券和 push

尽快促其完成首单，沉默用户数可通过短信和专属优惠将其召回。

需注意，不同平台对于自身沉默和流失的用户定义不同，有些平台会通过活跃来判断，也有些是通过交易来判断。此次仅做参考，需结合自身平台特点和业务诉求进行制定。

2.2 流量

用户进来后，最先承接用户的就是各级页面，包括 APP 的访问、首页、各活动页、商详页以及各页面内元素的点击，这些就是流量统计的关键指标。

1) APP

APP 打开人数；

APP 打开次数；

各 tab 曝光 UV；

各 tab 曝光 PV；

各 tab 点击 UV；

各 tab 点击 PV；

各 tab 的 UV 点击率=各 tab 点击 UV/各 tab 曝光 UV；

各 tab 的 PV 点击率=各 tab 点击 PV/各 tab 曝光 PV。

一般情况下，我们定义打开 APP 为活动人数，所以需要通过统计 APP 打开人数和次数来衡量活跃情况。

当前的 APP 多存在多个底部 tab，例如淘宝有首页、逛逛、消息、购物车和我的淘宝。

每个 tab 都会有对应的曝光和点击，并可依此计算点击率。



页面中各模块的点击流量

各tab的点击流量

产品小球

2) 活动页

页面曝光 UV;

页面曝光 PV;

页面点击 UV;

页面点击 PV;

页面 UV 点击率=页面点击 UV/页面曝光 UV;

页面 PV 点击率=页面点击 PV/页面曝光 PV;

人均曝光=页面曝光 PV/页面曝光 UV;

人均点击=页面点击 PV/页面点击 UV。

活动页包括首页、各级活动页、频道页，对于页面的统计流量，最重要的是曝光和点击，依此可以判断一个活动页面的客流量。

也可以参考人均点击、点击率等指标去判断单活动页面对用户的吸引力。

3) 活动页元素

元素曝光 UV

元素曝光 PV

元素点击 UV

元素点击 PV

元素 UV 点击率=元素点击 UV/元素曝光 UV

元素 PV 点击率=元素点击 PV/元素曝光 PV

人均曝光=元素曝光 PV/元素曝光 UV

人均点击=元素点击 PV/元素点击 UV

活动页元素，是指上述页面中的元素，例如 banner、icon、feed 流、广告位、弹窗和 floating 等。

这是活动页流量监控的重要数据，只有知道每个元素的点击率，才能知道用户对页面中哪些内容更感兴趣，依此指导运营进行页面的设计和调整。

尤其是广告位的投放，很多平台对于广告位投放内容，是需要进行内部竞争的。竞争的主要依据之一就是点击率，如果投放的内容点击率高，说明投放价值相对更高，当然这个还需要结合后续转化来查看。

4) 商详

商详页 UV;

商详页 PV;

添加购物车点击 UV;

添加购物车点击 PV;

立即购买点击 UV;

立即购买点击 PV。

商品详情页是下单支付流程的重要页面，主要记录用户对商品的浏览和跳转下单情况。除了基本的浏览数据、点击加车和购买数据。

其实商详页还有许多模块，例如优惠券弹层，用户是否点击查看有哪些优惠券并点击领取等。在实际的埋点中，所有的弹层浏览和点击都应该有埋点，根据业务需要，可以对页面上相应模块的浏览和点击数据都进行观测。



页面中各模块的点击流量

用户主要路径行为：加车、购买的点击流量

5) 订单页面

确认订单页 UV;

确认订单页 PV;

提交订单点击 UV;

提交订单点击 PV。

订单确认页同样是下单支付流程的核心页面，用户不管是通过商详页还是购物车页面，要下单支付时，都必须经过订单确认页，确认订单相关信息后，再跳转至支付。

其中，页面浏览数据和点击提交订单数据是最重要的，这是下单流程漏斗的关键数据指标。还有其他的模块也可以适当关注，例如优惠券弹层，用户是否点击查看有哪些优惠券并点击使用；选择收货地址弹层等。



页面中各模块的点击流量

用户主要路径行为：提交订单
的点击流量

6) 支付页面

支付页 UV；

支付页 PV；

支付订单点击 UV；

支付订单点击 PV。

支付页是下单支付流程的最后一个页面，也是成交的关键，用户只有最终成交了才意味着 GMV 的提升。支付页面一般也叫收银台页面。

页面浏览数据和点击支付数据是最重要的，这是下单流程漏斗的关键数据指标。很多平台在收银台还会开放其他功能，例如选择不同支付方式，支持选择分期支付并选择不同分期数，可以通过用户的点击去观察用户的支付喜好。



页面中各模块的点击流量

用户主要路径行为：支付订单
的点击流量

产品小球

2.3 搜索

在电商业务中，搜索是一个非常重要的业务入口，当用户无明确购买目标时，可能会浏览 feed 流的推荐商品。但是当用户有非常明确的购买目标时，搜索永远是用户的第一大入口。

1) 搜索基本情况

搜索曝光 UV；

搜索曝光 PV；

搜索点击 PV；

搜索点击 PV；

搜索 UV 使用率=搜索点击 UV/搜索曝光 UV；

搜索 PV 使用率=搜索点击 PV/搜索曝光 PV；

人均搜索次数=搜索点击 PV/搜索点击 UV；

人均搜索词量=(搜索有结果词量+搜索无结果词量)/搜索点击 UV。

搜索入口会存在于多个页面，一般多在首页，让用户明显能看到，所以需要记录其曝光和点击的数据，分析用户对搜索的使用情况。

除此之外，可以观测人均搜索次数和词量。如果用户搜索很多，一方面说明其来到该平台的目的明确，一方面也需要关注是否相关推荐不够精准，导致用户只能频繁使用搜索。



用户主要路径行为：点击搜索的点击流量

页面中各模块的点击流量

2) 搜索有结果

搜索有结果 UV;

搜索有结果 PV;

搜索有结果点击结果 UV;

搜索有结果点击结果 PV;

搜索有结果无点击 UV;

搜索有结果无点击 UV 占比 = 搜索有结果无点击 UV / 搜索有结果 UV;

搜索有结果词量;

搜索有结果无点击词量;

搜索有结果无点击词量占比 = 搜索有结果无点击词量 / 搜索有结果词量。

根据搜索结果可分为搜索有结果和搜索无结果，搜索有结果时可以查看其搜索有结果的人数和次数，以及其出结果后是否点击。

如果搜索有结果，但无点击占比很高，那可能需要关注搜索结果是否不够准确，搜索出来的商品并不是用户想要的，所以用户不想点击。



搜索结果的点击流量

3) 搜索无结果

搜索无结果 UV;

搜索无结果 PV;

搜索无结果人数占比=搜索无结果 UV / 搜索点击 UV;

搜索无结果词量;

搜索无结果词量占比=搜索无结果词量 / (搜索有结果词量+搜索无结果词量)。

相对于搜索有结果，搜索无结果则需要重点关注词量问题。

如果搜索无结果词量很多，且占比很高，则意味着当前商品短缺或搜索匹配算法需要优化。

当用户整体搜索无结果人数占比提升时，则需要马上关注搜索产品的优化了，如果越来越多的用户来搜索却没有得到结果反馈，那么用户将流失得越来越

多。

在搜索数据中，观测思路可以是：

先关注无结果数据，降低搜索无结果占比，确保用户搜得到东西；

再关注有结果数据，降低有结果无点击占比，确保用户搜到的东西是他想要的；

最后关注有结果点击数据，判断用户搜到且点击的东西最终有没有形成转化。

2.4 商品

商品主要看其销量和利润，销量通过下单支付指标，而利润则通过毛利指标。

1) 商品基本情况

商品数量；

商品上新数量；

商品有库存数量；

商品无库存数量；

商品有销售数量；

商品无销售数量；

商品上新率=商品上新数量 /商品数量；

商品动销率=商品有销售数量 /商品数量；

商品滞销率=商品无销售数量 /商品数量；

商品倾销率=商品无库存数量 /商品数量。

商品的数量体现了电商中“货”的能力，先有货才能有成交。

但除此之外，需要关注几大指标，商品上新率反馈平台是否有持续迭代更新商品。

商品动销率确保不仅有商品，而且还有人买。

如果视频滞销率过高，则需要采取相应措施，提升长尾商品的曝光。

如果商品倾销率很高，一方面可以体现平台用户喜好，一方面也需要注意供应链是否完备，是否需要迭代优化以支撑商品销售。

2) 商品销售

下单人数；

下单数量；

下单金额；

支付人数；

支付单数；

支付金额。

商品的下单和支付数据指标主要反馈商品的销售情况。

3) 商品利润

自营商品毛利=成交金额-采购成本-仓储成本-物流成本+返利；

pop 商品毛利=成交金额*扣点比例-平台促销费用。

商品的利润可分为自营商品和 pop 商品来计算。对于大平台来说，既有自营商品，也会有店铺入驻，通过分佣扣点的方式来计算平台利润。

自营商品毛利需要在商品成交金额上扣除采购成本、仓储成本和物流成本。除此之外还可能与品牌方有合作，销售某商品可获得对应返利，这也是收入的一部分。需注意的是，此处仅考虑商品的毛利，都是从商品的角度出发，像人力成本则不考虑在内。

对于 **pop 商品，即入驻店铺销售商品**，一般通过扣点比例计算佣金，同时还要扣除平台花费的促销费用，例如平台发放的优惠券金额等。

2.5 订单

订单是电商体系的核心，是一切转化和统计的关键。订单全流程包括下单、支付、成单、售后。

1) 下单

下单数量；

下单人数；

下单金额。

2) 成交

成交数量；

成交人数；

成交金额；

成交件单价=成交金额/成交数量；

成交客单价=成交金额/成交人数；

成交人数转化率=成交人数/下单人数；

成交订单转化率=成交数量/下单数量。

除了基本订单成交指标，还需要观测客单价，客单价是衡量用户价值的关键，也可以了解用户的消费习惯。

此外，通过成交的转化率可查看订单的流失情况，进而优化下单支付的流程漏斗。

3) 关单

关单数量；

关单人数；

关单金额；

自动关单数量；

手动关单数量。

订单关单包括用户主动取消订单或者系统因为各种原因取消订单。除了关注数量，需要关注关单的具体原因，例如用户主动取消订单的原因分类，是价格过高还是质量不好等。相当于一个用户调研，以此根据用户反馈不断迭代。

4) 售后

售后单数量；

售后单人数；

售后单成功数量；

售后单失败数量。

售后形式一般包括退款、退货和换货。观测售后单量和人数可以确认订单的履约质量，如果售后单量过高，说明商品质量和服务存在问题。同时还要关注售后单的结果，判断是否为用户提供了良好的售后服务。

2.6 营销

电商营销多通过优惠券和各类促销活动来完成，包括单品直降活动、满减满折优惠活动、拼团活动等。不同活动的基本衡量指标类似，但优惠券因为设置上不同所以对应指标也存在一些差距。

1) 优惠券

发行量；

领券人数；

领券数量；

领取率=领券数量/发行量；

用券人数；

用券数量；

用券率=用券数量/领券数量；

用券下单人数；

用券下单数量；

用券下单金额；

用券支付人数；

用券支付单量；

用券支付金额；

优惠券投入金额=用券支付单量*优惠券面额；

优惠券 ROI=优惠券投入金额/用券支付金额。

优惠券的领取率和用券率可以帮助运营发现漏斗问题，是用户不领券还是领完券

不用券下单。

优惠券的 ROI 是衡量优惠券活动价值的关键，通过用券支付金额可以确认优惠券带来的订单金额，再通过消耗的优惠券总数量与优惠券面额相乘得到总投入成本，最终计算得到投入产出比。

2) 活动

活动下单人数；

活动下单数量；

活动下单金额；

活动支付人数；

活动支付单量；

活动支付金额；

活动投入金额；

活动 ROI=活动投入金额/活动支付金额；

活动的指标与优惠券类似，主要以 ROI 作为活动价值的衡量。以上流量、订单、商品、营销等指标，既可以用于平台自营，也可以用于平台入驻商家的运营衡量。如果像京东这样的既有自营又有 pop 商家的平台，只需将不同指标应用于不同对象即可。

3. 本章小结&面试要点

3.1 本章小结

本章从互联网运营的视角出发，给大家介绍了用户获取、活跃、留存、营销、传播、营收相关的业务指标，这些指标都是互联网公司常见的业务衡量指标。在此基础上，给大家重点介绍了电商数据的指标体系，该体系可以理解为通用的指标体系，也是各个公司常用的指标体系。

3.2 面试要点

1. 请简要说明一下常见的 5 种广告付费模式。
2. 在成熟的运营体系下，会将活跃用户再细分为哪些类别？
3. 请解释一下什么是留存率？新增留存率、活跃留存率怎么计算？15 日留存率、15 日内留存率的区别是什么？
4. 请简要介绍一下衡量用户价值指数的 RMF 模型。
5. 请解释一下客单价、GMV、复购率等指标的含义。

6. 请介绍一下电商中常见的用户指标（10 种）。
7. 请介绍一下电商中常见的流量指标（10 种）。
8. 请介绍一下电商中常见的搜索指标（10 种）。
9. 请介绍一下电商中常见的商品指标（10 种）。
10. 请介绍一下电商指标体系中常见的订单交易指标（10 种）。

二、企业级电商产品设计策略

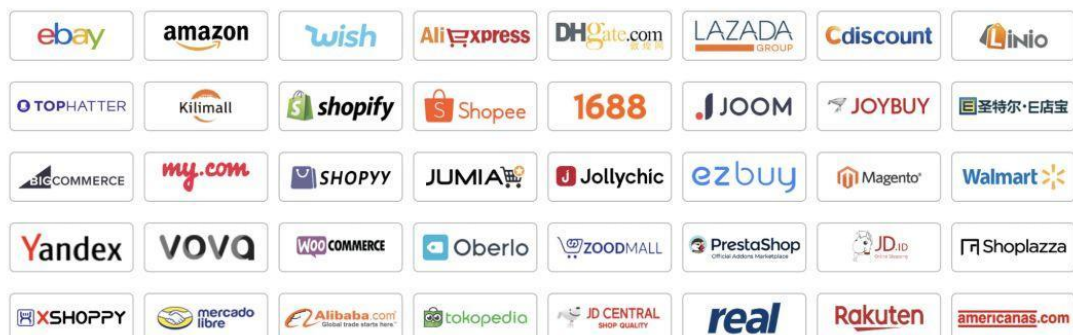
1. 百花齐放的电商平台

电商发展二十多年，电商平台数不胜数。哪里有流量，商家就会往哪里去。我列举了国内主要的电商平台，以及跨境主流的电商平台。大家看下听过哪些平台？

以天猫、京东、拼多多为主的国内电商：国内电商平台 80+



以 Amzon、Ebay 为主的跨境电商平台：



2. 电商中的人、货、场

在传统行业中，人们在进行交易时虽然不知道“人、货、场、内容”理论到底是什么，也没有理论做支撑，但是会默默地遵守这套理论模型。比如，大壮想开一家服装店，假设他的资金充足，他需要做以下几件事情：

- (1)调研商品，选品。他要清楚地知道用户对服装类型的购买意向，是选择女装、童装、冬装，还是选择西装等。
 - (2)在选好服装类型后，他其实已经限定了用户类型，知道了自己的商品会卖给哪些用户。
 - (3)他需要进行店铺选址。他要优先选择人流量大的位置，比如核心商圈店铺、小区店铺或者在校园内的店铺。
 - (4)他要进行店铺营销。一些商家会印刷很多小卡片或者传单，在上面描述一些促销信息，在线下采用人工的形式扩散。促销人员一般在商店附近发放小卡片或者传单，并直接把客户带入店内消费。
- 线下零售的过程如图所示。



2.1 如何定义人、货、场

人、货、场，分别代表选人、选品、选址。

- (1)选品。挑选合适的商品类型，如餐饮类、服装类、家居类或服务类商品。
- (2)选人。针对正确的用户，如男性、女性、儿童、发烧友等。
- (3)选址。寻找合理的销售地址，如店铺位置、摊位等。

这是在传统意义上的人、货、场，而在互联网中的新型“人、货、场”应该是找到全网最精准的用户，为这些用户定制个性化的服务和商品，最后通过合理的营销手段触达这些用户。

1. 熟悉用户特征

每个用户都有自己的喜好。例如，小明喜欢打游戏，是游戏宅男；花花酷爱美妆，每天都会看美妆直播，甚至自己也做直播。互联网会精准地记录用户特征，我们能够通过数据全方位地了解用户，建立用户画像。

比如，花花在经过银泰城附近时，她的手机会收到一条消息，告诉她银泰城内的YSL有打折优惠，然后电商产品可以结合花花的消费情况，再赠送她一张对应面额的优惠券。这就是目前的新零售电商模式。通过数据，电商产品培养了用

户的消费习惯。

2.用数据改造货物

用户在互联网中沉淀的数据能够让平台全方位地了解用户。比如，在开发新品的时候，我们发现在平台中有 34%的用户对补水和保湿很感兴趣，恰好公司正在研究面膜产品，我们就可以针对补水和保湿多做工作，这样我们的产品将会有不错的竞争力。这就是 C2B，通过用户反馈反向推动品牌开发合理的新品。

3.场的塑造

场的塑造是最关键的步骤,我们需要让货物通过合适的营销场触达精准的人。场可以是时间场，也可以是空间场。

前者通过合适的时间推送营销信息，比如在“双 11”狂欢节、年货节、圣诞节或情人节等特殊的日子推送。

后者则通过可以面向用户的场所推送营销信息，在线上有活动会场、促销页面或店铺，在线下有门店、摊位。

时间和空间结合可以塑造最完美的营销场，人、货、场结合则可以获得最大的利益。

4.内容“种草”

在做销售时，很多商家都会很困惑，即使用了五花八门的促销手段也有增长瓶颈。这时商家就需要使用其他额外的手段，比如以内容方式将商品信息渗透给用户。

商品的评价和问答是用户间沟通的桥梁，用户通过商品的评价和问答可以判断商品的质量。帖子、直播和短视频不断地通过内容信息告知用户商品的好坏，类似于广告，在用户心中“种草”。

人、货、场的内容理论是本章内容的核心重点论断。它们的结合可以帮助电商平台提升 GMV，核心的营销手段是在前期用内容不断渗透，在中期找到精准的人，并提供最适合的商品，再用促销手段刺激用户，从而将货物销售出去。无论是人找货还是货找人，都离不开这套理论模型。

2.2 电商的数据化思维

在互联网行业中，数据是指导产品优化的重要依据。我们在做业务决策时必须先了解数据情况，在有了数据支撑后，再有针对性地制定相应的策略。比如，在卖货的会场中，我们发现产品在上架后页面的点击率很低，只有 40%，而同级别产品的点击率可以达到 70%，此时就需要进行实时调整，具体方案如下：

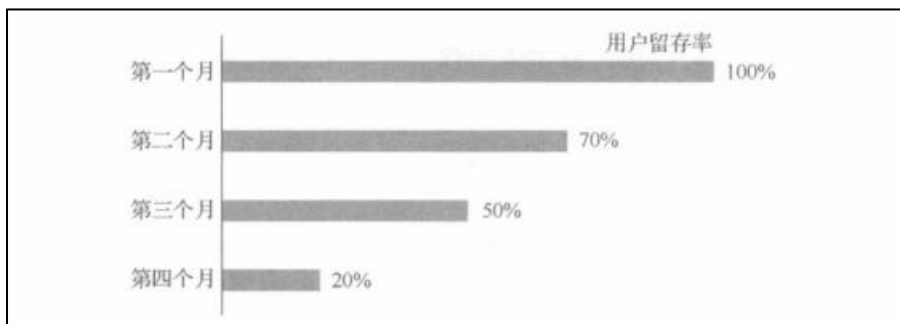
(1)把权益模块(比如，领券模块、购物津贴模块等)放置到页面的上方，在第一屏的中间位置展示。

(2)内容全部用个性化推荐,根据用户在电商平台的喜好数据向用户推荐和预测他喜欢的内容。

数据化思维以数据作为参考依据,对已有的数据进行分析、挖掘、处理,给出合理的解决方案,而不能通过拍脑门或靠经验做决策。数据化思维方法的应用大体上可以分为三个步骤,以电商 App 留存为例。

第一步,根据数据情况观察产品情况。主要观察问题的表象,如果可以把问题写下来,那么问题就已经解决了 50%。通过数据分析,我们能够将问题拆解得清晰且明确。

比如,在近三个月内,平台的用户量下降明显,如图所示。产品的用户留存率从开始的 100%逐渐降到第四个月的 20%,下降趋势也有所增加。



第二步,数据分析。通过电话访问、线上调研等方式,总结用户所遇到的问题,了解用户是如何思考的和用户为什么离开平台。

第三步,根据第二步的分析结果,给出有针对性的结论和解决办法。比如,发现绝大多数用户离开的原因是服务质量问题,平台不提供 x 天无理由退货,所以用户流失了,App 留存率降低。

在电商环境中,我们需要重点关注 GMV、转化率、DAU/MAU、用户留存率、拉新和复购 6 大指标。

1.GMV (这里转化率指下单人数/独立访客人数)

GMV 是电商行业最重要的指标之一,计算公式为 $GMV = \text{客单价} \times \text{转化率} \times UV$ 。所以要想提高 GMV,最有效的手段是出售高端的高客单价产品,提高购买转化率和购买人数。在电商中,GMV 是一个公司炫耀的资本,比如在双 11 狂欢节期间天猫的 GMV 为 1500 亿美元。其实,GMV 是一个非常宽泛的口径,只要用户在前台提交了订单,无论其是否付款、是否取消都会记录到 GMV 中,所以就产生了刷单、刷 GMV 的情况,尤其是电商的黑产。

2.转化率

转化率是电商业务的核心指标。转化率提高意味着订单量增加、利润提高。电商用户的转化漏斗如图所示,是天然的金字塔形漏斗。曝光流量虽然数量大,但是实际上最终进行转化的只有部分流量。所以,我们更需要把非转化流量变成

转化流量。



3.DAU/MAU

DAU/MAU(Daily ActiveUser/Month ActiveUser, 日活跃用户数量/月活跃用户数量)是电商产品的基础指标, 一般用于表明产品的运营情况。如上图所示, 如果我们想要提高转化流量, 那么需要有足够的 DAU, 即曝光流量。

4.用户留存率

用户留存率是产品的用户黏性指标, 我们主要考核产品的次日留存率、7 日留存率和月度留存率。

5. 拉新

拉新是产品永恒的话题。无论产品发展到什么阶段都需要获取新用户, 持续开拓新的流量入口。用户增长不仅包括用户量的累积, 而且应该囊括产品生命周期各个阶段的重要指标。

6.复购

复购即重复购买, 曝光流量在成为转化流量后, 拉新就成功了, 但是我们需要继续跟进这些用户, 通过相应的手段让新客变成老客, 让他们继续在平台中产生下单行为。某平台的数据表明, 63%的 GMV 是由老客贡献的, 可想而知复购的重要性。

除此之外, 还有访问时长、点击率指标等, 我们需要根据目标选择适合的考量标准。比如, 对页面效率, 我们需要考核点击率;对用户活跃, 我们需要考核 DAU。

3. 电商领域——人

3.1 会员体系

目前, 有两种主要的会员体系类型, 第一种为免费会员体系, 第二种为付费会员体系。我们所讲述的主要为免费会员体系。

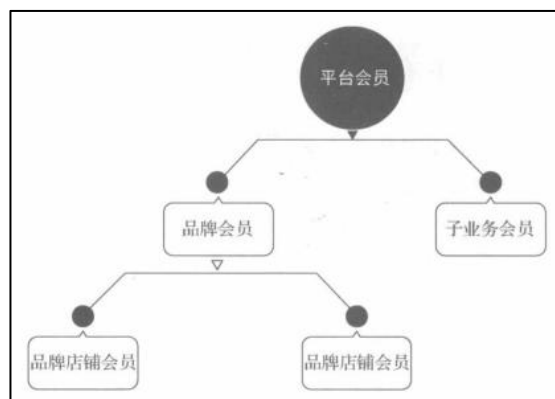
其实，电商中的各种产品形态都是现实生活中的缩影，会员体系也不例外。在传统零售中，到处都可以看到会员体系的身影。例如，山姆会员店会员、宜家会员，前者为付费、有门槛的会员，后者则为免费、无门槛的会员。当然，付费、有门槛的会员可以得到更多好处，如果购买次数较多，那么入会费是完全可以赚回来的。

用户一旦成为付费会员，那么在会员期内，这个用户就会有很大概率再次发生购买行为。对平台来说，这样提高了用户黏性，扩大了销售规模；对用户来说，用户在入会后可获得更多优质的商品和优惠的价格。

会员体系是用户运营中非常重要的一环，不但能够给产品带来源源不断的新用户，还能促使老用户变得更加活跃。不过，并不是在任何阶段都需要会员体系，或者基于会员做其他产品的设计，具体问题要具体分析。

当产品处于初期时，我们是不需要会员体系的。产品运营的重心应该是拉新的策略。比如，通过哪些手段、带来哪些收益。此时，产品的用户量可能还没有那么高，徒增会员体系也枉然。当产品处于发展期时，我们就可以考虑引入会员体系。在这个阶段，产品的用户量已经有了一定的积累，存量用户需要被促活，增量用户需要被刺激。当产品处于成熟期时，我们不但应该大力发展会员体系，而且还要做更多会员营销方面的事情。

当产品发展到一定阶段时，为了自身业务的发展，我们会发现，很多子业务也需要发展会员体系，比如店铺会员、B2B 会员等，这些会员体系呈树状结构，如图所示。



在淘宝系内，有公域和私域之分，从字面上也能理解，前者是公共的流量资源，是由官方管控的渠道，如首页的资源位、导购页的流量，后者为商家自己可以控制的流量渠道，如店铺、消息等。

1. 平台会员

平台会员是在整个公域内建立起来的会员体系，如天猫会员、京东会员。会员等级的定义、会员权益的发放管理以及会员信息的维护等字段都是由平台控制的，决定权属于平台。

2.品牌会员

品牌会员是商家在私域内建立起来的会员体系，与平台会员不同的是，对会员的管理权属于商家自己，平台只提供会员工具，不负责最终的管理。在这个模式中，商家的会员设置更灵活，能够按照自己品牌的调性个性化地配置相应的会员政策。

3.品牌店铺会员

很多大品牌从事的其实是集团性的业务，旗下会有很多子品牌，以宝洁为例，它的旗下有飘柔、舒肤佳、玉兰油、帮宝适等子品牌。母品牌和子品牌可能会经营两个旗舰店，并拥有不同的粉丝群体。由于不同子品牌的受众用户不同，所以它们需要分别使用不同的运营策略和市场营销打法。最后，它们可能培养两套会员体系，也就是品牌店铺会员，它属于更细颗粒度的会员，适用性不强。

4.子业务会员

子业务会员可以被理解为在电商环境中，不同业务线的会员体系。例如，京东商城有平台会员，下属于业务沃尔玛也有一套自己的会员体系；天猫旗下的银泰体系也有自己的会员体系。

表中所示为会员体系的详细定义。

	平台会员	品牌会员	品牌店铺会员	子业务会员
运营主体	平台	商家	商家	平台
会员等级	京东 PLUS 会员、淘宝超级会员	自定义会员	自定义会员	业务会员
成长体系	京东 PLUS 会员：收费会员。淘宝超级会员：根据购买、评价等行为综合考量	自定义体系	自定义体系	业务自定义

针对会员做一些可增值的事情，在日常生活中，我们会遇到一些商家推荐我们办会员卡的事情。比如，在你家楼下的理发店，洗剪吹原价为 50 元，你办一张会员卡需要 300 元，能用 8 次；附近的超市推出新的会员卡，消费可以攒积分，积分能够当钱花；办理招商银行信用卡，如果首笔刷卡满 499 元，就可以免年费。线下的会员营销玩法如下：

- (1) 会员可以享受商品或者服务的折扣价格。
- (2) 办会员卡，可以享受购物返积分。
- (3) 会员做了某些事情，在达成指标后能够得到相应的奖励。

电商的很多业务是现实的缩影，只是用线上的方式表达而已。我们上面提到的那些玩法，也可以用在线上，而且已经有很多平台在使用了，如京东和淘宝，效果也都不错。

随着流量红利期进入尾声，精细化运营才是未来的主要营销模式。会员玩法就是精细化运营的一部分，其主要方式是通过利益钩子获取用户的忠诚度并增加他对平台的认知。通过积分的小成本事件，我们可以撬动更多的用户资源。我们

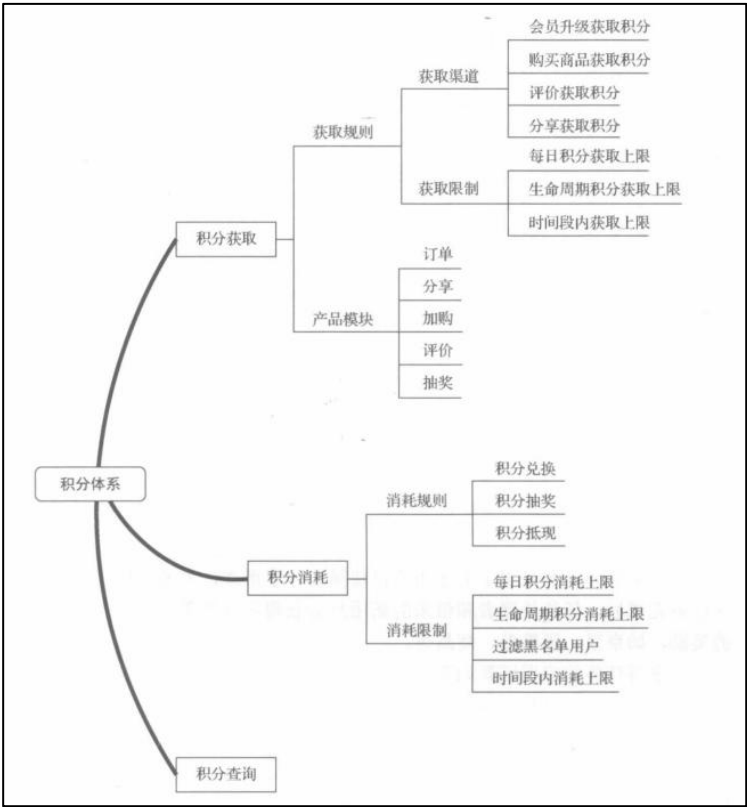
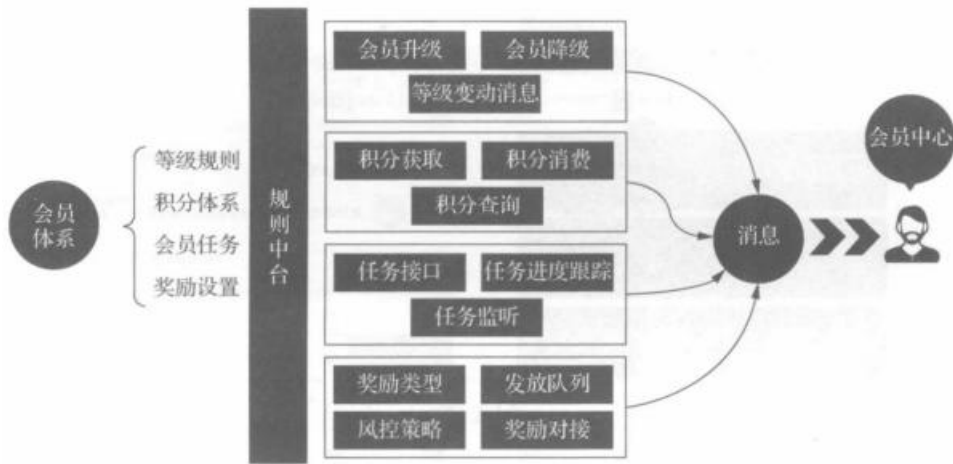
接下来了解一下基于会员的玩法。下图所示为会员体系的系统架构图，其最主要的四个模块如下。

(1)等级规则。该模块包含会员升级、会员降级的具体规则(包括会员达到什么条件可以升级，在什么时候判定他降级)和如何让用户提前感知升级和降级的等级变动消息。

(2)积分体系。有会员体系当然就会有积分体系，涉及积分就会有消费场景，也就衍生出了积分获取、积分消费、积分查询三个产品模块。

(3)会员任务。会员任务包含任务接口、任务进度跟踪，以及任务监听。

(4)奖励设置。奖励设置包含奖励类型、发放队列、风控策略和奖励对接。



3.2 用户画像(☆☆重点记忆)

简单来说，用户画像就是人的数据标签。这些标签是基于用户的日常行为信息、兴趣爱好以及社会属性计算出来的模型。多个标签的组合就形成了用户画像。

电商的用户画像把用户的行为电商化，统计了用户的交易信息、浏览信息等。例如，如果你没有填写个人信息，那么系统是不知道你的性别的。但是，你在接下来的一个月里，每天都会浏览男性使用的商品。这时，系统会给你贴一个标签--“喜欢男性商品”，如果你仍然长时间继续浏览男性使用的商品，系统就会给你追加一个性别标签-“男性”。所以，电商系统判断你是一个什么人，不是根据你说的，而是根据你做的事情判断。

毫无疑问，在构建用户画像的过程中，最重要的是数据。这些数据是用户在网上留下的痕迹，包含电商行为数据、电商交易数据、用户基本数据等，如图所示。

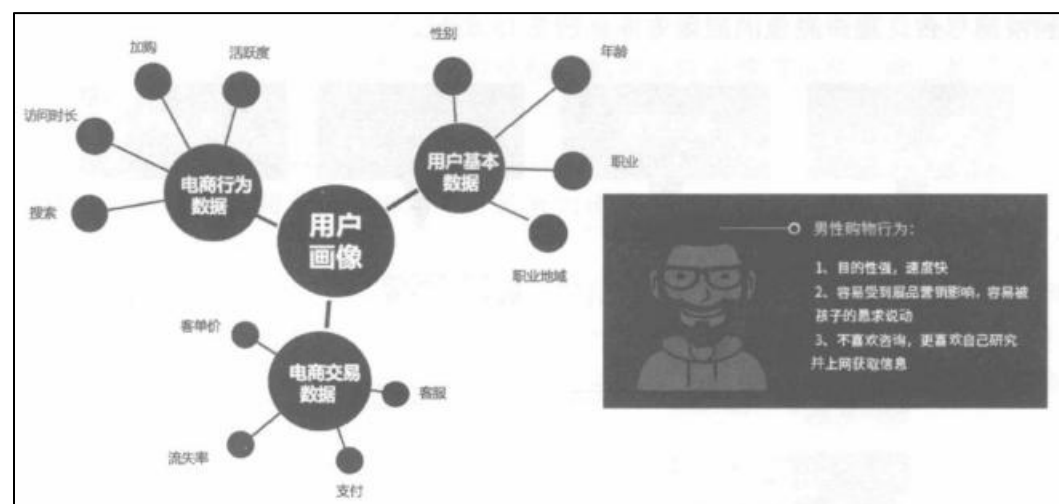
(1) 电商行为数据包含搜索、访问时长、加购、活跃度等行为数据。

(2) 电商交易数据包含客单价、流失率、支付、客服等交易数据。

(3) 用户基本数据包含性别、年龄、职业和职业地域等数据。

每个人在这个世界上都是独立存在的，不存在完全一样的人，他们的数据标签也不同，例如图中的用户画像可以表述为：

凌苏，性别为男性，留有短发和络腮胡子，戴眼镜，小眼睛。他在购物时目的性强、速度快，容易受到展品营销影响，在生活中容易被孩子的恳求说动，不喜欢咨询，更喜欢自己研究并上网获取信息。

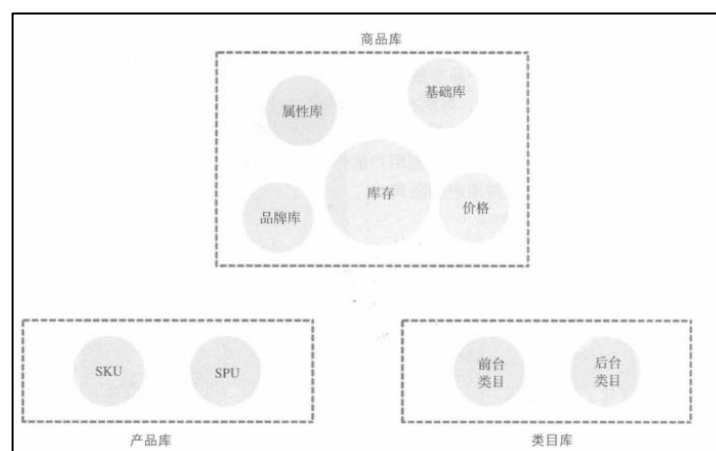


4. 电商领域——货

4.1 商品模型

商品是人类社会生产力发展到一定阶段的产物，是用于交换的劳动产品，是交易的介质，能够满足人类的需要。商品的存在形态不限于实物形态，也包含虚拟类商品(如延保服务、售后服务等)。

电商产品把线下的商品形态搬到了线上环境，把线下庞大的商品数据同样拿到了线上。当商品规模较小时，平台管理起来并不复杂，但是随着商家增多，商品的数量会呈现指数增长。此时，如果没有好的商品管理方式，平台的运营就会面临巨大的压力。这时，我们就需要一套“用得舒服”的商品管理系统而商品管理系统的根基就是商品的类目库。



商品管理系统可以分为以下三个部分。

(1)商品库。包含商品的基础库、属性库、品牌库、库存和价格。

(2)产品库。产品库可细分为SKU和SPU两种模型。

(3)类目库。类目库分为前台类目和后台类目。

电商的商品管理系统参考了传统行业的管理体系。它的意义在于，用户能够迅速找到商品，查看商品详情；平台可以减少运营工作量，提升效率。现在，用户在淘宝和京东上挑选商品时可以通过类目导航迅速定位到喜欢的商品，也可以通过关键词搜索商品，平台利用大数据给用户推荐他们喜欢的商品。用户还可以通过商品详情页面和商品评价等内容了解商品的全部信息，而这些信息要比线下更加丰富，这些能力都要归功于商品管理系统。

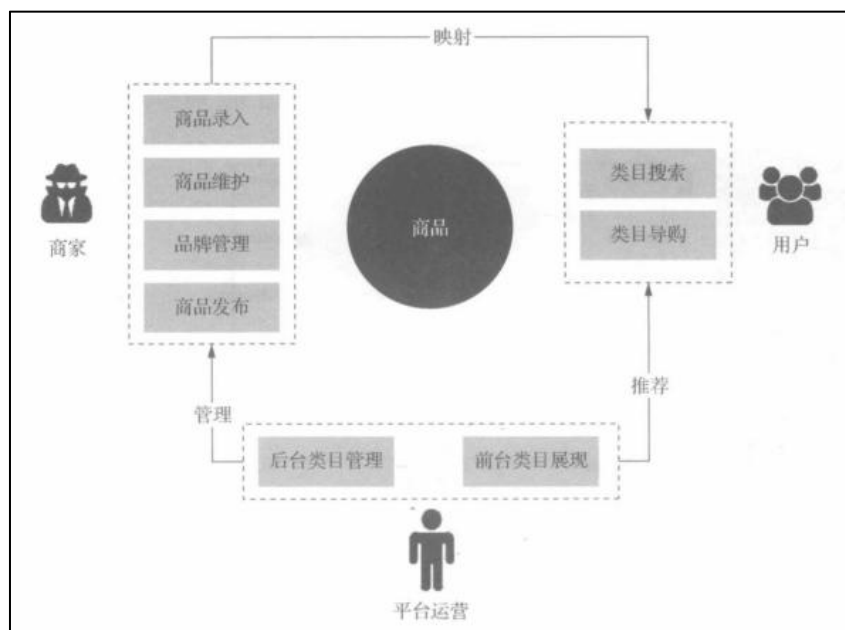
4.1.1 商品类目是根本(☆☆☆重点掌握)

在产品初期的时候，由于商品数量和种类不多，用两级分类即可描述商品类目，如一级分类为手机，二级分类为品牌，或者直接将具体商品放到导航上，以单品形式作为类目，如小米电商网站的类目导航，如图所示。



如果商品种类繁杂，涉及的商品数量巨大，类目层级就会变得越来越多。比如，一级分类为服饰，二级分类为男装，三级分类为牛仔衣，四级分类为款式，五级分类为大小，依此类推。这么多的类目层级会让运营和用户难以理解，用户在页面上难以找到深层级的商品，运营也没办法维护如此巨大的数据量。所以，先“吃螃蟹”的淘宝制定了一套规则，首次引入了类目属性的概念，制定了“**类目主导+属性辅助**”的商品管理形态，并沿用至今，而且规定了类目的层级不能超过四级，这套规则在业界广泛流传。

电商的商品管理面向平台运营、商家和用户三个角色。平台运营设置后台的类目管理标准，并根据情况设定前台类目的展现形式。商家根据平台运营设置的后台类目管理标准，进行店铺的商品录入、商品维护、品牌管理、商品发布。用户在前台进行类目搜索，根据类目查找商品，通过平台运营设定的前台类目展现形式，查看特定场景的商品组合。电商商品管理的角色和能力如图所示。



电商的商品类目管理灵感来源于线下实体店。线下实体店经过多年的发展，总结出一套高效的商品管理模式，即大前台小后台。简单来说，它把商品类目分

为前台类目和后台类目。前台类目面向用户，突出导购内容，让用户看得到，吸引他购买。后台类目面向平台运营或商家，负责商品底层的维护建设，提高他们的操作效率，方便管理。

1.前台类目

前台类目也同样呈树状结构，从一级类目到叶子类目。一级类目是最基础的类目，叶子类目是最尾部的类目，前者类似于树根，后者则类似于树叶。为了用户使用户查找商品，类目层级一般不能超过四级，这一点在电商中貌似达成了共识。不同级别类目的关联关系如图所示。



2.后台类目

后台类目主要面对的是商家和平台运营，他们负责搭建类目，挂载商品到叶子类目下。一些比较知名的公司会把一部分店铺运营工作交给第三方公司，如代运营公司，即外包公司。

当我们负责的电商产品 A 影响力较大时，会吸引平台型电商 B 公司入驻，例如苏宁入驻天猫、一号店入驻京东。B 公司的商品数据量也是巨大的，此时我们需要通过其他方式获取商品，并对应到电商产品 A 的后台类目上。我们一般通过一套通用的对接方案，结合类目预测和类目纠错算法进行匹配，当然，还需要人工参与。按照使用情况，后台类目可以分为三种状态：

- (1) 可用状态。类目是可以使用的正常状态。
- (2) 屏蔽状态。类目如果处于屏蔽状态，那么商家是无法看到的。
- (3) 不可用状态。类目已经无法使用了，在一般情况下商品后台会给商家提示改状态。

下图为淘宝的后台类目。每个商品在创建时，都必须有对应的归属类目。当选择类目后，我们才可以填写商品属性。一般来说，为了防止后台数据冗余和杂乱无章，商品后台类目层级不超过四级。



4.1.2 商品的关键信息--属性

当商品的数据规模增大时，类目树纵向成长会变得越来越深，给用户和运营造成了的巨大困扰。仅仅通过类目树表达商品信息是不够的，仍然需要属性补充信息。

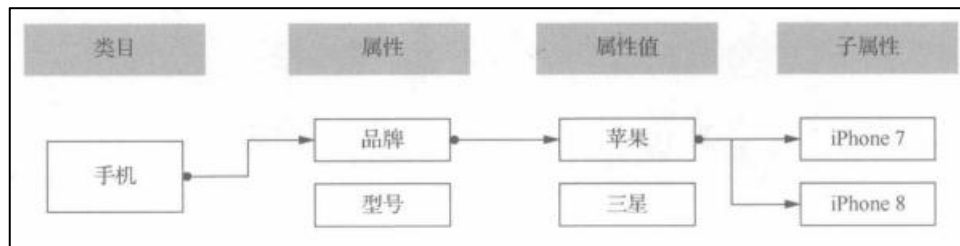
1. 属性

什么是商品属性?商品属性指的是商品的特征信息。型号、颜色、尺码、容量都是属性，属性是一类商品具有的特征。与属性相搭配的是属性值，代表属性的枚举值。例如，对于品牌(如苹果和三星)来说，品牌是属性，苹果和三星就是属性值。对于服装颜色(如红色)来说，服装颜色是属性，红色是属性值。下图为京东的商品属性图，左侧是商品的属性，右侧是属性值。



2. 子属性

属性有属性值，属性值还可以有属性，被称为子属性。例如，类目下有品牌，品牌是苹果，苹果下还有 iPhone 7 和 iPhone 8。其中，品牌是属性，苹果是属性值，iPhone7 就是子属性。品牌(属性)是没有子属性的，苹果(属性值)才有子属性。图中能够直观地解释上述概念。



3.公共属性

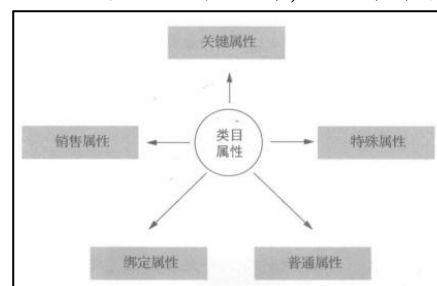
属性的产生解决了类目冗杂的问题，却衍生了更多问题。商品的属性库极其庞大，品牌和型号可以是属性，整洁程度也可以是属性，导致商品属性错乱、繁杂，运营的“主观性”属性让商品更加难以管理。随后，属性逐渐变得标准化，通用的公共属性对类目管理更加有效。

什么是公共属性?商品的很多属性是固定不变的，无论外界如何变化，这些属性都不会发生变化。比如，对于颜色来说标准色是固定不变的，尺码属性也是固定不变的。公共属性能够应用在不同类目下，形成一套通用的属性模板。商家在创建类目商品时，可以直接选择属性模板，挂载公共属性。

4.类目属性

不同的类目根据特色能够抽离出该类目下所有商品的共同属性，这部分公共属性称为类目属性。例如，手机品类都具有品牌、屏幕尺寸、分辨率、内存等属性，这些属性可以作为手机的类目属性。商家在创建商品时，这些作为必填的属性项。同时，类目属性还能够在前台友好地支持搜索和商品筛选。

后台的类目属性按照类型可以分为以下五种，如图所示。



4.1.3 SPU 和 SKU 模型

标准化产品单元(Standard Product Unit, SPU)是一组可复用、易检索的标准化信息的集合。该集合描述了一个“产品”的特性。

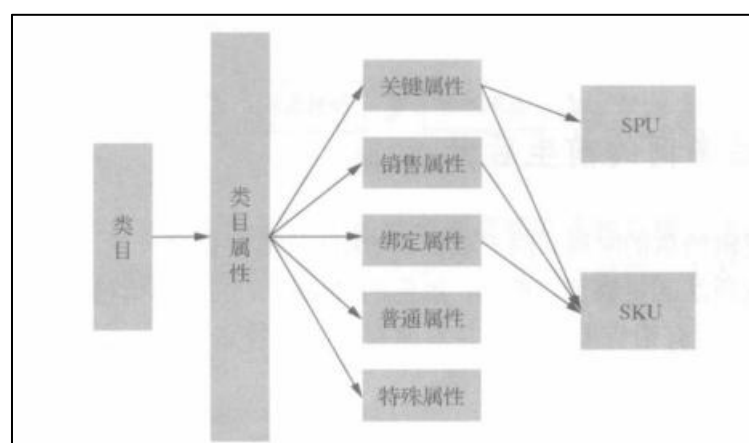
同一类商品的集合称为产品，上文我们提到了关键属性的概念，一个或多个关键属性能够确定 SPU。例如，手机品牌是苹果，型号是 iPhone7，那么这个 SPU 就确定了，它由两个关键属性组成，分别是苹果和 iPhone 7，绑定属性是屏幕大小、内存、分辨率等。

属性确定的标类产品相对比较容易定义出 SPU，例如 3C 产品的品牌、型号、

产地、尺寸、分辨率等都是标准的，汽车品牌 and 型号也是标准的。属性不容易确定的非标类产品，则比较难定义 SPU，国家或电商公司很难抽象出一套通用的模型规范非标类产品，如手工艺品类型和型号等，所以商家就自定义了一套符合场景的 SPU，即 SKU(StockKeepingUnit，库存量单位)。

SKU 即库存进出计量的基本单元，可以以件、盒、托盘等为单位。SPU 包含多个 SKU。SKU 是 SPU 的子集。例如 SPU 是 iPhone7，SKU 就是银色的 iPhone 7 十豪金的 iPhone 7、黑色的 iPhone 7。SPU 能代表一款鞋，SKU 则是这款鞋的其中一双。

SKU/SPU 与属性的关系如图所示。



商品系统还有一种商品类型叫套装，这是复杂一点儿的场景。套装由多个 SKU 组成，不同的 SKU 需要在创建时进行关联，然后会生成一个新的 SKU，叫虚拟组套。SKU 之间的关系是在空间上进行关联的。在列表页和商品详情页中，如果不用文字和图片就无法区分是单个 SKU 还是虚拟组套。

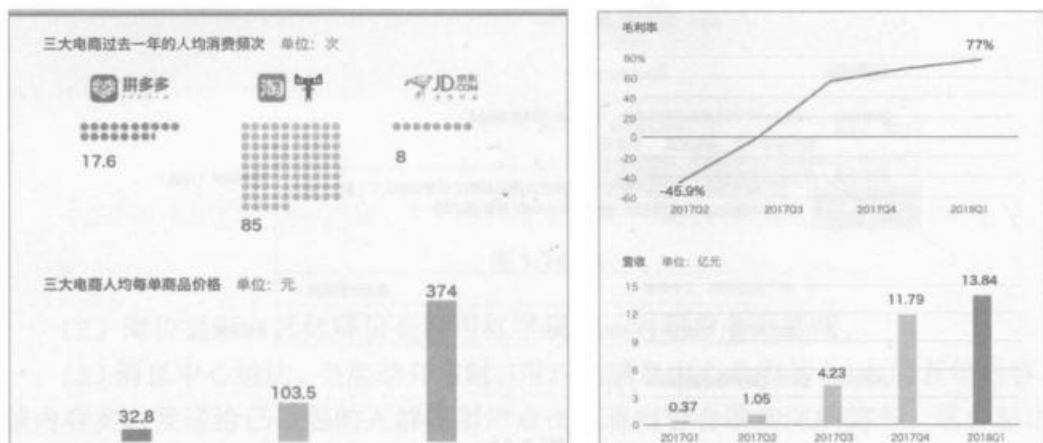
4.2 订单流转

4.2.1 电商城订单如何生成和流转

前面介绍了商品是怎么来的，详细地描述了商品最基础的底层设计逻辑，现在沿着货的交易链路，介绍一下订单系统的产品设计逻辑。

订单系统是电商后台产品最核心的一环，也是衡量电商公司业务能力的重要维度。根据订单情况，我们能够从订单信息中拆解出不同维度的数据指标。

比如，2018 年，拼多多、淘宝、京东的年人均消费频次分别为 17.6 次、85 次、8 次，拼多多、淘宝、京东的人均每单商品价格分别为 32.8 元、103.5 元、374 元。如图所示为拆解出来的不同纬度的数据指标。



电商为了销售的目标本质不改变,所以任何业务都不可能完全脱离订单交易域。订单在任何业务场景中都会存在,比如,你负责的是内容推荐流、直播、视频或帖子等产品,即使主要目标是阅读量、分发渠道等,但最后也还是会和购物场结合,比如用直播推荐商品、用帖子推荐商品。

4.2.2 最基本的订单知识(☆☆☆重点掌握)

订单管理中心(Order Center, OC)保存了交易记录,里面包含了所有的交易信息。订单内的各种字段都是用户在下单前在结算页内选择或输入的内容。

用户究竟进行了哪些操作才生成了订单呢?以我为例,我一般在有购买需求时才会打开购物 App。我首先要搜索商品。据说搜索可以贡献 60%的销售额,搜索包含关键字搜索和类目搜索。在搜索商品后,紧接着选择商品,进入商品详情页,查看商品信息、

评价和问答。把商品加入购物车,勾选商品,结算。在结算页中输入地址、联系方式,选择支付方式、配送方式、权益资产、发票类型。在检查无误后,提交订单。系统生成订单信息,并提示物流状态。

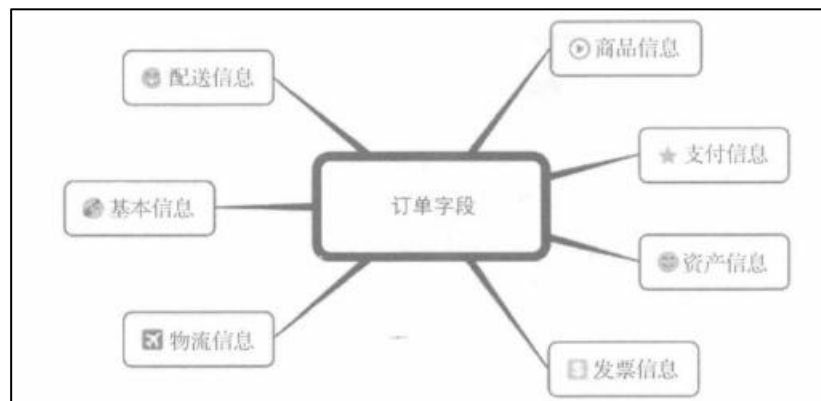
用户下订单路径如图所示:



这看似是一条很漫长的路径,但是每一步都必不可少。在未来,用户下单可能只需要一步。下面讲解订单的所有知识。

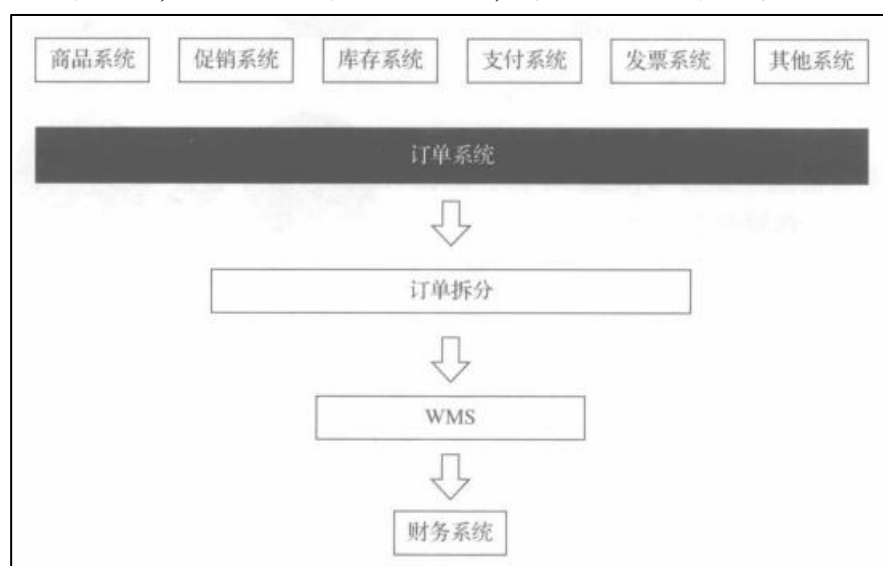
1. 订单字段

根据上述的下单路径，我们能够拆解出各项订单的字段，如图所示。



- (1) 基本信息。包含订单号、订单时间、订单状态等信息。
- (2) 商品信息。包含商品价格、商品名称、商品链接等信息。
- (3) 支付信息。包含支付方式、支付状态、支付时间、支付单号等信息。
- (4) 配送信息。包含是否包邮、不包邮时配送公司是哪个等信息。
- (5) 资产信息。包含红包、卡券、积分、京豆等虚拟资产等信息。
- (6) 发票信息。包含发票类型选择，是增值税普通发票还是增值税专用发票，是开电子发票还是不开发票等。
- (7) 物流信息。包含具体的物流状态，即在哪个时间点位于哪个站点/仓、由哪位配送员配送、是否签收、节点时间等信息。

订单包含如此多的字段，需要和下游多个系统对接。商品信息从商品系统中获取，促销信息从促销系统中获取，库存信息从库存系统中获取，支付信息从支付系统中获取，发票信息从发票系统中获取，其他信息需要从其他系统中获取，订单的生成流程如图所示。在生成订单后，订单系统还要进行拆分流程，包含优惠拆分和订单拆分，紧接着订单进入 WMS，最后进入财务开票流程。



2. 订单类型

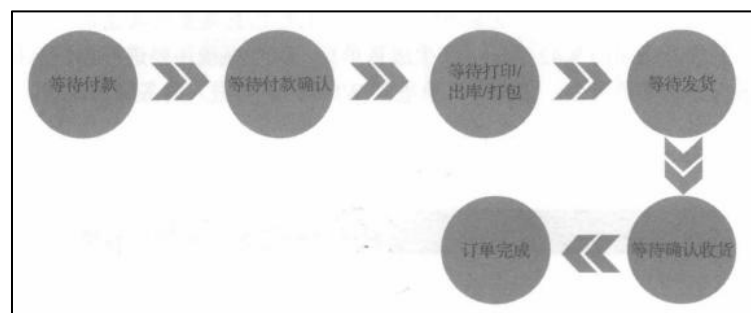
根据订单中商品的类型，我们可以将订单分为实物订单和虚拟订单。

实物订单是指订单中为实物商品，发货需要物流的一些商品订单(比如，订单中有冰箱、笔记本、手表)就是实物订单。如果客户购买的商品是苹果、香蕉等，那么该订单就是生鲜订单。用户在京东夺宝岛下的订单，被称为夺宝岛订单。把实物订单划分为不同的业务订单，主要是为了划分不同的业务，便于拆分业绩统计。

虚拟订单是指不需要物流发货，商品是虚拟物品的订单。商品可以是Q币、充值卡、点卡、礼品卡等。因为虚拟订单没有物流状态，所以订单流转和结算流程比实物订单相对简单。

3. 订单状态

订单也有生命周期，在不同的节点展示不同的状态信息。由于不同公司的业务模式不同，订单状态的划分可能也不同。以京东为例，下面介绍具体的订单状态是如何流转变化的。



(1)等待付款。如果是先款订单，那么用户需要在提交订单后支付订单金额。如果是未付款状态或者财务还没有对账完成，那么订单状态为等待付款。在一般情况下，等待付款的订单可以保留 24 小时，而大促期间的订单可能不到半小时就会被释放。不过，也有另类的需求场景，如企业客户的订单期限可以达到最长 15 天。

(2)等待付款确认。该状态为后台状态，用户在前台是无法看到的。等待付款确认指的是在用户付款之后财务系统需要进行财务对账。财务人员要对比台账进出时是否有变化，如果没有问题，这个状态就会发生改变。

(3)等待打印/出库/打包。订单在对账后，会迅速进入库房生产。为了保证时效，上述操作最短可以在 1 分钟内完成，这可能是因为该订单包含的商品就在离出库机器非常近的位置。随后，库房工作人员将订单商品打包整理。

(4)等待发货。订单商品在打包后，配送卡车会将订单商品配送到站点，在未装车前的订单状态是等待发货。这段时间可长可短，与订单时效有关，如果在晚上或者未到用户选择的发货时间，那么这个状态会一直持续。

(5)等待确认收货。订单商品在发货后，订单状态即更新为等待确认收货。
当用户收到货物之后,在第 7 天或第 10 天(用户可能会手动延长确认收货的时间)订单会自动确认。如果是商家与用户的交易，那么此时系统会将订单金额全部打给商家。

(6)订单完成。用户主动确认收货或者在第 7 天或第 10 天后订单自动确认，状态即变为订单完成。

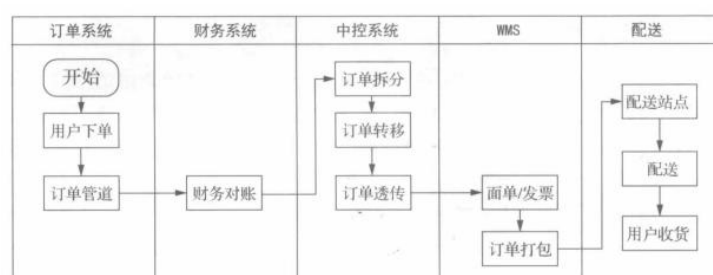
虚拟订单的状态比较简单，返回给用户的状态只有两种，即等待付款和完成。假设虚拟订单必须立即支付，订单则只有一种状态，即订单完成状态。

4.2.3 订单正逆向流程

订单状态的变更实际上是订单系统内部流转的过程，遍历了电商内部各个后台系统。

1. 订单的正向流程

自营订单从生产到配送至用户手里，经历了极其复杂的流程。订单的正向流程简化版一般如图所示。



- (1)订单系统。用户提交订单，订单系统接单，开始检查订单的合法性。然后，订单系统存储订单信息任务，并通过订单管道服务写订单业务标识。最后，订单系统通过状态机向上、下游业务同步订单状态信息。
- (2)财务系统。在用户提交订单时，订单系统还会调用台账服务写一笔台账，即在虚拟的账本上记一笔账，并记录在财务系统数据库中。如果订单为 10 元，台账就记录这笔账是 10 元。财务系统需要确认用户是否付款，并且要进行对账，确认写的台账和用户支付的金额是否能够匹配。
- (3)中控系统。京东把这个系统叫订单履约系统(OFC)。中控系统主要负责订单拆分、订单转移和订单透传。关于订单拆分，下面会详细说明。
- (4)WMS。WMS 负责面单和发票打印(随货发的发票打印)，打包商品，完成进出库任务。
- (5)配送。如果是自营仓模式，那么配送中心一般会将商品发送到各个地区的站点，再由站点的配送员送货。

货到付款的逻辑与在线支付的逻辑是完全不同的，前者是先把货送到用户手里，用户再付款，只要用户提交了订单，库房就进行订单生产；后者是只有用户付款了，库房才能生产订单，用户在收货后订单自动设置为已完成。

这是两者最大的区别，不过在售后退款时，在线支付的款项可以以原支付方式返回。货到付款的支付方式为刷卡和现金支付。对于支付方式为刷卡方式的订单，款项能够返回原卡；对于支付方式为现金支付的订单，款项只能退回到平台的虚拟资产。

2. 订单的逆向流程

任何产品都无法保证不产生售后问题。其实，用户选择你的产品可能是因为你的产品质量好、性价比高，也可能是因为对售后服务比较放心。目前，逆向流程指的是单 SKU 维度的逆向流程。因为订单中可能有多款商品，但用户只需要退部分商品，所以不能够采用整个订单维度的逆向流程。每次在逆向流程发起后，系统都会生成服务单。每个订单可能会产生多个服务单，并且单个 SKU 其实可以发起多次逆向服务。我们对不同的订单取消场景需要采取不同的产品策略。

1) 按订单取消类型

订单取消可以分为用户取消和系统取消，下面主要介绍系统取消。系统取消包含订单逾期取消、风控取消、客服取消等。订单取消和售后都属于逆向流程，但是产品动作是不一样的。未付款的订单取消逆向流程叫取消订单，已付款的订单取消逆向流程叫返修或退款。

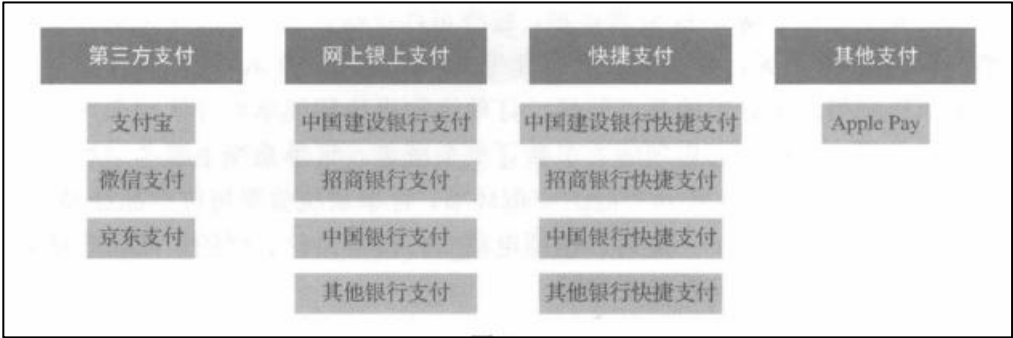
(1) 订单逾期取消。在 toC 的订单中，在一般情况下如果用户 24 小时未付款，系统就会触发取消任务。在“双 11”大促时，订单取消的时间可能会缩短为半小时或 1 小时，订单逾期则取消。在 toB 的订单中，订单取消的时间比较长，而且可以延期。

(2) 风控取消。风控取消是指系统如果判断用户进行了刷单、黄牛下单或者恶意下单，就会拦截该订单，并进行取消处理。系统可以通过下单频率、下单地址、下单账户判断，如果某些账户多次通过同一个地址下单，再取消订单，或者发空盒快递等，系统就会自动拦截有这些行为的订单。任何一家公司都没有办法避免恶意下单的行为，用户的恶意下单属于违规操作，平台需要了解每个用户的下单行为情况，然后再设定黑名单和白名单。

(3) 客服取消。客服每天需要处理大量的用户订单问题，所以需要被赋予超级权限，包含可以对订单进行增、删、改、查的能力。当遇到用户要求取消订单时，客服需要按需确定是否帮忙处理订单问题。当客服进行取消订单操作后，客户可以实时看到订单的状态，并且在订单的操作日志里能够查到具体的订单取消原因和操作人员。

4.2.4 订单支付方式

21 世纪后，中国银联成立，重新建立了支付行业的格局。随着电子商务的发展，整个支付行业进步飞速。为了提高支付效率，免除陌生人交易障碍，马云创办了支付宝。在当时，支付宝仅仅是淘宝的一个支付模块，后来进化为市值百亿美元的独角兽企业。2013 年，微信发布 5.0 版本，正式进入移动支付的浪潮。为了快速达成交易，提高订单转化率，电商平台必须要接入多种类型的支付方式，支付方式可以划分为以下几种，如图所示。



1. 第三方支付

支付宝支付、微信支付、京东支付等都是第三方支付方式。这些支付公司与银行签约，完成资金流转结算流程。目前，支付宝支付和微信支付已经占据了第三方支付的 90%。很多城市已经进入了无现金时代，例如，在杭州，人们交停车费、去超市和商场购物全部可以采用支付宝支付。

2. 网上银行支付

网上银行又称为网络银行，用户如果想申请开通网上银行服务，那么需要本人去银行柜台办理手续，如中国建设银行、招商银行或中国银行等。目前，用户已经可以在各银行 App 内申请开通网上银行服务，但是手续非常繁杂。

用户在订单的收银台选择对应的银行，在页面跳转至第三方银行网站的支付页面后，填写银行卡号、身份证号、手机号等信息，确认支付。在支付成功后，用户可以点击弹窗按钮返回订单页面。在支付完成后，用户卡里的钱会流向卖家的银行卡或者第三方支付平台的账户。网上银行支付一直被诟病，支付成功率仅为 65%，极易造成用户流失，并且经常会有重定向的钓鱼网站泄露用户的银行卡号和密码。

3. 快捷支付

快捷支付是指用户在电商平台上发起支付时，只需要提供银行卡号和手机号，银行会发送验证码给平台账户绑定的手机号，用户凭短信验证码即可完成支付。与网上银行支付相比，快捷支付的支付成功率高。用户仅需两步即可完成订单的支付流程，无须 U 盾，降低了操作门槛，并减少了被“钓鱼”的巨大风险。

用户在首次进行快捷支付时，需要进行绑卡、签约操作。这些操作只需要进行一次，无须重复，以后无须绑卡就可支付。常用的快捷支付有中国建设银行快捷支付、招商银行快捷支付和中国银行快捷支付等。

4.其他支付

有些手机厂商生产的手机会自带支付方式，如 ApplePay(目前在国内的市场份额很小)。另外，还有代扣、收单等类型，由于在 C 端不常见，这里就不展开描述了。

5. 电商领域——场

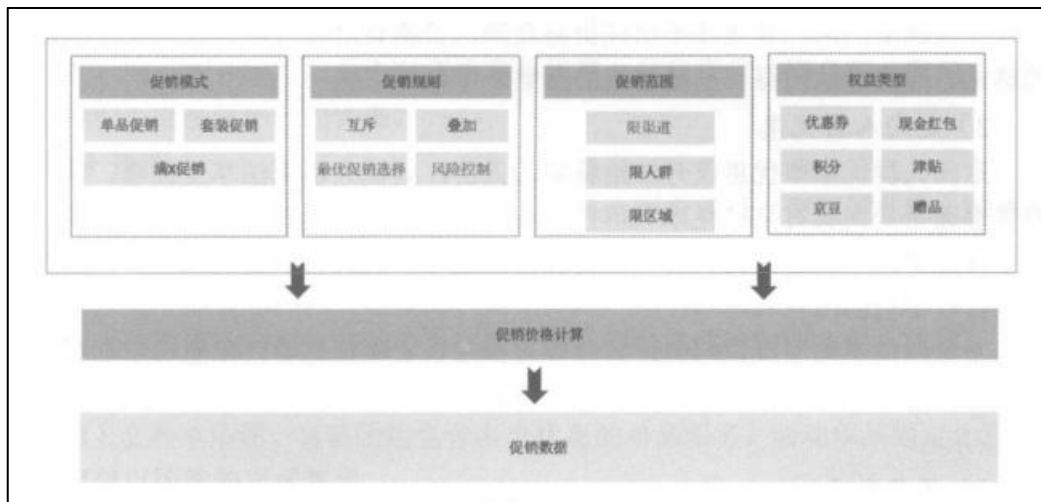
5.1 促销产品

5.1.1 促销产品的设计方法

我们在沟通需求时，经常会提到“场景”，比如在什么场景中遇到的问题、产品有哪些应用场景等，脱离了场景说问题没有任何意义。电商的“人、货、场”要强调场的作用，但是这还不够全面，应该为“人、货、场景”。“场景”其实可以分为“场”和“景”。前者表示提供的场所，含有空间感的概念，如超市的柜台、营销页面、交易所等。后者表示在场中可以实施的动作、行动、措施等。

我们把“人、货、场景”简单定义为找对用户，为他提供相应的货物，在某个场所售卖，通过一些手段刺激他购买。在电商系统中，促销产品的使命是刺激用户发生购买行为。什么是促销呢？从字面意思来看，促销是促进销售的手段。利益驱动的手段是最直接的。大多数人应该都是价格敏感型用户。价格因素在很大程度上影响了用户是否购买商品。

从用户视角来看，促销系统所表达的是促销前台交互流程和各种各样的促销模式。比如，用户在商品详情页中可以了解到买这个商品能够得到赠品、在购物车中可以切换促销模式等。实际上，促销产品复杂得多。促销系统一般可以分为促销模式、促销规则、促销范围、权益类型、促销价格计算和促销数据这几个模块。



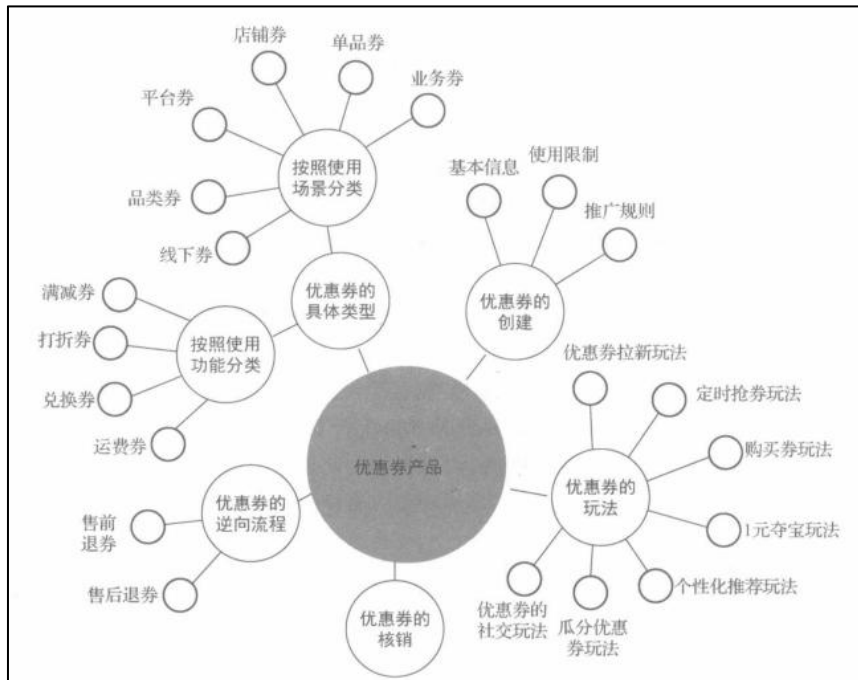
简单来说，促销模式是一个用户在特定的场景下得到什么利益，是优惠的获取模式。比如，满 199 元减 100 元的优惠券的意思是当订单商品的价格超过 199 元时用户获得减 100 元的资格。用户订单商品的价格超过 199 元是特定场景，减 100 元是用户得到的利益。

所有的促销模式都可以按照这个思路设计，目前在市面上最多的促销模式分别为单品促销、套装促销、满 x 促销。

5.1.2 优惠券的玩法

优惠券是促销产品向用户表达促销优惠的最重要介质之一，但优惠券系统和营销系统是两套不同的架构。对于优惠券来说，营销工具的玩法是分发场景之一，而不是全部。优惠券是促销的介质，能够抵扣一定的订单金额，是市面上常见的促销模式。1894 年，可口可乐创始人手工绘制了优惠券，用于促销。由于效果比较明显，优惠券逐渐发展至今。随着电商的发展，实体的优惠券变成了用户的虚拟资产，并逐渐产生了各式各样的玩法。

下图所示为优惠券产品的整体架构图，包含优惠券的具体类型、优惠券的创建、优惠券的玩法、优惠券的核销和优惠券的逆向流程。

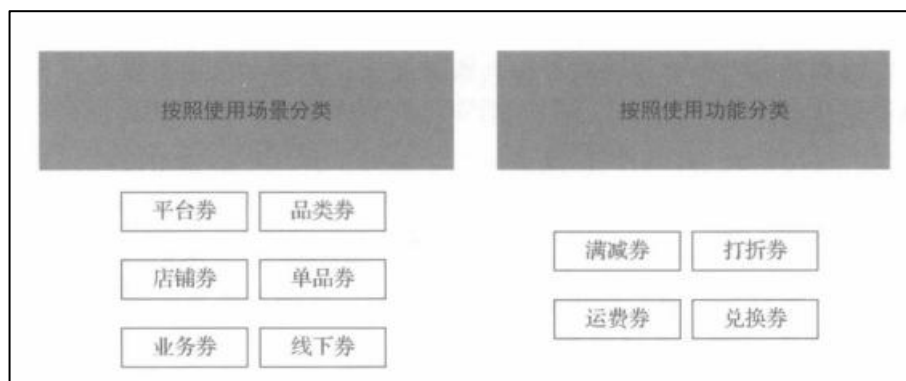


很多人曾经质疑过为什么电商存在优惠券产品，难道不可以让商品直接降价吗？尤其在“双11”和“618”大促时，优惠券叠加各种促销会让用户难以理解，用户完全不清楚到底如何才能享受最优惠的价格。其实，优惠券的存在是有必要的。

第一，有一种心理学现象叫沉没成本效应，指的是你付出得越多，成本越高。这里的成本既包括金钱成本，又包括时间、精力、资源等沉没成本。当你想要放弃时，因为你已经付出了很多沉没成本，所以你会做出不一样的选择。同理可证，优惠券需要用户经过一番努力后才能获取，成本越高证明其价值越高，使得优惠券的转化率就越高。

第二，单纯的降价打折的玩法单一，扩展性不好，价值认可度低。用户对普适性的折扣不买账，比如定时折扣，在规定时间内打折，也称为秒杀，有的商家把原价标得很高，把800元的商品标为原价是10000元，然后打0.8折，结果还是800元，所以用户看到的可能是“假打折”。而由于优惠券是用户主动领取的，用户只有领取才可以享受折扣，这样会让用户觉得很值，感觉这是很真实的促销。

第三，优惠券可以让电商玩法变得乐趣横生，是传递价值的载体。用户在升级会员后得到优惠券，在分享商品后得到优惠券，在玩了互动游戏后抽取优惠券。在限定的时间内抢优惠券。在结合了玩法后，优惠券的价值更加凸显，物以稀为贵，如果用户不参与这些互动，就拿不到“独有”的优惠券。优惠券可以做如下分类：



1.按照使用场景分类

在不同的使用场景中有不同的优惠券类型，如平台券、品类券、店铺券、单品券、业务券和线下券等。

(1)平台券。平台券是指平台通用型优惠券，适用范围为整个平台，还可以被称为全品类通用券。比如，京东全平台通用、跨店铺使用。此类券的价值比较高，用户很难获取大额券。平台券可以由平台出资创建，也可以由商家自行创建。但是，考虑到平台券的安全性问题，平台都需要限制平台券的使用时间。

天猫的购物津贴从严格意义上来说也是优惠券的一种类型，属于平台券。购物津贴只有在大促场景下才能使用，在其他时间购物津贴会失效。购物津贴的适用范围是全平台报名的商家，购物津贴需要商家自行提报创建。当购物津贴核销时，促销系统需要按照订单商品的金额平摊购物津贴费用。

京东全品类通用券的适用范围为自营商品，该券为官方券，由平台出资补贴，用户在下单时抵扣这部分金额。

(2)品类券。与平台券相比，品类券的适用范围缩小为某个品类或某些品类。例如，限母婴类目、限服饰类目、限 3C 商品使用的优惠券。品类券也是比较常见的优惠券类型，一般由平台出资。

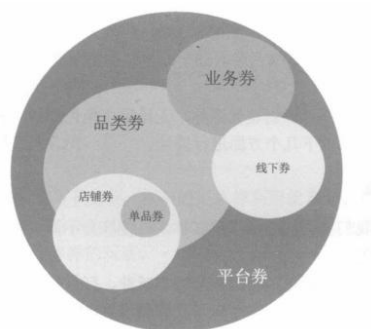
(3)店铺券。店铺券的适用范围为店铺，可以跨多个店铺使用，也可以在一个店铺中使用。商家可以根据自身情况自行创建此类型优惠券，预算需要由商家自行承担。

(4)单品券。单品券是由商家或平台创建的优惠券，仅适用于部分商品。比较紧俏的商品、稀有商品或新品在首发时一般才会设置单品券。例如，苹果产品在首发时，平台会赠送一些单品白条免息券或单品满减券。

(5)业务券。电商的业务类型复杂多样，某些业务会根据自己的业务类型设定优惠券。比如，生鲜业务的生鲜券、天猫超市业务的超市券等。其实，业务券也属于平台券的子集。

(6)线下券。现在是新零售时代，许多线下门店已经支持使用电商的优惠券，即新零售券。线下券的限定条件是某品牌门店。线下券的使用场景是用户在门店

挑选商品，付款时出示优惠券的二维码，导购员通过 POS 机扫码，对优惠券进行核销。同时，在线上会生成一笔订单。早在 O2O 时期，这类券便已经广泛被使用，如美团的订单券、猫眼的电影券。



这几种优惠券的关系如图所示，平台券覆盖范围最广，品类券和店铺券有一定交集，如购物津贴其实就是店铺券的变种，但也能在多家店铺使用。店铺券和单品券是包含关系。品类券、业务券和线下券属于并列关系，从严格意义上说这三种优惠券也有交集，并且这三种券还可以继续细分到店铺和单品上。

5.2 营销策略

5.2.1 营销广告的那些概念

在广告行业中，有些名词特别不易区分，比如收费模式可以分为 CPA、CPC CPM 和 CPS 等。这些英文缩写不好区分，但是前两个字母都是 CP，所以我们只需要记住最后一个字母。

1.CPA(Cost Per Action)

CPA 即按照实际效果收费，广告系统需要用户产生预期的行为，比如 App 下载次数或下订单的数量等行为。与 CPC 等收费模式不同，CPA 的效果数据更加真实，对于广告主来说更公平，广告主在获得收益后向广告平台付费。

2.CPC(Cost Per Click)

CPC 即按用户点击的次数收费。例如，用户每点击一次收费一元，不点击不收费。该收费模式被 Google 和百度广泛应用于搜索竞价模式。因为按点击次数收费，所以只要有点击就会收费，有时可能会出现很多恶意攻击的情况，比如友商发起的刷次数的动作，导致广告主的余额迅速下降。在电商场景中，假设按照用户每点击一次收费一元计费，用户点击了 1000 次广告，成交了 10 单，转化率为 1%，如果每单利润达到 100 元，投入与回报就可以平衡。

3.CPM(Cost Per Mille)

CPM 即按千次展现收费，比如向 1000 个用户曝光了广告，不管用户有什么行为，只要用户看了就会对广告主收费。这种模式常见于线下广告牌和线上的优

质资源位。与 CPC 模式相比, CPM 模式对于平台来说更加合理。毕竟, 用户只要看了广告就会影响其心智。目前, 电商广告基本上都是用这种模式收费的。

4.CPS(Cost Per Sale)

CPS 即按照销售分成收费。CPS 其实是 CPA 的一个分支, A 即 Action, 任何行动(如购买、加购等)都可以被看成 A。在现实中, 销售人员会通过售卖商品得到佣金, 比如, 销售人员每卖一辆车就能得到 3% 的佣金。

同理, 在淘宝中, 淘宝客到处为商家发送商品广告, 为商家引流成交 100 单, 成交金额为 10 万元, 他就可以得到 3% 的佣金, 即 3000 元, 这部分佣金就是广告主支付的广告费用。

以上是广告的收费模式, 广告主在投放广告后, 广告平台还要对广告负责, 包括如何调整广告、广告的效果如何, 所以我们需要了解下面的指标。

(1)CTR(Click Through Rate)。CTR 是广告的点击率。广告主把各种各样的内容投放给用户, 但是用户可能并不会有点击行为。如果广告主把素材 A 投放给 10 万个用户, 得到的 CTR 是 10%, 把素材 B 投放给另外 10 万个用户, 得到的 CTR 是 34%, 那么说明素材 B 的广告效果更好。CTR 是衡量线上广告传播效果的重要指标, 线下广告则没有这个数据。

(2)ROI(Return On Investment)。ROI 是投入产出比。 $ROI = \text{收入金额} / \text{投入金额}$ 。比如, 广告主投入 10 万元做线上广告, 最后只收入了 1 万元, ROI 是 0.1。反之, 如果收入了 100 万元, 那么 ROI 是 10。ROI 可能无限大, 也可能无限小, 广告主主要在提升产品质量和服务的同时, 运用好线上广告。ROI 是评估投放收益的最重要指标。

(3)A/B 测试。对于同一个功能或玩法, 运营人员可以把两套不同的产品机制 A 和 B(如不同的文案、不同的交互、不同的价格等)发送给等量的两类人群, 通过两类人群的数据反馈, 衡量 A、B 两个方案的优劣。在广告投放中, 如果广告主使用素材 A 得到的 CTR 只有 10%, 那么他肯定是不满意的, 所以使用了素材 B, CTR 迅速提升到 34%。这种方式是广告投放的素材 A/B 测试。

以上这些是在互联网广告中比较常见的概念, 我们需要先了解这些概念, 然后才能更好地理解接下来的内容。互联网广告是非常“烧钱”的, 每一次投放都带有广告主的期许, 高效投放是当务之急。

5.2.2 品牌的人群结构和人群策略

我们经常听到某地的特色商品, 如南非的钻石、景德镇的瓷器。以前, 用户如果需要这些特色商品, 那么只能去当地采买或者找供应商拿货, 这属于人找货的模式, 货就在那里, 需要人发现和寻找。在移动互联网时代, 电商通过类目和其他导购产品帮助用户寻找到他想要的商品, 这也是人找货的思路。

用户的浏览深度毕竟有限，可以发现的商品只是少数的。“不甘寂寞”的商品也希望得到曝光的机会，因此就商生了“货找人”的模式，基于商品本身的属性去配适合的人群。比如，美妆商品的定位人群自然是爱美的女性、机械键盘的定位人群是数码爱好者。人群即爱美的女性和数码爱好者，我们的任务就是如何找到这些人群。无论是“人找货”还是“货找人”，都是为了缩短用户和商品之间的距离，向精准用户推送他有购买倾向的商品，这是提升转化率的关键。

1.品牌的人群长什么样

获取新用户是互联网产品永恒的话题，产品无论到了什么阶段都要有拉新的手段。电商产品更应该重视拉新的需求。拉新的整体思路是维护好老用户，获取更多新用户。不过，拉新的成本非常高。对于老用户来说，我们可能花费 1 元的成本就可以让他下单；而对于新用户来说，我们则可能要付出多于老用户 20 倍或 50 倍的成本。

按照购买频次和购买时间，老用户还可以细分为复购人群、沉睡人群、潜在流失人群、流失人群等，对于不同的老用户人群，运营策略是不同的。

2.品牌对应的人群策略

我们对不同人群要使用不同的策略，我们可以使用哪些策略让无交集人群变成潜在人群？

(1)线上广告:我们可以大面积地投放广告。当初的淘宝网因为无法把广告投放到 4 大门户网站，所以投放到了各个小众网站上，逐渐被人们熟知。

(2)线下广告:我们可以把广告投放到所有大流量的资源平台，如地铁的广告位、公交站的广告位、电视节目等。我们投放线下广告的目的是不断地吸引用户关注，让用户记住你的品牌。史玉柱曾经说过，广告是对用户大脑的投资。

如何将品牌的潜在人群转化为新用户呢？

我们要让用户记住品牌相对比较容易，但是要让用户购买商品是很困难的，尤其在现在竞争如此激烈的环境下。我们可以按以下两点来做：

(1)提升产品和服务质量。有了金刚钻才能揽瓷器活，打铁需自身硬。虽然苹果产品的价格高，但是产品质量和用户体验极优。小米产品的价格亲民，质量上乘，性价比高。头部品牌都应该知道只有完善自身产品，才能吸引用户。

(2)刺激用户。潜在用户纠结要不要购买商品的原因可能是价格高，也可能是在和竞品对比。如果在用户购买的临界节点时，我们给用户一点儿刺激，这笔生意很可能就成交了。我们可以给用户什么刺激呢？利益刺激是最主要的，比如我正在犹豫是否买小米手机，突然商家就给我赠送了 100 元优惠券，那么我自然就下单了。其他刺激就比较虚幻了，在大多数情况下我们可以通过正面事件影响用户心智，如用户在观看世界杯比赛时突然看到了是小米冠名的，他的购买倾向也会

受到影响。

下面的策略可以让新用户转为复购人群，也就是让用户多次购买。大家电和家居之类商品的复购场景确实少，这里不多说了。下面以电子和快消商品为例，我们可以使用以下的策略。

(1)提前通知用户购买。很多商品是有使用周期的，我们需要预知用户的使用习惯，在用户发生第二次购买前，提前通知他商品快要用完了，需要补货。

(2)利益刺激。当然，仅仅通知用户购买，还是挺苍白无力的，很多用户会反感，如果此时你再发放一张不错的优惠券，势必事半功倍，提升用户的转化率。

5.2.3 如何触达这些人群

区分人群是第一步，接下来我们要通知这些人群。

1.短信通知

2.定向海报：我们要根据之前设置的人群策略，触达不同素材海报。不同人群可以在店铺首页中查看个性的海报内容，这样可以提升访客的转化率。定向海报可以做到千人千面，支持定向分发给不同人群的策略，对不同的人说不同的话。

3.EDM 营销：EDM 营销也叫电子邮件营销。邮箱类产品使用 EDM 营销较多，如网易邮箱经常在邮箱中打一些网易严选的广告。EDM 营销需要针对精准的人群，否则会对用户产生非常不好的影响。

4.媒体：媒体是指用户与品牌内容接触的触点，包括以淘宝、天猫、聚划算、口碑为代表的电商消费类媒体，以优酷土豆为代表的视频媒体，以新浪微博、陌陌、钉钉为代表的信息流媒体，以 UC 浏览器、高德为代表的移动信息流与搜索引擎媒体等。

5.手机 Push：桌面 Push 为强提醒，我们可以把消息直接发送到系统桌面；消息中心为弱提醒，我们如果把发送的消息沉淀到消息中心，那么用户只有在主动点开的时候才会看到具体的消息内容。按照场景划分，Push 可以分为产品 Push 和营销 Push。前者为系统自动发送给有条件的人，如系统给已经订阅某活动的用户发送消息；后者在营销活动时才会被使用，系统自选人群，自由发送消息。不过，我们需要注意发送消息的频次，以免用户关闭整个产品的通知渠道。

6.微信消息：微信的消息渠道是最有效的通知方式。微信拥有非常大的社交流量入口，用户会经常打开 App，当有消息通知时，用户会看到聊天框上有红点提示。微信消息在消息通知中以卡片形式展现，内容明确且易操作。

6. 本章小结&面试要点

6.1 本章小结

本章从电商产品的视角出发，给大家介绍了电商中的人、货、场相关产品理论，通过这些理论的学习，可以充分了解业务产品的设计思路、帮助我们理解数据建设的底层业务逻辑。电商领域中，是一切围绕用户来设计，核心的目标是在线上 APP 或者 PC 端网站激发用户的购买欲，从而达到营收的目的。

6.2 面试要点

- 1.请解释一下电商中的“人-货-场”对应的具体含义。
- 2.在电商环境中，业务通常需要重点关注哪 6 大类指标。
- 3.请简要描述一下什么是用户画像？用户画像通常包括哪些数据？
- 4.电商的商品管理面向哪三类角色，他们在平台上分别具备什么操作能力？
- 5.商品类目分为前台类目和后台类目，两种类目在使用场景上有什么不同？
- 6.SPU 和 SKU 分别是什么？举个例子说明一下。
- 7.请简述一下用户下订单路径，并且说明一下在各个环节涉及到哪些数据？
- 8.请简述一下营销活动触达给用户的 5 种常见模式。

三、企业级电商业务系统介绍(☆☆☆重点掌握)

1. 基本业务架构

电商发展至今，出现了各种各样的叫法，有平台电商、自营电商、B2B、B2C、生鲜电商、社交电商、兴趣电商等等。不论电商的形式如何多样，其本质都是买卖双方围绕商品交易履约的过程。

谈到电商，第一时间想到天猫、京东、拼多多。电商按类型分类，分为 B2C 型和 B2B2C 型，天猫、京东、拼多多均属于 B2B2C 型，即“平台对商家，商家对用户”；而 B2C 型，即“商家对用户”，一般为某企业直采直销型，不通过第三方平台，如华为商城、小米商城。

在开始探讨之前，我们先圈定一下探讨的方向，本文主要以自营电商为探讨方向。因为与之相对的平台型电商的核心其实是流量生意，系统的责任更多的是将海量的商品和消费者做匹配，平台以此向商家收取交易佣金和广告费。

而自营电商除了需要具备平台电商的能力之外，由于自采自销的业务模式还涉及采购、仓储、履约等实体环节，其系统模块更多更长更复杂，基本上能覆盖平台型电商的核心系统模块，因此以自营电商为主要探讨方向可以了解的更为全面。而其他诸如 B2B、B2C、生鲜电商等只是领域或表现形式不同，不属于同一个维度。

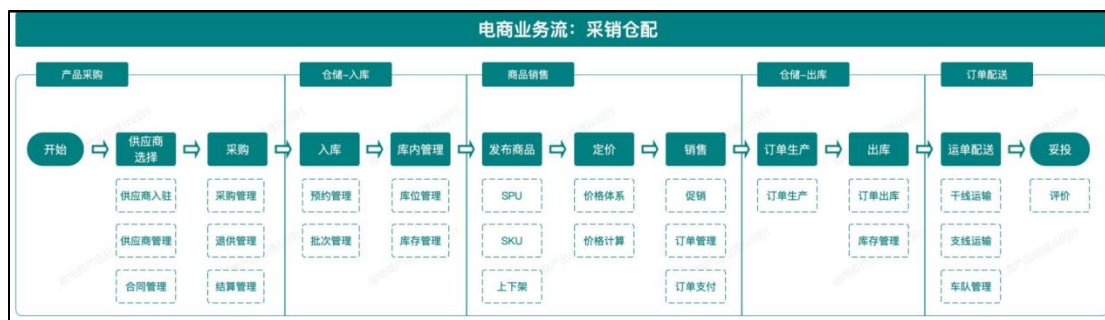
1.1 业务架构(☆☆理解记住)



那么一个完整的自营电商业务是怎么样的？将一件商品交付给消费者需要经历哪些环节呢。通过以上业务场景图可以看出，一件商品卖给消费者需要经历的环节非常多，包括线下实体环节和线上系统环节，按照线下实体环节进一步抽象后，可以将自营电商业务划分为以下 4 个部分，分别是：

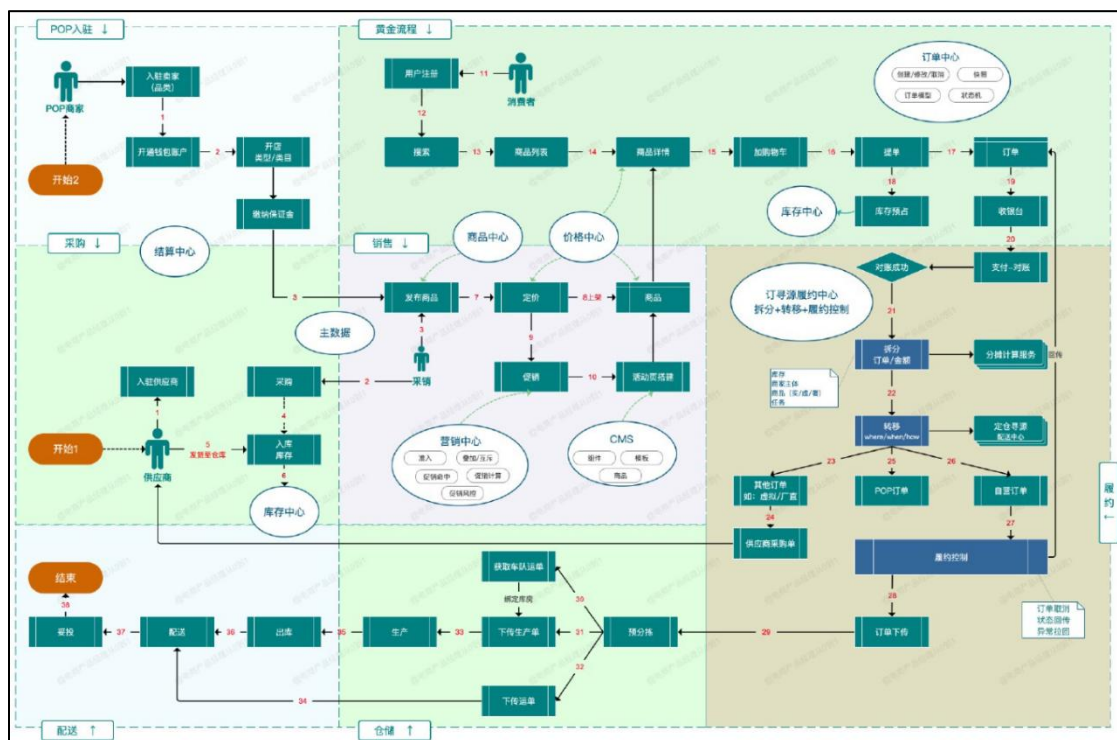
- 从供应商处采购产品
- 采购产品入仓存储管理
- 商品上架到电商平台销售
- 根据销售订单进行履约配送

用业务流程图表示如下：



1.2 系统流程

了解完业务流程后，基于业务流程再来梳理这些业务环节背后所需的系统就比较容易了。基于上面的业务梳理结果，我们将业务流程和所需系统或功能结合起来，看看每个业务环节都有哪些系统或功能，同时，我们将整个系统流程按**采购仓配**的业务边界进行划分，再结合 C 端购买流程，就能得到一张完整的大型电商系统流程图，如下：



上图就是一个较为复杂的电商业务在系统中的流转逻辑（参考京东），从供应商/卖家入驻平台到采购商品到收货入库到上架销售再到配送履约，通过这张图就可以比较清晰的看出每个业务环节所对应的系统或功能大概有哪些了。为了便于初学者理解，有必要对流程图中的的部分节点进行解释说明（可结合流程图图对比阅读）。

1.2.1 入驻&采购流程

- **入驻**：包括供应商入驻和卖家入驻，入驻动作主要包含选择入驻企业类型、基于类型提交相应材料（营业执照、资质等），签订合同，经过平台审核通过后，再开通企业账户钱包。
- **开店**：对于卖家而言，入驻成功后即可创建店铺，创建店铺主要是根据营业执照的经营范围选择主营类目、填写店铺基本信息、缴纳质保金等。
- **供应商入驻**：采购供应商入驻后，自营采销在采购系统中下采购订单，订单通过 EDI 或者线下的方式推送给供应商。
- **供应商发货**：供应商收到采购订单后，根据采购单中的信息（商品、收货仓库等）发货，发货后即产生采购在途库存。
- **库存**：无论是在途库存，还是实物入库后产生的实物库存，以及前端的可售库存等，均由库存中心控制。

1.2.2 销售流程

- **发布商品**。即创建商品 sku，包括填写商品参数，商品详情介绍等信息，POP 卖家会直接设置价格，自营因品类而异，创建商品时会调用主数据，即 spu。
- **定价**。发布商品时会通过定价系统制定销售价格，销售价格与采购价、库存成本价、促销价等一系列价格组成复杂的价格模型，并一起记录在价格中心，形成完善的价格体系。
- **上架**。发品并定价完成后，可通过上下架功能控制商品上架销售。
- **促销**。即通过营销工具做促销活动，包括单品类、总价类、优惠券等，所有的促销规则均通过营销中心控制，包括活动准入规则、不同活动之间的叠加互斥规则、促销命中规则、优惠计算、促销风控等。
- **活动页搭建**。促销商品有时候会在特定的活动专题页面集中展示（满减专区、特价专区等），通过 CMS 系统可以随时随地通过拖拽组件的方式搭建一个任意样式的专题页面，而不用临时开发。

1.2.3 黄金流程

指在用户视角下购买商品时所经历的页面，包括搜索—列表—商品详情页—购物车—提单页，很多公司内部也叫交易主流程或交易动线；

黄金流程是价格和促销的主要体现环节，因此在黄金流程中，对于价格和促销的设计尤为重要，包括优惠价格的高亮突出，促销标识的设计，购物车的凑单分组等，对转化有直接的影响。

- **生成订单**：生成订单在后端是一个非常复杂的过程，包括校验库存、价格、库存、用户信息、促销信息等，订单由订单中心负责创建生成。

- 预占库存：生成订单的同时，会与库存中心交互预占库存。
- 收银台：订单生成和支付是两个独立的环节，常见的支付方式有【在线支付】和【货到付款】，部分业务（B2B）会设置【对公转账】支付方式，后面两种支付方式以线下的形式完成。对于在线支付的订单，则可以将多种支付方式聚合到收银台页面，包括自由支付、网银支付和三方支付（微信支付、支付宝支付、京东支付等）。
- 支付对账。对于先款后货订单，各类支付方式均设有支付额度限制，对公转账的方式也无法保证用户一定按照订单实收支付，因此存在实际支付金额不一定等于订单应收金额，所以需要经过对账系统完成对账。

1.2.4 订单寻源履约中心

介于交易流程和仓储流程之间的系统——将电商系统生成的数以万计的订单，按时下发给最适合的库房进行生产，就是订单寻源履约系统的职责。订单寻源履约中心由多个子系统或服务组成，包括拆分系统、分摊计算服务、转移系统、履约控制中心等。

- 订单拆分：对账成功后，订单会进入到寻源履约中心的第一个系统——拆分系统。拆分的场景主要有：生产维度（仓库不同、商家不同、配送方式不同...）、业务类型维度（实物订单、虚拟订单、生鲜订单...）等，该系统的职责是按照不同的规则将父订单拆分成多个子订单进行生产。拆分时会调用分摊计算服务计算每个新子订单的金额。
- 订单转移。订单拆分完成之后会流转至转移流程，订单拆分完成之后会生成不同类型的订单，经过转移之后，不同的订单要执行不同的生产时机、不同的生产地点、不同的生产流程。
- 针对虚拟订单，不用经由库房实际生产，直接由转移给订单中心或者虚拟业务对应的系统进行处理。
- 针对厂商直发订单和 POP 订单，因为这类订单都是由三方卖家发货或者工厂直接发货，转移系统会与对应的业务系统交互，生成供应商采购单或下传订单给 POP 对应的系统。
- 针对自营的订单，由于订单商品所属的仓库不同、配送时效不同、商品的类型不同（如生鲜如要走冷链库房生产）、用户指定送达时间等原因，转移系统会控制订单生产的时机（如一周后送达订单则需要控制不要立刻下传库房）、生产的流程，生产的仓库。
- 履约控制：履约控制系统负责控制订单生产的流程，将订单推送至对应的库房，并回传生产节点（拣货、复核、打包出库...）给前台系统；针对取消逆向订单，根据订单流转的不同节点做对应的控制：如订单未流转至仓库，

则负责暂停订单下传，订单未出库则负责通知仓库终止生产、订单未派件则通知配送系统终止派件等。

- 订单下传。订单经过拆分、转移、履约控制后，按时下传到对应的库房进行生产。

以上就是电商全流程的介绍，全流程中的每个流程节点都是一个独立的、庞大的系统，本文只是介绍各个系统之间的流转关系和基本职责。

2. 业务系统核心功能(☆☆理解记住)



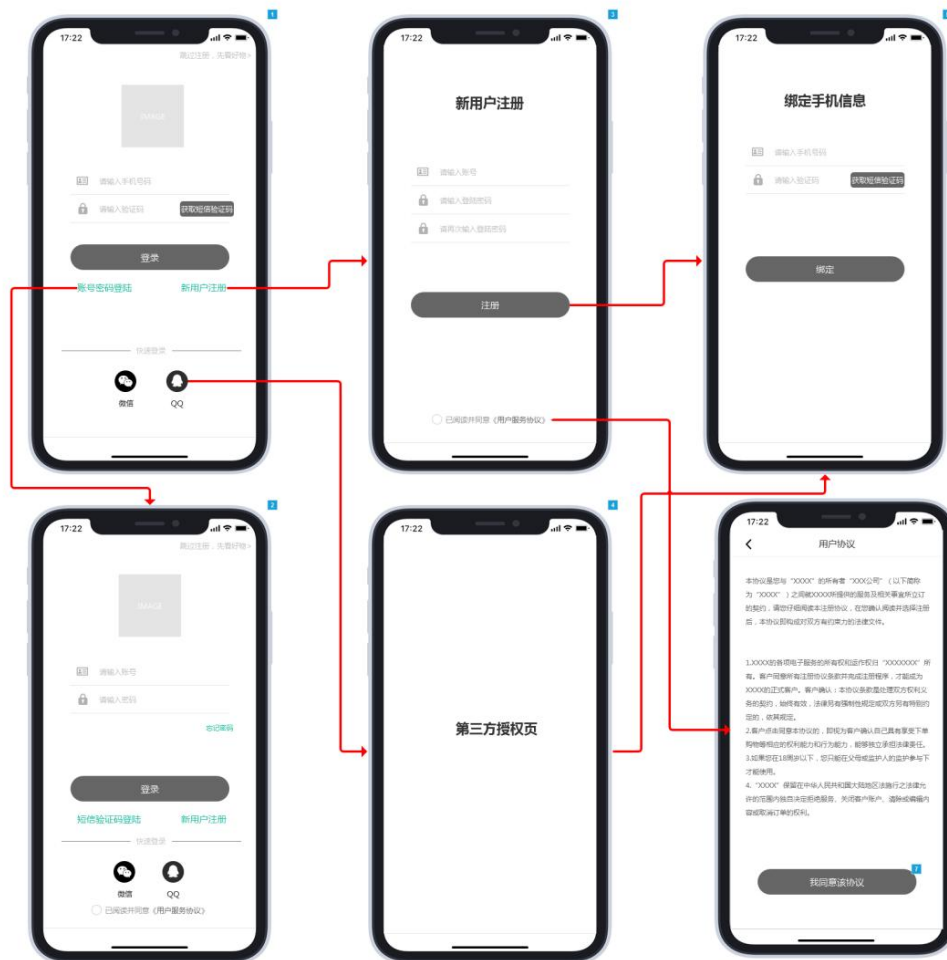
- **用户端系统**：主要负责用户选购商品的需求，核心系统包括注册/登录、黄金流程和个人中心等；用户端系统属于前台系统，在产品设计上更注重用户体验、数据分析等。
- **运营系统**：主要承载了内部运营的能力，核心系统包括用户管理、商品管理、价格管理以及营销管理等。
- **交易履约系统**：交易履约系统是一个中枢系统，向上承载订单交易，向下控制生产履约，核心系统有订单中心和寻源履约中心，属于电商系统中比较黑盒的部分，界面较少，更多的是底层逻辑。

- **供应链与生产系统**：供应链系统即进销存系统，在很多企业中统称 ERP，主要负责商品的采购、库存的管理等、仓储管理（WMS）以及运输管理（TMS）。
- **基础平台**：基础平台是业务系统之外的系统，主要包括员工账号管理、主数据、财务系统、商家管理系统、以及服务市场和开放平台等。
- **BI 系统**：平行于电商的其他系统，采集各个系统产生的数据，加工处理后反哺其他系统，提供各类数据分析能力。

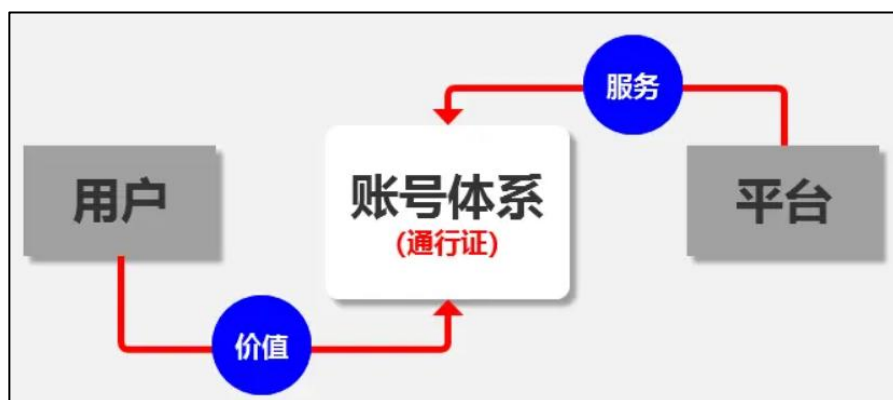
如果将电商系统比作一个大型超市。

- 用户端系统就是可以看见的超市本身，首页就是超市的入口，搜索就是导购牌，列表就是货架，购物车就是购物车，提单就是出口收银台，用户端系统负责承载消费者的选购下单需求；
- 运营系统就是超市的营销导购人员，负责引导消费者购物、负责制定商品价格、发布促销活动，CMS 系统就是超市的 DM 单、易拉宝，运营系统向上支撑各渠道、各终端的销售需求，向下对接订单系统和交易履约系统；
- 交易履约系统就是超市的调度员，对消费者的订单负责，当超市缺货时，及时从附近仓库调货，或者向工厂订货，保证消费者的订单能按时履约；
- 供应链与生产系统就是超市的幕后工作者，包括采购员、库管、配送员等。

2.1 登录模块与账号体系



账号体系是用户在各平台上的通行证。平台给与用户可持续的服务，用户在平台上获取价值，中间的媒介，便是账号体系。我们可以将其理解为维系用户与平台之间的枢纽。注：本文中，账号=账户，二者概念一致。



我们都知道，产品的最终服务方是“人”，那么，人拿什么来证明，我是这个产品的服务对象呢？可以想象一下，我们去银行的时候，银行让我们开“户”，提供了身份证，办了银行卡，卡号为 6367*****6318，那么，银行卡便是我们的账户；我们去屈臣氏的时候，结账时让我们办个卡，提供了手机号，办了会员卡，卡号为 QCS3***3，那么，会员卡便是我们的账户。

在这两个例子当中，身份证跟手机号都是我们在这个世上唯一的标识，可以作为注册账号的凭据。而我们提供了个人的唯一识别码（例如身份证、手机号等）后，不同的平台，将其转换为平台用户唯一的标识码（例如上文提到的银行卡号及会员卡号等）。

那么，有了账户，我们便可凭借着“通行证”在平台上获取服务。账号体系存在的价值在于：

- 提升系统的持续迭代能力，为后期产品扩张预先铺垫；
- 加强账户相关功能模块的可行性，保证系统的稳定性；
- 账户融合能力提升，产品生态圈布局一站式账户通行证。

并且，随着产品周期的逐步递进，账号体系的长尾价值会日益明显。

账号体系分为多种，从互联网发展至今，可归纳为四种，**自定义账号、邮箱账号、手机号账号、第三方登录账号**。

1. 自定义账号

自定义账号为“账号+密码”的形式，一般账号为用户自行定义，是比较久远的账号体系。在互联网野蛮生长的前期，各大平台尚未建立，且智能手机还未出世，使用平台均为PC端，则账号需用户自行拟定。

缺点的话，相信经历过的伙伴记忆尤深，经常拿着个小本子记录着不同的账号密码。QQ一个，某游戏平台一个，某社交平台一个……若设置得不同，则增加了用户的记忆负担。

放在互联网发展初期，则较为适用，而对于现在如此多的平台，“账号+密码”的方式已经开始慢慢逐步被替代。

2. 邮箱账号

在互联网蛮荒时代的过去，为了解决“降低用户记忆负担”的问题，适逢各类邮箱盛行，QQ邮箱、网易邮箱、新浪邮箱等各大巨头争相抢占市场，邮箱已成为当时用户的常用工具，记忆难度远比随意取得账号名更为简单，所以被用以“邮箱号+密码”的登录方式。

而邮箱作为注册方式，优点是像对于自定义账号的体系，邮箱更能够降低用户记忆负担；缺点也很明显，邮箱的认证环节较为冗长，需要打开邮箱接收邮件并确认，容易在这个过程中流失用户。

且在以往，PC端占据主导，如今，移动端占领大半江山，大多数用户不会在手机上安装一个邮箱应用，那么，在移动端接收邮箱验证信息的时候，难度增加，流失率也会增多。

所以对于目前移动端主导市场的情况来说，“邮箱+密码”的账号体系，也已经开始慢慢地淡出人们视野。

3. 手机号账号体系

据数据资料显示,截止 2018 年,智能手机的市场份额占比已经达到 98.6%,那么,可以说基本人手一机。每个人可能没有社交账号(微信、QQ 等),但一定会有手机号。

优点的话,对于用户来说,记住账号可能困难,但记住自身的手机号较为简单,且安全性高;对于平台来说,利用手机号登录,可以搭建起自身的账号体系,便于后续的精细化运营(活动通知,用户召回等等);由于目前手机号都是运营商实名,可减少垃圾营销号的账号。

缺点的话,其一,“手机号+验证码”登录需要支付一定的短信费用,好在价格不多,但需要防范黑客的恶意攻击,这里也涉及到账号体系的风控机制,后续会谈到;其二,在信号差的地方,接收不到验证码;其三,手机号存在换号情况,需要考虑解绑逻辑及换号逻辑。

注意点:手机号账号体系的便捷点在于,可将注册与登录流程合为一体,若未注册,则在获取验证码后登录的同时,系统自动帮用户注册,减少了用户的操作成本,提高便捷性。

但同时,由于其便捷性,也有其局限性,用户可能变更手机号,所以需要在用手机号注册之后,引导用户设置密码,减少风险。

4. 第三方账号体系

随着各大超级 APP 的诞生及趋于成熟,衍生出了“第三方登录”的这个概念。

第三方账号体系是指用户通过授权,将第三方资料绑定到自身账号体系上。当查询到用户第三方的帐号已经绑定了平台的某个 user_id 时,直接登录对应的帐号,实现一键注册与一键登录。

优点的话,对于用户来说,授权方便,当拥有主流社交账号(如微信、QQ、微博等),即可一键注册及一键登录,无需浪费太多时间。且可获取第三方平台的一些基础资料,填充了账号前期的空白(例如头像、昵称等)

缺点的话,主要针对于平台,无法形成自身的账号体系,依托于第三方,有一定的风险(第三方倒闭之类的),但可通过后续进入到平台,引导绑定手机号等操作来弥补。

2.2 商品系统与商品中心

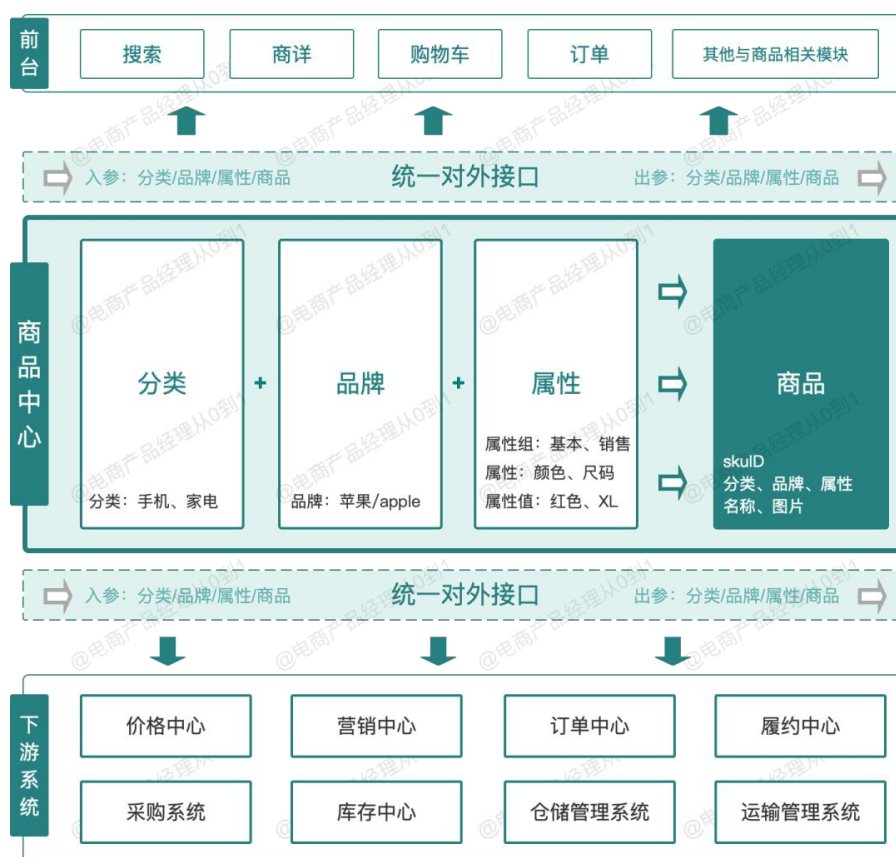
什么是商品系统?顾名思义,商品系统是负责管理与商品有关的数据和能力的系统,可以用以下三句话概括:

- 负责创建和管理商品的系统

- 负责创建和管理分类、品牌、属性以及其他所有与商品相关的数据的系统
- 负责对前台商品展示以及相关上下游系统提供商品能力的系统

在电商产品体系中，商品是其他所有相关系统的基础，用技术的术语讲，商品系统是其他系统的依赖：从商品采购到库存管理，从商品自身的创建、上下架管理到商品定价以及前台展示，从促销活动到下单支付，从订单生产到履约配送，电商中的每个流程都与商品相关。

而在这些所有上下游系统之间，商品数据必须是统一的、通用的、标准化的，比如采购系统中的商品 A 和订单系统中的商品 A 需要是同一个 A，所以商品系统在很大程度上必须具备中心化服务的能力，因此商品系统也是所有系统中最优先需要设计成中心化服务的系统。



上图中间的部分就是商品中心，商品中心的核心模块可以抽象成 4 个，分别是：

- 商品分类：用于创建分类、管理分类，为商品以及其他系统提供分类能力
- 商品品牌：用于创建品牌、管理品牌，为商品以及其他系统提供品牌能力
- 商品属性：用于创建属性、管理属性，为商品以及其他系统提供属性能力
- 商品：用于创建商品、管理商品，为其他系统提供商品能力

分类管理、品牌管理、属性管理与商品管理在系统架构上相互独立存在，单向依赖，是独立的系统模块，每个模块独立负责各自的职能，既能单独对外提供

服务，又能打包对外提供服务，各个模块之间又通过特定的对象相互关联。

分类、品牌和属性作为商品的基础组成部分，三者共同组成了商品。

商品中心最基本的职能是向上支撑前台各个环节的商品查询、展示功能，同时向下为价格中心、营销中心、订单中心、履约中心、采购系统、库存中心、仓储以及运输等下游系统提供商品能力和服务，保证电商体系下所有系统对于商品数据的输入和输出的统一。

2.3 商家价值与商家后台

查阅各大电商平台的财报和日常积累，给出各大电商平台商家数量。

阿里系：2015 年达到千万级，目前虽然面临拼多多，美团，头条系和快手的竞争，但是推出淘特与之对应，商家数量也维持在这个级别之上。而品牌商家入驻的天猫，品牌数量在 30 万左右，与京东基本持平。

京东：得益于疫情线上催化以及国家对于二选一垄断的打破，京东品牌商家与天猫基本持平，达到 30 万左右。另外京东基于平台调性和品质把控要求，第三方商家要求远远比阿里、拼多多严格，结合面向下沉市场的京喜和面向性价比人群的京东秒杀供应链，京东平台商家预计在百万级别。

拼多多：2019 年 510 万，2020 年 860 万，2021 年预计达到千万级。

美团：2020 年底活跃商家数 680 万，到了 21 年第二季度，活跃商户数增至 770 万，预计目前美团商家数在 900 万左右，也快达到千万级。

抖音电商：仅仅抖音企业号数量已经接近千万，我们看企业号数量恐怖式增长：19 年 11 月 100w，20 年 5 月 300w+，20 年 8 月 400w+，20 年 11 月 500w+，21 年 7 月 800w+，那么到了 22 年 Q1，应该接近千万。另外还有数百万个人主播卖家，抖音商家生态已经形成。

商家对平台价值主要体现在：

- 贡献平台的 GMV
- 贡献平台的营收
- 贡献平台活跃性

贡献平台的 GMV 很好理解，平台数万亿交易流水主要是卖家提供商品和服务所达成的，即使像京东这样有部分自营业务，自营 GMV 占整体而言，份额不足 3 成，这些巨量规模也只有中国这个超大规模经济体（强大的制造业和超级辛勤的商家群体），超大规模消费群体（买家）才能达成。

这也是为什么中国可以出现多家超大规模电商平台的根本原因，在世界其他国家，即使是美国和印度，都不具备这样的条件。

商家通过各种佣金以及推广营销费用的形式给平台贡献营收，佣金一般根据

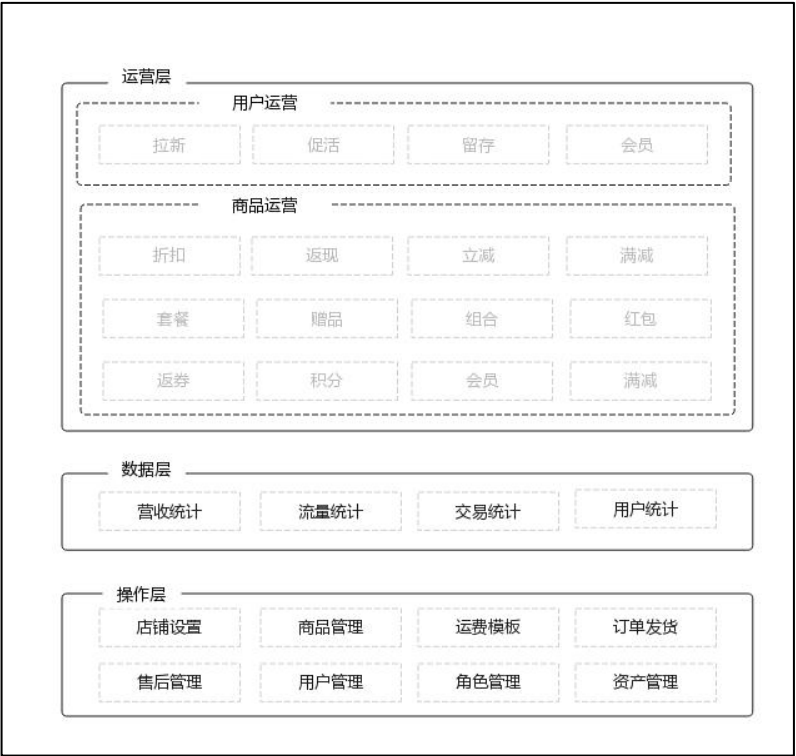
不同品类毛利水平而不同，毛利高的品类佣金高，毛利低的品类佣金低，佣金的高低也与品类内商家有关。

而推广营销费用主要是商家自助行为，当然承受力强的商家愿意支付更高的推广费用获客，这里的承受能力强，不仅仅是商家盈利能力强，也有可能是商家被迫无奈下的甩货、新店、新品等原因亏损推广。

商家的佣金扣点最多只能保本，而平台要盈利，主要来自商家推广营销费用，这一点考验平台的自然流量和付费流量的调控能力，也是平台基于经济学底层的核心能力，这部分内容在付费流量与免费流量协同中会重点说明。

贡献交易流水和营收，大家都好理解，但是贡献平台活跃性，这一点估计很多产品经理都没有想到，商品的经营和运营者是卖家的主要角色，但同时他们也作为平台买家的角色，因他们对平台的熟悉程度远远超过单纯的平台消费者，而且他们在平台上工作，用户时长和粘性要远远超过单纯的消费者。有些平台在启动初期，就是靠平台卖家第一批启动平台的活跃性，这一点阿里和拼多多都呈现类似现象。

作为平台型电商最重要的参与者之一，商家自然需要有一套独立的操作后台（系统）。商家后台涉及三个层面，且以由底往上的顺序流转进行，大体可以概括为此三层：操作层，数据层，运营层。



1. 操作层

(1) 店铺设置

店铺初次登入时，需要设置店铺信息，申请店铺认证，装修店铺。

要点：店铺认证，店铺装修

难点：店铺装修涉及到前端拖动式操作，前期建设时可设计成模板化操作，根据选择模板装修店铺，降低开发成本。

注意：店铺认证需要人工审核，后台须有审核通道，商家需要结果查询。

（2）运费模板

设置运费模板，物流设置。

要点：运费模板，物流设置

难点：运费模板。总结起来有三种：一是店铺运费模板；二是单品运费模板；三是混合运费模板。系统设计前期可先选择支持【店铺运费模板】，即可供店铺统一设置运费，应用到店铺的每个商品。运费计算方面还需要在系统设计初期定好规则，规则不是随意定，而是根据各大物流公司在各个区域的收费标准，在商家设置时动态计算运费。例如：顺丰与四通一达在广州的首重与续重收费标准不同。

（3）商品管理

要点：发布商品，商品上下架，库存管理。

难点：库存管理，库存分为两种：普通库存，活动库存。普通商品的正常销售调用的是普通库存，活动库存是从总库存划出的一部分，供活动时使用。库存管理的难点在于难以保证线上库存与实物库存一致。因为有时候业务需求是允许超卖，做预售，不同活动独占库存，不同渠道分配库存，就会造成线上库存与实物库存不一致。

（4）订单发货

要点：订单管理，订单推送

难点：订单可以说是整个电商流程最核心的一部分，在商家后台的设计中，好的订单设计可以提高商家使用成本，带来便捷的操作体验。订单包含商品，优惠，用户，收货信息，支付信息等一系列订单实时数据，通过订单中心，实现对订单的管理，支持订单接收，订单自动合并与拆分，自动匹配仓库，库存控制，自动匹配快递，结算与支付等订单生命周期中一系列协同作业。订单的难点在于它处于整个流程的核心位置，连接上下游，在各种业务场景下衍生出各种订单正向及逆向流程，是考验产品设计的一大难点。

（5）售后管理

要点：退货退款，换货，退款

难点：售后管理处理的是订单的逆向流程，在售后管理的设计上一定要充分考虑用户的体验，例如：在用户未签收时允许用户申请退款；难点在于订单的逆向流程在源于多个方面，例如：支付前取消；未签收退款；收货后退款，收货后换货，

收货后退货等。每一个场景下都对应着相应的订单状态，时间和优惠信息，如果是活动订单，还涉及售后订单优惠分摊的计算，这都需要前期的设计考虑更全面。

（6）用户管理

要点：用户管理，会员管理，会员营销

难点：系统设计上，不仅要支持平台会员体系，还需要支持商家构建以商家为中心的会员体系，这两套体系独立但也相关联。

（7）角色管理

要点：权限管理，角色管理，权限分配

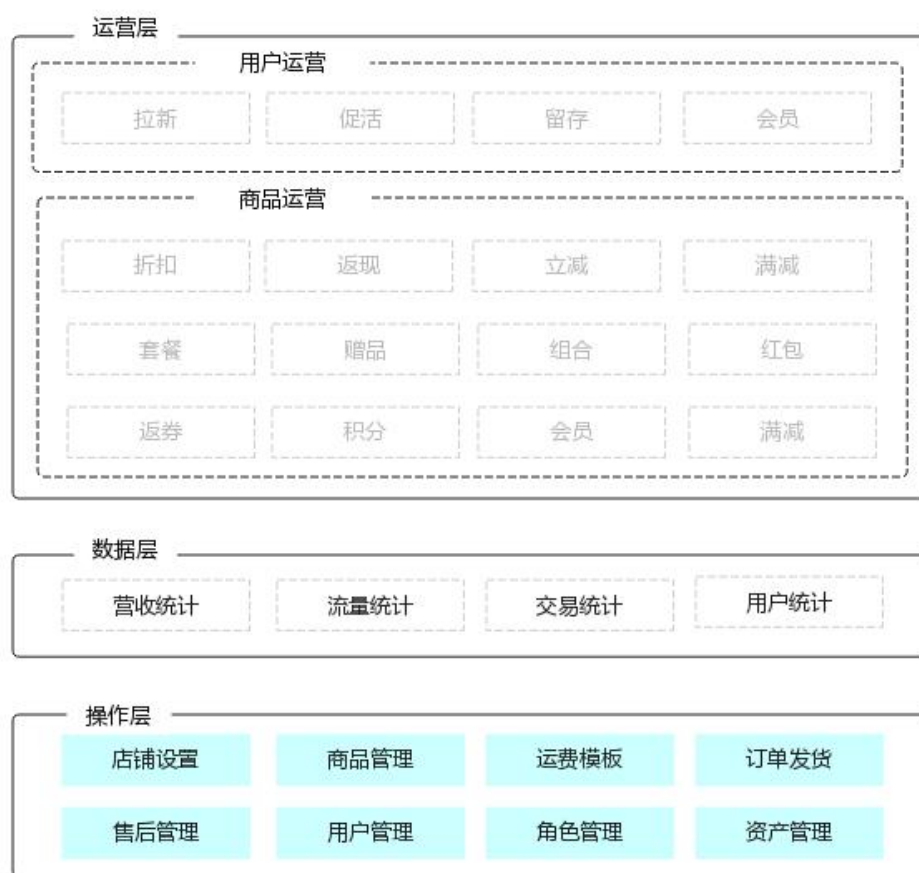
难点：角色是权限的载体，在控制用户操作功能权限的时候，可以通过授予不同角色的功能权限，然后通过对不同类型用户授予不同用户角色，就控制了不同用户之间的不同功能操作权限，形成了一个功能权限体系的闭环。怎么做到权限列表与实际功能一一对应？就需要在开发系统页面时，将统一的权限借口嵌入到页面中。

（8）资产管理

要点：资金明细，资金操作；

难点：每一条资金的来龙去脉都必须记录清楚，每一条资金记录都需与订单，用户绑定关系，方便追溯。

这样我们就简单把操作层的模块都讲完了，框架图更新如下：



2. 数据层

(1) 营收统计

要点：店铺收入，经营报表

难点：统计店铺收入，支持不同时间段查看不同数据。系统支持统计某个时间区间范围内的店铺数据，并做分析。例如经营周报：可列出一周概要，包括一周支付金额，转化率，客单价，访客数，付款人数等数据，并与上周每项数据对比，将结果可视化。

(2) 流量统计

要点：访客数量，页面浏览，访客地域，流量统计

难点：想要做好流量统计，前期设计就需要在用户端加入数据埋点，有了数据后对数据进行拆解归类，前期数据埋点越全面，后期流量统计就越全面，不仅可为商家提供访客数，还能提供具体页面的访客数量，访客地域分布等。

(3) 交易统计

要点：下单笔数，付款笔数，转化率，客单价，发货数量

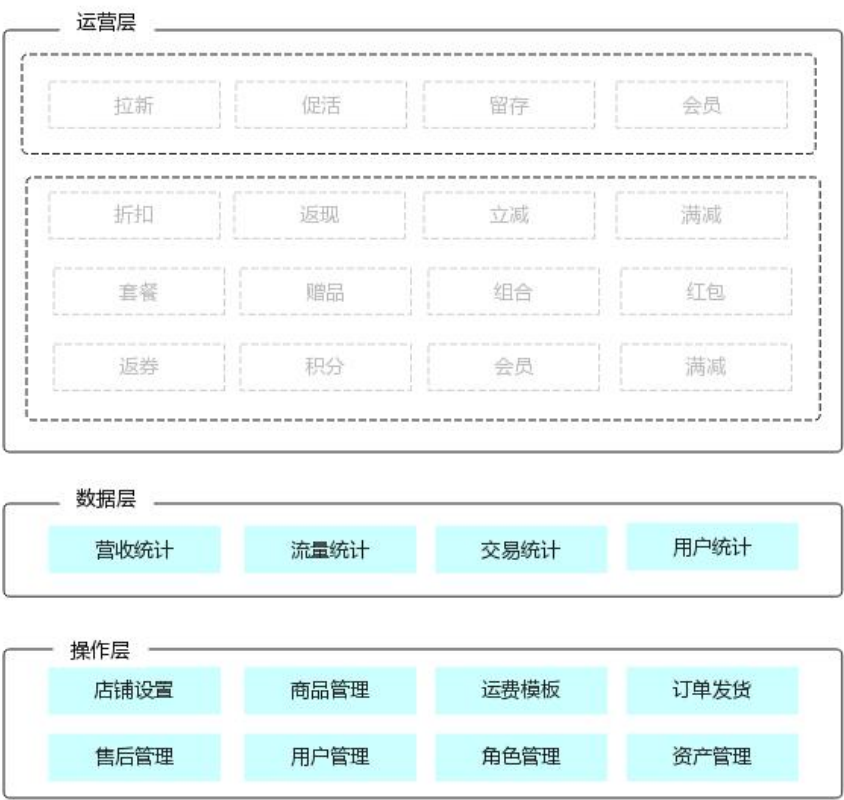
难点：统计商家的订单数据，根据订单数据计算转化率，客单价。交易数据支持

与昨日，或某固定时间区间内对比，提供经营建议

(4) 用户统计

要点：粉丝数量，增长趋势

难点：支持打通微信，微博，对用户粉丝进行统计，记录净增长粉丝，新增粉丝，流失粉丝，将增长趋势可视化，提供运营建议。



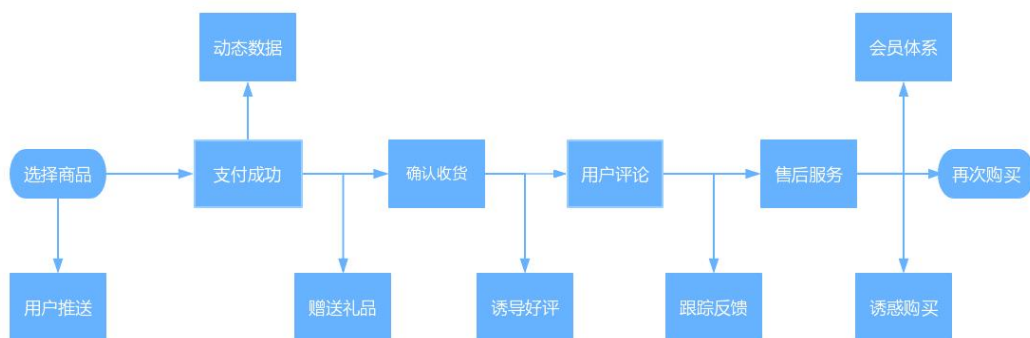
3. 运营层

正所谓产品运营不分家,好的产品运营一定能支持更好的运营,商家后台也一样,活动运营对于整个电商市场来说依然成为必不可少的一部分。

(1) 用户运营

要点：拉新，促活，留存

难点：目前整个市场的运营玩法五花八门，不过基本都是基于节日做出的活动促销，通过大量的优惠活动促进用户消费。要把握这些点必须要数据的支撑，大数据当眼睛，运营规则当指挥棒。用户运营大概如下图所示流程进行。



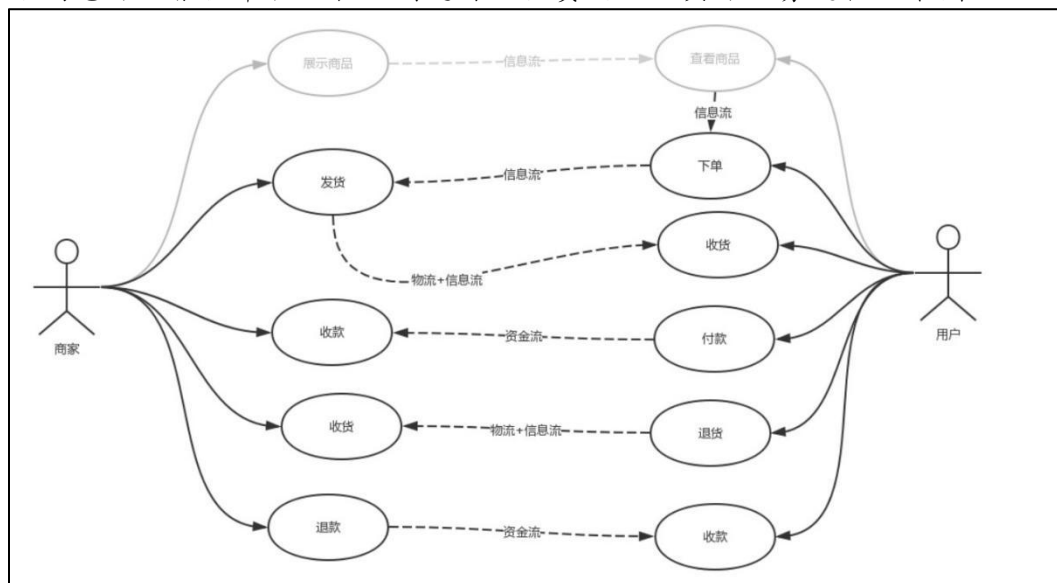
(2) 商品运营

要点：活动营销，丰富玩法

难点：用户运营是发现，引导，而商品运营更多的是控制，运作。商品运营的难点在于如何灵活支持丰富的营销活动，营销活动灵活性高，种类丰富，玩法多样化，而操作系统反而需要便捷，易操作，这正是系统设计的难点所在。

2.4 交易系统核心功能

交易的本质是一个信息流、物流和资金流的转换过程，商家通过平台展示商品信息，用户获取商品信息后做出购买决策，用户付钱交换商家提供的商品和服务，商家通过物流或快递把货权转移给用户，如果用户对商家交付的商品或服务不满意可以根据平台运营规则进行退换货处理。具体业务过程如下图：



而我们的交易系统就是要把整个交易过程中涉及的业务完整的记录下来，以备买卖双方随时进行跟踪、查看。为了让我们记录的信息成结构性，我们把承载不同阶段的业务信息的对象划分成如下几类：

- 订单——记录交易双方的主体信息、交易品信息、交易的关键节点信息以及

订单本身特有的信息；

- 发货单——记录卖家对订单履约过程的信息；
- 退款单——记录退换货信息处理过程的完整的信息；
- 资金流水——记录用户付款、商家收款、商家退款、用户收款以及与订单的关系的信息。

下面以以上三个单据为核心，对相关业务进行分解。

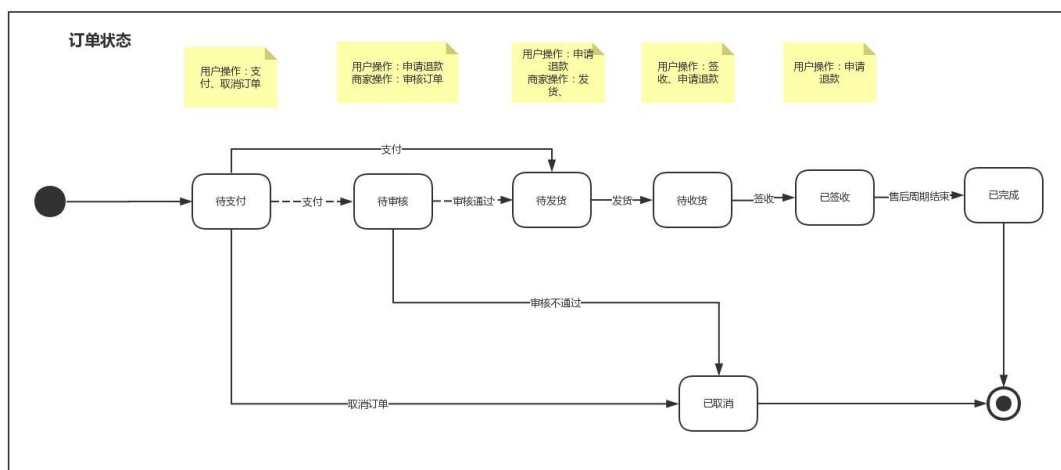
一、订单业务分析

首先，我们看下订单的基本信息结构，如下图：



在订单的基本信息中我们会包涵订单号、订单创建时间、订单状态等关键信息。其中订单一般会有主子订单号之分，这是订单拆单业务的产物（拆单在订单中是一个关键业务，后面单独进行说明）；

订单状态一般用于表达订单的整个生命周期，一般如下图：



- 其中待审核状态在有些业务中存在，有些业务又不存在，我们在做中台时该状态是一个可选的节点。
- 买家和卖家信息一般记录其相关的 ID、名称或等级等信息，卖家还需要记录

其店铺的信息。

- 开票信息指的是用户在下单是选择的开票类型以及提交的开票资料，普票和增值税、个人和企业其所需的开票资料不同。
- 支付信息只记录支付时间以及支付流水号，该流水号是能够贯穿交易、支付以及第三方的一个表示。
- 订单结算信息一般的结构如下图：

商品合计金额	实付金额	优惠券优惠	店铺优惠券优惠	活动优惠	店铺促销优惠	会员权益折扣	积分抵扣	运费
1000	911	20	10	20	10	30	5	6

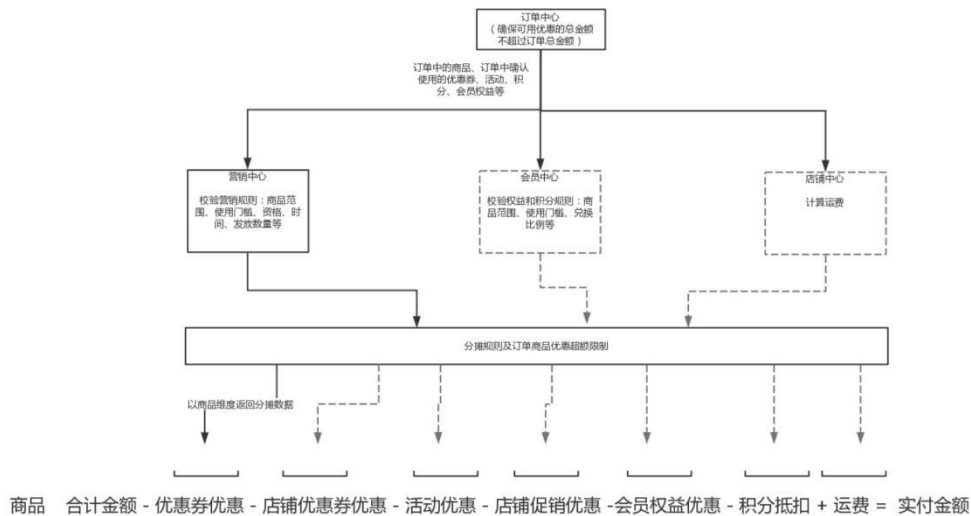
每一项优惠或者折扣都需要分开记录，而优惠、折扣和抵扣都是由对应的业务系统进行业务处理后的返回值。

- 收货人则指的是收货人的名称、手机号以及地址信息。物流信息明细一般都是通过第三方接口获取，再记录到系统中。
- 最后还有一部分则是交易品明细，需要注意，订单中记录的交易品明细一定是下单那一刻确认的交易品信息，包括交易品的基本信息、价格信息、数量等。这就要求我们在系统中需要有交易品版本的管理，能够准确的记录在某一个时间段内，任何一个交易品的准确信息。

订单中的交易品明细一般的信息结构如下：

商品名称	ID	数量	单价	合计金额	实付金额	优惠券优惠	店铺优惠券优惠	活动优惠	店铺促销优惠	会员权益折扣	积分抵扣	运费
不俱肥臀粗腿宽松吊裆针织裤哈伦裤	2184082 4184012 41	2	65	130								
欧洲站气质高腰针织七分阔腿裤	2184082 4184022 13	4	53	212								

在整个信息结构中最难的时如何获取优惠、折扣和抵扣，而在一般的系统设计的逻辑步骤如下（以优惠券优惠为例）：



在订单这部分的业务中有一个十分重要的业务逻辑需要进行说明，那就是订

单拆单。拆单主要为了满足在财务结算以及物流发货上的相关需求，也能够让每个商家更好的完成履约。在电商的交易、物流流程中有可能需要拆单的环节有：购物车、订单提交、发货/配送、发包裹。

前两个环节属于交易业务内，后两个环节属于物流配送环节，在此我们仅对前两个环节的拆单进行说明。在交易环节拆单主要有这几个常见的因素：交易模式（海外购）、品类（药物、生鲜或一些特殊品类）、店铺。

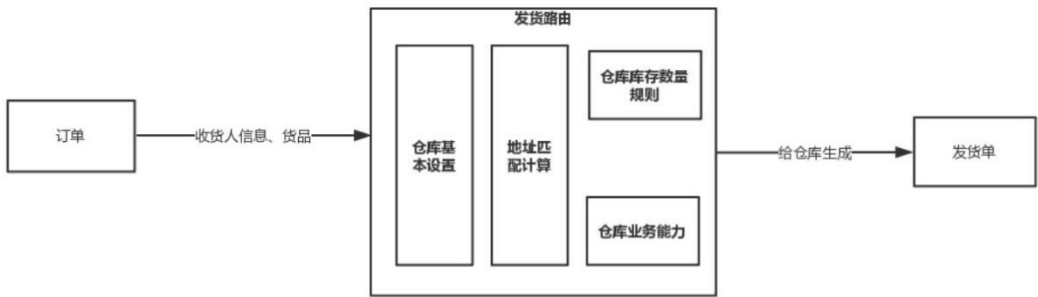
交易模式拆单是因为其在订单确认时需要进行操作和展示的信息完全与普通交易不一样。例如：海外购需要进行一些必要的资质认证、关税以及对金额的一些限制等等，会员店需要进行资质认证或有特殊的结算方式等。而**品类**主要会影响下单的环节，例如：药物需要先有处方，在审核通过后才能下单等。而**店铺**则是因为要分成不同的商家进行履约、结算而需要拆分订单。

二、订单发货履约

订单履约业务从商家进行发货操作开始，一直到用户签收并完成订单为止。严格意义上的履约应该包涵如下步骤：发货-拣货-出库-配送-签收这几个环节，但拣货-出库-配送这三个环节是属于物流管理系统（WMS）所负责的业务，我们在此不做过多的阐述。

在发货时，我们需要注意的是这是从信息流转换到物流的过程，在当前大家都追求线上线下融合的大趋势下，我们需要提供一个灵活的发货业务处理逻辑。

首先我们能够满足一个订单多次发货、多个订单合并发货这些场景；其次在电商系统对接了物流系统后，我们需要能够做到发货单具备路由的功能，即系统能够根据客户的地址和仓库的地址、货物在仓库的库存情况以及仓库业务处理的能力等因素进行自动选择，以达到库存效益和用户体验之间的最大收益。有发货路由的整体业务流程：



发货单需要记录：发货单基本信息、订单基本信息、收货人信息、货品信息、发货仓信息以及最终的物流配送信息。用户即能在订单中查看商品的物流情况，而一般的订单中的物流信息都是通过为在发货单上面记录的快递单号/运单号在通过第三方物流信息对接平台获取的。在商家进行发货操作后，用户能够对商品进

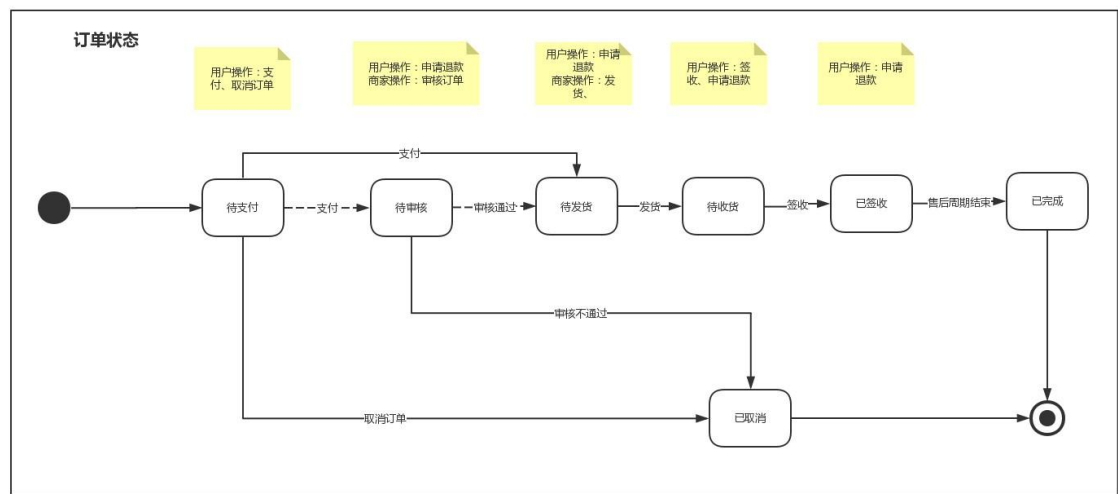
行签收，签收操作的目的是确定货物物权的转移，很多企业会以该节点作为财务中收入确认的触发节点。

三、退款相关的业务

1. 退款的整体介绍

退款业务可分为三个大的类型：仅退款、退货退款、换货，本期不实现换货业务。仅退款指的是对资金流进行处理，不产生物流；退货退款指的是需要处理物流及资金流。

那再什么情况下会产生退款业务呢？首先我们回顾下整个交易流程是怎样的，如图：



这是业务简化后的状态图，在业务中订单支付后只要进入待发货（如果有待审核，也可）状态以后，在订单状态为已完成之前，都能发生退款业务。具体场景如下：

- (1)商家还未发货，用户发现购买的商品不是自己需要的，申请退款，可对已提交的订单中的部分商品进行退款，也可整单进行退款；
- (2)商家已发货，但用户还未签收时用户进行退款申请，可对订单中的部分商品进行退款或整单进行退款；
- (3)用户已经签收，但订单还未进入已完成状态时用户进行退款申请，可对订单中的部分商品或整单进行退款；

2.5 营销系统核心功能

促销是电商运营最常见的手段，促销规则复杂多样，拥有着优惠券、限时折扣、满减送、限时特价、团购和预售等等多种形式的，其对提高客单价和客单量有很大帮助，所以了解促销体系对电商运营至关重要。要了解促销活动，首先需要了解促销活动的目的究竟是什么。

- 拉新：促销可以吸引新用户，尤其是用户较少的新平台和新店，从而吸引用户。
- 去库存：通过活动，可以清理库存，降低库存的占用成本。
- 扩大品牌知名度：与广告一起进行促销可以扩大品牌知名度。
- 推新：不少商家大力开展新品或热销产品的推广活动，增加店铺流量，也为其他产品提供曝光机会。
- 与其他平台竞争：现在有很多人造购物节，比如双 11、618，还有很多电商平台或企业主动或被动地进行促销活动。
- 提高客单价和客单量：在优惠券、满额折扣、满额赠品的催化下，用户会不由自主地进行凑单购买，增加购买金额，产生很多额外的购买行为。

在进行促销活动的产品设计时，应提前对运营活动成本进行核算，以及预估活动效果，选择最合适的促销手段。目前，互联网平台可使用的营销方式已经比较成熟；C 端在用户侧包装及展现形式上可能会存在变形和延伸，但对于 B 端而言，更需要关注的是营销系统底层逻辑，抽象出表象下的可复用规则，及各系统之间互相之间的关联作用。

1. 营销工具类型归纳

互联网目前出现的活动形式大约有 13 种，按照使用场景可归纳为三大类：

- 单品类活动：折扣，立减，买赠，特价，秒杀；
- 凑单类活动：阶梯满减，阶梯满折，阶梯满赠，M 元 N 件，买 M 件免 N 件，第 M 件 N 折，第 M 件 N 元；
- 传播类活动：拼团

单品类活动适用于提升转化率，凑单类活动适用于提升客单价及产生关联售卖，传播类活动则适用于拉新及进行站外活动。

2. 资源位活动

一般来说，资源位活动与“栏目”比较相似，这些“栏目”出现在可以由平台给与流量的位置，如：首页、首页轮播、搜索结果页等，可包含一种及一种以上的活动类型。

常见的“栏目”有秒杀、拼团、品牌特卖等，不同平台对“栏目”的主题和包装形式不同，栏目与活动形式可形成一对一、一对多及多对多的关系；

以京东首页为例，“京东秒杀”、“每日特价”、“品牌闪购”等，都可以定义为“栏目”，其中“品牌闪购”只是“栏目”名称，在二级页面内又可包含买赠及折扣等多种活动类型。



3. 业务数据库(☆☆理解记住)

我们的业务数据库就是基于产品的设计而设计出来的数据存储结构。它主要是存储各个后台系统的数据。这些数据都存储在 MySQL 数据库中，由后端开发人员进行日常开发和维护。这里列出电商业务系统种常见的 40+个表：

market_coupon	product_attr	product_spu	user_address
market_coupon_history	product_attr_group	product_spu_attr_value	user_userinfo
market_coupon_spu	product_brand	product_spu_desc	userbehavior
market_coupon_spu_category	product_category	regioninfo	
market_member_price	product_category_brand	shop_base_info	
market_seckill_promotion	product_comment	shop_spu_info	
market_seckill_session	product_comment_replay	trade_order	
market_seckill_sku	product_goodsbrand	trade_order_item	
market_seckill_sku_notice	product_goodscolor	trade_order_operate_history	
market_sku_bounds	product_goodsinfo	trade_order_return_apply	
market_sku_full_reduction	product_goodssize	trade_order_return_reason	
market_sku_ladder	product_sku	trade_order_setting	
orderdetail	product_sku_attr_value	trade_payment_info	
orderinfo	product_sku_images	trade_refund_info	

3.1 用户模块

3.1.1 账号体系概念

如何设计账号体系？

1. 账号体系的字段

要设计一个账号体系，那么需要先了解，账号体系包括哪些基础字段。

(1) 用户账号 (User Account, 包括自定义、邮箱、手机号)

用户账号 (User Account) 为系统平台内唯一的标识，不同的账号体系类型，用户账号的展现形式也不同。包括自定义账号，邮箱号，手机号三种类型。

自定义账号的，则用户账号由数字+字母或者中文组成。

邮箱的话，则为邮箱号，数字+英文组成。

手机号，则由 11 位数字组成。

通常处于隐私性及安全性的层面考虑，用户账号仅对个人可见，其余人不可见，是用户个人的通行证。

(2) 用户密码 (User Password)

用户密码 (User Password) 是基于“账号+密码”体系类的字段，一般由字母跟数字组成，部分密码会设置成包含符号。

(3) 用户名称 (UserName)

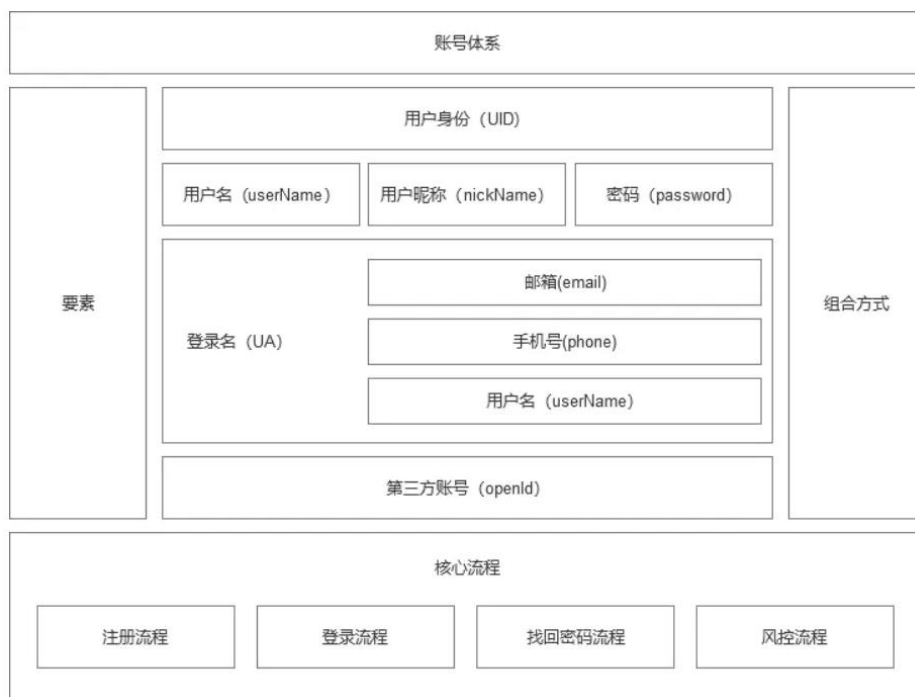
用户名称 (UserName) 为用户对外展示的名字标识。可以是用户自定义，也可以是系统先自行分配设置，后续提供修改的权限，若是采用第三方授权登录，则一般会以第三方的昵称作为用户名称，后续可修改。

(4) 用户唯一标识 (UserID, 简称 UID)

用户唯一标识 (UID) 为系统配置，一般为数字组成，不对用户可见。用户在注册为系统的账号后，则自动生成，不可更改，与每个用户一一对应。

(5) 开放账户 (OpenID)

借助第三方网站 url 验证用户身份，当用户第三方授权登录成功，系统通过获取用户的 open ID 生成 user ID,同时也读取用户在第三方网站的身份昵称以及头像，该昵称可作为首次的账号名。



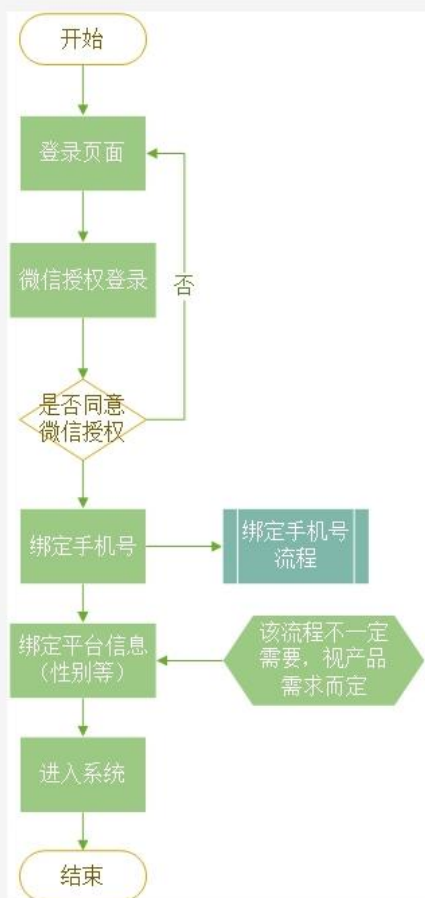
3.1.2 账号体系的核心流程

账号体系包含四个核心流程，分别为注册流程、登录流程、找回密码流程、风控流程。其中，注册流程与登录流程可合并为一起。

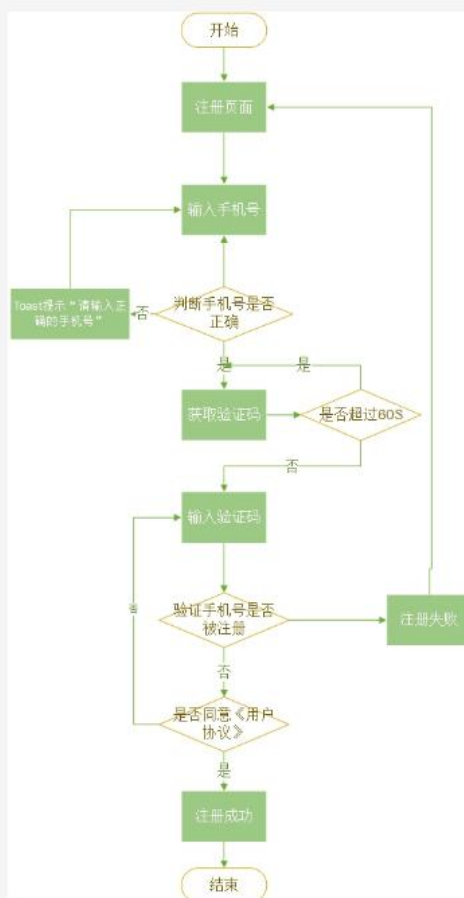
(1) 注册+登录流程

要设计一个登录注册流程，那么我们需要先简单的进行该流程的具体分析。账号的注册登陆流程，遵循三个原则：①安全性 ②普适性 ③便捷性。

第三方登录流程



手机号注册流程



(2) 找回密码流程

常见的几种找回密码的方式：手机号、邮箱、银行卡、人工申诉。

①手机号验证

若密码丢失，则可通过“原手机+验证码”的形式来进行重设密码，但会有运营商二次放号的情况，那么需要在系统设计中规划多个“受信任手机”或“受信任设备”的情况。

②邮箱找回

邮箱找回的方法是以往PC端的经典方法，PC时代=邮箱时代。QQ、网易等主流邮箱占据了整个市场，自然通过邮箱找回最为方便。但随着移动时代的到来，邮箱找回的方式逐渐被遗弃，且在账号体系若无通过邮箱为主的账号方式，则一般也很少会在体系当中要求用户绑定邮箱。在移动时代，邮箱找回较为麻烦且用处不大，但可作为手机号找回的辅助方式。

③验证银行卡

若在体系当中涉及到银行卡绑定的，则银行卡可作为个人的唯一标识，通过“银行卡+预留手机号+验证码”的验证方式来验证个人信息，从而明确是否本人

的方式，来找回密码。

④人工服务

在各类验证方式都走不通的情况下，只能通过人工服务来进行找回密码。需要先让用户上传身份证等个人确切信息之后，客服在一定的工作日内进行处理，会耗费相应的客服资源，且反馈时间较长。

(3) 风控流程

风控流程的本质在于确保用户身份的合法性及正确性，防止部分不法分子恶意攻击及注册，影响原本体系的安全。

账号作为用户登录系统的“通行证”，其重要性不言而喻。而保护好账号的安全，也是至关重要的。其在于注册、登录流程上，被黑客恶意注册，导致平台资源浪费；账号信息泄露，导致用户信息受损；不论是对于平台还是对于用户，都是极大的损失。对于风控措施，可从恶意注册账号、IP、盗取等方向来思考。下面列举了以下几项针对于账号体系的风控措施，可供参考。



3.1.3 账号体系的合并与打通

在各大系统中，存在多个系统，多个账号，互相交杂的情况，那么，延伸出了账号的合并问题与打通问题。

同个系统中，存在的问题是合并问题：同类型的账号合并；不同类型的账号合并。不同系统中，存在的问题是打通问题：单向数据打通；双向数据打通。

1. 账号体系的合并

账号的合并指的是，在一个系统内，一个用户的多个账号合并成为同一个账号。账号分为同类型与不同类型两种。同类型指的是注册类型相同，例如同一个人用两个手机号注册两个账号；不同类型指的是注册类型不同，例如有手机号注册，有微信注册，有邮箱注册，但其拥有者均为同一个人。

那么此时，用户发起合并，将其所拥有的账号合并为同一个。处理方式为：保留其中一个主账号，将数据累加到主账号当中，其余账号数据清空。



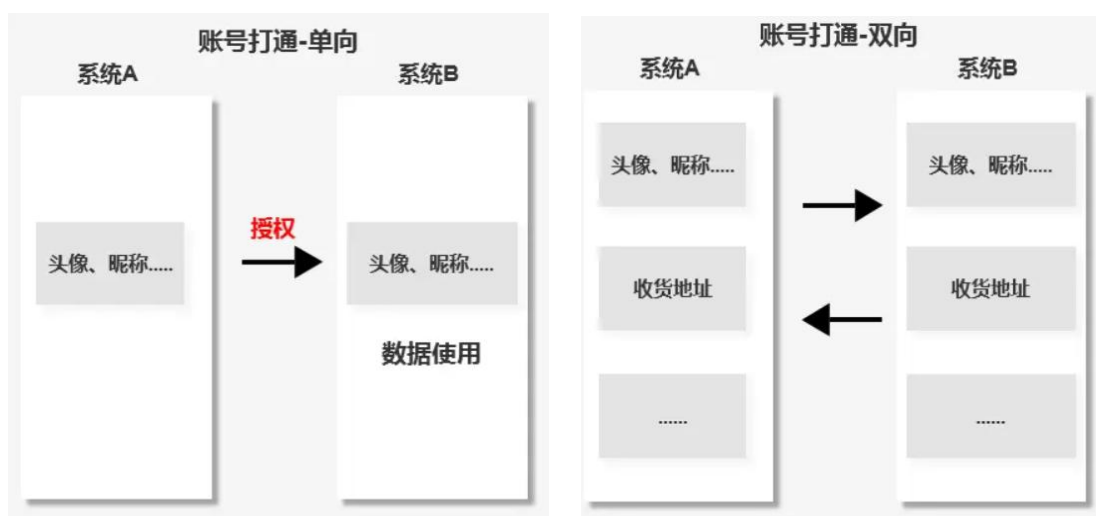
2. 账号体系的打通

账号的打通指的是，在不同的系统，账号数据的互通。在这当中，“互通”又分为单向打通与双向打通。

单向打通指的是一个系统有权使用另外一个系统的部分数据，这里的“有权使用”我们可以理解为“授权”。第三方授权便是这个原理，某 APP 授权微信，则由于微信的开放平台数据，某 APP 可获取其名称头像等数据，进而采用这些数据在系统中流转。

双向打通指的是两个不同的系统，数据相通，多用于合作模式的系统，情况不多。例如 A 与 B 两个系统，部分数据是公用的，那么，在 A 系统中修改了个人信息，在 B 系统中的个人信息也随之修改，共用的是同一套数据。

这当中的核心为数据流通，一个用户在多个系统内，局部数据流通（单向或双向），数据总量不变。



3.1.4 核心业务表

3.1.4.1 用户基础信息表

```
CREATE TABLE `user_userinfo` (  
  `userid` varchar(6) NOT NULL DEFAULT '-' COMMENT '用户 ID',  
  `username` varchar(20) NOT NULL DEFAULT '-' COMMENT '用户名称',  
  `userpassword` varchar(100) NOT NULL DEFAULT '-' COMMENT '密码',  
  `sex` int(11) NOT NULL DEFAULT '0' COMMENT '性别',  
  `usermoney` int(11) NOT NULL DEFAULT '0' COMMENT '账号余额',  
  `frozenmoney` int(11) NOT NULL DEFAULT '0' COMMENT '近一个月的花费的总的金额',  
  `addressid` varchar(20) NOT NULL DEFAULT '-' COMMENT '用户地址 ID, 0 表示没有获取地址',  
  `regtime` varchar(20) NOT NULL DEFAULT '-' COMMENT '注册时间',  
  `lastlogin` varchar(20) NOT NULL DEFAULT '-' COMMENT '最后登录时间',  
  `lasttime` date NOT NULL COMMENT '系统下载最后时间'  
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COMMENT='用户表';
```

3.2 商品模块

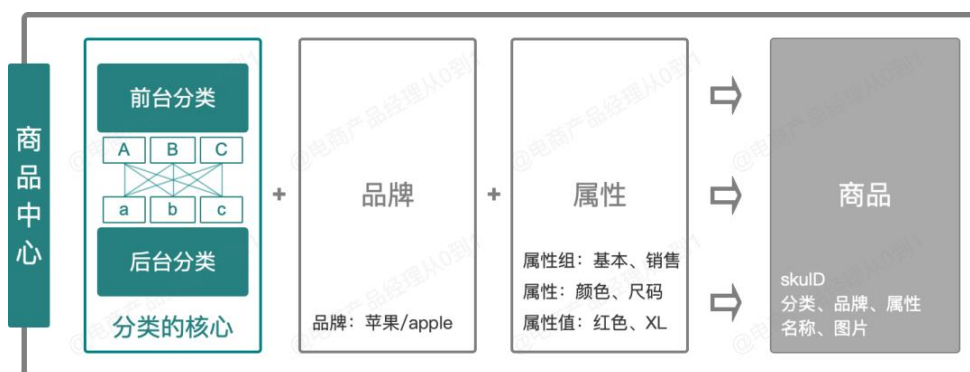
3.2.1 分类管理

电商平台其实可以看成是实体超市在虚拟网站上的映射，在超市的购物场景中，为了让消费者快速找到想要购买的商品，超市往往会将商品进行分门别类。

不同的区域放置不同的品类，每个区域用导购牌标识出来，不同的货架陈列不同的商品，这就是超市的分类，分类可以帮助消费者更方便快捷地找到想要的商品。而在电商的 sku 量级是实体商超的千倍万倍，在电商网站查询目标商品会变得更加的困难，因此超市的分类功能就很自然的应用到电商平台中。

对于用户端：搜索和分类导航是消费者寻找商品时使用的最多的两个途径（随着推荐技术应用程度的加深，搜索和分类的权重有一定程度的下降），分类导航可以帮助用户快速定位到目标商品，分类导航功能的实现依赖的就是商品的分类管理；

对于运营端：海量的商品管理对于平台和商家而言也有很大的成本，运营要快速定位到某一个商品除了搜索关键词外，分类管理也是重要的手段，分类可以提高管理商品的效率；



分类管理的核心设计思想是：前后台分离，多对多映射。

(1) 前后台分离

指前台分类和后台分类设计成两套分类体系，后台分类主要面向平台管理，其主要逻辑是结构化的管理逻辑（iPhone13∈手机分类，卫裤∈服装分类）；前台分类主要面向用户端使用，为运营提供运营空间，为用户购物提供便捷，其主要逻辑是运营和用户体验（如卫裤在前台的分类可以是”冬季新品“，主要考虑的是运营效果以及用户是否更好的找到商品）。

(2) 多对多映射

前台分类必须以后台分类作为分类基础，在创建前台分类时，必须要先选择后台分类；前台分类和后台分类之间是多对多的映射关系，即一个前台分类可选多个后台分类，不同前台分类可选同一个后台分类。

3.2.2 品牌管理

品牌管理主要包含两部分功能，即品牌的创建和品牌的品牌管理。

(1) 品牌创建

品牌的创建场景主要有两种：一种是平台侧主动创建，初期一般是按照各个品类直接初始化到系统中去；另一种平台型电商比较常见的申请方式，即由入驻到平台的三方商家申请品牌（如阿里京东平台中，如果新入驻的商家是自有新品牌或者未被平台收录，则都需要先走品牌申请流程，如早期的淘品牌韩都衣舍等），平台审核通过后落入品牌库。

(2) 品牌管理

品牌管理就是品牌自身的查询、修改、停用等功能，即基础的增查改，需要注意的是品牌在修改或者停用时，也要校验品牌下没有关联的数据（商品、分类等），否则不能操作修改或者停用（部分信息的修改如果不影响其他数据的展示，则可以修改）。

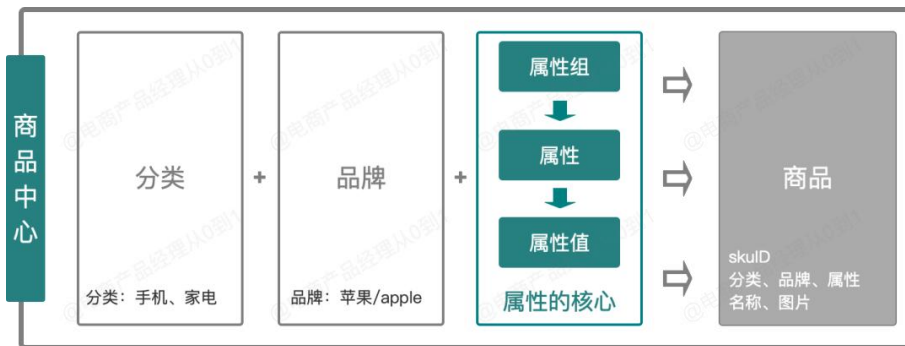
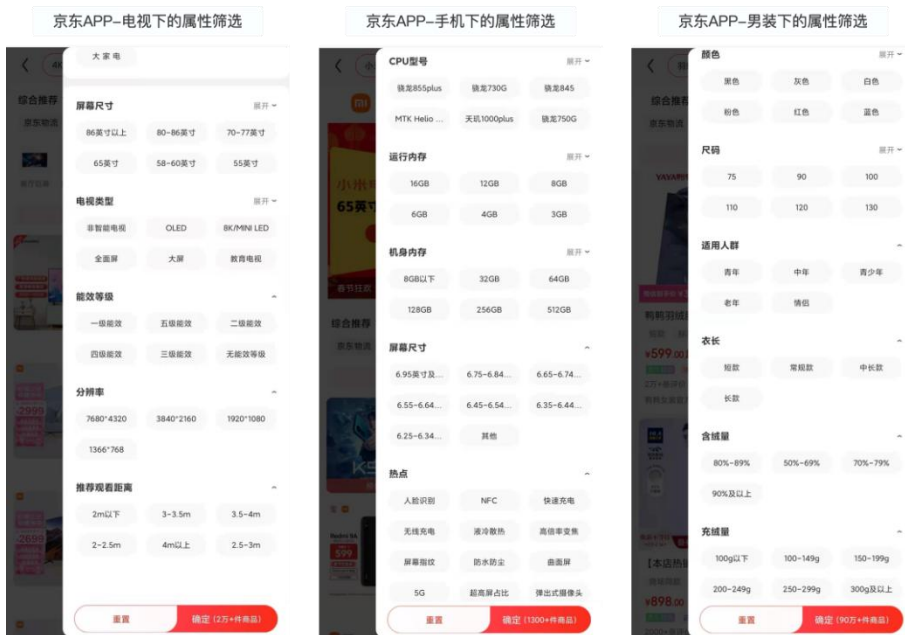
3.2.3 属性管理

属性是什么？——属性指商品所特有的性质，如颜色属性、尺码属性、规格

属性等。

属性的作用？——属性也是将商品进行分类和聚合的必要且重要的方法，类目和品牌只能在一定程度上将海量的商品进行分类，比如用户在京东购买“家电”，可以通过类目（如：电视、空调、冰洗...）、品牌（如：格力、小米...）来查询想要的产品（如：小米电视），但是当用户想进一步快速定位到最准确的意向结果（如小米电视 55 寸 4K 版），类目和品牌就显得不够用了，因此如果想实现更细颗粒度的划分，只能通过属性的方式来实现。

一个商品所拥有的所有性质，都是这个商品的属性，如：屏幕尺寸、分辨率、能效等级等。而不同类目下的商品，其拥有的属性又不一样（比如服装没有屏幕尺寸等属性，手机没有尺码属性），且随着品类的丰富，属性的数据也随之变得非常庞大，为了更好地管理不同类型的商品的属性，使其在前台应用时可以有清晰简单的逻辑，属性管理的意义就显得非常重要了。



属性从结构上可拆分成 3 块，分别是：属性组>属性>属性值

- 属性组：将大量属性分类，便于管理，如手机的“外观属性”、“性能属性”等；

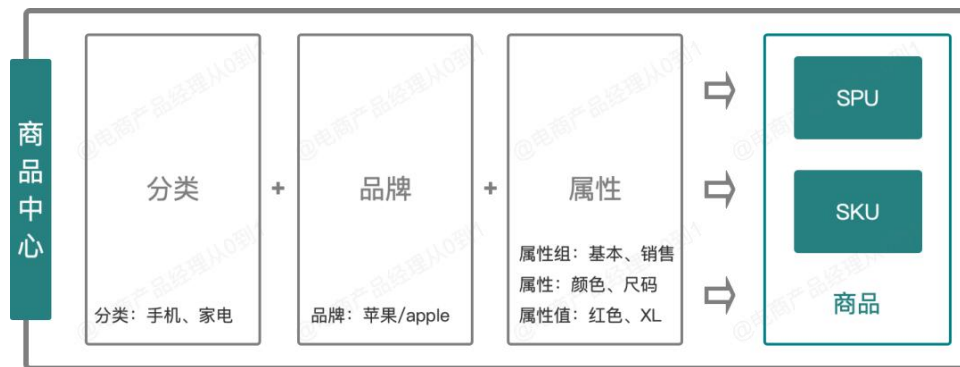
- 属性：即属性名，指具体的属性，如颜色、版本、运行内存；
- 属性值：指属性的具体值，如颜色属性的属性值有：红色、黄色、绿；

以上 3 个模块就组成了属性管理，有时候为了在业务层面沟通方便或者口径统一，又或者其他的应用场景，还会引入属性分类的设计，在属性组之上将属性在进行分类（相对抽象的分类），一般分为基础属性、销售属性、仓储属性、物流属性、扩展属性（关于属性分类，没有标准设计，结合自身的平台特性划分即可，也可不设计属性分类）。

3.2.4 商品管理

商品管理模块主要负责创建商品、管理商品，为其他系统提供商品能力，是商品中心最核心的模块，同时商品管理也依赖分类、品牌、属性三个模块。

以创建商品为例，创建商品时需要填写的字段中，最重要的就是选择分类、品牌、属性，可以说商品是建立在分类模块、品牌模块和属性模块之上的模块。



两个概念——SPU&SKU

在具体说明商品模块之前，有必要先说明两个名词“SPU 和 SKU”，不止电商平台，大部分涉及交易的系统都可能涉及 SPU 和 SKU 相，相信大部分人都接触过这两个名称。以下是百度百科对 SPU 和 SKU 的解释：

SPU(Standard Product Unit)——标准化产品单元。是商品信息聚合的最小单位，是一组可复用、易检索的标准化信息的集合，该集合描述了一个产品的特性。通俗点讲，属性值、特性相同的商品就可以称为一个 SPU

SKU(stock keeping unit)——SKU 即库存进出计量的单位，可以是以件、盒、托盘等为单位；SKU 是一个可以售卖的商品。

通俗地讲，SPU 定位一个产品，SKU 定位为商品。例如手机这个类目下，iPhone 13 就是一个 SPU，iPhone 13 256G 黑色就是一个 SKU（京东的 SKU 还会加上公开版等属性）一般情况下，基础属性可以看成是 spu 的属性，如手机的屏幕尺寸、CPU 型号、像素等基础属性；销售属性以及仓储物流属性存储在 sku 上，如颜色、版本等销售属性。

3.2.5 商品创建流程

创建商品功能从界面上看比较简单，就是填写一堆字段，然而根据平台的大小、复杂程度、垂直专业程度等因素的不同，可以抽象成两种设计方案：

方案 1：同时创建多个 sku，同步生成关联 spu，整体方案是直接创建 sku，维护多个不同的属性即意味着生产维护多个 sku；该方案适用于大部分 2C 类综合型电商平台（如淘宝和京东 POP 创建商品就是这种方式）

方案 2：先创建 spu，基于 spu 创建 sku，整体方案是由平台的主数据团队负责维护 spu，由商家（包括自营和 POP）运营（或采销）基于 spu 维护 sku，创建 sku 时，先选择 spu（spu 中的基础属性已经由数据团队维护完毕），在 spu 基础上维护销售属性、物流属性等，然后生成 sku；该方案适用于专业性非常强的垂直 B2B 行业，如汽车、医药等。

产生两种方向的原因是：垂直 B2B 平台的商家（行业传统，商家老板年纪较大）运营能力有限，维护商品属性的错误率远高于 2C 平台的商家，而平台对于商品结构化的管控要求较高，为了避免同一个商品被不同的商家维护成多个不同的属性（如汽车轮胎的胎面宽、尺寸等属性），平台往往选择由专门的数据团队维护商品的基础属性，即维护 spu。且 B2B 垂直电商的品类少、sku 量级相对较小，品类标准化程度高，由平台统一维护的可行性高。

而综合电商成千上万个品类，靠平台的数据团队统一维护不太现实，或者像服装这样的非标品类，对于商品的结构化管理诉求低，因此综合平台（京东、淘宝）与垂直平台的设计方向就出现了差异。

实际情况下，即使是综合平台（如京东），不同的品类也会出现不同的设计方法，有的品类垂直纵深，也是采用的平台维护 spu，商家创建 sku 的方式。

（1）创建 sku 设计方案，以方向 1 说明

这种方法比较常见，见过淘宝、京东等主流电商后台的同学应该有一定的印象，逻辑就是直接创建 sku，生成 sku 的同时，生成相应的 spu，如下图的淘宝商家后台：

商品优化

7探索

待整改 0

待优化 7

基础信息

销售信息

售后服务

物流信息

支付信息

图文描述

了解全店商品? 全店诊断报告

飞机种类

请输入自定义值

+ 添加

☒ 固定翼

☒ 直升机

☒ 其它类型

开始排序

批量填充

颜色分类	飞机种类	颜色分类图	价格(元)	数量(件)	商家编码	商品条形码
橙绿色	固定翼	选择图片	8000.00	0		
	直升机	选择图片	90000.00	0		
	其它类型	选择图片	10000.00	20		
黑色	固定翼	选择图片	8000.00	0		
	直升机	选择图片	10000.00	0		
	其它类型	选择图片	80000.00	18		
白色	固定翼	选择图片	8000.00	0		
	直升机	选择图片		0		
	其它类型	选择图片	70000.00	188		

一口价

100000.00

元

本类目常规价格范围是0.01元-100000.00元之间, 请规范价格行为

提交宝贝信息

基础属性相同的多个sku对应同一个spu

颜色属性+飞行种类属性确定一个sku

价格和库存存储在sku上(即使批量填充一样的价格和库存, 在数据底层也是按照sku的维度存储)

从图中可以看出，基础属性相同的多个 sku 对应同一个 spu，颜色属性+飞行种类属性确定一个 sku，价格和库存存储在 sku 上（即使批量填充一样的价格和库存，在数据底层也是按照 sku 的维度存储；而在前台则以 spu 的维度展示（体验更好），切换销售属性即是在同一个 spu 维度下，切换不同的 sku：

下面为天猫 PC 端-iPhone13 的商品详情页，可以看出，iPhone13+黑色+128G 和 iPhone13+粉色+128G 是两个独立的 SKU（skuid 不同），两个 sku 对应的 SPU 是同一个（spuid 相同）。

天猫首页

购物车

SPU

SKU

我的天猫

商家支持

Apple Store 官方旗舰店

Mac

iPhone

iPad

Watch

AirPods

HomePod mini

iPod touch

AirTag

教育专区

配件

Apple/苹果 iPhone 13

价格

¥5999.00

运费

上海 至 北京 海运区 快速: 0.00

付款后3天内发货

累计评价 161795

送天猫积分 599

版本

iPhone 13 mini

iPhone 13

网络类型

无SIM卡的版

机身颜色

粉色

蓝色

午夜色

星光色

红色

存储容量

128GB

256GB

512GB

数量

1

花呗分期

第12期免息, 可免449.93元, 每期499.92元(每日16.66元)

什么是花呗分期

¥1999.66x3期

¥999.83x6期

¥499.91x12期

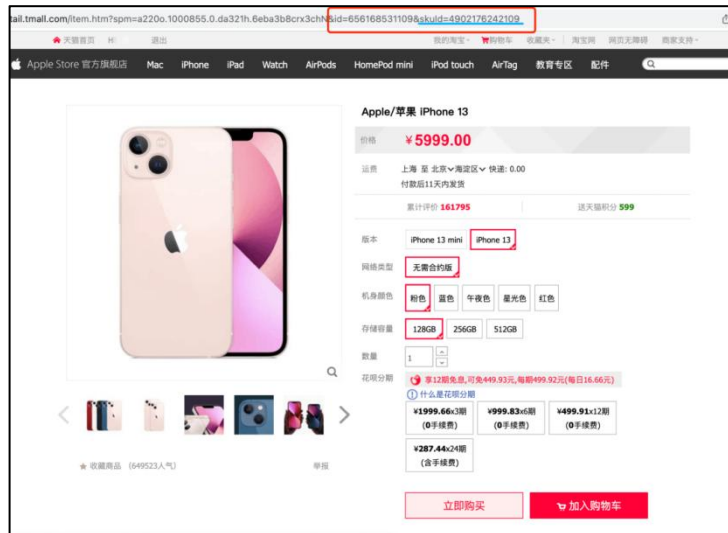
¥287.44x4期

立即购买

加入购物车

收藏商品 (649523人气)

举报



以上就是方案 1 的核心设计逻辑：即创建商品的时候，维护不同的销售属性就是维护了多个不同的 sku，但最终同步生成了同一个 spu。

了解了设计逻辑以后，对于功能设计就非常简单的了，不论是自营后台还是三方 pop 后台，创建商品的界面大同小异，可按照流程大约分为 7 个步骤：



第一步：选择类目（商品分类），注意选择类目时，调用的是后台分类（这里就用上了分类管理的服务了）：

当选择分类后，系统自动加载该分类下对应的品牌数据（有的类目则是在填写商品信息时选择品牌）。后面的步骤比较简单，只是填写对应的字段内容即可，就不过多展开了，直接附上一张后台截图供参考：

The screenshot shows a product management interface. The left sidebar contains navigation links: 待上架 0, 待优化 8, 基础信息, 销售信息, 售后服务, 物流信息, 支付信息, and 图文描述. The main content area is divided into two tabs: 基础信息 (Basic Information) and 销售信息 (Sales Information).

基础信息 (Basic Information):

- 宝贝类型 (Product Type):** 全新 (New) / 二手 (Second-hand)
- 宝贝标题 (Product Title):** 智能飞行器 (Smart Flying Robot)
- 导购标题 (Sales Title):** (0/30) 展示效果: 详细和描述未填写查看商品详情工具: 商品名称工具
- 类目属性 (Category Attributes):** 4/4 填写一项即可发布上架
- 品牌 (Brand):** BBS/宝贝星
- 型号 (Model):** 24X01024234
- 玩具类型 (Toy Type):** 电动玩具
- 适用性别 (Applicable Gender):** 中性

销售信息 (Sales Information):

- 颜色分类 (Color Classification):** 军绿色 (Military Green), 灰色 (Grey), 白色 (White)
- 宝贝销售规格 (Product Sales Specifications):** 在标题框中输入或选择内容可以进行筛选和批量填充

颜色分类	飞机种类	颜色分类图	价格(元)	数量(件)	商家编码	商品条形码
军绿色	固定翼	选择图片	8000.00	0		
	直升机	选择图片	90000.00	0		
	其它类型	选择图片	10000.00	20		
灰色	固定翼	选择图片	8000.00	0		
	直升机	选择图片	10000.00	0		
	其它类型	选择图片	80000.00	18		

以上就是创建商品的设计方案，方案 2 与方案 1 的区别是：创建 sku 时，需要基于 spu 创建 sku，即 spu 由内部数据运维团队负责创建、维护，运营以及三方商家创建 sku 时需要先选择 spu（如果没有则需要提交申请），具体界面与方案 1 区别不大。

3.2.6 核心业务表

3.2.6.1 SKU 信息表

```
CREATE TABLE `product_sku` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'skuId',  
  `spu_id` bigint(20) DEFAULT NULL COMMENT 'spuId',  
  `name` varchar(255) DEFAULT NULL COMMENT 'sku 名称',
```

```

`category_id` bigint(20) DEFAULT NULL COMMENT '所属分类 id',
`brand_id` bigint(20) DEFAULT NULL COMMENT '品牌 id',
`default_image` varchar(255) DEFAULT NULL COMMENT '默认图片',
`title` varchar(255) DEFAULT NULL COMMENT '标题',
`subtitle` varchar(2000) DEFAULT NULL COMMENT '副标题',
`price` decimal(18,4) DEFAULT NULL COMMENT '价格',
`weight` int(20) DEFAULT NULL COMMENT '重量（克）',
PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=11 DEFAULT CHARSET=utf8 COMMENT='sku 信息';

```

3.2.6.2 SKU 属性表

```

CREATE TABLE `product_sku_attr_value` (
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'id',
  `sku_id` bigint(20) DEFAULT NULL COMMENT 'sku_id',
  `attr_id` bigint(20) DEFAULT NULL COMMENT 'attr_id',
  `attr_name` varchar(200) DEFAULT NULL COMMENT '销售属性名',
  `attr_value` varchar(200) DEFAULT NULL COMMENT '销售属性值',
  `sort` int(11) DEFAULT NULL COMMENT '顺序',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=31 DEFAULT CHARSET=utf8 COMMENT='sku 销售属性&值';

```

3.2.6.3 SPU 信息表

```

CREATE TABLE `product_spu` (
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '商品 id',
  `name` varchar(200) DEFAULT NULL COMMENT '商品名称',
  `category_id` bigint(20) DEFAULT NULL COMMENT '所属分类 id',
  `brand_id` bigint(20) DEFAULT NULL COMMENT '品牌 id',
  `publish_status` tinyint(4) DEFAULT NULL COMMENT '上架状态[0 - 下架, 1 - 上架]',
  `create_time` datetime DEFAULT NULL COMMENT '创建时间',
  `update_time` datetime DEFAULT NULL COMMENT '更新时间',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=12 DEFAULT CHARSET=utf8 COMMENT='spu 信息';

```

3.2.6.4 SPU 属性表

```

CREATE TABLE `product_spu_attr_value` (
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'id',
  `spu_id` bigint(20) DEFAULT NULL COMMENT '商品 id',
  `attr_id` bigint(20) DEFAULT NULL COMMENT '属性 id',
  `attr_name` varchar(200) DEFAULT NULL COMMENT '属性名',
  `attr_value` varchar(200) DEFAULT NULL COMMENT '属性值',
  `sort` int(11) DEFAULT NULL COMMENT '顺序',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=37 DEFAULT CHARSET=utf8 COMMENT='spu 属性值';

```

3.2.6.5 商品类别表

```
CREATE TABLE `product_category` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '分类 id',  
  `name` char(50) DEFAULT NULL COMMENT '分类名称',  
  `parent_id` bigint(20) DEFAULT NULL COMMENT '父分类 id',  
  `status` tinyint(4) DEFAULT NULL COMMENT '是否显示[0-不显示, 1 显示]',  
  `sort` int(11) DEFAULT NULL COMMENT '排序',  
  `icon` char(255) DEFAULT NULL COMMENT '图标地址',  
  `unit` char(50) DEFAULT NULL COMMENT '计量单位',  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB AUTO_INCREMENT=1433 DEFAULT CHARSET=utf8mb4 COMMENT='商品三级分类';
```

3.2.6.6 商品品牌表

```
CREATE TABLE `product_brand` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '品牌 id',  
  `name` char(50) DEFAULT NULL COMMENT '品牌名',  
  `logo` varchar(2000) DEFAULT NULL COMMENT '品牌 logo',  
  `status` tinyint(4) DEFAULT NULL COMMENT '显示状态[0-不显示; 1-显示]',  
  `first_letter` char(1) DEFAULT NULL COMMENT '检索首字母',  
  `sort` int(11) DEFAULT NULL COMMENT '排序',  
  `remark` longtext COMMENT '备注',  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8 COMMENT='品牌';
```

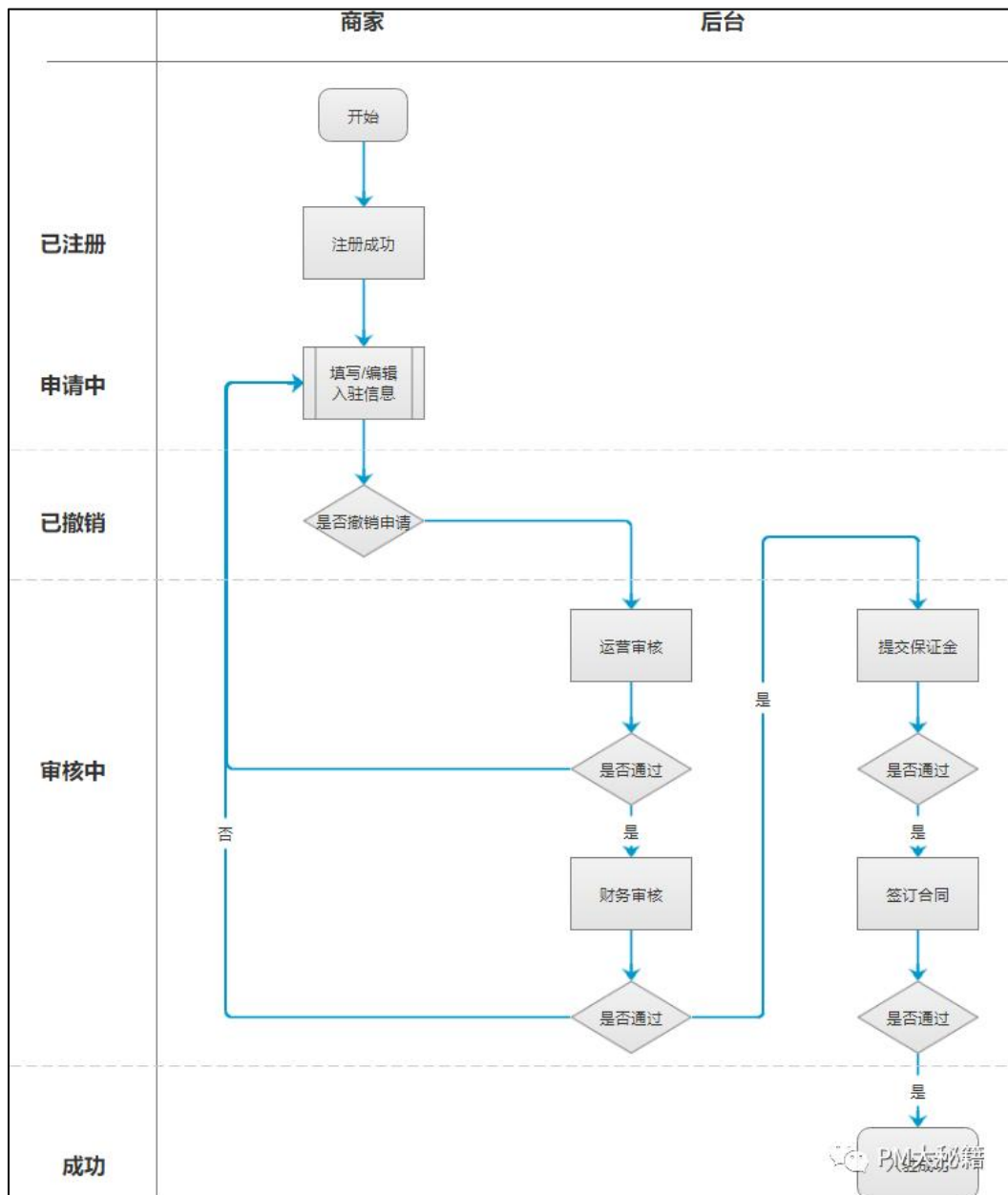
3.3 商家模块

3.3.1 商家入驻流程详解

接下来我们按照商家入驻状态流向和退店状态流向，对商家店铺的整个生命周期过程的相关事项展开了讲解。

全球互联网电商市场依旧欣欣向荣，移动购物还是那么的势不可挡，有实力、有资金的公司搭建自己的自营电商平台，而其他公司选择值得信赖的第三方电商平台入驻，比如某东、某宝、某多多。

商家按其规则自主入驻平台，平台将店铺经营权、定价权交还给商家，让商家直接面对客户，由市场对商家进行优胜劣汰。平台负责制定规则，对商家行为进行监督和奖惩，促进市场繁荣发展。最后，平台按与商家协商好的佣金方式进行相互结算。



商家入驻状态流转分为已注册、申请中、已撤销、审核中、审核失败、成功等。

1. 已注册状态

商家成功注册平台账号，但是未进行任何入驻信息操作；

2. 申请中

商家注册平台账号后，进行入驻信息的填写，也就是商家成功完成第一步操作后的状态；还有一种情况，已撤销状态的商家，又进行入驻操作。

3. 已撤销

商家入驻信息填写过程中，分为好几个步骤，当商家不想继续入驻时，可执行【撤销商户入驻】操作，那此时的状态应为已撤销，要区分已注册状态。

4. 审核中

该状态分为运营审核、财务审核、保证金审核、合同审核。

- 运营审核：主要负责检查商家添加填写的信息是否正确，比如企业资质、店铺信息、品牌信息是否准确，还有各种文件照片是否按照平台要求进行上传。
- 财务审核：负责检查商家提供的对公银行账号是否准确，具体操作是财务人员想向商家银行账号一笔小额款项，一般是0.01~0.2元(具体按财务要求来)。第二天查看银行流水，若款项没有退回的话，则就默认款项已经打到对方账户，后台进行同意操作；若款项被退回公司账号，则说明商家银行账号有问题，后台应拒绝通过，并附上原因让商家自行修改。
- 保证金审核：商家按照保证金规则计算出来的金额，向平台进行汇款操作，并提交汇款证明图片。财务人员过1~2天，查看银行流水是否该笔保证金款项到账。
- 合同审核：签订合同有两种方式，一种是使用第三方电子合同签名平台进行操作，另一种是平台提供电子版合同，让商家填写成功敲公章，并以图片的形式进行上传。

5. 审核失败

当审核被拒绝的时候，应当注明拒绝原因，商家可以按要求修改；

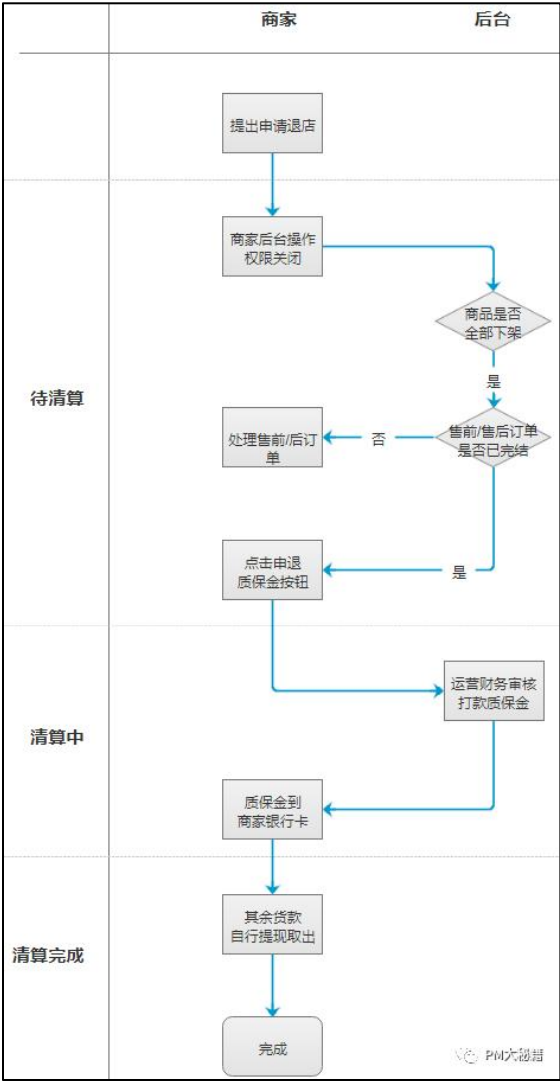
6. 成功

商家和平台合同审核通过后，即商家入驻平台成功，此时店铺处于正常营业中。

商家入驻步骤大体可以分为七个步骤：

- 入驻须知环节；
- 企业资质环节；
- 店铺信息环节；
- 品牌信息环节；
- 经营概况环节；
- 平台审核环节；
- 签订合同环节；

3.3.2 商家退店流程详解



商家退店状态流转分为待清算、清算中、清算完成。

当商家因自身等不可抗拒原因造成不想在平台继续运营店铺,那么商家可以自行发起店铺退店操作,页面需要弹窗让商家进行二次确认。再次确认后,店铺状态由正常运营(成功)流转成待清算状态,平台客服接到商家退店事项,需要了解商家具体原因,以改善后续的平台服务及提升技术服务。

还有一些情况会导致店铺被动退店操作,应事先通知到商家:

- 1) 若店铺考核不达标进入清算,比如:入驻 90 个自然日内在架商品小于一定数量且无成交额,特殊品类除外。
- 2) 商家违反平台规章制度,且情节严重,需提前告知商家并进行店铺清算过程中。比如:店铺存在假货等违规。
- 3) 达到入驻年限未续签清退。
- 4) 其他平台规定清算规则

1. 待清算

店铺处于待清算过程中，以下权限将关闭：

- 1)商家后台无法发布商品、编辑商品、上架商品；
- 2)店铺商品均下架操作，搜索无法检到该店商品；下架操作可有商家执行，也可有程序自动执行。
- 3)商家后台若是对接第三方仓储 api，相关操作应将关闭；

2. 清算中

店铺清算中状态，必须同时满足以下情况：

- 1) 所有待发货订单要操作发货，或者与用户协商处理；
- 2) 所有售后订单必须要完结；
- 3) 所有上架商品必须下架处理；

然后满足后，店铺后台显示申退保证金的按钮，商家点击后，显示清算中状态，平台财务进行审核确认，向商家进行打款，退回保证金操作。

tips：上面提到商家点击“显示申退保证金的按钮”，这一环节也可以通过程序自动来实现：当店铺满足清算中条件时，无需操作，自动显示清算中状态，然后平台进行审核确认操作。

3. 清算完成 (店铺关闭)

这里指的清算仅仅指清算保证金、不涉及到清算商品结算的货款（每个平台规定不一样，无需纠结）。当财务查看银行流水，确认款项已打到商家银行账号，那么保证金清算流程算是完成了。

还有一点要考虑，商家退店成功后，重新入驻需要间隔多久，要不要设置间隔时间按平台实际情况做决定。

3.3.3 核心业务表

3.3.3.1 店铺信息表

```
CREATE TABLE `shop_base_info` (  
  `shop_id` varchar(20) NOT NULL DEFAULT '-' COMMENT '店铺 ID',  
  `shop_name` varchar(20) NOT NULL DEFAULT '-' COMMENT '店铺名称',  
  `shop_status` int(11) NOT NULL DEFAULT '0' COMMENT '营业状态(0 在线营业 1 暂时歇业 2 停业)',  
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COMMENT='店铺表';
```

3.4 交易模块

3.4.1 交易业务流程

整个订单的产生到结束的，主要有以下 3 个流程：这里没加退款。

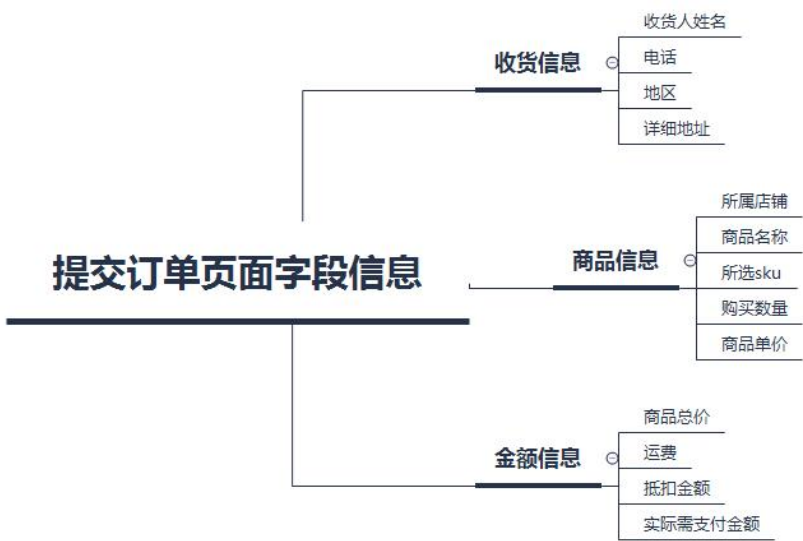
主流程:

操作人员:



3.4.2 提交订单流程

在该页面需要提交的字段信息如下图所示：



用户一系列操作对各项数据的影响：

1. 选择收货地址

用户选择的收货地址中的地区会涉及到运费的计算（每个商家有自己的 n 套运费计算模板，添加商品时会选择对应的运费模板，不同地区的运费计算会根据商品重量或件数进行计算，相应的介绍在商家系统一文中阐述）

2. 选择是否抵扣

我们平台业务上暂未涉及优惠券，但会通过一些渠道获得相应的金币，而这些金币是可以用来抵扣订单金额的，但是抵扣有多种方式。

3. 选择是否朋友代付

代付是一套比较复杂的业务场景，会涉及到微信的分享，获取微信的 openid 之类的流程，具体在另外的文章中我会写出来。

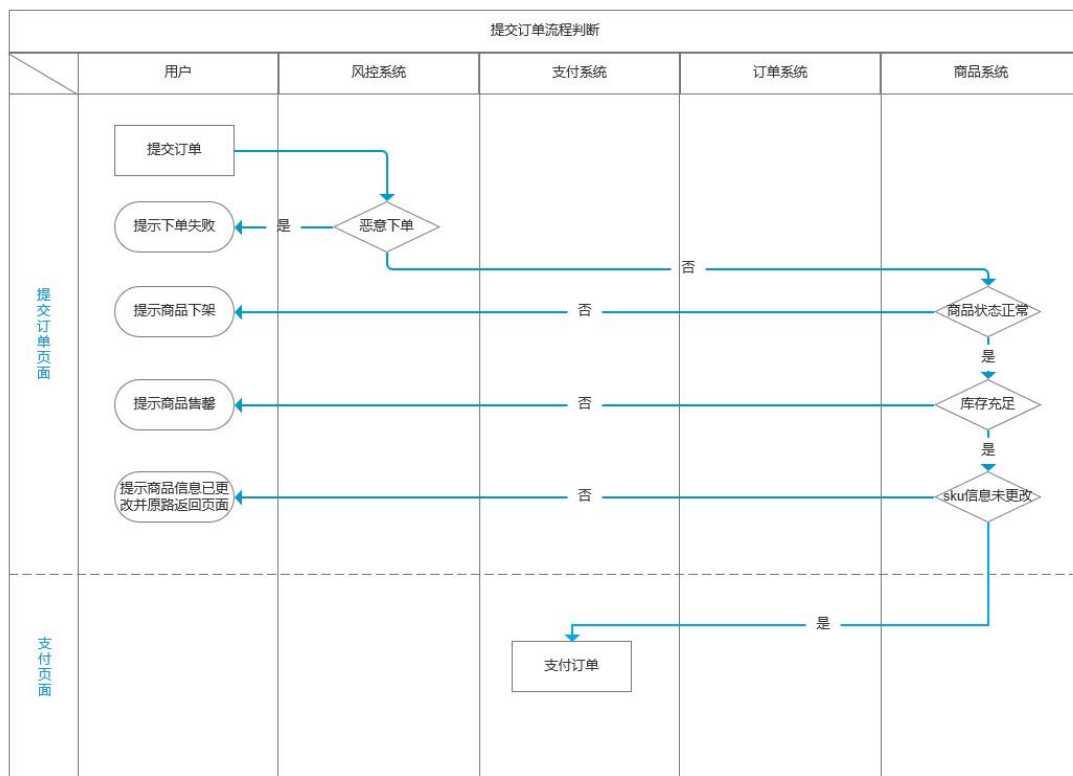
4. 选择是否匿名购买

当用户选择是后，用户对该商品的评价可以根据一定的显示规则来隐藏昵称

5. 填写订单备注

需要注意的一点是，订单备注是跟着商家走的，也就是跟着订单走的（因为会涉及到以商家为维度拆单），所以需要每个商家有一个输入框。

判断流程：点击提交订单按钮后，此时后台会进行一系列的判断。



1. 风控验证

判断此次下单是否恶意可以通过一些维度，比如该账号或 ip 是否一段时间内重复下单多次，是否有过恶意下单记录等，这里不做展开。

2. 验证商品状态

如果当用户停留在提交订单页面期间，商家下架了该商品，处于已下架状态，或者用户选择的该 sku 已售罄；所以需要在提交订单页面验证一次。

3. 验证 sku 库存

比如用户在选择 sku 时，购买数量为 2 件，库存也剩 2 件，但此时有另外的用户抢先购买了 1 件；所以需要在提交订单页面判断一次库存是否大于该用户的购买数量（因为在提交订单后就会锁定库存，所以并不会出现在支付页面显示商品库存不足的情况）。

4. 验证 sku 信息是否更改

当用户停留在提交订单页面期间，商家更改了商品的 sku 信息（比如直接删掉或修改了金额）并成功上架，此时应提示用户“商品信息已更改，请重新下单”，并原路返回页面。

3.4.3 支付订单流程

当用户把订单提交后此时后台会有两步操作：

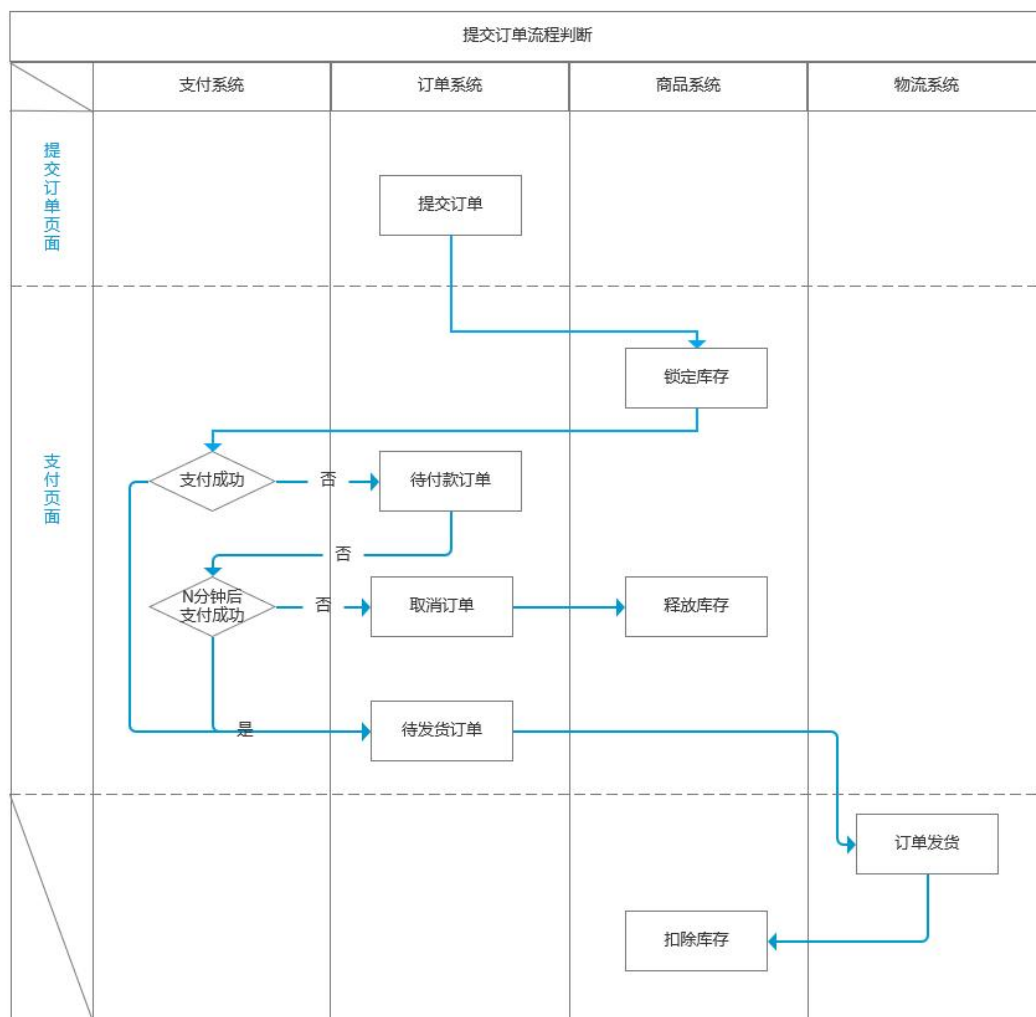
1) 拆单

由购物车进入提交订单页面时可能有多商家多商品的情况，一旦提交了订单就会涉及到拆单（不管是否成功支付），一般来说最简单的是按商家拆，拆完后分别流转至相应的商家后台，用户在客户端的订单列表也会看到多个子订单；如果业务场景要求的话可以再按仓库等维度拆，这里不做展开；

2) 生成账单

生成账单的目的是为了记录该笔母订单的金额，如商品金额、抵扣总金额、各商品分别抵扣金额、用户需支付金额等，用户将要支付的是母订单的账单，当该笔账单已完成，则各子订单状态跳转为待发货；

注意，如果用户在支付页面退出，此时账单也会随着商家拆分成各子账单，因为用户可以在订单列表里分别对拆分后的子订单进行支付。



下面是支付页面的字段和各项判断流程：

一、支付方式

1. 支付宝/微信等三方支付

由开发同学对接好三方支付平台的接口即可，这里不做展开。

2. 余额支付

用户在平台会通过一定的方式获取余额（非充值，也非用来抵扣的金币，是一种支付方式）

二、锁定库存：两种方案

1) 提交订单即锁库存

这样做的优点是用户的体验较好，我提交了订单这个商品就是我的了，我可以慢慢付款；

缺点是可能会导致真正有购买需求的用户无法购买，比如甲用户先提交订单锁定了库存他还在考虑中，不一定会买，但是乙用户想立即购买却发现没货了（也不排除有人恶意下单锁定库存）

所以待付款订单一般都会有剩余支付时间，比如 30 分钟，到了时间自动取消订单并释放库存，或者在添加商品的 sku 时设置单人限购数量，这样一个账号只能在某一段时间内购买 n 次，同时技术上也可以做限制，同一 ip 只能购买 n 次

2) 支付成功才锁库存

这样做的优点是可以筛掉恶意下单的情况；缺点是用户的体验会差一些，可能付款慢一点就会失去购买的机会。

4. 是否支付成功

支付成功即生成待发货订单，立即锁定库存。

支付失败则还是为待付款订单，然后开始倒计时；一般平台商品库存充足倒计时可长一点，对用户会友好一点，库存不怎么充足或者平台上入驻的小商家居多，平台无法控制商家的库存或者下架之类的操作；

如果也考虑到时间给用户带来的紧迫感的话，时间可以短一些；时间一到订单状态就应变成已关闭状态，用户无法支付，同时释放库存。

订单成功支付后就需要商家处理订单了，同时用户也可以进行一些操作。

3.4.4 处理订单流程

首先，用户选好商品之后会提交订单，经过一系列判断之后，用户成功提交了订单，此时订单状态为待付款，并且会写入数据库（多商家的订单就按商家拆单后写入）；然后用户就开始支付订单，经过一系列判断成功支付后，这时就需要商家处理订单，进行发货等操作，同时用户也可以进行一些操作。

我们先来看看完整订单字段：

1. 订单信息

1) 订单编号

2) 订单类型

可以分为“自己购买/好友代付/任务活动”等。

标记订单类型有两个目的：一是不同的订单类型在客户端和后台可能有不同的页面展示和操作流程，二是可以进行数据统计并分析；所以订单类型可以分得越细越好。

3) 订单备注

这个字段在后台订单详情里可以放在较显眼的位置，以防工作人员漏掉。

2. 商品信息

1) 商品编号

添加商品时的编号，方便查找商品（但在数据库里不是商品的唯一 ID，因为商家数量够大时会产生重复的情况，但又不能做防止重复的限制）。

2) 商品名称

商品名称是较大概率会产生重复的情况；从商家的角度来说，名称怎么取，与搜索引擎和推荐商品的匹配程度具有相当大的关联。

3) sku（商品的属性规格）

4) 购买数量

5) 商品来源

分为普通商品/活动区域一/活动区域；活动区域是一个替代名词，在我们平台会有几个不同的活动区域，在后台的营销系统里，可以直接往不同的活动区域里添加不同的商品，并设置相应的如活动价格/单人限购数量等信息；区分的作用主要是用于数据统计并分析。

3. 金额信息

商品结算价、抵扣、邮费、实际支付金额、支付方式

4. 用户信息

1) 账户信息（昵称/账号）

2) 收货人信息（收货人姓名/电话/地区/详细地址）

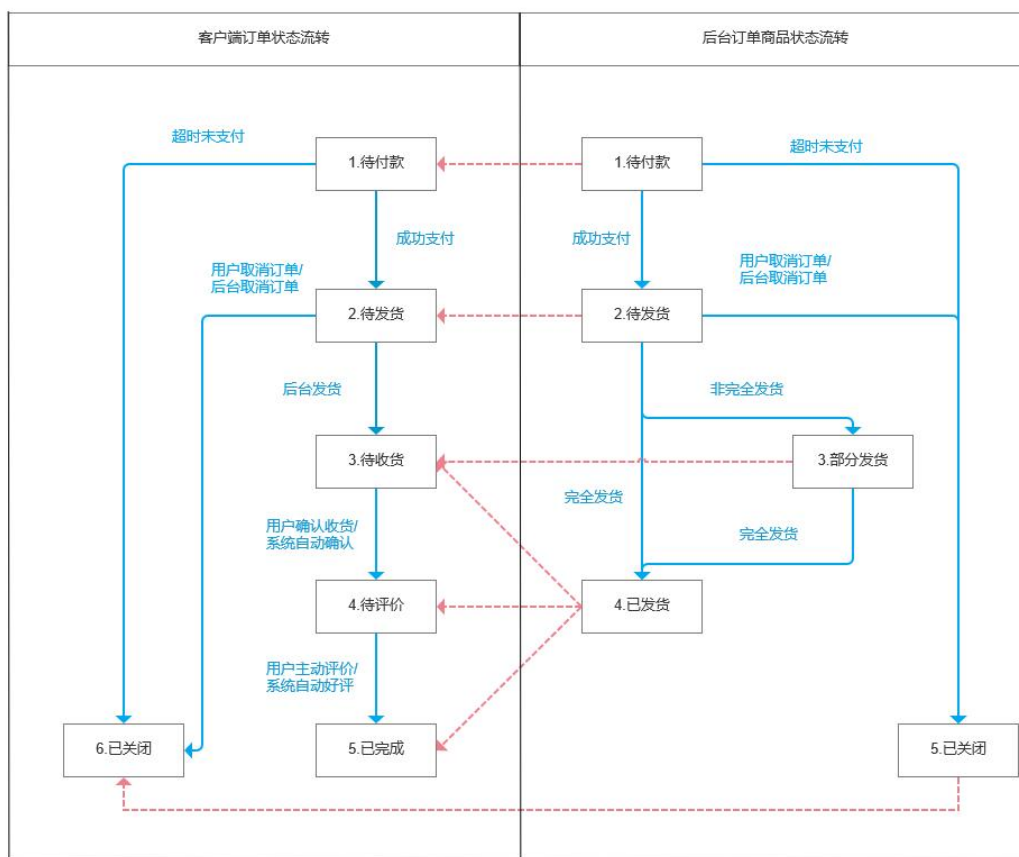
5. 时间信息

下单时间、支付时间、发货时间、确认收货时间

6. 操作信息

操作账号、操作时间、操作内容

注：红色虚线为客户端订单状态与对应的后台订单商品状态



3.4.5 退货退款流程

售后类型基本可以分为四种情况：仅退款（未收到货）、退货退款（已收到货）、换货、补寄。这里仅讨论退货退款的情况，我将从申请售后、退货退款单状态与操作、退货分支流程、涉及其他版块等 4 个维度来讲解。

1. 申请售后

首先，申请售后时需要选择申请原因、填写问题描述、上传凭证图片等三项操作，提交之后就会生成一条售后单记录，然后进入后台，待工作人员进行操作。

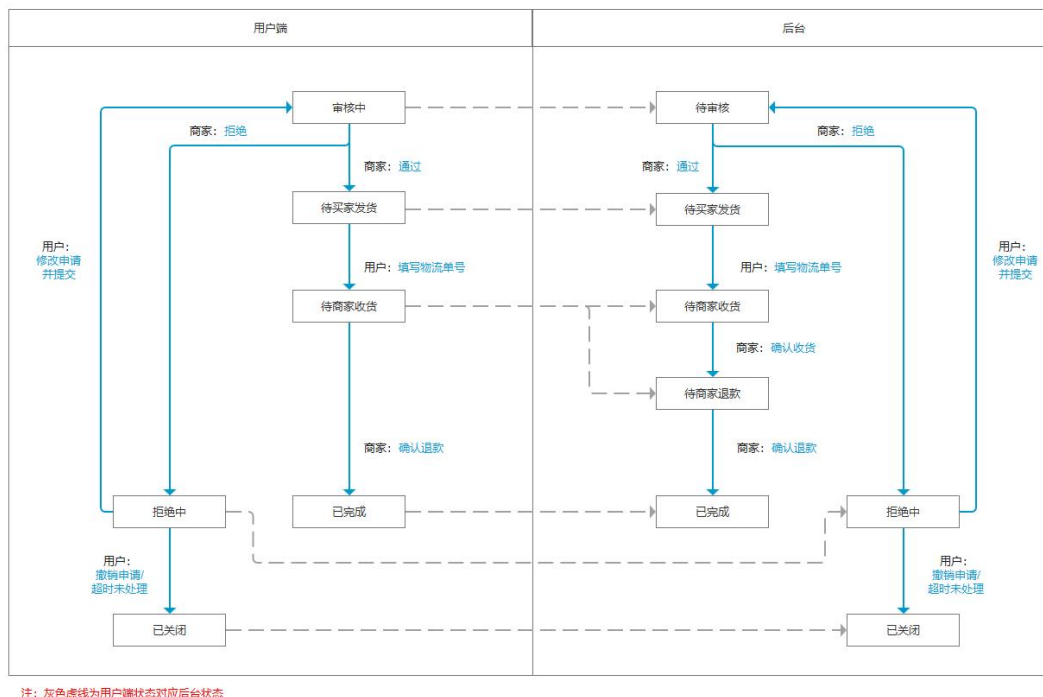
2. 退货退款单状态对应操作

(1) 用户端状态对应操作

审核中（修改申请、撤销申请）、待买家发货（填写物流单号、撤销申请）、待商家收货（无）、已完成（无）、拒绝中（修改申请、撤销申请）、已关闭（无）

(2) 后台状态对应操作

待审核（通过、拒绝）、待买家发货（无）、待商家收货（确认收货、查看物流）、待商家退款（确认退款、查看物流）、已完成（查看物流）、已拒绝（无）、已关闭（无）



如上图，用户端和后台的退货单状态除了“待商家退款”都能一一对上（不像订单那样复杂，因为一个售后单只会对应一个商品），而用户端省略“待商家退款”的目的主要是从用户考虑，一是简少用户的认知成本，二是缓解用户的焦虑。而后台为什么要有这一状态是因为对商家来说，收货是商品部干的事，退款是财务干的事，不同部门的权限不一样，当然如果要做细一点，甚至可以财务有一套审批的流程（审批、打款等操作由财务和出纳等角色操作），这里不做展开。

3.4.6 核心业务表

3.4.6.1 订单表

```
CREATE TABLE `trade_order` (
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'id',
  `user_id` bigint(20) DEFAULT NULL COMMENT 'member_id',
  `order_sn` char(32) DEFAULT NULL COMMENT '订单号',
  `coupon_id` bigint(20) DEFAULT NULL COMMENT '使用的优惠券',
  `create_time` datetime DEFAULT NULL COMMENT '创建时间',
  `username` varchar(200) DEFAULT NULL COMMENT '用户名',
  `total_amount` decimal(18,4) DEFAULT NULL COMMENT '订单总额',
  `pay_amount` decimal(18,4) DEFAULT NULL COMMENT '应付总额',
  `freight_amount` decimal(18,4) DEFAULT NULL COMMENT '运费金额',
  `promotion_amount` decimal(18,4) DEFAULT NULL COMMENT '促销优化金额（促销价、满减、阶梯价）',
  `integration_amount` decimal(18,4) DEFAULT NULL COMMENT '积分抵扣金额',
  `coupon_amount` decimal(18,4) DEFAULT NULL COMMENT '优惠券抵扣金额',
  `discount_amount` decimal(18,4) DEFAULT NULL COMMENT '后台调整订单使用的折扣金额',
```

```

`pay_type` tinyint(4) DEFAULT NULL COMMENT '支付方式【1->支付宝; 2->微信; 3->银联; 4->货到付款; 】',
`source_type` tinyint(4) DEFAULT NULL COMMENT '订单来源[0->PC 订单; 1->app 订单]',
`status` tinyint(4) DEFAULT NULL COMMENT '订单状态【0->待付款; 1->待发货; 2->已发货; 3->已完成; 4->已关闭; 5->无效订单】',
`delivery_company` varchar(64) DEFAULT NULL COMMENT '物流公司(配送方式)',
`delivery_sn` varchar(64) DEFAULT NULL COMMENT '物流单号',
`auto_confirm_day` int(11) DEFAULT NULL COMMENT '自动确认时间(天)',
`integration` int(11) DEFAULT NULL COMMENT '可以获得的积分',
`growth` int(11) DEFAULT NULL COMMENT '可以获得的成长值',
`bill_type` tinyint(4) DEFAULT NULL COMMENT '发票类型[0->不开发票; 1->电子发票; 2->纸质发票]',
`bill_header` varchar(255) DEFAULT NULL COMMENT '发票抬头',
`bill_content` varchar(255) DEFAULT NULL COMMENT '发票内容',
`bill_receiver_phone` varchar(32) DEFAULT NULL COMMENT '收票人电话',
`bill_receiver_email` varchar(64) DEFAULT NULL COMMENT '收票人邮箱',
`receiver_name` varchar(100) DEFAULT NULL COMMENT '收货人姓名',
`receiver_phone` varchar(32) DEFAULT NULL COMMENT '收货人电话',
`receiver_post_code` varchar(32) DEFAULT NULL COMMENT '收货人邮编',
`receiver_province` varchar(32) DEFAULT NULL COMMENT '省份/直辖市',
`receiver_city` varchar(32) DEFAULT NULL COMMENT '城市',
`receiver_region` varchar(32) DEFAULT NULL COMMENT '区',
`receiver_address` varchar(200) DEFAULT NULL COMMENT '详细地址',
`confirm_status` tinyint(4) DEFAULT NULL COMMENT '确认收货状态[0->未确认; 1->已确认]',
`delete_status` tinyint(4) DEFAULT NULL COMMENT '删除状态【0->未删除; 1->已删除】',
`use_integration` int(11) DEFAULT NULL COMMENT '下单时使用的积分',
`payment_time` datetime DEFAULT NULL COMMENT '支付时间',
`delivery_time` datetime DEFAULT NULL COMMENT '发货时间',
`receive_time` datetime DEFAULT NULL COMMENT '确认收货时间',
`comment_time` datetime DEFAULT NULL COMMENT '评价时间',
`modify_time` datetime DEFAULT NULL COMMENT '修改时间',
`remark` varchar(500) DEFAULT NULL COMMENT '订单备注',
PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=30836 DEFAULT CHARSET=utf8 COMMENT='订单';

```

3.4.6.2 订单项表

```

CREATE TABLE `trade_order_item` (
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'id',
  `order_id` bigint(20) DEFAULT NULL COMMENT 'order_id',
  `order_sn` char(32) DEFAULT NULL COMMENT 'order_sn',
  `spu_id` bigint(20) DEFAULT NULL COMMENT 'spu_id',
  `spu_name` varchar(255) DEFAULT NULL COMMENT 'spu_name',
  `spu_pic` varchar(500) DEFAULT NULL COMMENT 'spu_pic',
  `spu_brand` varchar(200) DEFAULT NULL COMMENT '品牌',
  `category_id` bigint(20) DEFAULT NULL COMMENT '商品分类 id',
  `sku_id` bigint(20) DEFAULT NULL COMMENT '商品 sku 编号',

```

```

`sku_name` varchar(255) DEFAULT NULL COMMENT '商品 sku 名字',
`sku_pic` varchar(500) DEFAULT NULL COMMENT '商品 sku 图片',
`sku_price` decimal(18,4) DEFAULT NULL COMMENT '商品 sku 价格',
`sku_quantity` int(11) DEFAULT NULL COMMENT '商品购买的数量',
`sku_attrs_vals` varchar(500) DEFAULT NULL COMMENT '商品销售属性组合（JSON）',
`promotion_amount` decimal(18,4) DEFAULT NULL COMMENT '商品促销分解金额',
`coupon_amount` decimal(18,4) DEFAULT NULL COMMENT '优惠券优惠分解金额',
`integration_amount` decimal(18,4) DEFAULT NULL COMMENT '积分优惠分解金额',
`real_amount` decimal(18,4) DEFAULT NULL COMMENT '该商品经过优惠后的分解金额',
`gift_integration` int(11) DEFAULT NULL COMMENT '赠送积分',
`gift_growth` int(11) DEFAULT NULL COMMENT '赠送成长值',
PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COMMENT='订单项信息';

```

3.4.6.3 支付信息表

```

CREATE TABLE `trade_payment_info` (
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'id',
  `order_sn` char(32) DEFAULT NULL COMMENT '订单号（对外业务号）',
  `order_id` bigint(20) DEFAULT NULL COMMENT '订单 id',
  `trade_no` varchar(50) DEFAULT NULL COMMENT '交易流水号',
  `total_amount` decimal(18,4) DEFAULT NULL COMMENT '支付总金额',
  `subject` varchar(200) DEFAULT NULL COMMENT '交易内容',
  `payment_status` varchar(20) DEFAULT NULL COMMENT '支付状态',
  `create_time` datetime DEFAULT NULL COMMENT '创建时间',
  `confirm_time` datetime DEFAULT NULL COMMENT '确认时间',
  `callback_content` varchar(4000) DEFAULT NULL COMMENT '回调内容',
  `callback_time` datetime DEFAULT NULL COMMENT '回调时间',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COMMENT='支付信息表';

```

3.4.6.4 退款信息表

```

CREATE TABLE `trade_refund_info` (
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'id',
  `order_return_id` bigint(20) DEFAULT NULL COMMENT '退款的订单',
  `refund` decimal(18,4) DEFAULT NULL COMMENT '退款金额',
  `refund_sn` varchar(64) DEFAULT NULL COMMENT '退款交易流水号',
  `refund_status` tinyint(1) DEFAULT NULL COMMENT '退款状态',
  `refund_channel` tinyint(4) DEFAULT NULL COMMENT '退款渠道[1-支付宝, 2-微信, 3-银联, 4-汇款]',
  `refund_content` varchar(5000) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COMMENT='退款信息';

```

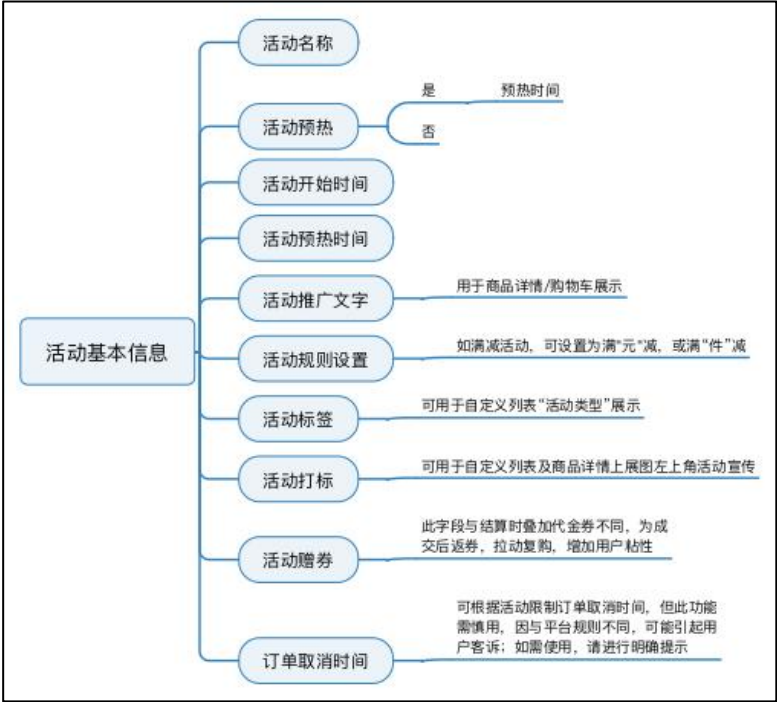
3.5 营销模块

促销活动的流程主要分成五个部分，明确促销目标，选择促销方式，产出促销方案，创建促销活动，促销活动开始。如图所示。



3.5.1 活动信息录入

在这里列举一些活动基本信息内可共用的字段，也有少部分根据不同工具所匹配的字段信息，如凑单类所需的凑单页，可以选择系统自动生成或自定义跳转，可根据具体情况添加字段，在此不再赘述。



促销活动的基本设置主要包括活动名称、促销编码、促销时间、促销平台、促销渠道、促销用户范围、促销链接等。如图所示，为有赞平台促销基本设置所示：

- 活动名称：由用户设置，可以设置副标题。
- 促销编码：自动生成，相当于活动的唯一代码。
- 促销时间：设置活动的有效时间段，前端活动标签将在此时间段内显示。如果是单一的产品促销类型，您可以选择提前显示，页面详细信息会通过特殊显示（如倒计时）进行标记。
- 推广渠道：一般电商平台包括 app、H5 商城、PC 商城。你可以选择推广活动的渠道，比如 app 的专享价。
- 限购数量：默认不限购。如果需要设置限购数量，则应将其分为两个维度。对于单个用户，应设置单个活动商品的购买限额和起始数量；就整体活动而言，需要设置分配给该活动的商品库存数量。所有存货都参与该活动。
- 推广用户范围：主要指所有用户。新用户（第一次发送购买行为）可以享受指定用户级别的活动资格。用户级别主要由成员系统模块定义。
- 推广链接：活动建立后，生成专属活动和特殊页面，自动生成活动链接。活动页面配置可以与活动设置分开。

3.5.2 促销规则设置

针对不同类型的促销活动有不同的活动规则，这在产品设计和订单计算中会有所不同。以下是对不同类型促销活动的分析。

1) 满减促销

满减促销主要是推动用户凑单购买商品。主要有两种形式：阶梯满减和循环满减。阶梯满减是指按级别设置优惠金额，例如满 100 减 10、满 300 减 50、满 500 减 80 等。每层之间要增加判断逻辑，循环满减，也是分级设置优惠，不过规则较为简单。例如设置“每满 200 减 20”，则订单金额 200 元实付 180 元，订单金额 410 元实付 370 元。

2) 满赠促销

满赠促销和赠品促销的区别在于，满赠促销是根据订单中主要商品的价格来区分的，赠品涨价可以分阶段设置。有两种形式“满 X 元送某商品”和“满 X 元送某商品加 Y 元”。

3) 单品促销

单品促销很简单，可显示折扣或直接设置商品促销价格。例如显示促销商品“8 折”，或者显示“原价 100 元、促销价 80 元”。促销价不能高于正常售卖价。

4) 套装促销

组合商品套装出售。注意组合的是商品，而不是 SKU。例如手机 2000 元，手机支架 200 元，套装购买 2100 元。这种形式有点类似组合 SKU，只是实现方式有所区别。

5) 赠品促销

购买主商品之后赠送商品。既要选择主商品，也要设置赠品。

获得赠品的条件有两种方式，从主商品中任意购买都可以获得赠品，或从主商品中购买 N 件以上可获得赠品。

对于赠品，也有多种赠送方式——全部赠送或者选一件赠品。这种赠品促销的方式在现实生活中比较常见。

6) 多买优惠促销

这种促销方式参考一些线下卖场的活动形式，有“M 元任选 N 件”、“M 件 N 折”两种优惠形式。

在设置时，选择对应类型的优惠形式，设置促销规则，例如选择“M 元任选 N 件”形式，让用户在固定商品池中任选 N 件 M 元。当检测到参与多买优惠活动的商品在购物车中或被下单时，应自动将优惠金额计算进订单总额中。

7) 定金促销

定金促销有定金预购和定金杠杆两种常见的模式。

定金预购指全款缴纳定金，交了定金相当于就已经确认订单支付完成，只要商品到货就可以直接发货；定金杠杆，指预售期交付一定金额的定金可以折抵更多金额，例如定金 10 元可抵扣 30 元，当商品正式卖的时候恢复正常价格，而交过定金的补齐尾款就行。

3.5.3 活动下发方式

活动可手动下发，也可选择系统下发方式，区别在于规则是否具有高度一致性和活动是否具有固定的节奏。

举例：如节奏及时间概念比较强烈的秒杀活动，如果每个单独的时间点都有业务手动下发，不仅辛苦，而且效率很低，这种情况下可设置系统按照日，周等不同时间维度进行自动下发。

3.5.4 活动选品及提报

1) 选品模板

这里需要注意不同活动所需要提供的活动信息是不同的，所以提报时的选品模板需要按活动类型进行处理，除了常规的活动价，活动库存，限购数量外，如立减有立减金额字段（可针对单个 sku 进行修改）等。

2) 可提报商品及提报校验

选品、提报应与活动规则模板相关联，规则即是选品及提报时的部分校验逻辑；判断商品是否为可提报商品，以及是否满足活动模板要求；这一步可以帮助业务进行选品及提高入选几率。

通用校验还包含如商品状态等，诸如售罄状态是否可提报，活动开始前是否需要二次校验；

3) 活动审批流

审批流需考虑正向及逆向流程，审批流与时间节点密切相关，会影响审核状态：

正向流程中：需考虑参与角色及进度是否一致性；如在平台活动中，涉及多个部门，则需要考虑各部门之间的审批流是否独立；如果超时未审核，是否自动驳回等。

逆向流程中：需考虑打回结果及二次提报审批流；如在“打回修改”后，操作权限由谁来承接，二次提报的流程是否可根据不同的活动信息，进行不同的设计等。

3.5.5 参与促销的订单计算

促销品的计算主要涉及两个方面：购物车和订单计算。

首先判断所选商品是否参与促销活动，其次判断是否满足促销条件，然后根据促销规则计算订单金额，判断是否可以同时与其他优惠同时享用。

当产品参与各种促销活动时，如果存在冲突，通常根据优先原则自动选择最佳优惠方案进行计算。

3.5.6 营销数据报表

“业务报表的核心价值是掌握事实、发现问题、分析原因、产生对策。产品经理要和业务人员一起，关注完整的体系化指标建设，设计有使用价值的报表。观察、分析问题的视角和思路是报表设计的核心，绚丽的交互只是次要的存在。报表数据可以帮助业务规划、决策、分析活动，作用范围覆盖了活动创建、活动中、活动后，是运营人员不可或缺的工具。

1) 提供客观、准确的活动数据

我们可以大致从流量数据和交易数量两个维度进行规划：

流量数据：如新客数，活动及商品 UV 等——流量型数据需注意前端的数据埋点；

交易数据：如销售额，销售件数，支付用户数，转化率，客单价等——交易型数据需注意订单金额的准确性。

在营销系统中，不同活动关注的的数据不同，如拼团活动还需关注开团订单数，参团订单数，成团订单数等

2) 发现数据指标变化

这里需要对数据结果进行处理，进一步对比数据值，观察数据曲线；总体数据可加入一些对比分析数值：如销售环比，与昨日，与上周等带有时间性质的数据字段，可以更直观的感受数据变化。如设定范围数值，当数据出现不正常的波动时进行预警，便于业务团队分析数据变化原因。

3.5.7 核心业务表

3.5.7.1 优惠券表

```
CREATE TABLE `market_coupon` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'id',  
  `coupon_type` tinyint(1) DEFAULT NULL COMMENT '优惠券类型[0->全场赠券；1->会员赠券；2->购物赠券；3->注册赠券]',  
  `coupon_img` varchar(2000) DEFAULT NULL COMMENT '优惠券图片',  
  `coupon_name` varchar(100) DEFAULT NULL COMMENT '优惠券名字',  
  `num` int(11) DEFAULT NULL COMMENT '数量',  
  `amount` decimal(18,4) DEFAULT NULL COMMENT '金额',  
  `per_limit` int(11) DEFAULT NULL COMMENT '每人限领张数',  
  `min_point` decimal(18,4) DEFAULT NULL COMMENT '使用门槛',  
  `start_time` datetime DEFAULT NULL COMMENT '开始时间',  
  `end_time` datetime DEFAULT NULL COMMENT '结束时间',  
  `use_type` tinyint(1) DEFAULT NULL COMMENT '使用类型[0->全场通用；1->指定分类；2->指定商品]',  
  `note` varchar(200) DEFAULT NULL COMMENT '备注',  
  `publish_count` int(11) DEFAULT NULL COMMENT '发行数量',  
  `use_count` int(11) DEFAULT NULL COMMENT '已使用数量',  
  `receive_count` int(11) DEFAULT NULL COMMENT '领取数量',  
  `enable_start_time` datetime DEFAULT NULL COMMENT '可以领取的开始日期',  
  `enable_end_time` datetime DEFAULT NULL COMMENT '可以领取的结束日期',  
  `code` varchar(64) DEFAULT NULL COMMENT '优惠码',  
  `member_level` tinyint(1) DEFAULT NULL COMMENT '可以领取的会员等级[0->不限等级，其他-对应等级]',  
  `publish` tinyint(1) DEFAULT NULL COMMENT '发布状态[0-未发布，1-已发布]',  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB AUTO_INCREMENT=101 DEFAULT CHARSET=utf8 COMMENT='优惠券信息';
```

3.5.7.2 优惠券与产品关系表

```
CREATE TABLE `market_coupon_spu` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'id',  
  `coupon_id` bigint(20) DEFAULT NULL COMMENT '优惠券 id',  
  `spu_id` bigint(20) DEFAULT NULL COMMENT 'spu_id',  
  `spu_name` varchar(255) DEFAULT NULL COMMENT 'spu_name',  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COMMENT='优惠券与产品关联';
```

4. 本章小结&面试要点

4.1 本章小结

本章主要介绍了企业级电商系统的核心系统功能，给大家介绍了其中的产品设计原型、核心的业务流程。通过对本章知识的学习，主要是帮助大家了解企业里面常见的业务系统、数据库结构、数据字段信息。

4.2 面试要点

1. 请讲一下电商业务架构的核心模块。
2. 请简述一下电商业务系统的核心流程。
3. 请简述一下商品系统、商品中心的核心功能。
4. 请简述一下商家后台的核心功能。
5. 请简述一下交易系统的核心功能。
6. 请简述一下营销系统的核心功能。
7. 请说明一下商品信息表里面有哪些核心的字段。
8. 请举例说明 SPU 与 SKU 的区别
9. 请说明一下用户在交易过程中涉及的业务过程，并对每个过程展开描述。
10. 请说一下你在做交易数据过程中核心用到的数据表有哪些？

四、企业级数据仓库项目架构

1. 项目背景

1.1 业务背景

电商数据建设的业务场景分为两部分：

- 1. 为管理层提供公司运营数据、支持业务决策
- 2. 为业务运营人员提供数据支撑、可视化看板，优化日常运营策略
- 3. 为商家提供运营动作效果数据，支撑商家日常的店铺运营

1.2 技术背景

1.项目人员

业务产品、产品运营、销售支持、数据分析师、数据产品、数仓开发

2.主题域建设

（1）用户主题建设：以平台增长业务团队当前对用户主题数据的需求为契机，完善底层和应用层的用户数据建设，满足在业务侧在不同场景下的用数需求。这里主要涉及 dws、ads 层的建设。

（2）商品主题建设：由于历史原因，公司存在两套商品系统，老系统最后会合并到新系统里面去，但是在数据层面，需要同时兼容处理两套商品数据；将散落在各个地方的商品数据进行整合，统一构建商品主题的维表，在公司层面数据的一致性、准确性会得到保障。

（3）商家主题建设：以商家运营业务团队当前对用户访问商家浏览数据的需求为契机，建设用户访问商品的流量数据、构建公司级别的统一商户数据，主要是为 B 端的商家用户提供业务运营数据。这里主要涉及 ads 层的建设，数据加工完成后会回流到商家后台系统。

（4）交易数据建设：数据侧需要将订单下单、支付、退款等多个流程的数据整合起来，形成完整的交易数据体系；交易数据需要进行通用性地建设，用来服务于其他主题域、支持跨域数据的建设。

（5）营销数据建设：数据侧需要对优惠券主题数据进行建设，满足业务方的日常用数需求；优惠券数据与交易数据密切相关，建设时会引用订单数据。

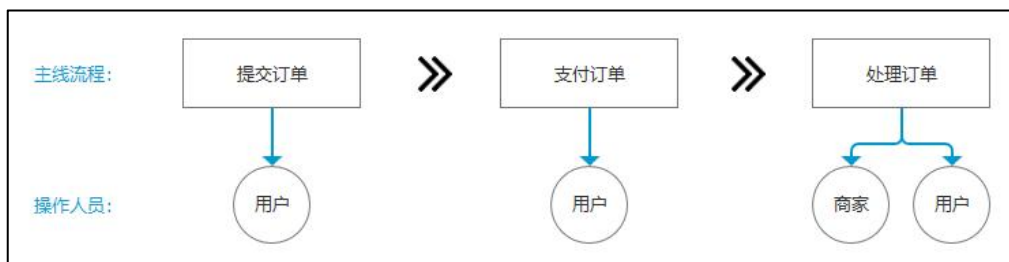
1.3 项目规划(☆☆☆重点记忆)

项目事项	具体内容	工时	备注
需求沟通	与业务方进行需求细节沟通	5 天	这里需要与业务方多轮沟通，明确需求细节、数据交付形式、交付节点

方案设计	设计技术方案，包括业务过程整理、数仓架构设计	5 天	按照维度建模的方式，设计数仓架构、hive 表结构
用户主题建设	根据用户流量数据、交易数据等进行用户数据建模	30 天	
商品主题建设	建设商品主题的数据	20 天	
商户主题建设	建设商户主题的数据	20 天	
交易主题建设	建设交易主题的数据	20 天	
营销主题建设	建设营销主题的数据	15 天	
数据交付文档	对交付的数据进行文档说明	5 天	
项目总结与回顾	对项目开发的过程中遇到的问题进行总结；并整理项目的建设成果	7 天	

2. 技术方案设计

2.1 业务过程梳理(☆☆☆重点记忆)



维度模型设计期间主要涉及 4 个主要的过程:

- (1)选择业务过程
- (2)声明粒度
- (3)确认维度
- (4)确认事实

在建模期间，需要考虑业务需求以及协作建模阶段涉及的底层数据源。从上述维度建模步骤来看，重点在解决数据粒度、维度设计和事实表设计问题。



声明粒度，为业务最小活动单元或不同维度组合。以共同粒度从多个组织业务过程合并度量的事实表称为合并事实表，需要注意的是，来自多个业务过程的事实合并到合并事实表时，它们必须具有同样等级的粒度。

例如：常见的电商下单环节，每个用户提交一笔订单（仅限一个物品），就

对应于一条订单记录。

【业务过程】：下订单

【粒度】：每笔订单（拆分为单个物品）

【维度】：地域、年龄、渠道等（可供分析的角度）

【事实/度量】：订单金额等（可用于分析的数据）

2.2 总线架构(☆☆重点理解)

根据阿里巴巴 OneData 方法论，在明确每个数据域中有哪些业务过程后，您需要开始定义维度，并基于维度构建总线矩阵。

1. 定义维度

在划分数据域、构建总线矩阵时，需要结合对业务过程的分析定义维度。以 A 电商公司的营销业务板块为例，在交易数据域中，我们重点分析确认收货（交易成功）的业务过程。

在确认收货的业务过程中，维度所依赖的业务角度主要有两个，即商品和收货地点（地域）。本教程中，假设收货和购买是同一个地点。

从商品角度分析，我们可以定义出以下维度：

- 商品 ID（主键）
- 商品名称
- 商品交易价格
- 商品类目 ID
- 商品类目名称
- 品类 ID
- 品类名称
- 买家 ID
- 商品状态：0 正常；1 用户删除；2 下架；3 未上架
- 商品所在城市
- 商品所在省份

从地域角度分析，我们可以定义出以下维度：

- 城市 code
- 城市名称
- 省份 code
- 省份名称

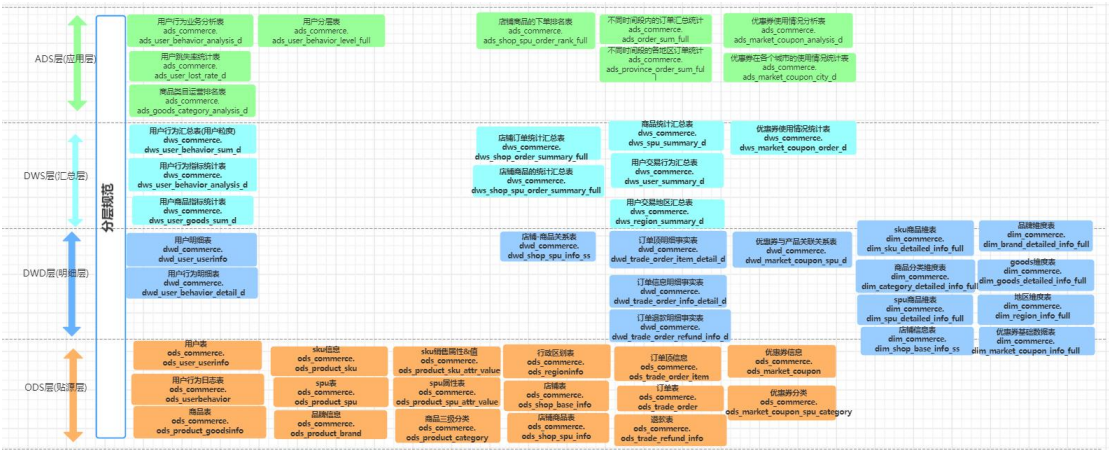
作为维度建模的核心，维度在企业级数据仓库中必须具有唯一性。维度在每个业务板块内必须具有唯一性，即每个维度在所属业务板块内有且只有一种定

义。例如项目里面的省份维度，对于营销业务板块内的任何业务过程所传达的信息都是一致的。其中 Y 表示包含该维度，N 表示不包含。

数据域/过程		一致性维度					
		购买省份	购买城市	类目	品牌	商品	成交金额
交易域	下单	Y	Y	Y	Y	Y	N
	支付	Y	Y	Y	Y	Y	N
	发货	Y	Y	Y	Y	Y	N
	确认收货	Y	Y	Y	Y	Y	Y

2.3 数仓模型架构

项目中涉及的核心表 50 个，其中 ods 层 17 个，dwd 层 7 个，dim 层 8 个，dws 层 9 个，ads 层 9 个。



2.3.1 ods 层



2.3.2 dim 层



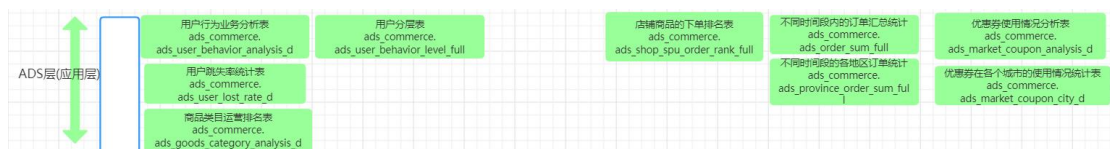
2.3.3 dwd 层



2.3.4 dws 层



2.3.5 ads 层



3. 数据分主题建设

3.1 用户主题建设

1. 通过使用数仓建模的方式进行数据加工处理，分析某个时间段内发生的 100 万条用户消费行为，业务侧需要尝试从数据中提取出有价值的业务建议。
2. 平台增长业务团队需要统计用户在平台的操作行为路径，判断用户在各个节点的流失率情况，为用户运营提供参考。
3. 平台增长业务团队需要对用户进行分层分类管理，便于监测、提升用户活跃度。
4. 以平台增长业务团队当前对用户主题数据的需求为契机，完善底层和应用层的用户数据建设，满足在业务侧在不同场景下的用数需求。这里主要涉及 dws、ads

层的建设。

3.2 商品主题建设

- 1.商家运营业务团队需要统计各个商品的流量情况，为平台的店铺提供日常运营数据。
- 2.目前商品的相关数据都分散在多个系统里面，业务侧从商品维度进行分析时，需要从多处获取数据，而且信息统一和整合，不同的业务方获取的数据存在差异。
- 3.下游业务需要从商品的各个维度去统计相关的指标时，无法获取完整、准确的数据，导致数据存在多套不同的统计结果。

3.3 商户主题建设

- 1.在电商平台的商家版 APP 中，平台会为商家搭建数据指标体系，科学地指导商家的经营发展方向以及提供经营状况概览。
- 2.提升 B 端商家产品体验，通过数据呈现商家的运营动作效果，为业务产生增值收入。可提供增值服务，为平台创造营收，比如提供同类店铺的对比数据。

3.4 交易主题建设

- 1.公司领导层需要关注用户在平台的下单情况、营收情况，财务部分需要根据订单量和订单金额统计公司的财务状况。
- 2.交易和支付业务团队需要关注订单的转化率、订单支付状态等数据。

3.5 营销主题建设

- 1.营销概况数据：商家系统内会嵌入营销数据，并进行可视化呈现形式：
- 2.营销运营：运营团队的同事会关注优惠券类的营销活动的带来的效果，需要数据侧对优惠券数据进行统计。

4. 本章小结&面试要点

4.1 本章小结

本章主要介绍了电商数据建设项目的业务背景、技术背景。然后基于当前项目需求对业务过程进行了数量，盘点了核心的总线架构，设计出了该项目主题的数仓架构，对于每个主题域的建设进行了初步介绍。

4.2 面试要点

1. 请介绍一下项目的业务背景、技术背景。
2. 请介绍一下项目中核心的业务过程。
3. 请介绍一下项目整体架构，并对每个主题域的数据建设做简要说明。

五、用户主题数据域建设

1. 建设背景

1.1 业务背景

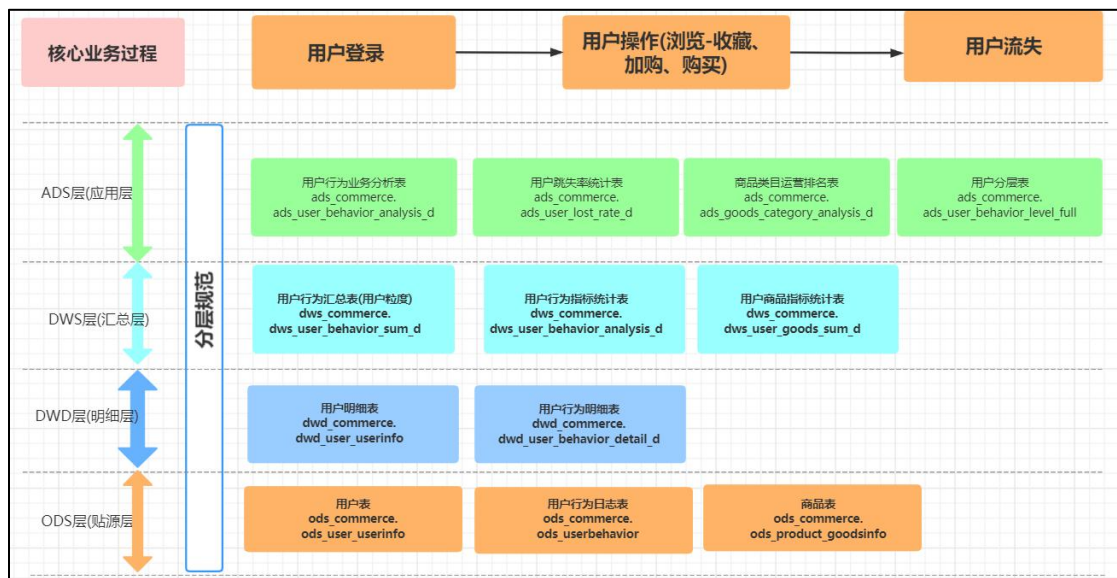
1. 通过使用数仓建模的方式进行数据加工处理，分析某个时间段内发生的 100 万条用户消费行为，业务侧需要尝试从数据中提取出有价值的业务建议。
2. 平台增长业务团队需要统计用户在平台的操作行为路径，判断用户在各个节点的流失率情况，为用户运营提供参考。
3. 平台增长业务团队需要对用户进行分层分类管理，便于监测、提升用户活跃度。
4. 商家运营业务团队需要统计各个商品的流量情况，为平台的店铺提供日常运营数据。

1.2 技术背景

1. 以平台增长业务团队当前对用户主题数据的需求为契机，完善底层和应用层的用户数据建设，满足在业务侧在不同场景下的用数需求。这里主要涉及 dws、ads 层的建设。
2. 以商家运营业务团队当前对用户访问商家浏览数据的需求为契机，建设用户访问商品的流量数据，主要是为 B 端的商家用户提供业务运营数据。这里主要涉及 ads 层的建设，数据加工完成后会回流到商家后台系统。

2. 数仓架构

这里是用户主题数据建设里面涉及的核心业务过程、数仓分层架构。



3. 数据建设

3.1 ods 层数据

这部分的数据主要有两个来源：第一个是来自用户系统的用户基础信息数据，第二个是来自平台日志系统的用户流量数据。

3.1.1 核心表的数据结构

表 1：用户基础信息表

```

DROP TABLE IF EXISTS ods_user_userinfo;

CREATE TABLE IF NOT EXISTS ods_user_userinfo (
    userid STRING COMMENT '用户 ID',
    username STRING COMMENT '用户名称',
    userpassword STRING COMMENT '密码',
    sex INT COMMENT '性别',
    usermoney INT COMMENT '钱包',
    frozenmoney INT COMMENT '近一个月的花费的总的金额',
    addressid STRING COMMENT '用户地址 ID, 0 表示没有获取地址',
    regtime STRING COMMENT '注册时间',
    lastlogin STRING COMMENT '最后登录时间',
    lasttime string COMMENT '系统下载最后时间'
) COMMENT '用户表' ROW FORMAT DELIMITED FIELDS TERMINATED BY '#' STORED AS textfile;

```

表 2：用户行为记录表

```

DROP TABLE IF EXISTS ods_userbehavior;

CREATE TABLE IF NOT EXISTS ods_userbehavior (
    user_id STRING COMMENT '用户标识',
    item_id STRING COMMENT '商品标识',

```



```
category_id STRING COMMENT '商品分类标识',
type STRING COMMENT '用户对商品的行为类型,浏览、收藏、加购物车、购买, 对应取值分别是 1、2、3、4',
action_time STRING COMMENT '行为时间'
) COMMENT '用户行为表' ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS textfile;
```

3.1.2 核心表的开发代码

该表直接从业务库里面同步过来, 可以采用 datax、cannal、flume 等数据同步工具获取到与业务库一样的数据。

3.1.3 核心表的数据介绍

这里对数据表的字段进行探查, 获取其分布情况。

--1. 查看用户量

```
select
    sex
    ,count(userid) as user_cnt
    ,count(userid) as user_cnt1
from ods_commerce.ods_user_userinfo
group by sex
;
```

--2. 查看用户行为表的分区情况

```
show partitions ods_commerce.ods_userbehavior
```

--3. 查看用户行为表的枚举值分布

```
select type,count(*) as pv,count(distinct user_id) as uv
from ods_commerce.ods_userbehavior
group by type
;
```

执行结果:

结果 1:

sex	user_cnt	user_cnt1
0	1000	1000

结果 2:

user_id	item_id	category_id	type	action_time
767038	4936937	2945933	pv	1512212223
767038	3318967	2945933	pv	1512212306
767038	344628	2945933	pv	1512212344
767038	1353441	64179	cart	1512226082
767038	791776	64179	pv	1512226104
767038	1371525	3776866	pv	1512226213
767038	3150019	3164550	buy	1512226484

结果 3:

type	pv	uv
buy	2015839	672404
fav	2888258	389823
pv	89716264	984114
cart	5530446	738996

3.2 dwd 层数据

3.2.1 核心表的数据结构

表 1：用户表

```
USE dwd_commerce;

DROP TABLE IF EXISTS dwd_user_userinfo;

CREATE TABLE IF NOT EXISTS dwd_user_userinfo (
    user_id STRING COMMENT '用户 ID'
    ,user_name STRING COMMENT '用户名称'
    ,sex INT COMMENT '性别'
    ,user_money decimal(20,4) COMMENT '钱包'
    ,frozen_money decimal(20,4) COMMENT '近一个月的花费的总的金额'
    ,address_id STRING COMMENT '用户地址 ID, 0 表示没有获取地址'
    ,reg_time STRING COMMENT '注册时间'
    ,last_login STRING COMMENT '最后登录时间'
) COMMENT '用户表' STORED AS ORC;
```

表 2：

```
DROP TABLE IF EXISTS dwd_commerce.dwd_user_behavior_detail_d;

CREATE TABLE IF NOT EXISTS dwd_commerce.dwd_user_behavior_detail_d (
    user_id STRING COMMENT '用户标识'
    ,item_id STRING COMMENT '商品标识'
    ,category_id STRING COMMENT '商品分类标识'
    ,type STRING COMMENT '用户对商品的行为类型,浏览、收藏、加购物车、购买,对应取值分别是 1、2、3、4'
    ,action_time STRING COMMENT '行为时间'
)
COMMENT '用户行为明细表'
PARTITIONED BY (dt string comment '日期')
STORED AS orc;
```

3.2.2 核心表的开发代码

【表 1：用户表】

核心代码：

```
insert overwrite table dwd_user_userinfo

select

    userid as user_id
    ,username as user_name
    ,sex
```

```

,cast(usermoney as decimal(20,4)) as user_money
,cast(frozenmoney as decimal(20,4)) as frozen_money
,addressid as address_id
,from_unixtime(cast(regtime as bigint),'yyyy-MM-dd HH:mm:ss') as reg_time
,from_unixtime(cast(lastlogin as bigint),'yyyy-MM-dd HH:mm:ss') as last_login
from ods_commerce.ods_user_userinfo
;

```

数据验证:

表详情	日志	查询结果 (100)	查询历史	(预览) ods_market_c...	下载csv	下载xls	固化表格
user_id	user_name	sex	user_money	frozen_money	address_id	reg_time	last_login
231625	18768998529	0	0.0000	0.0000	0	2017-06-01 00:01:58	2017-06-01 00:01:58
231626	15993269807	0	0.0000	0.0000	0	2017-06-01 00:02:25	2017-06-01 00:02:25
231627	15256885559	0	0.0000	0.0000	0	2017-06-01 00:02:40	2017-06-01 00:25:34
231628	15842014644	0	0.0000	0.0000	0	2017-06-01 00:03:04	2017-06-01 00:03:04
231629	劳劳淘	0	0.0000	0.0000	59792	2017-06-01 00:04:00	2017-06-01 00:04:00
231630	15696898035	0	0.0000	0.0000	0	2017-06-01 00:05:37	2017-06-01 00:05:37

表 2: 用户行为明细表

```

set hive.exec.dynamic.partition.mode=nonstrict;
insert overwrite table dwd_commerce.dwd_user_behavior_detail_d partition(dt)
select
    user_id
, item_id
, category_id
, type
, from_unixtime(cast(action_time as bigint),'yyyy-MM-dd HH:mm:ss') as action_time
, to_date(from_unixtime(cast(action_time as bigint),'yyyy-MM-dd')) as dt
from ods_commerce.ods_userbehavior
;

```

数据验证:

表详情	日志	查询结果 (100)	查询历史	(预览) ods_market_c...	下载csv	下载xls	固化表格
user_id	item_id	category_id	type	action_time	dt		
674989	945533	2520377	pv	2016-01-01 08:05:56	2016-01-01		
674989	772185	2407396	pv	2016-01-01 08:12:41	2016-01-01		
674989	4320139	2407396	pv	2016-01-01 08:17:18	2016-01-01		
674989	1825058	2673821	pv	2016-01-01 08:20:32	2016-01-01		
674989	3149399	4350978	pv	2016-01-01 08:20:43	2016-01-01		
674989	1825058	2673821	pv	2016-01-01 08:20:58	2016-01-01		
469251	987361	2465336	pv	2016-01-05 11:49:49	2016-01-05		
469251	1185049	4690421	pv	2016-01-06 13:01:11	2016-01-06		

3.2.3 涉及的基础知识

- 1.日期函数: 参考: <https://blog.csdn.net/wangwangstone/article/details/110011860>
- 2.cast 函数的用法: 注意 from_unixtime(cast(regtime as bigint),'yyyy-MM-dd HH:mm:ss')
- 3.to_date 函数
- 4.会话参数: set hive.exec.dynamic.partition.mode=nonstrict; -- 影响到分区写入
- 5.动态分区、查看、删除分区:

```

show partitions dwd_commerce.dwd_user_behavior_detail_d;

```

```
alter table dwd_commerce.dwd_user_behavior_detail_d drop if exists partition (dt<'2016-01-01');  
alter table dwd_commerce.dwd_user_behavior_detail_d drop if exists partition (dt>='2022-05-20');
```

3.3 dws 层数据

3.3.1 核心表的数据结构

【表 1：用户行为汇总表】

```
--1.用户粒度的汇总模型  
DROP TABLE IF EXISTS dws_commerce.dws_user_behavior_sum_d;  
CREATE TABLE IF NOT EXISTS dws_commerce.dws_user_behavior_sum_d (  
    user_id STRING COMMENT '用户标识'  
    ,view_cnt BIGINT COMMENT '浏览次数'  
    ,fav_cnt BIGINT COMMENT '收藏次数'  
    ,cart_cnt BIGINT COMMENT '加购物车次数'  
    ,buy_cnt BIGINT COMMENT '购买次数'  
)  
COMMENT '用户行为汇总表(用户粒度)'  
PARTITIONED BY (dt string comment '日期')  
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','  
STORED AS orc;
```

【表 2：用户行为指标统计表】

```
--2.日期维度的汇总模型,基于用户粒度的向上汇总  
DROP TABLE IF EXISTS dws_commerce.dws_user_behavior_analysis_d;  
CREATE TABLE IF NOT EXISTS dws_commerce.dws_user_behavior_analysis_d (  
    view_cnt BIGINT COMMENT '浏览次数'  
    ,fav_cnt BIGINT COMMENT '收藏次数'  
    ,cart_cnt BIGINT COMMENT '加购物车次数'  
    ,buy_cnt BIGINT COMMENT '购买次数'  
    ,view_uv BIGINT COMMENT '浏览人数'  
    ,fav_uv BIGINT COMMENT '收藏人数'  
    ,cart_uv BIGINT COMMENT '加购物车人数'  
    ,buy_uv BIGINT COMMENT '购买人数'  
)  
COMMENT '用户行为指标统计表'  
PARTITIONED BY (dt string comment '日期')  
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','  
STORED AS orc;
```

【表 3：用户商品指标统计表】

```
--3. user_id*item_id 粒度的用户行为统计数据  
DROP TABLE IF EXISTS dws_commerce.dws_user_goods_sum_d;
```

```
CREATE TABLE IF NOT EXISTS dws_commerce.dws_user_goods_sum_d (  
    user_id STRING COMMENT '用户标识'  
    ,goods_id STRING COMMENT '商品标识'  
    ,goods_name STRING COMMENT '商品名称'  
    ,category_id STRING COMMENT '商品分类标识'  
    ,view_cnt BIGINT COMMENT '浏览次数'  
    ,fav_cnt BIGINT COMMENT '收藏次数'  
    ,cart_cnt BIGINT COMMENT '加购物车次数'  
    ,buy_cnt BIGINT COMMENT '购买次数'  
)  
  
COMMENT '用户商品指标统计表'  
  
PARTITIONED BY (dt string comment '日期')  
  
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','  
  
STORED AS orc;
```

3.3.2 核心表的开发代码

【表 1： 用户行为汇总表】

核心代码：

```
insert overwrite table dws_commerce.dws_user_behavior_sum_d partition(dt)  
select  
    user_id  
    ,sum(if(type='pv',1,0)) as view_cnt  
    ,sum(if(type='fav',1,0)) as fav_cnt  
    ,sum(if(type='cart',1,0)) as cart_cnt  
    ,sum(if(type='buy',1,0)) as buy_cnt  
    ,dt  
from dwd_commerce.dwd_user_behavior_detail_d  
where dt between '2017-11-01' and '2017-12-01'  
group by  
    dt  
    ,user_id  
;
```

数据验证：

user_id	view_cnt	fav_cnt	cart_cnt	buy_cnt	dt
1	6	0	0	0	2017-11-25
1000000	1	0	0	0	2017-11-25
1000019	2	0	1	1	2017-11-25
1000037	6	0	0	0	2017-11-25
1000046	2	0	0	0	2017-11-25
1000064	64	0	5	1	2017-11-25
1000073	7	0	0	0	2017-11-25
1000082	5	0	0	0	2017-11-25
1000109	6	0	0	0	2017-11-25
1000118	4	0	0	1	2017-11-25

【表 2： 用户行为指标统计表】

核心代码：

```
insert overwrite table dws_commerce.dws_user_behavior_analysis_d partition(dt)
select
    sum(if(type='pv',1,0)) as view_cnt
    ,sum(if(type='fav',1,0)) as fav_cnt
    ,sum(if(type='cart',1,0)) as cart_cnt
    ,sum(if(type='buy',1,0)) as buy_cnt
    ,count(distinct if(type='pv',user_id,null)) as view_uv
    ,count(distinct if(type='fav',user_id,null)) as fav_uv
    ,count(distinct if(type='cart',user_id,null)) as cart_uv
    ,count(distinct if(type='buy',user_id,null)) as buy_uv
    ,dt
from dwd_commerce.dwd_user_behavior_detail_d
where dt between '2017-11-01' and '2017-12-01'
group by
    dt
;
```

数据验证：

表详情	日志	查询结果 (1)	查询历史	(预览) ods_market_c...							下载csv	下载xls	图化表格
view_cnt	fav_cnt	cart_cnt	buy_cnt	view_uv	fav_uv	cart_uv	buy_uv	dt					
562	0	0	0	55	0	0	0	2017-11-03					

【表 3：用户商品指标统计表】

核心代码：

```
set hive.exec.dynamic.partition.mode=nonstrict;
insert overwrite table dws_commerce.dws_user_goods_sum_d partition(dt)
select
    a.user_id
    ,a.item_id as goods_id
    ,b.goodsname as goods_name
    ,a.category_id
    ,sum(case when a.type='pv' then 1 else 0 end) as view_cnt
    ,sum(case when a.type='cart' then 1 else 0 end) as cart_cnt
    ,sum(case when a.type='fav' then 1 else 0 end) as fav_cnt
    ,sum(case when a.type='buy' then 1 else 0 end) as buy_cnt
    ,a.dt
from dwd_commerce.dwd_user_behavior_detail_d as a
join ods_commerce.ods_product_goodsinfo as b
on a.item_id=b.goodsid
where a.dt between '2017-11-01' and '2017-12-01'
group by a.user_id,a.item_id,b.goodsname,a.category_id,a.dt;
```

数据验证：

user_id	goods_id	goods_name	category_id	view_cnt	fav_cnt	cart_cnt	buy_cnt	dt
1000014	2557285	衣服6189	1265358	1	0	0	0	2017-11-25
1000018	299218	衣服6376	2671397	1	0	0	0	2017-11-25
1000031	514099	衣服4828	4357323	1	0	0	0	2017-11-25
1000034	551803	衣服7130	4801426	1	0	0	0	2017-11-25
1000041	1157172	衣服4788	2157356	1	0	0	0	2017-11-25
1000077	1160964	衣服5252	3607361	1	0	0	0	2017-11-25
1000077	4517742	衣服6174	3738615	1	1	0	0	2017-11-25
1000084	4474837	衣服1752	144028	2	0	0	0	2017-11-25
1000089	2264365	衣服7972	4789432	1	0	0	0	2017-11-25
1000100	356171	衣服4440	2465336	1	0	0	0	2017-11-25

3.3.3 涉及的基础知识

1. 流量需求数据通常分为两部分来建设，主要包括可累加指标（比如浏览次数）、去重指标（比如浏览用户数）。
2. 常用维度退化写入汇总表：例如表 3 就把商品名称退化到汇总表中，作为常用的维度提供给下游直接使用，而无需重复关联维表。

3.4 ads 层数据

3.4.1 核心表的数据结构

【表 1：用户行为业务分析表】

```
-- 场景一：业务指标分析

DROP TABLE IF EXISTS ads_commerce.ads_user_behavior_analysis_d;

CREATE TABLE IF NOT EXISTS ads_commerce.ads_user_behavior_analysis_d (
    avg_view_cnt BIGINT COMMENT '人均浏览次数'
    ,avg_fav_cnt BIGINT COMMENT '人均收藏次数'
    ,avg_cart_cnt BIGINT COMMENT '人均加购物车次数'
    ,avg_buy_cnt BIGINT COMMENT '人均购买次数'
)
COMMENT '用户行为业务分析表'
PARTITIONED BY (dt string comment '日期')
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS orc;
```

【表 2：用户跳失率统计表】

```
--场景 2：跳失率：在 11 月 25 日至 12 月 3 日期间只有浏览行为的用户/总用户数 OK

DROP TABLE IF EXISTS ads_commerce.ads_user_lost_rate_d;

CREATE TABLE IF NOT EXISTS ads_commerce.ads_user_lost_rate_d (
    lost_rate decimal(20,4) COMMENT '跳失率'
)
COMMENT '用户跳失率统计表'
PARTITIONED BY (dt string comment '日期')
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS orc;
```


【表 3：商品类目运营排名表】

```
--场景 3：业务指标分析
--获取每个商品类目浏览量排名前 10 的商品、以及他们的浏览次数

DROP TABLE IF EXISTS ads_commerce.ads_goods_category_analysis_d;

CREATE TABLE IF NOT EXISTS ads_commerce.ads_goods_category_analysis_d (
    category_id STRING COMMENT '商品类别 ID'
    ,goods_id STRING COMMENT '商品标识'
    ,goods_name STRING COMMENT '商品名称'
    ,view_cnt BIGINT COMMENT '浏览次数'
    ,view_rnk BIGINT COMMENT '浏览排名'
)
COMMENT '商品类目运营排名表'
PARTITIONED BY (dt string comment '日期')
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS orc;
```

【表 4：用户分层表】

```
--场景 4：用户分层 这里需要注意 max_dd 的字段类型

DROP TABLE IF EXISTS ads_commerce.ads_user_behavior_level_full;

CREATE TABLE IF NOT EXISTS ads_commerce.ads_user_behavior_level_full (
    user_id STRING COMMENT '用户 ID'
    ,max_dd STRING COMMENT '最近访问日期'
    ,min_dd STRING COMMENT '最早访问日期'
    ,diff BIGINT COMMENT '浏览间隔天数'
    ,r_score BIGINT COMMENT '用户分层'
)
COMMENT '用户分层表'
STORED AS orc;
```

3.4.2 核心表的开发代码

【表 1：用户行为业务分析表】

核心代码：

```
insert overwrite table ads_commerce.ads_user_behavior_analysis_d partition(dt)
select
    view_cnt/view_uv as avg_view_cnt
    ,fav_cnt/fav_uv as avg_fav_cnt
    ,cart_cnt/cart_uv as av_cart_cnt
    ,buy_cnt/buy_uv as avg_buy_cnt
    ,dt
from dws_commerce.dws_user_behavior_analysis_d
```

```
where dt between '2017-11-01' and '2017-12-03'

;
```

数据验证:

avg_view_cnt	avg_fav_cnt	avg_cart_cnt	avg_buy_cnt	dt
4	null	null	null	2017-11-01
8	null	null	null	2017-11-02
10	null	null	null	2017-11-03

【表 2：用户跳失率统计表】

核心代码:

```
set hive.exec.dynamic.partition.mode=nonstrict;

insert overwrite table ads_commerce.ads_user_lost_rate_d partition(dt)

select

count(if(t.fav_cnt=0 and t.cart_cnt=0 and t.buy_cnt=0,user_id,null))/count(user_id)*100 as lost_rate

,dt

from dws_commerce.dws_user_behavior_sum_d

where dt between '2017-11-30' and '2017-12-03'

group by dt

;
```

数据验证:

lost_rate	dt
46.3722	2017-11-30
45.2660	2017-12-01
46.8227	2017-12-02
47.3333	2017-12-03

【表 3：商品类目运营排名表】

核心代码:

```
insert overwrite table ads_commerce.ads_goods_category_analysis_d partition(dt)

select category_id,goods_id,goods_name,view_cnt,rnk as view_rnk,dt

from (

select

dt

,category_id

,goods_id

,goods_name

,view_cnt

,RANK() OVER(PARTITION BY dt,category_id ORDER BY view_cnt desc) AS rnk

from (

select

dt

,category_id

,goods_id
```

```

,goods_name
,sum(view_cnt) as view_cnt
from dws_commerce.dws_user_goods_sum_d
where dt between '2017-11-03' and '2017-11-03'
group by dt,category_id,goods_id,goods_name
) as tmp
) as a where rnk=3
;

```

数据验证:

category_id	goods_id	goods_name	view_cnt	view_rnk	dt
1045172	5029495	衣服7586	3	3	2017-11-03
1102540	4994254	衣服1795	4	3	2017-11-03
1102540	4387812	衣服5591	4	3	2017-11-03
1379146	2776148	衣服9826	2	3	2017-11-03
2355072	813863	衣服1257	1	3	2017-11-03
2355072	1132032	衣服8195	1	3	2017-11-03
2355072	1325249	衣服9783	1	3	2017-11-03
2355072	1618601	衣服5677	1	3	2017-11-03
2355072	1999605	衣服1618	1	3	2017-11-03

【表 4：用户分层表】

核心代码:

```

-- 分层数据加工逻辑
with ta as (
select user_id,to_date(from_unixtime(cast(action_time as bigint),'yyyy-MM-dd')) as dd
from ods_commerce.ods_userbehavior
where type='pv'
)
,tb as (
select
user_id
,max(dd) as max_dd
,min(dd) as min_dd
,datediff(max(dd),min(dd)) as diff
from ta
group by user_id
)
insert overwrite table ads_commerce.ads_user_behavior_level_full
select
user_id
,max_dd
,min_dd
,diff
,(case when diff BETWEEN 0 and 2 then 4
when diff BETWEEN 3 and 4 then 3
when diff BETWEEN 5 and 6 then 2
when diff BETWEEN 7 and 8 then 1

```

```

else 0 end ) as r_score

from tb

;

```

数据验证：

```

--1.查看执行结果
select * from ads_commerce.ads_user_behavior_level_full limit 100;

--2.查看用户分层分布
select r_score,count(*) from ads_commerce.ads_user_behavior_level_full group by r_score;

--2.查看用户浏览天数分布
select diff,count(*) from ads_commerce.ads_user_behavior_level_full group by diff;

```

结果 1：

user_id	max_dd	min_dd	diff	r_score
1000026	2017-12-03	2017-11-25	8	1
100006	2017-12-03	2017-11-25	8	1
1000099	2017-12-03	2017-11-25	8	1
1000134	2017-12-03	2017-11-25	8	1
1000229	2017-12-03	2017-11-25	8	1
1000242	2017-12-03	2017-11-25	8	1
1000337	2017-12-03	2017-11-28	5	2
1000350	2017-12-03	2017-11-25	8	1
1000445	2017-12-03	2017-11-25	8	1
1000553	2017-12-03	2017-11-25	8	1

结果 2：

r_score	_c1
0	39638
1	801721
2	104106
3	32666
4	5983

结果 3：

diff	_c1
0	1342
1	1732
2	2909
3	11977
4	20689
5	35168
6	68938
7	170157
8	631564
9	32041
10	3519

3.4.3 涉及的基础知识

1. RANK()函数的用法。
2. With 语句的用法。
3. 比例型的指标数据放在 ads 层进行个性化计算。

4. 建设成果

1.业务层面：为公司的用户增长团队提供了基础的分析数据，原本他们每周需要花 2 天时间来做数据统计、图表分析，现在只需要 0.5 天即可获取到他们的数据，极大地提升了他们日常取数的效率。同时为平台合作的 1000 多万商家提供了日常经营分析数据，为他们日常的运营提供了科学有效的数据支撑。

2.数据层面：完成了用户行为特征的建设，主要是涉及用户浏览、收藏、加购物车、购买等业务场景下的用户特点，丰富了用户画像的行为特征库。

5. 知识要点

1.维度建模流程：

(1)需求搜集：主要是用户增长团队、商家运营团队对用户行为数据的需求。

(2)业务过程：用户登录 APP 后，浏览相关的商品、收藏或者加入购物车、最后产生购买行为。

(3)声明粒度：这里涉及了日期*用户粒度的行为统计数据、日期*用户*商品粒度的浏览统计数据。

(4)确定维度和事实：事实有可加性、半可加性、非可加性三种类型，需要将不可加性事实分解为可加的组件。这里涉及的主要维度包括日期、用户、商品，主要的事实（指标）包括浏览人数&次数、收藏人数&次数、加购人数&次数、购买人数&次数、流失率等。

2.数仓开发规范：ads、dws、dwd、ods 分别调用和依赖低层的模型。

六、商品主题数据域建设

1. 建设背景

1.1 业务背景

1.目前商品的相关数据都分散在多个系统里面，业务侧从商品维度进行分析时，需要从多处获取数据，而且信息统一和整合，不同的业务方获取的数据存在差异。

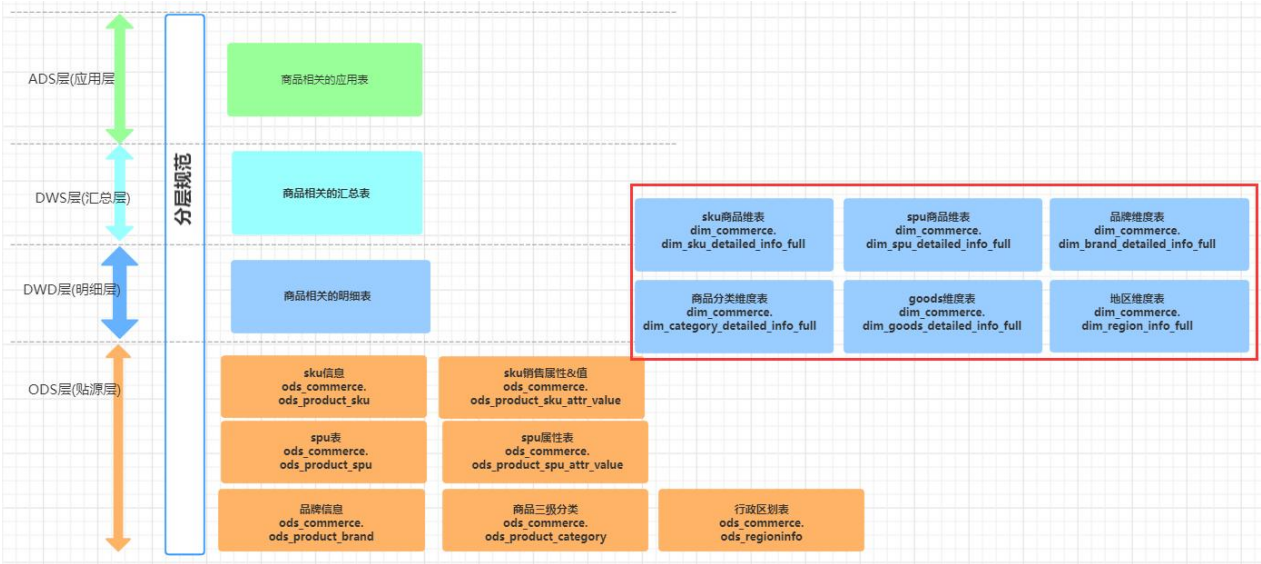
2.下游业务需要从商品的各个维度去统计相关的指标时，无法获取完整、准确的数据，导致数据存在多套不同的统计结果。

1.2 技术背景

- 1.由于历史原因，公司存在两套商品系统，老系统最后会合并到新系统里面去，但是在数据层面，需要同时兼容处理两套商品数据。
- 2.将散落在各个地方的商品数据进行整合，统一构建商品主题的维表，在公司层面数据的一致性、准确性会得到保障。

2. 数据架构

这里是商品主题数据建设里面涉及的核心数据源头表、加工得到的公共维表数据。



3. 数据建设

3.1 ods 层数据

3.1.1 核心表的数据结构

表 1: spu 信息表

```
DROP TABLE IF EXISTS ods_product_spu;

CREATE TABLE IF NOT EXISTS ods_product_spu (
  id BIGINT COMMENT '商品 id',
  name STRING COMMENT '商品名称',
  category_id BIGINT COMMENT '所属分类 id',
  brand_id BIGINT COMMENT '品牌 id',
  publish_status INT COMMENT '上架状态[0 - 下架, 1 - 上架]',
```

```
create_time STRING COMMENT '创建时间',  
update_time STRING COMMENT '更新时间'  
) COMMENT 'spu 信息' STORED AS orc;
```

表 2: spu 属性表

```
DROP TABLE IF EXISTS ods_product_spu_attr_value;  
CREATE TABLE IF NOT EXISTS ods_product_spu_attr_value (  
    id BIGINT COMMENT 'id',  
    spu_id BIGINT COMMENT '商品 id',  
    attr_id BIGINT COMMENT '属性 id',  
    attr_name STRING COMMENT '属性名',  
    attr_value STRING COMMENT '属性值',  
    sort INT COMMENT '顺序'  
) COMMENT 'spu 属性值' STORED AS orc;
```

表 3: sku 信息表

```
DROP TABLE IF EXISTS ods_product_sku;  
CREATE TABLE IF NOT EXISTS ods_product_sku (  
    id BIGINT COMMENT 'skuId',  
    spu_id BIGINT COMMENT 'spuId',  
    name STRING COMMENT 'sku 名称',  
    category_id BIGINT COMMENT '所属分类 id',  
    brand_id BIGINT COMMENT '品牌 id',  
    default_image STRING COMMENT '默认图片',  
    title STRING COMMENT '标题',  
    subtitle STRING COMMENT '副标题',  
    price DOUBLE COMMENT '价格',  
    weight INT COMMENT '重量（克）'  
) COMMENT 'sku 信息' STORED AS orc;
```

表 4: sku 销售属性表

```
DROP TABLE IF EXISTS ods_product_sku_attr_value;  
CREATE TABLE IF NOT EXISTS ods_product_sku_attr_value (  
    id BIGINT COMMENT 'id',  
    sku_id BIGINT COMMENT 'sku_id',  
    attr_id BIGINT COMMENT 'attr_id',  
    attr_name STRING COMMENT '销售属性名',  
    attr_value STRING COMMENT '销售属性值',  
    sort INT COMMENT '顺序'  
) COMMENT 'sku 销售属性&值' STORED AS orc;
```

表 5: 品牌信息表

```
DROP TABLE IF EXISTS ods_product_brand;  
CREATE TABLE IF NOT EXISTS ods_product_brand (  
    id BIGINT COMMENT '品牌 id',  
    name STRING COMMENT '品牌名',  
    logo STRING COMMENT '品牌 logo',
```



```

status INT COMMENT '显示状态[0-不显示; 1-显示]',
first_letter STRING COMMENT '检索首字母',
sort INT COMMENT '排序',
remark string COMMENT '备注'
) COMMENT '品牌' STORED AS orc;

```

表 6：商品分类表

```

DROP TABLE IF EXISTS ods_product_category;
CREATE TABLE IF NOT EXISTS ods_product_category (
    id BIGINT COMMENT '分类 id',
    name STRING COMMENT '分类名称',
    parent_id BIGINT COMMENT '父分类 id',
    status INT COMMENT '是否显示[0-不显示, 1 显示]',
    sort INT COMMENT '排序',
    icon STRING COMMENT '图标地址',
    unit STRING COMMENT '计量单位'
) COMMENT '商品三级分类' STORED AS orc;

```

表 7：行政区划表

```

DROP TABLE IF EXISTS ods_regioninfo;
CREATE TABLE IF NOT EXISTS ods_regioninfo (
    regionid STRING COMMENT '地区 ID',
    parentid STRING COMMENT '父级区域 ID',
    regionname STRING COMMENT '地区名称',
    regiontype INT COMMENT '区域类别（0 国家/1 省份/2 城市/3 区县）',
    agencyid INT COMMENT '无用字段',
    pt STRING COMMENT '系统更新时间'
) COMMENT '行政区划表' STORED AS orc;

```

3.1.2 核心表的开发代码

该表直接从业务库里面同步过来，可以采用 datax、cannal、flume 等数据同步工具获取到与业务库一样的数据。

3.1.3 核心表的数据介绍

这里对数据表的字段进行探查，获取其分布情况。

3.2 dim 层数据

3.2.1 核心表的数据结构

表 1：sku 商品维度表

```

-- 1.sku 商品维度表

```

```

DROP TABLE IF EXISTS dim_commerce.dim_sku_detailed_info_full;

CREATE TABLE IF NOT EXISTS `dim_commerce.dim_sku_detailed_info_full`(
  `sku_id` bigint COMMENT '商品 id',
  `sku_name` string COMMENT '商品名称',
  `category_id` bigint COMMENT '所属三级分类 id',
  `category_name` string COMMENT '所属三级分类名字',
  `brand_id` bigint COMMENT '品牌 id',
  `brand_name` string COMMENT '品牌名称',
  `sku_default_image` string COMMENT '默认图片',
  `sku_title` string COMMENT '标题',
  `sku_subtitle` string COMMENT '副标题',
  `sku_price` double COMMENT '价格',
  `spu_id` bigint COMMENT 'spu 商品 id',
  `spu_name` string COMMENT 'spu 商品名称',
  `sku_attrs` array<struct<attr_id:bigint,attr_name:string,attr_value:string>> COMMENT 'sku 平台属性'
) COMMENT 'sku 商品维度表'

STORED AS orc;

```

表 2: spu 商品维度表

```

-- 2.spu 商品维度表

DROP TABLE IF EXISTS dim_commerce.dim_spu_detailed_info_full;

CREATE TABLE IF NOT EXISTS `dim_commerce.dim_spu_detailed_info_full`(
  `spu_id` bigint COMMENT 'spu 商品 id',
  `spu_name` string COMMENT 'spu 商品名称',
  `spu_publish_status` int COMMENT '上架状态',
  `spu_create_time` string COMMENT '创建时间',
  `spu_update_time` string COMMENT '更新时间',
  `category_id` bigint COMMENT '所属三级分类 id',
  `category_name` string COMMENT '所属三级分类名字',
  `brand_id` bigint COMMENT '品牌 id',
  `brand_name` string COMMENT '品牌名称',
  `spu_attrs` array<struct<col1:bigint,col2:string,col3:string>>COMMENT 'spu 平台属性'
) COMMENT 'spu 商品维度表'

STORED AS orc;

```

表 3: 品牌维度表

```

-- 3.品牌维度表

DROP TABLE IF EXISTS dim_commerce.dim_brand_detailed_info_full;

CREATE TABLE IF NOT EXISTS `dim_commerce.dim_brand_detailed_info_full`(
  `id` bigint COMMENT '品牌 id',
  `name` string COMMENT '品牌名称',
  `logo` string COMMENT '品牌 logo',
  `update_time` string COMMENT '更新时间'
) COMMENT '品牌维度表'

STORED AS orc;

```

表 4：商品分类维度表

```
-- 4.商品分类维度表

DROP TABLE IF EXISTS dim_commerce.dim_category_detailed_info_full;

CREATE TABLE IF NOT EXISTS `dim_commerce.dim_category_detailed_info_full` (
  `category_3_id` bigint COMMENT '三级分类 id',
  `category_3_name` string COMMENT '三级分类名称',
  `category_2_id` bigint COMMENT '二级分类 id',
  `category_2_name` string COMMENT '二级分类名称',
  `category_1_id` bigint COMMENT '一级分类 id',
  `category_1_name` string COMMENT '一级分类名称'
) COMMENT '商品分类维度表'

STORED AS orc;
```

表 5：goods 维度表

```
-- 5.goods 维度表

DROP TABLE IF EXISTS dim_commerce.dim_goods_detailed_info_full;

CREATE TABLE IF NOT EXISTS `dim_commerce.dim_goods_detailed_info_full` (
  `goodsid` string COMMENT '商品 ID',
  `brand_id` string COMMENT '品类 ID',
  `markid` string COMMENT '专场 ID（商品售卖的位置）',
  `goodstag` string COMMENT '进货渠道，档口的名字',
  `brand_name` string COMMENT '品牌名称（不用，脱敏严重）',
  `customtag` string COMMENT '商品的详情',
  `goodsname` string COMMENT '竞价排名，BH 为公司的补货',
  `clickcount` int COMMENT '商品的点击次数',
  `clickcr` int COMMENT '-',
  `goodsnumber` int COMMENT '货号',
  `goodsweight` int COMMENT '商品重量',
  `marketprice` double COMMENT '进货价，成本',
  `shopprice` double COMMENT '售价',
  `addtime` string COMMENT '新款建档时间，在数据库里',
  `isonsale` int COMMENT '是否在售（1 在售，0 否）',
  `sales` int COMMENT '真实的销量+刷单的销量',
  `realsales` int COMMENT '实际销量',
  `extraprice` double COMMENT '特别价格（促销价）',
  `goodsno` string COMMENT '货号 ID，一个商品 ID 可能对应多个货号 ID',
  `update_time` string COMMENT '更新时间',
  `category_3_id` int COMMENT '三级分类 id',
  `category_3_name` string COMMENT '三级分类名称',
  `category_2_id` int COMMENT '二级分类 id',
  `category_2_name` string COMMENT '二级分类名称',
  `category_1_id` int COMMENT '一级分类 id',
  `category_1_name` string COMMENT '一级分类名称'
) COMMENT 'goods 维度表'

STORED AS orc;
```

表 6：地区维度表

```
-- 6.地区维度表

DROP TABLE IF EXISTS dim_commerce.dim_region_info_full;

CREATE TABLE IF NOT EXISTS `dim_commerce.dim_region_info_full` (
  `county_id` string COMMENT '区县 id',
  `county_name` string COMMENT '区县名称' ,
  `city_id` string COMMENT '城市 id' ,
  `city_name` string COMMENT '城市名称' ,
  `province_id` string COMMENT '省份 id' ,
  `province_name` string COMMENT '省份名称' ,
  `country_id` string COMMENT '国家 id',
  `country_name` string COMMENT '国家名称',
  `update_time` string COMMENT '更新时间'
)COMMENT '地区维度表'

STORED AS orc;
```

3.2.2 核心表的开发代码

表 1：sku 商品维度表

```
--核心代码

with product_sku as (
  select
    id,
    spu_id,
    name,
    catagory_id,
    brand_id,
    default_image,
    title,
    subtitle,
    price,
    weight
  from
    ods_commerce.ods_product_sku
),
product_spu as (
  select
    id as spu_id,
    name as spu_name
  from
    ods_commerce.ods_product_spu
),
product_category as (
```

```

select
    id as category_id,
    name as catagory_name
from
    ods_commerce.ods_product_category
),
product_brand as (
    select
        id as brand_id,
        name as brand_name
    from
        ods_commerce.ods_product_brand
),
sku_attr_value as (
    select
        sku_id,
        collect_set(named_struct('attr_id',attr_id, 'attr_name',attr_name, 'attr_value',attr_value)) as sku_attrs
    from
        ods_commerce.ods_product_sku_attr_value
    group by
        sku_id
)
insert overwrite table dim_commerce.dim_sku_detailed_info_full
select
    id as sku_id,
    name as sku_name,
    product_sku.catagory_id,
    product_category.catagory_name,
    product_sku.brand_id,
    product_brand.brand_name,
    default_image as sku_default_image,
    title as sku_title,
    subtitle as sku_subtitle,
    price as sku_price,
    product_sku.spu_id,
    product_spu.spu_name,
    sku_attr_value.sku_attrs
from
    product_sku
left join product_spu on product_sku.spu_id = product_spu.spu_id
left join product_category on product_sku.catagory_id = product_category.category_id
left join product_brand on product_sku.brand_id = product_brand.brand_id
left join sku_attr_value on product_sku.id = sku_attr_value.sku_id

```

;

数据验证：

sku_id	sku_name	catagory_id	catagory_name	brand_id	brand_name	sku_default_image
1	华为mate30pro 黑色,8G,128G	225	手机	2	华为	https://www.jd.com/2020-03-21/a3a0a224-caad-4af2-8eec-f62c0
2	华为mate30pro 黑色,8G,256G	225	手机	2	华为	https://www.jd.com/2020-03-21/f053e068-e43d-46e7-8d67-496
3	华为mate30pro 黑色,8G,128G	225	手机	2	华为	https://www.jd.com/2020-03-21/a3a0a224-caad-4af2-8eec-f62c0
4	华为mate30pro 黑色,8G,256G	225	手机	2	华为	https://www.jd.com/2020-03-21/f053e068-e43d-46e7-8d67-496
5	华为mate30pro 黑色,8G,128G	225	手机	2	华为	https://www.jd.com/2020-03-21/a3a0a224-caad-4af2-8eec-f62c0
6	华为mate30pro 黑色,8G,256G	225	手机	2	华为	https://www.jd.com/2020-03-21/f053e068-e43d-46e7-8d67-496
7	华为mate30pro 黑色,8G,128G	225	手机	2	华为	https://www.jd.com/2020-03-21/a3a0a224-caad-4af2-8eec-f62c0
8	华为mate30pro 黑色,8G,256G	225	手机	2	华为	https://www.jd.com/2020-03-21/f053e068-e43d-46e7-8d67-496
9	华为mate30pro 黑色,8G,128G	225	手机	2	华为	https://www.jd.com/2020-03-21/a3a0a224-caad-4af2-8eec-f62c0
10	华为mate30pro 黑色,8G,256G	225	手机	2	华为	https://www.jd.com/2020-03-21/f053e068-e43d-46e7-8d67-496

表 2: spu 商品维度表

```
-- 核心代码

with product_spu as (

    select

        id as spu_id,

        name as spu_name,

        category_id,

        brand_id,

        publish_status as spu_publish_status,

        create_time as spu_create_time,

        update_time as spu_update_time

    from

        ods_commerce.ods_product_spu

),

product_category as (

    select

        id as category_id,

        name as category_name

    from

        ods_commerce.ods_product_category

),

product_brand as (

    select

        id as brand_id,

        name as brand_name

    from

        ods_commerce.ods_product_brand

),

spu_attr_value as (

    select

        spu_id,

        collect_set(struct(attr_id, attr_name, attr_value)) as spu_attrs

    from

        ods_commerce.ods_product_spu_attr_value

    group by
```

```

        spu_id
    )
insert overwrite table dim_commerce.dim_spu_detailed_info_full
select
    product_spu.spu_id,
    spu_name,
    spu_publish_status,
    spu_create_time,
    spu_update_time,
    product_spu.category_id,
    product_category.category_name,
    product_spu.brand_id,
    product_brand.brand_name,
    spu_attr_value.spu_attrs
from
    product_spu
left join product_category on product_spu.category_id = product_category.category_id
left join product_brand on product_spu.brand_id = product_brand.brand_id
left join spu_attr_value on product_spu.spu_id = spu_attr_value.spu_id
;

```

数据验证:

spu_id	spu_name	spu_publish_status	spu_create_time	spu_update_time	category_id	category_name	brand_id	brand_name	spu_attrs
7	华为mate30pro	1	2020-03-21 00:54:21	2020-03-21 00:54:21	225	手机	2	华为	[[{"col1": "value1"}]]
8	华为mate30pro	1	2020-03-21 12:19:34	2020-03-21 12:19:34	225	手机	2	华为	[[{"col1": "value2"}]]
9	华为mate30pro	1	2020-03-21 12:34:56	2020-03-21 12:34:56	225	手机	2	华为	[[{"col1": "value3"}]]
10	华为mate30pro7	1	2020-03-21 12:46:59	2020-03-21 12:46:59	225	手机	2	华为	[[{"col1": "value4"}]]
11	华为mate30pro7	1	2020-03-21 13:07:59	2020-03-21 13:07:59	250	平板电视	2	华为	[[{"col1": "value5"}]]

表 3: 品牌维度表

```

-- 核心代码
insert overwrite table dim_commerce.dim_brand_detailed_info_full
select
    id,
    name,
    logo,
    '20220529' AS update_time
from
    ods_commerce.ods_product_brand
union all
select
    cast(supplierid as bigint) as id,
    brandtype as name,
    '' as logo,
    pt as update_time
from
    ods_commerce.ods_product_goodsbrand;

```


数据验证：

id	name	logo	update_time
182	蜜糖小米		20200601
195	VIVI SHOW		20200601
196	Left&Right		20200601
199	合美公社		20200601
201	FST		20200601
202	BKSY		20200601
203	皇后新衣		20200601
204	VIVI SHOW ROOM		20200601
205	panpan		20200601
206	卡菲		20200601

表 4：商品分类维度表

```
--核心代码

insert overwrite table dim_commerce.dim_category_detailed_info_full

select

  a.id as category_3_id,

  a.name as category_3_name,

  a.parent_id as category_2_id,

  b.name as category_2_name,

  b.parent_id as category_1_id,

  c.name as category_1_name

from

  ods_commerce.ods_product_category a

  inner join ods_commerce.ods_product_category b on a.parent_id = b.id

  inner join ods_commerce.ods_product_category c on b.parent_id = c.id;
```

数据验证：

category_3_id	category_3_name	category_2_id	category_2_name	category_1_id	category_1_name
165	电子书	22	电子书刊	1	图书、音像、电子书刊
166	网络原创	22	电子书刊	1	图书、音像、电子书刊
167	数字杂志	22	电子书刊	1	图书、音像、电子书刊
168	多媒体图书	22	电子书刊	1	图书、音像、电子书刊
169	音乐	23	音像	1	图书、音像、电子书刊
170	影视	23	音像	1	图书、音像、电子书刊
171	教育音像	23	音像	1	图书、音像、电子书刊
172	少儿	24	英文原版	1	图书、音像、电子书刊
173	商务投资	24	英文原版	1	图书、音像、电子书刊
174	英语学习与考试	24	英文原版	1	图书、音像、电子书刊

表 5：goods 维度表

```
insert overwrite table dim_commerce.dim_goods_detailed_info_full

select

  goodsid,

  typeid as brand_id,

  markid,

  goodstag,

  brandtag as brand_name,

  customtag,

  goodsname,

  clickcount,

  clickcr,
```

```

goodsnumber,
goodsweight,
marketprice,
shopprice,
addtime,
isonsale,
sales,
realsales,
extraprice,
goodsno,
dt as update_time,
nvl(category_base.category_3_id, 650) as category_3_id,
nvl(category_base.category_3_name, 'T 恤') as category_3_name,
nvl(category_base.category_2_id, 76) as category_2_id,
nvl(category_base.category_2_name, '女装') as category_2_name,
nvl(category_base.category_1_id, 9) as category_1_id,
nvl(category_base.category_1_name, '服饰内衣') as category_1_name
from
ods_commerce.ods_product_goodsinfo2 goods
left join dim_commerce.dim_category_detailed_info_full category_base on goods.goodsname = category_base.
category_3_name

```

数据验证：

goodsid	brand_id	markid	goodstag	brand_name	customtag	goodsname
404408	221	3981	十三行写字楼	上草玄田	灯光下拍照微色差以实物为准、(图中是一个色) 中高腰; 无弹力	【上草玄田】 5667
404360	201	3981	十三行写字楼	上草玄田		【上草玄田】 659
404363	268	3980	南城实力档口	MS.LEE	模特图有滤镜颜色参考实拍为主灯光问题有轻微色差	BH 【MS.LEE】 6732 复古民族风一字肩上
404314	268	3980	南城实力档口	MS.LEE	模特图有滤镜颜色参考实拍为主灯光问题有轻微色差	BH 【MS.LEE】 6857 碎花蕾丝系带露肩上
404246	268	3980	南城实力档口	w	以实拍为主灯光问题有轻微色差	bh 【W】 302 连帽网纱露衫 XJ
404341	225	3980	南城实力档口	ISSGL	模特图有滤镜颜色参考实拍为主灯光问题有轻微色差无弹力	【ISSGL】 8086 涂鸦印花字母高腰牛仔半
404326	225	3980	南城实力档口		以实拍为主灯光问题有轻微色差无弹性身高163大腿中部	【奇点】 9852 重工腰带不规则高腰半身裙
404316	225	3980	南城实力档口	优然	以实拍为主灯光问题有轻微色差无弹性	【优然】 882 纯色高腰裙裤 XJ
404315	225	3980	南城实力档口		模特图有滤镜颜色参考实拍为主灯光问题有轻微色差	【大牌小物】 77056 棉麻排扣高腰半身裙
404294	225	3980	南城实力档口		模特图有滤镜颜色参考实拍为主灯光问题有轻微色差模特身高158	【Small】 302 字母刺绣不规则牛仔高腰半

表 6：地区维度表

```

--核心代码
insert overwrite table dim_commerce.dim_region_info_full
select
a.regionid as county_id,
a.regionname as county_name,
b.regionid as city_id,
b.regionname as city_name,
c.regionid as province_id,
c.regionname as province_name,
d.regionid as country_id,
d.regionname as country_name,
a.pt as update_time
from
ods_commerce.ods_regioninfo a

```

```

inner join ods_commerce.ods_regioninfo b on a.parentid = b.regionid
inner join ods_commerce.ods_regioninfo c on b.parentid = c.regionid
inner join ods_commerce.ods_regioninfo d on c.parentid = d.regionid
;

```

数据验证：

3402	庐阳区	3401	合肥	3	安徽	1	中国	20200601
3403	瑶海区	3401	合肥	3	安徽	1	中国	20200601
3404	蜀山区	3401	合肥	3	安徽	1	中国	20200601
3405	包河区	3401	合肥	3	安徽	1	中国	20200601
3406	长丰县	3401	合肥	3	安徽	1	中国	20200601
3407	肥东县	3401	合肥	3	安徽	1	中国	20200601
3408	肥西县	3401	合肥	3	安徽	1	中国	20200601
2912	和平区	343	天津	27	天津	1	中国	20200601
2913	河西区	343	天津	27	天津	1	中国	20200601
2914	南开区	343	天津	27	天津	1	中国	20200601

3.2.3 涉及的基础知识

- 1.函数 collect_set、named_struct、cast 的使用。
- 2.多表关联的写法。
- 3.表拼接一般有两种方式，上下拼接（union all）和左右拼接（join），上下拼接在字段数不一致的情况下的处理方法。
- 4.维度退化的使用，从一张表内拆出多个字段。

4. 建设成果

- 1.业务层面：在公司层面构建了完整、统一的商品主题数据维表，这些表目前服务于 10 多条业务线，为各个业务团队提供了统一的商品数据、品牌数据，保障了业务日常工作中商品主题维度下的数据一致性。
- 2.数据层面：完成了商品维表、商品分类维表、地区维表等数据的统一建设，通过这次构建的一致性维度，保障了维度数据的一致性，也降低了这些维度的运维成本。

5. 知识要点

1.知识点回顾：表级别的整合主要有两种形式：

- 1.垂直整合，即不同来源表包含相同的数据集，只是存储的信息不同，可以整合到同一个维度模型中。
- 2.水平整合，即不同来源表包含不同的数据集，这些子集之间无交叉或存在部分交叉，如果有交叉则去重；如果无交叉，考虑不同子集的自然键是否冲突，不冲突则可以将各子集自然键作为整合后的自然键，或者将各自然键加工成一个超自

然键。

2.知识点回顾：将维度的属性层次合并到单个维度中的操作称为反规范化。

3.知识点回顾：维度设计基本方法

- 1.选择或者新建一个维度，通过之前总线矩阵的构建掌握了目前数仓架构中的维度。
- 2.确定主维表。此处主维表一般是 ODS 表，直接与业务系统同步。
- 3.确定相关维表。数仓是业务源系统的数据整合，不同业务系统或者同一业务系统中的表之间存在关联性。跟据对业务的梳理，我们可以确认哪些表和主维表存在关联关系，并选择其中的某些表用于生成维度属性。
- 4.确定维度属性。本步骤分为两阶段，第一阶段是从主维表中选择维度属性或生成新的维度属性；第二阶段是从相关维表中选择维度属性或生成新的维度属性。

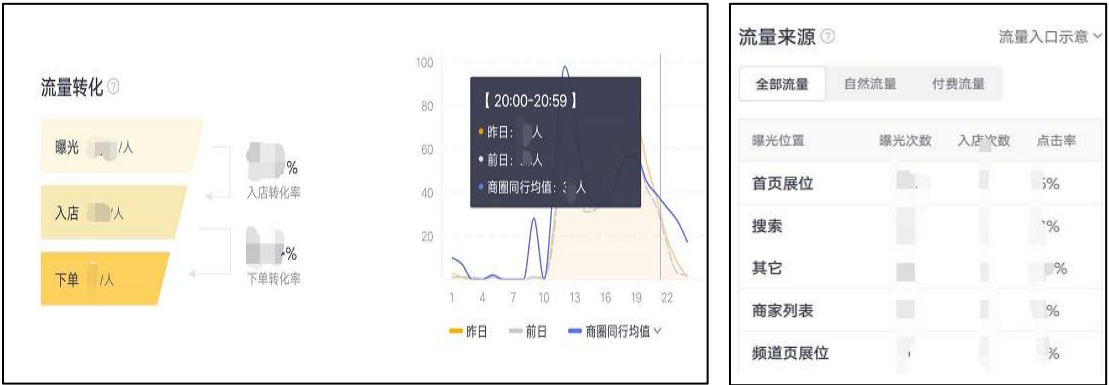
七、商户主题数据域建设

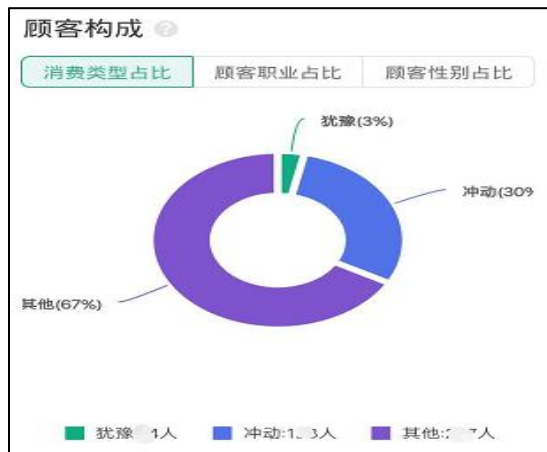
1. 建设背景

1.1 业务背景

- 1.在电商平台的商家版 APP 中，平台会为商家搭建数据指标体系，科学地指导商家的经营发展方向以及提供经营状况概览。
- 2.提升 B 端商家产品体验，通过数据呈现商家的运营动作效果，为业务产生增值收入。可提供增值服务，为平台创造营收，比如提供同类店铺的对比数据。

以下是商户关注的数字指标搭建的具体概况，经营数据一共分为以下几个板块进行建设：





具体数据指标:

1. 流量转化

曝光人数: 统计时间内, 看到店铺的顾客数。

进店转化率: 统计时间内, 看到店铺的顾客中, 进入店铺的顾客。

下单转化率: 统计时间内, 进入店铺的顾客中, 下单的顾客占比。

2. 流量来源

(1) 全部流量: 包括本店的自然流量与付费流量之和。

(2) 自然流量: 平台根据用户喜好推荐给用户或用户主动搜索店铺产生的流量, 也包括新店加权或平台奖励的流量。

(3) 曝光位置:

首页展位: 店铺在首页“顶部横幅”等位置的曝光。

搜索: 店铺在用户主动搜索商品或店铺时的曝光。

商家列表: 店铺在“首页”、“附近商家”和“附件热卖”列表中的曝光。

频道页展示：店铺在“家电”、“手机”等频道中的曝光。

其他：店铺在所有非上述渠道的曝光，包括：天降红包，订单，代金券等。



3.顾客构成

冲动的顾客下单前所花时间较少，犹豫的顾客下单前所花时间较多。

消费类型占比、顾客职业占比、顾客性别占比。

4.新老客统计

下单人数：统计时间内，下单的顾客数；

新客人数：统计时间内，首次下单的顾客数；

老客人数：统计时间内，非首次下单的顾客数。

5.复购分析

复购率：统计时间内，所有下单顾客中，在本店下单两次及以上的顾客占比。

新客复购率：统计时间内，所有首次下单的顾客中，在本店下单两次及以上的顾客占比。

老客复购率：统计时间内，所有非首次下单的顾客中，在本店下单两次及以上的顾客占比。

复购人数：统计时间内，在本店下单两次及以上的顾客数。

复购新客：统计时间内，在本店下单两次及以上的顾客中，首次下单的顾客数。

复购老客：统计时间内，在本店下单两次及以上的顾客中，非首次下单的顾客数。

6.留存分析

活跃顾客：统计时间内，下单的顾客数最后一次成交在近 30 天内的顾客数。

沉默顾客：最后一次成交在 30-60 天内的顾客数。

流失顾客：最后一次成交在 60-90 天内的顾客数。

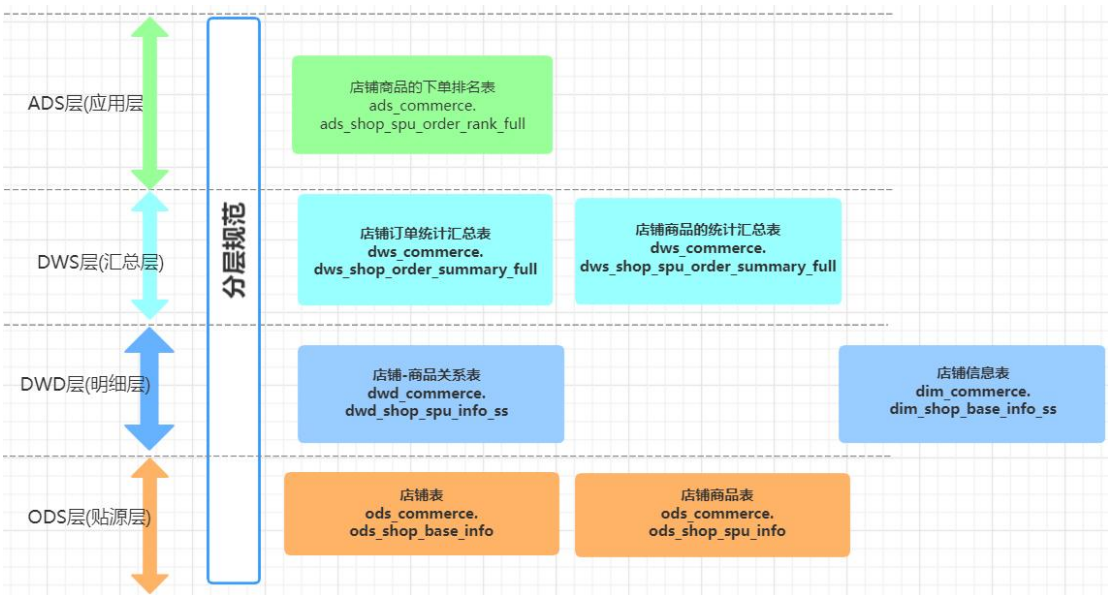
1.2 技术背景

1.需要构建公司级别的统一商户数据，以该数据为基础，继续进行用户、交易等数据域的建设。

2.该主题域的建设通常与其他数据一起，共同组成高度复用性的数据模型。

2. 数据架构

这里只列出与店铺相关的部分核心表的架构，其他跨数据域建设的数据会在其他章节讲到。



3. 数据建设

3.1 ods 层数据

3.1.1 核心表的数据结构

表 1：店铺表

```
DROP TABLE IF EXISTS ods_shop_base_info;
```



```
CREATE TABLE IF NOT EXISTS ods_shop_base_info (
  shop_id STRING COMMENT '店铺 ID',
  shop_name STRING COMMENT '店铺名称',
  shop_status int COMMENT '营业状态(0 在线营业 1 暂时歇业 2 停业)'
) COMMENT '店铺表' STORED AS textfile;
```

表 2：店铺商品表

```
DROP TABLE IF EXISTS ods_shop_spu_info;
CREATE TABLE IF NOT EXISTS ods_shop_spu_info (
  shop_id STRING COMMENT '店铺 ID',
  shop_name STRING COMMENT '店铺名称',
  spu_id int COMMENT '商品 ID'
) COMMENT '店铺商品表' STORED AS textfile;
```

3.1.2 核心表的开发代码

该表直接从业务库里面同步过来，可以采用 datax、cannal、flume 等数据同步工具获取到与业务库一样的数据。

3.1.3 涉及的基础知识

这里对数据表的字段进行探查，获取其分布情况。

3.2 dim 层&dwd 层数据

3.2.1 核心表的数据结构

表 1：店铺信息表

```
-- part1:店铺表的构建
DROP TABLE IF EXISTS dim_commerce.dim_shop_base_info_ss;
CREATE TABLE IF NOT EXISTS dim_commerce.dim_shop_base_info_ss (
  shop_id STRING COMMENT '店铺 ID',
  shop_name STRING COMMENT '店铺名称',
  shop_status int COMMENT '营业状态(0 在线营业 1 暂时歇业 2 停业)'
)
COMMENT '店铺信息表'
PARTITIONED BY (dt string comment '数据日期')
STORED AS orc;
```

表 2：店铺商品表

```
-- part2:店铺-商品关系表的构建
DROP TABLE IF EXISTS dwd_commerce.dwd_shop_spu_info_ss;
CREATE TABLE IF NOT EXISTS dwd_commerce.dwd_shop_spu_info_ss (
  shop_id STRING COMMENT '店铺 ID',
  shop_name STRING COMMENT '店铺名称',
  spu_id int COMMENT '商品 ID',
```

```

    spu_name int COMMENT '商品名称'
)
COMMENT '店铺商品表'
PARTITIONED BY (dt string comment '数据日期')
STORED AS orc;

```

3.2.2 核心表的开发代码

表 1：店铺信息表

```

--核心代码
set hive.exec.dynamic.partition.mode=nonstrict;
insert overwrite table dim_commerce.dim_shop_base_info_ss partition(dt)
select
    shop_id,
    shop_name,
    shop_status,
    date_sub(current_date(),1) as dt
from ods_commerce.ods_shop_base_info
;

```

表 2：店铺商品表

```

--核心代码
set hive.exec.dynamic.partition.mode=nonstrict;
insert overwrite table dwd_commerce.dwd_shop_spu_info_ss partition(dt)
select
    a.shop_id,
    a.shop_name,
    a.spu_id,
    c.goodsname as spu_name,
    date_sub(current_date(),1) as dt
from ods_commerce.ods_shop_spu_info as a
join (
    select shop_id,shop_status
    from dim_commerce.dim_shop_base_info_ss
    where dt=date_sub(current_date(),1) and shop_status in (0,1)
) as b
on a.shop_id=b.shop_id
left join (
    select goodsid,max(goodsname) as goodsname
    from ods_commerce.ods_product_goodsinfo
    where goodsname is not null
    group by goodsid
) as c -- 此次对脏数据进行处理
on a.spu_id=c.goodsid

```

```
;
```

3.2.3 涉及的基础知识

这里对数据表的字段进行探查，获取其分布情况。

1. 日期函数的使用：date_sub(current_date(),1)
2. 脏数据的处理方法。

3.3 dws 层数据

3.3.1 核心表的数据结构

表 1：店铺订单统计汇总表

```
--part3:店铺订单统计汇总表
DROP TABLE IF EXISTS dws_commerce.dws_shop_order_summary_full;
CREATE TABLE IF NOT EXISTS dws_commerce.dws_shop_order_summary_full (
    `shop_id` STRING COMMENT '店铺 ID',
    `shop_name` STRING COMMENT '店铺名称',
    `order_cnt` BIGINT COMMENT '下单次数',
    `order_num` BIGINT COMMENT '下单件数',
    `order_coupon_cnt` BIGINT COMMENT '使用优惠券下单次数',
    `order_total_amount` DECIMAL(20,4) COMMENT '下单订单总额',
    `order_pay_amount` DECIMAL(20,4) COMMENT '下单应付总额',
    `order_freight_amount` DECIMAL(20,4) COMMENT '下单运费金额',
    `order_promotion_amount` DECIMAL(20,4) COMMENT '下单促销优化总金额（促销价、满减、阶梯价）',
    `order_integration_amount` DECIMAL(20,4) COMMENT '下单积分抵扣总金额',
    `order_coupon_amount` DECIMAL(20,4) COMMENT '下单优惠券抵扣总金额',
    `order_discount_amount` DECIMAL(20,4) COMMENT '下单后台调整订单使用的折扣总金额',
    `refund_payment_cnt` BIGINT COMMENT '被退款次数',
    `refund_payment_num` BIGINT COMMENT '被退款件数',
    `refund_payment_amount` DECIMAL(20,4) COMMENT '被退款金额',
    `browse_cnt` BIGINT COMMENT '商品浏览次数',
    `collection_cnt` BIGINT COMMENT '商品收藏次数',
    `shopping_cart_cnt` BIGINT COMMENT '商品加入购物车次数'
)COMMENT "店铺订单统计汇总表"
stored as orc
;
```

表 2：店铺商品的统计汇总表

```
DROP TABLE IF EXISTS dws_commerce.dws_shop_spu_order_summary_full;
CREATE TABLE IF NOT EXISTS dws_commerce.dws_shop_spu_order_summary_full (
    `shop_id` STRING COMMENT '店铺 ID',
```

```

`shop_name` STRING COMMENT '店铺名称',
`spu_id` STRING COMMENT 'spu_id',
`order_cnt` BIGINT COMMENT '下单次数',
`order_num` BIGINT COMMENT '下单件数'
)COMMENT "店铺商品的统计汇总表"
stored as orc
;

```

3.3.2 核心表的开发代码

表 1：店铺订单统计汇总表

```

-- 核心代码
set hive.exec.dynamic.partition.mode=nonstrict;
insert overwrite table dws_commerce.dws_shop_order_summary_full
select
    shop_id,
    shop_name,
    sum(order_cnt) as order_cnt,
    sum(order_num) as order_num,
    sum(order_coupon_cnt) as order_coupon_cnt,
    sum(order_total_amount) as order_total_amount,
    sum(order_pay_amount) as order_pay_amount,
    sum(order_freight_amount) as order_freight_amount,
    sum(order_promotion_amount) as order_promotion_amount,
    sum(order_integration_amount) as order_integration_amount,
    sum(order_coupon_amount) as order_coupon_amount,
    sum(order_discount_amount) as order_discount_amount,
    sum(refund_payment_cnt) as refund_payment_cnt,
    sum(refund_payment_num) as refund_payment_num,
    sum(refund_payment_amount) as refund_payment_amount,
    sum(browse_cnt) as browse_cnt,
    sum(collection_cnt) as collection_cnt,
    sum(shopping_cart_cnt) as shopping_cart_cnt
from
    dwd_commerce.dwd_shop_spu_info_ss a
left join dws_commerce.dws_sku_summary_full b on a.spu_id = b.spu_id
group by
    shop_id,
    shop_name;

```

表 2：店铺商品的统计汇总表

```

-- 核心代码
insert overwrite table dws_commerce.dws_shop_spu_order_summary_full
select
    a.shop_id,
    a.shop_name,

```

```

a.spu_id,
sum(order_cnt) as order_cnt,
sum(order_num) as order_num
from
dwd_commerce.dwd_shop_spu_info_ss a
left join dws_commerce.dws_sku_summary_full b on a.spu_id = b.spu_id
group by
a.shop_id,
a.shop_name,
a.spu_id
;

```

3.3.3 涉及的基础知识

1. 不同粒度的表的构建方法

3.4 ads 层数据

3.4.1 核心表的数据结构

表 1：店铺商品的下单排名表

```

DROP TABLE IF EXISTS ads_commerce.ads_shop_spu_order_rank_full;
CREATE TABLE IF NOT EXISTS ads_commerce.ads_shop_spu_order_rank_full (
    `shop_id` STRING COMMENT '店铺 ID',
    `shop_name` STRING COMMENT '店铺名称',
    `spu_id` STRING COMMENT 'spu_id',
    `order_cnt` BIGINT COMMENT '下单次数',
    `order_num` BIGINT COMMENT '下单件数',
    `rnk` BIGINT COMMENT '排名'
) COMMENT "店铺商品的下单排名表"
stored as orc
;

```

3.4.2 核心表的开发代码

表 1：店铺商品的下单排名表

```

--核心代码
insert overwrite table ads_commerce.ads_shop_spu_order_rank_full
select
    shop_id,
    shop_name,
    spu_id,
    order_cnt,
    order_num,

```

```
rnk
from (
select
shop_id,
shop_name,
spu_id,
order_cnt,
order_num,
row_number() over (partition by shop_id order by order_cnt desc) as rnk
from dws_commerce.dws_shop_spu_order_summary_full
) as tmp where rnk<=3;
```

3.4.3 涉及的基础知识

1. 窗口函数的使用

4. 建设成果

- 1.业务层面：为公司的商户运营团队提供了基础的商户数据，为不同的业务部门提供了统一的商户数据，保障了商户维度的统一。
- 2.数据层面：与商户相关的其他数据域一起建设、完成了包括商户主题在内的多个主题域的数据建设，构建完成后输出到商户运营后台，为 1000 多万家商户提供日常经营支持。

5. 知识要点

1.知识点回顾：

声明粒度，声明粒度是维度设计的重要步骤。粒度用于确定某一事实表中的行表示什么。在选择维度或事实前必须声明粒度，因为每个候选维度或事实必须与定义的粒度保持一致。在从给定的业务过程获取数据时，原子粒度是最低级别的粒度。强烈建议从关注原子级别粒度数据开始设计，因为原子粒度数据能够承受无法预期的用户查询。

八、交易主题数据域建设

1. 建设背景

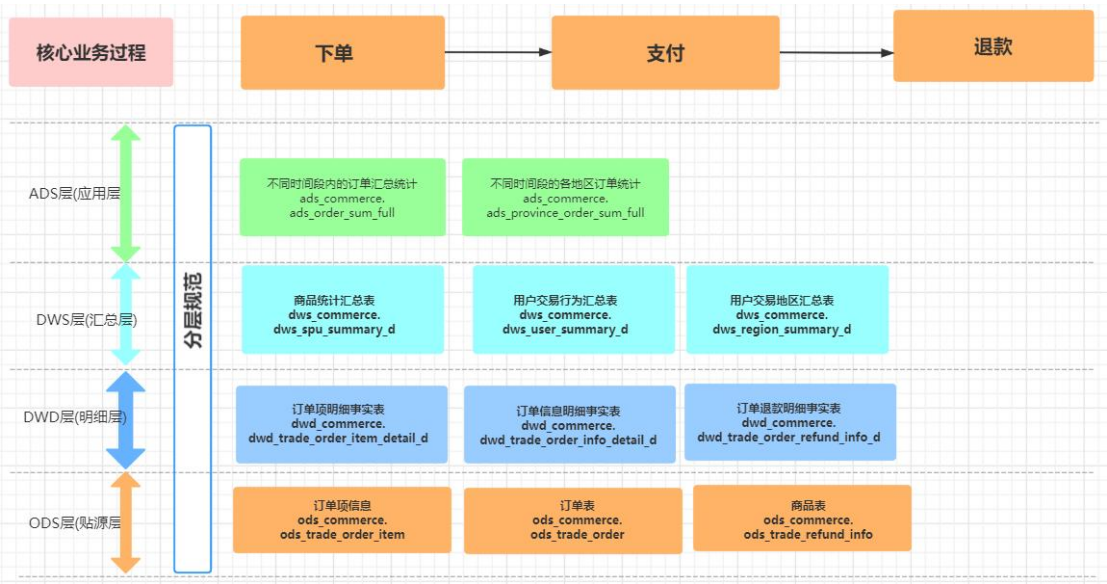
1.1 业务背景

- 1.公司领导层需要关注用户在平台的下单情况、营收情况，财务部分需要根据订单量和订单金额统计公司的财务状况。
- 2.交易和支付业务团队需要关注订单的转化率、订单支付状态等数据。

1.2 技术背景

- 1. 数据侧需要将订单下单、支付、退款等多个流程的数据整合起来，形成完整的交易数据体系。
- 2. 交易数据需要进行通用性地建设，用来服务于其他主题域、支持跨域数据的建设。

2. 数据架构



3. 数据建设

3.1 ods 层数据

3.1.1 核心表的数据结构

表 1：订单项信息表

```
DROP TABLE IF EXISTS ods_trade_order_item;

CREATE TABLE IF NOT EXISTS ods_trade_order_item (

  id BIGINT COMMENT 'id',

  order_id BIGINT COMMENT 'order_id',

  order_sn STRING COMMENT 'order_sn',

  spu_id BIGINT COMMENT 'spu_id',

  spu_name STRING COMMENT 'spu_name',

  spu_pic STRING COMMENT 'spu_pic',

  spu_brand STRING COMMENT '品牌',

  category_id BIGINT COMMENT '商品分类 id',

  sku_id BIGINT COMMENT '商品 sku 编号',

  sku_name STRING COMMENT '商品 sku 名字',

  sku_pic STRING COMMENT '商品 sku 图片',

  sku_price DOUBLE COMMENT '商品 sku 价格',

  sku_quantity INT COMMENT '商品购买的数量',

  sku_attrs_vals STRING COMMENT '商品销售属性组合（JSON）',

  promotion_amount DOUBLE COMMENT '商品促销分解金额',

  coupon_amount DOUBLE COMMENT '优惠券优惠分解金额',

  integration_amount DOUBLE COMMENT '积分优惠分解金额',

  real_amount DOUBLE COMMENT '该商品经过优惠后的分解金额',

  gift_integration INT COMMENT '赠送积分',

  gift_growth INT COMMENT '赠送成长值'

) COMMENT '订单项信息' ROW FORMAT DELIMITED FIELDS TERMINATED BY '#' STORED AS textfile;
```

表 2：订单表

```
DROP TABLE IF EXISTS ods_trade_order2;

CREATE TABLE IF NOT EXISTS ods_trade_order2 (

  id BIGINT COMMENT 'id',

  user_id BIGINT COMMENT 'member_id',

  order_sn STRING COMMENT '订单号',

  coupon_id BIGINT COMMENT '使用的优惠券',

  create_time STRING COMMENT '创建时间',

  username STRING COMMENT '用户名',

  total_amount DOUBLE COMMENT '订单总额',

  pay_amount DOUBLE COMMENT '应付总额',

  freight_amount DOUBLE COMMENT '运费金额',

  promotion_amount DOUBLE COMMENT '促销优化金额（促销价、满减、阶梯价）',

  integration_amount DOUBLE COMMENT '积分抵扣金额',

  coupon_amount DOUBLE COMMENT '优惠券抵扣金额',

  discount_amount DOUBLE COMMENT '后台调整订单使用的折扣金额',

  pay_type INT COMMENT '支付方式【1->支付宝；2->微信；3->银联； 4->货到付款；】',

  source_type INT COMMENT '订单来源[0->PC 订单；1->app 订单]',
```

```

status INT COMMENT '订单状态【0->待付款；1->待发货；2->已发货；3->已完成；4->已关闭；5->无效订单】',
delivery_company STRING COMMENT '物流公司(配送方式)',
delivery_sn STRING COMMENT '物流单号',
auto_confirm_day INT COMMENT '自动确认时间（天）',
integration INT COMMENT '可以获得的积分',
growth INT COMMENT '可以获得的成长值',
bill_type INT COMMENT '发票类型[0->不开发票；1->电子发票；2->纸质发票]',
bill_header STRING COMMENT '发票抬头',
bill_content STRING COMMENT '发票内容',
bill_receiver_phone STRING COMMENT '收票人电话',
bill_receiver_email STRING COMMENT '收票人邮箱',
receiver_name STRING COMMENT '收货人姓名',
receiver_phone STRING COMMENT '收货人电话',
receiver_post_code STRING COMMENT '收货人邮编',
receiver_province STRING COMMENT '省份/直辖市',
receiver_city STRING COMMENT '城市',
receiver_region STRING COMMENT '区',
receiver_address STRING COMMENT '详细地址',
confirm_status INT COMMENT '确认收货状态[0->未确认；1->已确认]',
delete_status INT COMMENT '删除状态【0->未删除；1->已删除】',
use_integration INT COMMENT '下单时使用的积分',
payment_time STRING COMMENT '支付时间',
delivery_time STRING COMMENT '发货时间',
receive_time STRING COMMENT '确认收货时间',
comment_time STRING COMMENT '评价时间',
modify_time STRING COMMENT '修改时间',
remark STRING COMMENT '订单备注'
) COMMENT '订单' STORED AS textfile;

```

表 3：退款信息表

```

DROP TABLE IF EXISTS ods_trade_refund_info;
CREATE TABLE IF NOT EXISTS ods_trade_refund_info (
    id BIGINT COMMENT 'id',
    order_return_id BIGINT COMMENT '退款的订单',
    refund DOUBLE COMMENT '退款金额',
    refund_sn STRING COMMENT '退款交易流水号',
    refund_status INT COMMENT '退款状态',
    refund_channel INT COMMENT '退款渠道[1-支付宝，2-微信，3-银联，4-汇款]',
    refund_content STRING COMMENT ''
) COMMENT '退款信息' STORED AS textfile;

```

3.1.2 核心表的开发代码

该表直接从业务库里面同步过来，可以采用 datax、cannal、flume 等数据同

步工具获取到与业务库一样的数据。

3.1.3 涉及的基础知识

这里对数据表的字段进行探查，获取其分布情况。

3.2 dwd 层数据

3.2.1 核心表的数据结构

表 1：订单项明细事实表

```
DROP TABLE IF EXISTS dwd_commerce.dwd_trade_order_item_detail_d;

CREATE TABLE IF NOT EXISTS dwd_commerce.dwd_trade_order_item_detail_d(

  `order_id` bigint COMMENT '订单 id',
  `order_sn` string COMMENT '订单编号',
  `user_id` string COMMENT 'user_id',
  `spu_id` bigint COMMENT 'spu_id',
  `spu_name` string COMMENT 'spu_name',
  `spu_pic` string COMMENT 'spu_pic',
  `spu_brand` string COMMENT '品牌',
  `category_id` bigint COMMENT '商品分类 id',
  `sku_id` bigint COMMENT '商品 sku 编号',
  `sku_name` string COMMENT '商品 sku 名字',
  `sku_pic` string COMMENT '商品 sku 图片',
  `sku_price` double COMMENT '商品 sku 价格',
  `sku_quantity` int COMMENT '商品购买的数量',
  `color_id` string COMMENT '颜色 ID',
  `size_id` string COMMENT '尺码 ID',
  `coupon_id` bigint COMMENT '使用的优惠券',
  `create_time` string COMMENT '创建时间',
  `total_amount` double COMMENT '订单总额',
  `pay_amount` double COMMENT '应付总额',
  `freight_amount` double COMMENT '运费金额',
  `promotion_amount` double COMMENT '促销优化金额（促销价、满减、阶梯价）',
  `integration_amount` double COMMENT '积分抵扣金额',
  `coupon_amount` double COMMENT '优惠券抵扣金额',
  `discount_amount` double COMMENT '后台调整订单使用的折扣金额',
  `receiver_province` string COMMENT '省份/直辖市',
  `receiver_city` string COMMENT '城市',
  `receiver_region` string COMMENT '区',
  `receiver_address` string COMMENT '详细地址'

) COMMENT '订单项明细事实表'

PARTITIONED BY (dt string comment '日期')

STORED AS orc;
```

表 2：订单信息明细事实表

DROP TABLE IF EXISTS dwd_commerce.dwd_trade_order_info_detail_d;	
CREATE TABLE IF NOT EXISTS dwd_commerce.dwd_trade_order_info_detail_d(	
`order_id` bigint COMMENT '订单 id',	
`user_id` bigint COMMENT 'member_id',	
`order_sn` string COMMENT '订单号',	
`coupon_id` bigint COMMENT '使用的优惠券',	
`create_time` string COMMENT '创建时间',	
`username` string COMMENT '用户名',	
`total_amount` double COMMENT '订单总额',	
`pay_amount` double COMMENT '应付总额',	
`freight_amount` double COMMENT '运费金额',	
`promotion_amount` double COMMENT '促销优化金额（促销价、满减、阶梯价）',	
`integration_amount` double COMMENT '积分抵扣金额',	
`coupon_amount` double COMMENT '优惠券抵扣金额',	
`discount_amount` double COMMENT '后台调整订单使用的折扣金额',	
`pay_type` int COMMENT '支付方式【1->支付宝；2->微信；3->银联； 4->货到付款；】',	
`source_type` int COMMENT '订单来源【0->PC 订单；1->app 订单】',	
`status` int COMMENT '订单状态【0->待付款；1->待发货；2->已发货；3->已完成；4->已关闭；5->无效订单】',	
`delivery_company` string COMMENT '物流公司(配送方式)',	
`delivery_sn` string COMMENT '物流单号',	
`auto_confirm_day` int COMMENT '自动确认时间（天）',	
`integration` int COMMENT '可以获得的积分',	
`growth` int COMMENT '可以获得的成长值',	
`receiver_name` string COMMENT '收货人姓名',	
`receiver_phone` string COMMENT '收货人电话',	
`receiver_post_code` string COMMENT '收货人邮编',	
`receiver_province` string COMMENT '省份/直辖市',	
`receiver_city` string COMMENT '城市',	
`receiver_region` string COMMENT '区',	
`receiver_address` string COMMENT '详细地址',	
`confirm_status` int COMMENT '确认收货状态【0->未确认；1->已确认】',	
`delete_status` int COMMENT '删除状态【0->未删除；1->已删除】',	
`use_integration` int COMMENT '下单时使用的积分',	
`payment_time` string COMMENT '支付时间',	
`delivery_time` string COMMENT '发货时间',	
`receive_time` string COMMENT '确认收货时间',	
`comment_time` string COMMENT '评价时间',	
`modify_time` string COMMENT '修改时间',	
`is_refund` int COMMENT '是否为退款订单',	
`refund` double COMMENT '退款金额',	
`refund_sn` string COMMENT '退款交易流水号',	
`refund_status` int COMMENT '退款状态',	

```

`refund_channel` int COMMENT '退款渠道[1-支付宝, 2-微信, 3-银联, 4-汇款]'
) COMMENT '订单信息明细事实表'
PARTITIONED BY (dt string comment '日期')
STORED AS orc;

```

表 3：订单退款明细事实表

```

DROP TABLE IF EXISTS dwd_commerce.dwd_trade_order_refund_info_d;
CREATE TABLE IF NOT EXISTS dwd_commerce.dwd_trade_order_refund_info_d(
    `id` bigint COMMENT 'id',
    `order_return_id` bigint COMMENT '退款的订单',
    `refund` double COMMENT '退款金额',
    `refund_sn` string COMMENT '退款交易流水号',
    `refund_status` int COMMENT '退款状态',
    `refund_channel` int COMMENT '退款渠道[1-支付宝, 2-微信, 3-银联, 4-汇款]',
    `refund_content` string COMMENT '',
    `receiver_province` string COMMENT '省份/直辖市',
    `receiver_city` string COMMENT '城市',
    `receiver_region` string COMMENT '区',
    `user_id` bigint COMMENT 'member_id',
    `username` string COMMENT '用户名'
) COMMENT '订单退款明细事实表'
PARTITIONED BY (dt string comment '日期')
STORED AS orc;

```

3.2.2 核心表的开发代码

表 1：订单项明细事实表

```

set hive.exec.dynamic.partition.mode=nonstrict;
insert overwrite table dwd_commerce.dwd_trade_order_item_detail_d partition(dt)
select
    a.order_id,
    a.order_sn,
    a.user_id,
    a.spu_id,
    a.spu_name,
    a.spu_pic,
    a.spu_brand,
    a.category_id,
    a.sku_id,
    a.sku_name,
    a.sku_pic,
    a.sku_price,
    a.sku_quantity,

```

```

a.color_id,
a.size_id,
b.coupon_id,
b.create_time,
b.total_amount,
b.pay_amount,
b.freight_amount,
b.promotion_amount,
b.integration_amount,
b.coupon_amount,
b.discount_amount,
b.receiver_province,
b.receiver_city,
b.receiver_region,
b.receiver_address,
b.dt
from
ods_commerce.ods_trade_order_item a
left join ods_commerce.ods_trade_order2 b on a.order_id = b.id;

```

表 2：订单信息明细事实表

```

-- 核心代码
set hive.exec.dynamic.partition.mode=nonstrict;

insert overwrite table dwd_commerce.dwd_trade_order_info_detail_d partition(dt)
select
a.id as order_id,
user_id,
order_sn,
coupon_id,
create_time,
username,
total_amount,
pay_amount,
freight_amount,
promotion_amount,
integration_amount,
coupon_amount,
discount_amount,
pay_type,
source_type,
status,
delivery_company,
delivery_sn,
auto_confirm_day,

```

```

integration,
growth,
receiver_name,
receiver_phone,
receiver_post_code,
receiver_province,
receiver_city,
receiver_region,
receiver_address,
confirm_status,
delete_status,
use_integration,
payment_time,
delivery_time,
receive_time,
comment_time,
modify_time,
if(order_return_id is not null, 1, 0) as is_refund,
refund,
refund_sn,
refund_status,
refund_channel,
a.dt
from ods_commerce.ods_trade_order2 a
left join ods_commerce.ods_trade_refund_info b on a.id = b.order_return_id;

```

表 3：订单明细事实表

```

-- 核心代码
set hive.exec.dynamic.partition.mode=nonstrict;

insert overwrite table dwd_commerce.dwd_trade_order_refund_info_d partition(dt)
select
a.id,
a.order_return_id,
a.refund,
a.refund_sn,
a.refund_status,
a.refund_channel,
a.refund_content,
b.receiver_province,
b.receiver_city,
b.receiver_region,
b.user_id,
b.username,

```



```

b.dt
from ods_commerce.ods_trade_refund_info a
left join ods_commerce.ods_trade_order2 b on b.id = a.order_return_id;

```

3.2.3 涉及的基础知识

1. 不同业务过程数据的融合

选择业务过程，业务过程是组织完成的操作型活动。理清业务过程，就是需要建立或获取性能度量，并转换为事实表中的事实。多数事实表关注某一业务过程的结果。过程的选择是非常重要的，因为过程定义了特定的设计目标以及对粒度、维度、事实的定义。

3.3 dws 层数据

3.3.1 核心表的数据结构

表 1：商品统计汇总表

```

DROP TABLE IF EXISTS dws_commerce.dws_spu_summary_d;
CREATE TABLE IF NOT EXISTS dws_commerce.dws_spu_summary_d(
    `spu_id` bigint COMMENT 'spu_id',
    `order_cnt` bigint COMMENT '下单次数',
    `order_num` bigint COMMENT '下单件数',
    `order_coupon_cnt` bigint COMMENT '使用优惠券下单次数',
    `order_total_amount` decimal(20,4) COMMENT '下单订单总额',
    `order_pay_amount` decimal(20,4) COMMENT '下单应付总额',
    `order_freight_amount` decimal(20,4) COMMENT '下单运费金额',
    `order_promotion_amount` decimal(20,4) COMMENT '下单促销优化总金额（促销价、满减、阶梯价）',
    `order_integration_amount` decimal(20,4) COMMENT '下单积分抵扣总金额',
    `order_coupon_amount` decimal(20,4) COMMENT '下单优惠券抵扣总金额',
    `order_discount_amount` decimal(20,4) COMMENT '下单后台调整订单使用的折扣总金额',
    `refund_payment_cnt` bigint COMMENT '被退款次数',
    `refund_payment_num` bigint COMMENT '被退款件数',
    `refund_payment_amount` decimal(20,4) COMMENT '被退款金额',
    `browse_cnt` bigint COMMENT '商品浏览次数',
    `collection_cnt` bigint COMMENT '商品收藏次数',
    `shopping_cart_cnt` bigint COMMENT '商品加入购物车次数'
)COMMENT "商品统计汇总表"
PARTITIONED BY (dt string comment '日期')
stored as orc;

```

表 2：用户行为汇总表

```

DROP TABLE IF EXISTS dws_commerce.dws_user_summary_d;
CREATE TABLE IF NOT EXISTS dws_commerce.dws_user_summary_d(

```

```

`user_id` bigint COMMENT 'member_id',
`browse_cnt` bigint COMMENT '商品浏览次数',
`collection_cnt` bigint COMMENT '商品收藏次数',
`shopping_cart_cnt` bigint COMMENT '商品加入购物车次数',
`purchase_cnt` bigint COMMENT '商品购买次数',
`order_cnt` bigint COMMENT '下单次数',
`order_coupon_cnt` bigint COMMENT '使用优惠券下单次数',
`order_total_amount` decimal(20,4) COMMENT '下单订单总额',
`order_pay_amount` decimal(20,4) COMMENT '下单应付总额',
`order_freight_amount` decimal(20,4) COMMENT '下单运费金额',
`order_promotion_amount` decimal(20,4) COMMENT '下单促销优化总金额（促销价、满减、阶梯价）',
`order_integration_amount` decimal(20,4) COMMENT '下单积分抵扣总金额',
`order_coupon_amount` decimal(20,4) COMMENT '下单优惠券抵扣总金额',
`order_discount_amount` decimal(20,4) COMMENT '下单后台调整订单使用的折扣总金额',
`refund_payment_cnt` bigint COMMENT '被退款次数',
`refund_payment_num` bigint COMMENT '被退款件数',
`refund_payment_amount` decimal(20,4) COMMENT '被退款金额'
) COMMENT '用户行为汇总表'
PARTITIONED BY (dt string comment '日期')
stored as orc;

```

表 3：地区汇总表

```

DROP TABLE IF EXISTS dws_commerce.dws_region_summary_d;
CREATE TABLE IF NOT EXISTS dws_commerce.dws_region_summary_d(
`province` string COMMENT '省份/直辖市',
`city` string COMMENT '城市',
`region` string COMMENT '区',
`order_cnt` bigint COMMENT '下单次数',
`order_num` bigint COMMENT '下单件数',
`order_coupon_cnt` bigint COMMENT '使用优惠券下单次数',
`order_total_amount` decimal(20,4) COMMENT '下单订单总额',
`order_pay_amount` decimal(20,4) COMMENT '下单应付总额',
`order_freight_amount` decimal(20,4) COMMENT '下单运费金额',
`order_promotion_amount` decimal(20,4) COMMENT '下单促销优化总金额（促销价、满减、阶梯价）',
`order_integration_amount` decimal(20,4) COMMENT '下单积分抵扣总金额',
`order_coupon_amount` decimal(20,4) COMMENT '下单优惠券抵扣总金额',
`order_discount_amount` decimal(20,4) COMMENT '下单后台调整订单使用的折扣总金额',
`refund_payment_cnt` bigint COMMENT '被退款次数',
`refund_payment_num` bigint COMMENT '被退款件数',
`refund_payment_amount` decimal(20,4) COMMENT '被退款金额',
`browse_cnt` bigint COMMENT '商品浏览次数',
`collection_cnt` bigint COMMENT '商品收藏次数',
`shopping_cart_cnt` bigint COMMENT '商品加入购物车次数',
`purchase_cnt` bigint COMMENT '商品购买次数'
)COMMENT '地区汇总表'

```

```
PARTITIONED BY (dt string comment '日期')
stored as orc;
```

3.3.2 核心表的开发代码

表 1: 商品统计汇总表

```
set hive.exec.dynamic.partition.mode=nonstrict;

with order_refund as (
  select
    b.spu_id,
    count(1) as refund_payment_cnt,
    sum(b.skud_quantity) as refund_payment_num,
    sum(refund) as refund_payment_amount,
    a.dt
  from
    dwd_commerce.dwd_trade_order_refund_info_d a
  left join dwd_commerce.dwd_trade_order_item_detail_d b on b.order_id = a.order_return_id
  group by
    b.spu_id,a.dt
),
order_info as (
  select
    spu_id,
    count(order_id) as order_cnt,
    sum(skud_quantity) as order_num,
    sum(if(coupon_id is not null, 1, 0)) as order_coupon_cnt,
    sum(total_amount) as order_total_amount,
    sum(pay_amount) as order_pay_amount,
    sum(freight_amount) as order_freight_amount,
    sum(promotion_amount) as order_promotion_amount,
    sum(integration_amount) as order_integration_amount,
    sum(coupon_amount) as order_coupon_amount,
    sum(discount_amount) as order_discount_amount,
    dt
  from
    dwd_commerce.dwd_trade_order_item_detail_d a
  group by
    spu_id,dt
),
user_behavior as (
  select
    cast(item_id as bigint) as spu_id,
    sum(if(type = 1, 1, 0)) as browse_cnt,
```

```

sum(if(type = 2, 1, 0)) as collection_cnt,
sum(if(type = 3, 1, 0)) as shopping_cart_cnt,
dt
from
dwd_commerce.dwd_user_behavior_detail_d
where
type in (1, 2, 3)
group by
item_id,dt
),
union_all_table as (
select
spu_id,
order_cnt,
order_num,
order_coupon_cnt,
order_total_amount,
order_pay_amount,
order_freight_amount,
order_promotion_amount,
order_integration_amount,
order_coupon_amount,
order_discount_amount,
0 as refund_payment_cnt,
0 as refund_payment_num,
0 as refund_payment_amount,
0 as browse_cnt,
0 as collection_cnt,
0 as shopping_cart_cnt,
dt
from
order_info
union all
select
spu_id,
0 as order_cnt,
0 as order_num,
0 as order_coupon_cnt,
0 as order_total_amount,
0 as order_pay_amount,
0 as order_freight_amount,
0 as order_promotion_amount,
0 as order_integration_amount,
0 as order_coupon_amount,

```

```

    0 as order_discount_amount,
    refund_payment_cnt,
    refund_payment_num,
    refund_payment_amount,
    0 as browse_cnt,
    0 as collection_cnt,
    0 as shopping_cart_cnt,
    dt
from
    order_refund
union all
select
    spu_id,
    0 as order_cnt,
    0 as order_num,
    0 as order_coupon_cnt,
    0 as order_total_amount,
    0 as order_pay_amount,
    0 as order_freight_amount,
    0 as order_promotion_amount,
    0 as order_integration_amount,
    0 as order_coupon_amount,
    0 as order_discount_amount,
    0 as refund_payment_cnt,
    0 as refund_payment_num,
    0 as refund_payment_amount,
    browse_cnt,
    collection_cnt,
    shopping_cart_cnt,
    dt
from
    user_behavior
),
sum_table as (
select
    spu_id,
    sum(order_cnt) as order_cnt,
    sum(order_num) as order_num,
    sum(order_coupon_cnt) as order_coupon_cnt,
    sum(order_total_amount) as order_total_amount,
    sum(order_pay_amount) as order_pay_amount,
    sum(order_freight_amount) as order_freight_amount,
    sum(order_promotion_amount) as order_promotion_amount,
    sum(order_integration_amount) as order_integration_amount,

```

```

sum(order_coupon_amount)      as order_coupon_amount,
sum(order_discount_amount)    as order_discount_amount,
sum(refund_payment_cnt)      as refund_payment_cnt,
sum(refund_payment_num)      as refund_payment_num,
sum(refund_payment_amount)    as refund_payment_amount,
sum(browse_cnt)              as browse_cnt,
sum(collection_cnt)          as collection_cnt,
sum(shopping_cart_cnt)       as shopping_cart_cnt,
dt
from union_all_table
group by spu_id,dt
)
insert overwrite table dws_commerce.dws_spu_summary_d partition(dt)
select
spu_id,
order_cnt,
order_num,
order_coupon_cnt,
order_total_amount,
order_pay_amount,
order_freight_amount,
order_promotion_amount,
order_integration_amount,
order_coupon_amount,
order_discount_amount,
refund_payment_cnt,
refund_payment_num,
refund_payment_amount,
browse_cnt,
collection_cnt,
shopping_cart_cnt,
dt
from sum_table
;

```

表 2：用户行为汇总表

```

set hive.exec.dynamic.partition.mode=nonstrict;
with user_behavior as (
select
cast(user_id as bigint) as user_id,
count(1) as login_cnt,
sum(if(type = 1, 1, 0)) as browse_cnt,
sum(if(type = 2, 1, 0)) as collection_cnt,
sum(if(type = 3, 1, 0)) as shopping_cart_cnt,
sum(if(type = 4, 1, 0)) as purchase_cnt,

```

```

    dt
from
    dwd_commerce.dwd_user_behavior_detail_d
group by
    user_id,dt
),
user_order as (
    select
        cast(user_id as bigint) as user_id,
        count(order_id) as order_cnt,
        sum(if(coupon_id is not null, 1, 0)) as order_coupon_cnt,
        sum(total_amount) as order_total_amount,
        sum(pay_amount) as order_pay_amount,
        sum(freight_amount) as order_freight_amount,
        sum(promotion_amount) as order_promotion_amount,
        sum(integration_amount) as order_integration_amount,
        sum(coupon_amount) as order_coupon_amount,
        sum(discount_amount) as order_discount_amount,
        dt
    from
        dwd_commerce.dwd_trade_order_item_detail_d a
    group by
        user_id,dt
),
user_order_refund as (
    select
        a.user_id,
        count(1) as refund_payment_cnt,
        sum(b.skus_quantity) as refund_payment_num,
        sum(refund) as refund_payment_amount,
        a.dt
    from
        dwd_commerce.dwd_trade_order_refund_info_d a
    left join dwd_commerce.dwd_trade_order_item_detail_d b on b.order_id = a.order_return_id
    group by
        a.user_id,a.dt
),
union_all_table as (
    select
        user_id,
        0 as browse_cnt,
        0 as collection_cnt,
        0 as shopping_cart_cnt,
        0 as purchase_cnt,

```



```
order_cnt,  
order_coupon_cnt,  
order_total_amount,  
order_pay_amount,  
order_freight_amount,  
order_promotion_amount,  
order_integration_amount,  
order_coupon_amount,  
order_discount_amount,  
0 as refund_payment_cnt,  
0 as refund_payment_num,  
0 as refund_payment_amount,  
dt
```

```
from
```

```
    user_order
```

```
union all
```

```
select
```

```
    user_id,  
    browse_cnt,  
    collection_cnt,  
    shopping_cart_cnt,  
    purchase_cnt,  
    0 as order_cnt,  
    0 as order_coupon_cnt,  
    0 as order_total_amount,  
    0 as order_pay_amount,  
    0 as order_freight_amount,  
    0 as order_promotion_amount,  
    0 as order_integration_amount,  
    0 as order_coupon_amount,  
    0 as order_discount_amount,  
    0 as refund_payment_cnt,  
    0 as refund_payment_num,  
    0 as refund_payment_amount,  
    dt
```

```
from
```

```
    user_behavior
```

```
union all
```

```
select
```

```
    user_id,  
    0 as browse_cnt,  
    0 as collection_cnt,  
    0 as shopping_cart_cnt,  
    0 as purchase_cnt,
```

```

0 as order_cnt,
0 as order_coupon_cnt,
0 as order_total_amount,
0 as order_pay_amount,
0 as order_freight_amount,
0 as order_promotion_amount,
0 as order_integration_amount,
0 as order_coupon_amount,
0 as order_discount_amount,
refund_payment_cnt,
refund_payment_num,
refund_payment_amount,
dt
from
    user_order_refund
),
sum_table as (
select
    user_id,
    sum(browse_cnt)          as browse_cnt,
    sum(collection_cnt)      as collection_cnt,
    sum(shopping_cart_cnt)   as shopping_cart_cnt,
    sum(purchase_cnt)        as purchase_cnt,
    sum(order_cnt)           as order_cnt,
    sum(order_coupon_cnt)    as order_coupon_cnt,
    sum(order_total_amount)  as order_total_amount,
    sum(order_pay_amount)    as order_pay_amount,
    sum(order_freight_amount) as order_freight_amount,
    sum(order_promotion_amount) as order_promotion_amount,
    sum(order_integration_amount) as order_integration_amount,
    sum(order_coupon_amount) as order_coupon_amount,
    sum(order_discount_amount) as order_discount_amount,
    sum(refund_payment_cnt)   as refund_payment_cnt,
    sum(refund_payment_num)   as refund_payment_num,
    sum(refund_payment_amount) as refund_payment_amount,
    dt
from union_all_table
group by user_id,dt
)
insert overwrite table dws_commerce.dws_user_summary_d partition(dt)
select
    user_id,
    browse_cnt,
    collection_cnt,

```

```

shopping_cart_cnt,
purchase_cnt,
order_cnt,
order_coupon_cnt,
order_total_amount,
order_pay_amount,
order_freight_amount,
order_promotion_amount,
order_integration_amount,
order_coupon_amount,
order_discount_amount,
refund_payment_cnt,
refund_payment_num,
refund_payment_amount,
dt
from sum_table
;

```

表 3: 地区汇总表

```

set hive.exec.dynamic.partition.mode=nonstrict;
with order_refund as (
    select
        a.receiver_province,
        a.receiver_city,
        a.receiver_region,
        count(1) as refund_payment_cnt,
        sum(b.skus_quantity) as refund_payment_num,
        sum(refund) as refund_payment_amount,
        b.dt
    from
        dwd_commerce.dwd_trade_order_refund_info_d a
        left join dwd_commerce.dwd_trade_order_item_detail_d b on b.order_id = a.order_return_id
    group by
        a.receiver_province,
        a.receiver_city,
        a.receiver_region,
        b.dt
),
order_info as (
    select
        receiver_province,
        receiver_city,
        receiver_region,
        count(order_id) as order_cnt,

```

```

sum(sku_quantity) as order_num,
sum(if(coupon_id is not null, 1, 0)) as order_coupon_cnt,
sum(total_amount) as order_total_amount,
sum(pay_amount) as order_pay_amount,
sum(freight_amount) as order_freight_amount,
sum(promotion_amount) as order_promotion_amount,
sum(integration_amount) as order_integration_amount,
sum(coupon_amount) as order_coupon_amount,
sum(discount_amount) as order_discount_amount,
dt

```

```

from

```

```

dwd_commerce.dwd_trade_order_item_detail_d a

```

```

group by

```

```

receiver_province,

```

```

receiver_city,

```

```

receiver_region,

```

```

dt

```

```

),

```

```

--应该需要做个窗口函数去重

```

```

user_address as (

```

```

select

```

```

user_id,

```

```

receiver_province,

```

```

receiver_city,

```

```

receiver_region,

```

```

dt

```

```

from

```

```

dwd_commerce.dwd_trade_order_info_detail_d

```

```

group by

```

```

user_id,

```

```

receiver_province,

```

```

receiver_city,

```

```

receiver_region,

```

```

dt

```

```

),

```

```

user_behavior as (

```

```

select

```

```

receiver_province,

```

```

receiver_city,

```

```

receiver_region,

```

```

sum(if(type = 1, 1, 0)) as browse_cnt,

```

```

sum(if(type = 2, 1, 0)) as collection_cnt,

```

```

sum(if(type = 3, 1, 0)) as shopping_cart_cnt,

```

```

sum(if(type = 4, 1, 0)) as purchase_cnt,

```

```

    a.dt
from
    dwd_commerce.dwd_user_behavior_detail_d a
left join user_address b
on cast(a.user_id as bigint) = b.user_id
group by
    receiver_province,
    receiver_city,
    receiver_region,
    a.dt
),
union_all_table as (
select
    receiver_province,
    receiver_city,
    receiver_region,
    order_cnt,
    order_num,
    order_coupon_cnt,
    order_total_amount,
    order_pay_amount,
    order_freight_amount,
    order_promotion_amount,
    order_integration_amount,
    order_coupon_amount,
    order_discount_amount,
    0 as refund_payment_cnt,
    0 as refund_payment_num,
    0 as refund_payment_amount,
    0 as browse_cnt,
    0 as collection_cnt,
    0 as shopping_cart_cnt,
    0 as purchase_cnt,
    dt
from
    order_info
union all
select
    receiver_province,
    receiver_city,
    receiver_region,
    0 as order_cnt,
    0 as order_num,
    0 as order_coupon_cnt,

```

```
0 as order_total_amount,  
0 as order_pay_amount,  
0 as order_freight_amount,  
0 as order_promotion_amount,  
0 as order_integration_amount,  
0 as order_coupon_amount,  
0 as order_discount_amount,  
refund_payment_cnt,  
refund_payment_num,  
refund_payment_amount,  
0 as browse_cnt,  
0 as collection_cnt,  
0 as shopping_cart_cnt,  
0 as purchase_cnt,  
dt
```

```
from
```

```
order_refund
```

```
union all
```

```
select
```

```
receiver_province,  
receiver_city,  
receiver_region,  
0 as order_cnt,  
0 as order_num,  
0 as order_coupon_cnt,  
0 as order_total_amount,  
0 as order_pay_amount,  
0 as order_freight_amount,  
0 as order_promotion_amount,  
0 as order_integration_amount,  
0 as order_coupon_amount,  
0 as order_discount_amount,  
0 as refund_payment_cnt,  
0 as refund_payment_num,  
0 as refund_payment_amount,  
browse_cnt,  
collection_cnt,  
shopping_cart_cnt,  
purchase_cnt,  
dt
```

```
from
```

```
user_behavior
```

```
),
```

```
sum_table as (
```

```

select
    receiver_province,
    receiver_city,
    receiver_region,
    sum(order_cnt)          as order_cnt,
    sum(order_num)          as order_num,
    sum(order_coupon_cnt)   as order_coupon_cnt,
    sum(order_total_amount) as order_total_amount,
    sum(order_pay_amount)   as order_pay_amount,
    sum(order_freight_amount) as order_freight_amount,
    sum(order_promotion_amount) as order_promotion_amount,
    sum(order_integration_amount) as order_integration_amount,
    sum(order_coupon_amount) as order_coupon_amount,
    sum(order_discount_amount) as order_discount_amount,
    sum(refund_payment_cnt) as refund_payment_cnt,
    sum(refund_payment_num) as refund_payment_num,
    sum(refund_payment_amount) as refund_payment_amount,
    sum(browse_cnt)        as browse_cnt,
    sum(collection_cnt)    as collection_cnt,
    sum(shopping_cart_cnt) as shopping_cart_cnt,
    sum(purchase_cnt)      as purchase_cnt,
    dt
from union_all_table
group by
    receiver_province,
    receiver_city,
    receiver_region,
    dt
)
insert overwrite table dws_commerce.dws_region_summary_d partition(dt)
select
    receiver_province as province,
    receiver_city as city,
    receiver_region as region,
    order_cnt,
    order_num,
    order_coupon_cnt,
    order_total_amount,
    order_pay_amount,
    order_freight_amount,
    order_promotion_amount,
    order_integration_amount,
    order_coupon_amount,
    order_discount_amount,

```



```
refund_payment_cnt,  
refund_payment_num,  
refund_payment_amount,  
browse_cnt,  
collection_cnt,  
shopping_cart_cnt,  
purchase_cnt,  
dt  
from sum_table  
;
```

3.3.3 涉及的基础知识

1. 指标体系的构建
2. 常见交易指标的加工

3.4 ads 层数据

3.4.1 核心表的数据结构

表 1：订单汇总统计表

表 2：各地区订单统计

3.4.2 核心表的开发代码

表 1：订单汇总统计表

```
--part1:订单汇总统计  
drop table if exists ads_commerce.ads_order_sum_full;  
create table ads_commerce.ads_order_sum_full  
stored as orc  
as  
select  
    recent_days,  
    sum(order_cnt) as order_cnt,  
    sum(order_total_amount) as order_total_amount,  
    sum(order_user_cnt) as order_user_cnt  
from  
(  
    select  
        1 as recent_days,  
        sum(order_cnt) as order_cnt,
```

```

sum(order_total_amount) as order_total_amount,
sum(if(order_total_amount>0,1,0)) as order_user_cnt
from
dws_commerce.dws_spu_summary_d
where dt between '2019-10-31' and '2019-10-31' group by 1
union all
select
3 as recent_days,
sum(order_cnt) as order_cnt,
sum(order_total_amount) as order_total_amount,
sum(if(order_total_amount>0,1,0)) as order_user_cnt
from
dws_commerce.dws_spu_summary_d
where dt between '2019-10-29' and '2019-10-31' group by 1
union all
select
7 as recent_days,
sum(order_cnt) as order_cnt,
sum(order_total_amount) as order_total_amount,
sum(if(order_total_amount>0,1,0)) as order_user_cnt
from
dws_commerce.dws_spu_summary_d
where dt between '2019-10-25' and '2019-10-31' group by 1
) as tmp
group by
recent_days;

-- 数据验证: OK
select * from ads_commerce.ads_order_sum_full limit 10;

```

表 2: 各地区订单统计

```

--part2:各地区订单统计
drop table if exists ads_commerce.ads_province_order_sum_full;
create table ads_commerce.ads_province_order_sum_full
stored as orc
as
select
recent_days,
province,
sum(order_cnt) as order_cnt,
sum(order_total_amount) as order_total_amount,
sum(order_user_cnt) as order_user_cnt
from
(

```

```

select
    1 as recent_days,
    province,
    sum(order_cnt) as order_cnt,
    sum(order_total_amount) as order_total_amount,
    sum(if(order_total_amount>0,1,0)) as order_user_cnt
from
    dws_commerce.dws_region_summary_d
where dt between '2019-10-31' and '2019-10-31'
group by province
union all
select
    3 as recent_days,
    province,
    sum(order_cnt) as order_cnt,
    sum(order_total_amount) as order_total_amount,
    sum(if(order_total_amount>0,1,0)) as order_user_cnt
from
    dws_commerce.dws_region_summary_d
where dt between '2019-10-29' and '2019-10-31'
group by province
union all
select
    7 as recent_days,
    province,
    sum(order_cnt) as order_cnt,
    sum(order_total_amount) as order_total_amount,
    sum(if(order_total_amount>0,1,0)) as order_user_cnt
from
    dws_commerce.dws_region_summary_d
where dt between '2019-10-25' and '2019-10-31'
group by province
) as tmp
group by
    recent_days,
    province;

```

3.4.3 涉及的基础知识

1. 企业里面通常会涉及不同时间段内的订单指标的加工，满足业务侧的分析诉求。
2. 订单数据与地理位置结合起来分析，有助于优化平台的分区域运营策略。

4. 建设成果

- 1.业务层面：为公司的管理层提供了公司的日常营收数据，为财务部门提供了财务营收数据，有效地支撑了上层决策。
- 2.数据层面：构建了完整的交易主题数据，涵盖了下单、支付、退款等核心业务流程。与用户、商户数据进行多主题的综合建设，覆盖了日常核心的分析场景，提升了数据分析团队的工作效率。

5. 知识要点

1.知识点回顾：

选择业务过程，业务过程是组织完成的操作型活动。理清业务过程，就是需要建立或获取性能度量，并转换为事实表中的事实。多数事实表关注某一业务过程的结果。过程的选择非常重要的，因为过程定义了特定的设计目标以及对粒度、维度、事实的定义。

2.知识点回顾：

确认维度（描述环境），维度提供围绕某一业务过程事件所涉及的“谁、什么、何处、何时、为什么、如何”等背景。维度表包含分析应用所需要的用于过滤及分类事实的描述性属性。牢牢掌握事实表的粒度，就能够将所有可能存在的维度区分开来。

3.知识点回顾：

确认事实（用于度量）事实，涉及来自业务过程事件的度量，基本上都是以数据值表示。一个事实表行与按照事实表粒度描述的度量事件之间存在一对一关系，因此事实表对应一个物理可观察的事件。在事实表内，所有事实只允许与声明的粒度保持一致。

九、营销主题数据域建设

1. 建设背景

1.1 业务背景

- 1.营销概况数据：商家系统内会嵌入营销数据，并进行可视化呈现形式：



数据指标:

营销订单: 本店补贴金额>0 的完成订单数;

营销订单比: 本店商家补贴金额>0 的完成订单数/本店完成订单数;

营销订单实付: 营销订单的顾客实际支付的金额;

商家补贴: 营销订单商家在该营销活动上的总补贴金额;

营销力度: 商家补贴总额/营销订单的顾客应付金额;

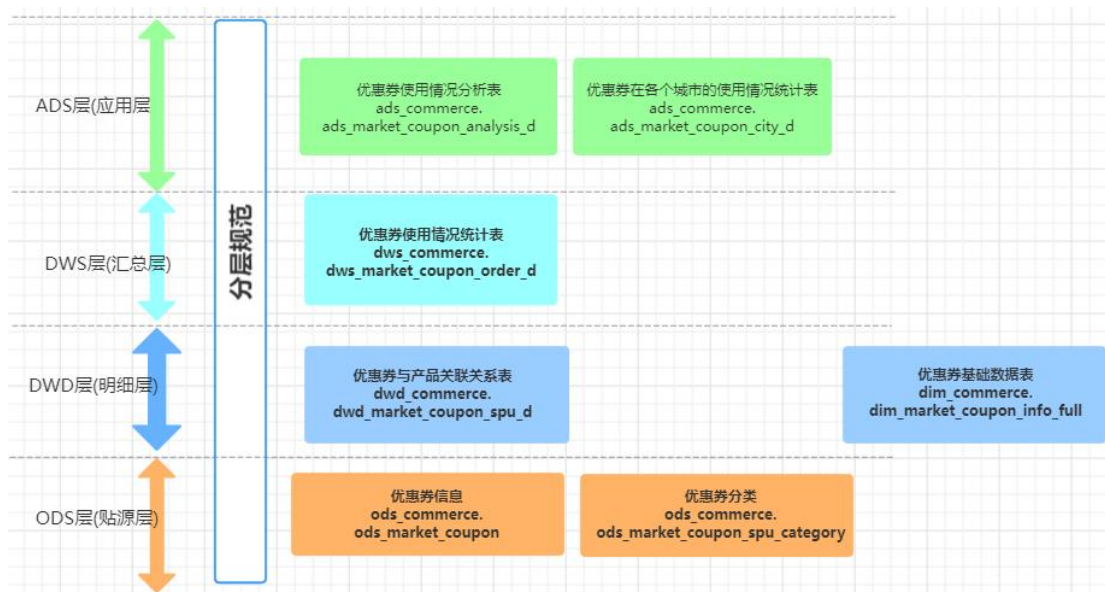
投入产出比: 营销订单顾客实际支付的金额/营销订单商家的总补贴金额。

2.营销运营: 运营团队的同事会关注优惠券类的营销活动的带来的效果, 需要数据侧对优惠券数据进行统计。

1.2 技术背景

1. 数据侧需要对优惠券主题数据进行建设, 满足业务方的日常用数需求。
2. 优惠券数据与交易数据密切相关, 建设时会引用订单数据。

2. 数据架构



3. 数据建设

3.1 ods 层数据

3.1.1 核心表的数据结构

表 1：优惠券信息表

```

DROP TABLE IF EXISTS ods_market_coupon;

CREATE TABLE IF NOT EXISTS ods_market_coupon (

  id BIGINT COMMENT 'id',
  coupon_type INT COMMENT '优惠券类型[0->全场赠券; 1->会员赠券; 2->购物赠券; 3->注册赠券]',
  coupon_img STRING COMMENT '优惠券图片',
  coupon_name STRING COMMENT '优惠券名字',
  num INT COMMENT '数量',
  amount DOUBLE COMMENT '金额',
  per_limit INT COMMENT '每人限领张数',
  min_point DOUBLE COMMENT '使用门槛',
  start_time STRING COMMENT '开始时间',
  end_time STRING COMMENT '结束时间',
  use_type INT COMMENT '使用类型[0->全场通用; 1->指定分类; 2->指定商品]',
  note STRING COMMENT '备注',
  publish_count INT COMMENT '发行数量',
  use_count INT COMMENT '已使用数量',
  receive_count INT COMMENT '领取数量',
  enable_start_time STRING COMMENT '可以领取的开始日期',
  enable_end_time STRING COMMENT '可以领取的结束日期',
  code STRING COMMENT '优惠码',

```

```

member_level INT COMMENT '可以领取的会员等级[0->不限等级, 其他-对应等级]',
publish INT COMMENT '发布状态[0-未发布, 1-已发布]'
) COMMENT '优惠券信息' STORED AS textfile;

```

表 2: 优惠券分类关联

```

DROP TABLE IF EXISTS ods_market_coupon_spu_category;
CREATE TABLE IF NOT EXISTS ods_market_coupon_spu_category (
    id BIGINT COMMENT 'id',
    coupon_id BIGINT COMMENT '优惠券 id',
    category_id BIGINT COMMENT '产品分类 id',
    category_name STRING COMMENT '产品分类名称'
) COMMENT '优惠券分类关联' STORED AS textfile;

```

3.1.2 核心表的开发代码

该表直接从业务库里面同步过来, 可以采用 datax、cannal、flume 等数据同步工具获取到与业务库一样的数据。

3.1.3 涉及的基础知识

这里对数据表的字段进行探查, 获取其分布情况。

3.2 dim 层&dwd 层数据

3.2.1 核心表的数据结构

表 1: 优惠券基础信息表

```

-- part1: 优惠券基础数据表的构建
DROP TABLE IF EXISTS dim_commerce.dim_market_coupon_info_full;
CREATE TABLE IF NOT EXISTS dim_commerce.dim_market_coupon_info_full (
    coupon_id BIGINT COMMENT 'id',
    coupon_type INT COMMENT '优惠券类型[0->全场赠券; 1->会员赠券; 2->购物赠券; 3->注册赠券]',
    coupon_img STRING COMMENT '优惠券图片',
    coupon_name STRING COMMENT '优惠券名字',
    num INT COMMENT '数量',
    amount DOUBLE COMMENT '金额',
    per_limit INT COMMENT '每人限领张数',
    min_point DOUBLE COMMENT '使用门槛',
    start_time STRING COMMENT '开始时间',
    end_time STRING COMMENT '结束时间',
    use_type INT COMMENT '使用类型[0->全场通用; 1->指定分类; 2->指定商品]',
    note STRING COMMENT '备注',
    publish_count INT COMMENT '发行数量',
    use_count INT COMMENT '已使用数量',
    receive_count INT COMMENT '领取数量',
    enable_start_time STRING COMMENT '可以领取的开始日期',

```

```

enable_end_time STRING COMMENT '可以领取的结束日期',
code STRING COMMENT '优惠码',
member_level INT COMMENT '可以领取的会员等级[0->不限等级, 其他-对应等级]',
publish INT COMMENT '发布状态[0-未发布, 1-已发布]',
category_id BIGINT COMMENT '产品分类 id',
category_name STRING COMMENT '产品分类名称'
)
COMMENT '优惠券基础信息表'
STORED AS orc;

```

表 2：优惠券与产品关联关系表

```

-- part2:优惠券-商品关系表的构建
DROP TABLE IF EXISTS dwd_commerce.dwd_market_coupon_spu_d;
CREATE TABLE IF NOT EXISTS dwd_commerce.dwd_market_coupon_spu_d (
    id BIGINT COMMENT 'id',
    coupon_id BIGINT COMMENT '优惠券 id',
    coupon_name BIGINT COMMENT '优惠券名称',
    spu_id BIGINT COMMENT '商品 id',
    spu_name STRING COMMENT '商品名称'
)
COMMENT '优惠券与产品关联关系表'
PARTITIONED BY (dt string comment '日期')
STORED AS orc;

```

3.2.2 核心表的开发代码

表 1：优惠券基础信息表

```

--核心代码
insert overwrite table dim_commerce.dim_market_coupon_info_full
select
    a.id as coupon_id,
    coupon_type,
    coupon_img,
    coupon_name,
    num,
    amount,
    per_limit,
    min_point,
    start_time,
    end_time,
    use_type,
    note,
    publish_count,
    use_count,

```



```

receive_count,
enable_start_time,
enable_end_time,
code,
member_level,
publish,
b.category_id,
b.category_name
from ods_commerce.ods_market_coupon as a
left join ods_commerce.ods_market_coupon_spu_category as b
on a.id=b.coupon_id
;

```

表 2：优惠券与产品关联关系表

```

set hive.exec.dynamic.partition.mode=nonstrict;
insert overwrite table dwd_commerce.dwd_market_coupon_spu_d partition(dt)
select
a.id,
a.coupon_id,
b.coupon_name,
a.spu_id,
a.spu_name,
date_sub(current_date(),1) as dt
from ods_commerce.ods_market_coupon_spu as a
left join dim_commerce.dim_market_coupon_info_full as b
on a.coupon_id=b.coupon_id
;

```

3.2.3 涉及的基础知识

1. 不同粒度的数据的构建

声明粒度，声明粒度是维度设计的重要步骤。粒度用于确定某一事实表中的行表示什么。在选择维度或事实前必须声明粒度，因为每个候选维度或事实必须与定义的粒度保持一致。在从给定的业务过程获取数据时，原子粒度是最低级别的粒度。强烈建议从关注原子级别粒度数据开始设计，因为原子粒度数据能够承受无法预期的用户查询。

3.3 dws 层数据

3.3.1 核心表的数据结构

表 1：优惠券使用情况统计表

```

DROP TABLE IF EXISTS dws_commerce.dws_market_coupon_order_d;

CREATE TABLE IF NOT EXISTS dws_commerce.dws_market_coupon_order_d (
    coupon_type INT COMMENT '优惠券类型[0->全场赠券; 1->会员赠券; 2->购物赠券; 3->注册赠券]',
    coupon_id BIGINT COMMENT '优惠券 id',
    coupon_name string COMMENT '优惠券名称',
    receiver_province string COMMENT '省份/直辖市',
    receiver_city string COMMENT '城市',
    coupon_amount double COMMENT '优惠券抵扣金额',
    coupon_order_cnt BIGINT COMMENT '用券订单数',
    coupon_refund_cnt BIGINT COMMENT '用券退单数'
)
COMMENT '优惠券使用情况统计表'
PARTITIONED BY (dt string comment '日期')
STORED AS orc;

```

3.3.2 核心表的开发代码

表 1: 优惠券使用情况统计表

```

set hive.exec.dynamic.partition.mode=nonstrict;

insert overwrite table dws_commerce.dws_market_coupon_order_d partition(dt)

select
    a.coupon_type
    ,a.coupon_id
    ,coalesce(a.coupon_name,concat('优惠券',a.coupon_id)) as coupon_name
    ,coalesce(b.receiver_province,'0') as receiver_province
    ,coalesce(b.receiver_city,'0') as receiver_city
    ,sum(b.coupon_amount) as coupon_amount
    ,count(distinct b.order_id) as coupon_order_cnt
    ,count(distinct if(b.is_refund=1,b.order_id,null)) as coupon_refund_cnt
    ,date_sub(current_date(),1) as dt
from dim_commerce.dim_market_coupon_info_full as a
join dwd_commerce.dwd_trade_order_info_detail_d as b
on a.coupon_id=b.coupon_id

group by
    a.coupon_type
    ,a.coupon_id
    ,coalesce(a.coupon_name,concat('优惠券',a.coupon_id))
    ,coalesce(b.receiver_province,'0')
    ,coalesce(b.receiver_city,'0')
;

```

3.3.3 涉及的基础知识

1. coalesce 函数的使用

2. concat 函数的使用

3.4 ads 层数据

3.4.1 核心表的数据结构

表 1：优惠券使用情况统计表

```
--场景 1：获取每种类别的消费券在每个省份、每个城市的下单数排名前 3 的数据，要求输出券类型、券 ID 和名称、省份、城市、
      下单订单数、排名

DROP TABLE IF EXISTS ads_commerce.ads_market_coupon_analysis_d;

CREATE TABLE IF NOT EXISTS ads_commerce.ads_market_coupon_analysis_d (
    coupon_type INT COMMENT '优惠券类型[0->全场赠券；1->会员赠券；2->购物赠券；3->注册赠券]',
    coupon_id BIGINT COMMENT '优惠券 id',
    coupon_name string COMMENT '优惠券名称',
    receiver_province string COMMENT '省份/直辖市',
    receiver_city string COMMENT '城市',
    coupon_order_cnt BIGINT COMMENT '用券订单数',
    rank BIGINT COMMENT '用券订单量排名'
)
COMMENT '优惠券使用情况统计表'
STORED AS orc;
```

表 2：优惠券在各个城市的使用情况统计表

```
--场景 2：获取消费券在每个省份、每个城市的下单数排名前 3 的数据，要求输出省份、城市、下单订单数、排名，目的是为了看优
      惠券在各个城市的投放效果

DROP TABLE IF EXISTS ads_commerce.ads_market_coupon_city_d;

CREATE TABLE IF NOT EXISTS ads_commerce.ads_market_coupon_city_d (
    receiver_province string COMMENT '省份/直辖市',
    receiver_city string COMMENT '城市',
    coupon_order_cnt BIGINT COMMENT '用券订单数',
    rank BIGINT COMMENT '用券订单量排名'
)
COMMENT '优惠券在各个城市的使用情况统计表'
STORED AS orc;
```

3.4.2 核心表的开发代码

表 1：优惠券使用情况统计表

```
--核心代码

with tmp as(

select
coupon_type
,coupon_id
```

```

,coupon_name
,receiver_province
,receiver_city
,coupon_order_cnt
,row_number() over (partition by coupon_type,receiver_province,receiver_city order by coupon_order_cnt de
    sc) as rnk
from dws_commerce.dws_market_coupon_order_d
)
insert overwrite table ads_commerce.ads_market_coupon_analysis_d
select
coupon_type
,coupon_id
,coupon_name
,receiver_province
,receiver_city
,coupon_order_cnt
,rnk
from tmp where rnk<=3
;

```

表 2：优惠券在各个城市的使用情况统计表

```

-- 核心代码
with ta as(
select
receiver_province
,receiver_city
,sum(coupon_order_cnt) as coupon_order_cnt
from dws_commerce.dws_market_coupon_order_d
group by
receiver_province
,receiver_city
)
,tb as(
select
receiver_province
,receiver_city
,coupon_order_cnt
,row_number() over (partition by receiver_province,receiver_city order by coupon_order_cnt desc) as rnk
from ta
)
insert overwrite table ads_commerce.ads_market_coupon_city_d
select
receiver_province
,receiver_city
,coupon_order_cnt

```

```
,rnk
from tb where rnk<=3
;
```

3.4.3 涉及的基础知识

1. with 语法
2. 窗口函数的使用

4. 建设成果

- 1.业务层面：构建了完善的优惠券数据，为营销团队提供了日常分析数据，原本他们每周需要花3天时间从不同的系统里面获取到优惠券的统计数据统计,现在只需要0.5天即可获取到他们的数据，极大地提升了他们日常取数的效率、有力地支撑了日常营销活动的效果回收。
- 2.数据层面：完成了优惠券数据的建设、丰富了营销主题域的数据。

5. 知识要点

- 1.dim 层建设：构建通用的维表，可以为不同的业务线提供数据服务。
- 2.跨主题域的数据建设：主要是营销域、交易域的联合建设，充分体现了分层建模式的优越性。

十、数据节点部署

1. SQL 任务调度部署

1.1.Dolpinscheduler 配置数据源

1) 点击数据源中心-创建数据源



2) 创建 Hive JDBC 数据源，创建完成后点击测试连接，显示成功，连接测试成

功

编辑数据源

* 数据源

HIVE/IMPALA

* 数据源名称

ds_hive

描述

hive数据源

* IP主机名

* 端口

10000

* 用户名

密码

请输入密码

* 数据库名

ods_commerce

jdbc连接参数

请输入格式为 {"key1": "value1", "key2": "value2"...} 连接参数

取消

测试连接

编辑

3) 创建 Spark Thrift Server 数据源，创建完成后点击测试连接，显示成功，连接测试成功

编辑数据源

* 数据源

HIVE/IMPALA

* 数据源名称

ds_spark_thrift

描述

* IP主机名

* 端口

10000

* 用户名

密码

请输入密码

* 数据库名

ods_commerce

jdbc连接参数

请输入格式为 {"key1": "value1", "key2": "value2"...} 连接参数

取消

测试连接

编辑

1.2 创建项目

点击项目管理->创建项目

DolphinScheduler

首页

项目管理

资源中心

数据源中心

监控中心

中文

ds_teacher

项目

创建项目

请输入关键词

编号	项目名称	所属用户	工作流定义数	正在运行的流程数	描述	创建时间	更新时间	操作
1	ds_warehouse_commerce	ds_teacher	1	0	演示大数据库-电商数仓项目	2022-07-31 11:03:41	2022-07-31 11:03:41	+ -

10条/页

1

前往 1 页

编辑

* 项目名称

ds_warehouse_commerce

* 所属用户

ds_teacher

描述

滄生大数据-电商数仓项目

取消

编辑

1.3 创建工作流 Demo

1) 点击对应的项目名称，进入项目管理界面

项目							
创建项目							
编号	项目名称	所属用户	工作流定义数	正在运行的流数	描述	创建时间	更新时间
1	ds_warehouse_commerce	ds_teacher	1	0	滄生大数据-电商数仓项目	2022-07-31 11:03:41	2022-07-31 11:03:41
10条/页 < 1 > 前往 1 页							

2) 点击工作流定义->创建工作流

工作流定义									
创建工作流 导入工作流									
☐	编号	工作流名称	状态	创建时间	更新时间	描述	创建用户	修改用户	定时状态
☐	14	dag_commerce_warehouse	上传	2022-07-31 11:13:31	2022-07-31 18:12:52	-	ds_teacher	ds_teacher	-
删除 导出 批量复制									
10条/页 < 1 > 前往 1 页									

创建工作流

工具栏

SHELL

SUB_PROCESS

PROCEDURE

SQL

SPARK

FUNK

MR

PYTHON

DEPENDENT

HTTP

DATA

PIGEON

SOOOP

CONDITIONS

SWITCH

WATERDROP

画布

缩放

1

0.2

3) 创建一个 shell task

当前节点设置

节点名称: 测试

运行状态: ☒ 正常 ☐ 禁止执行

描述: 请输入描述

任务优先级: MEDIUM

Worker分组: default 环境名称: 测试环境

失败重试次数: 0 (次) 失败重试间隔: 1 (秒)

定时执行时间: 0 (秒)

超时告警: ☐

脚本:

```
1. echo "hello"
```

资源: 请选择资源

自定义参数: [+](#)

前置任务: 请选择

4) 点击保存，需要指定 dag 名字

设置DAG图名称

请输入名称(必填)

请输入描述(选填)

选择租户: default

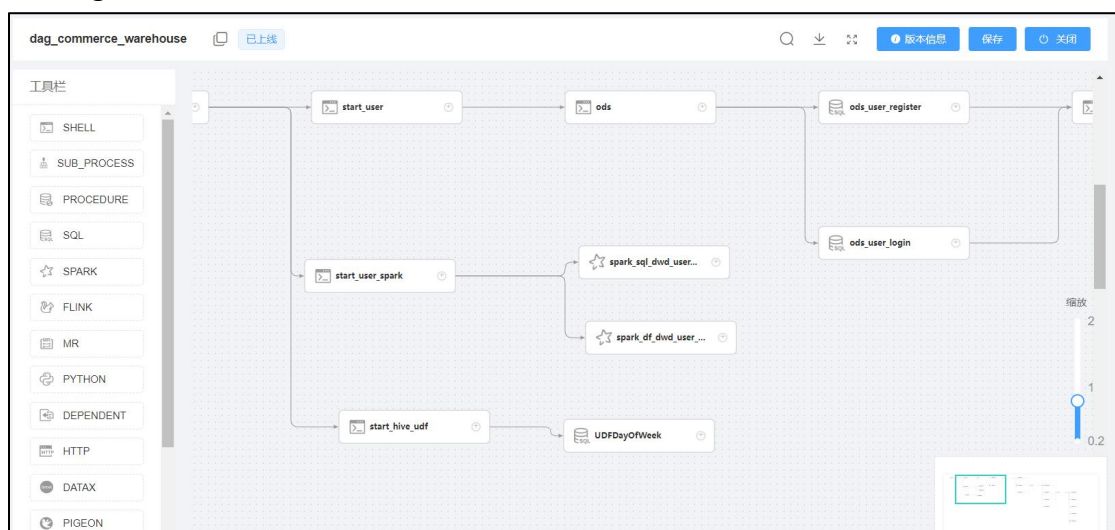
超时告警: ☐

设置全局: [+](#)

取消 添加

1.4 创建数仓项目 workflow

1) Dag 流程图



2) Shell 脚本 task 模板

当前节点设置

节点名称

运行标志 ☒ 正常 ☐ 禁止执行

描述

任务优先级 MEDIUM

Worker分组 default 环境名称 请选择

失败重试次数 1 (次) 失败重试间隔 1 (分)

延时执行时间 0 (分)

超时告警 ☒

超时策略 ☒ 超时告警 ☒ 超时失败

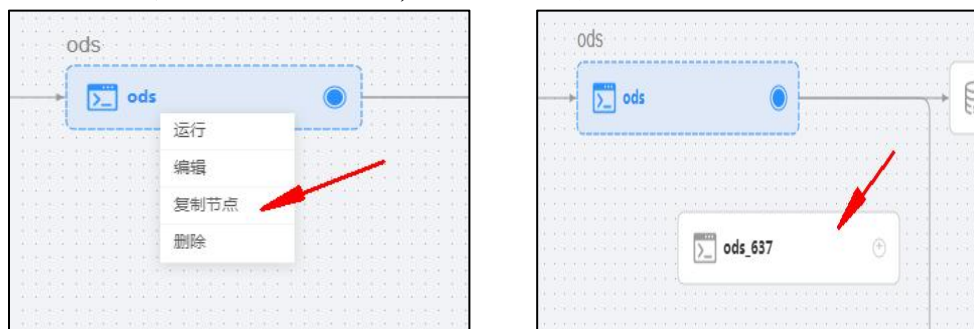
超时长 30 分

脚本

```
1 echo "commerce_warehouse_start"
```

资源 请选择资源

注：task 都是可以复制的，当我们需要创建另外一个 task 的时候，不需要新建，可以从已有的 task 复制一个，再进行编辑



3) Hive SQL task 模板

当前节点设置

节点名称: ods_user_login

运行标志: ☒ 正常 ☐ 禁止执行

描述: ods_user_login-导入

任务优先级: MEDIUM

Worker分组: default 环境名称: 请选择

失败重试次数: 3 (次) 失败重试间隔: 5 (分)

延时执行时间: 0 (分)

超时告警: ☒ 超时告警 ☐ 超时失败

超时策略: ☒ 超时告警 ☐ 超时失败

超时时长: 30 分

数据源: HIVE ds_hive

sql类型: 非查询

sql参数: 请输入格式为 key1=value1,key2=value2...

sql语句:


```

1 insert overwrite table ods_commerce.ods_user_login partition(dt)
2 select
3   user_id,
4   null as login_name,
5   null as nick_name,
6   null as name,
7   null as phone_num,
8   null as email,
9   null as user_level,
10  null as birthday,
    
```

4) Spark SQL 模板

当前节点设置

节点名称: dwd_user_register_full

运行标志: ☒ 正常 ☐ 禁止执行

描述: 用户注册累积全量表

任务优先级: MEDIUM

Worker分组: default 环境名称: 请选择

失败重试次数: 3 (次) 失败重试间隔: 5 (分)

延时执行时间: 0 (分)

超时告警: ☒ 超时告警 ☐ 超时失败

超时策略: ☒ 超时告警 ☐ 超时失败

超时时长: 30 分

数据源: HIVE ds_spark_thrift

sql类型: 非查询

sql参数: 请输入格式为 key1=value1,key2=value2...

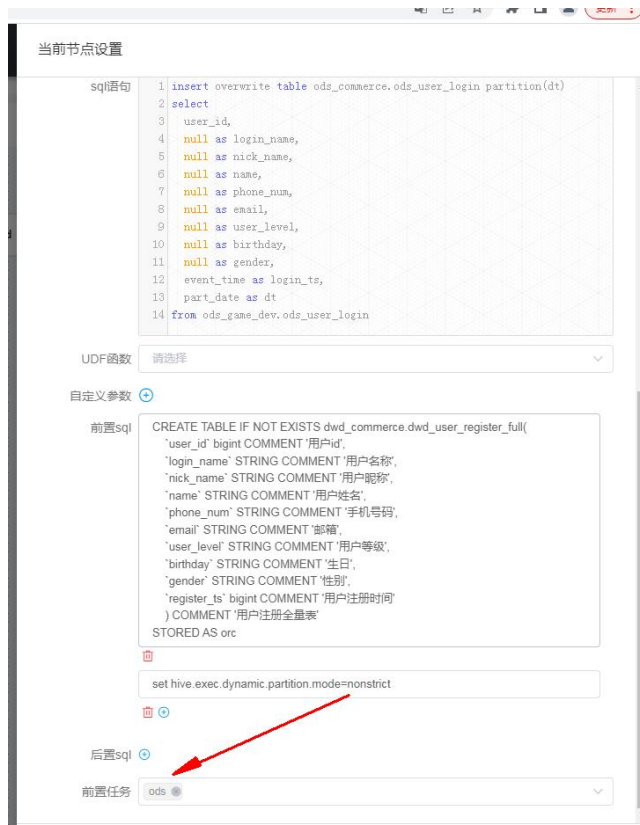
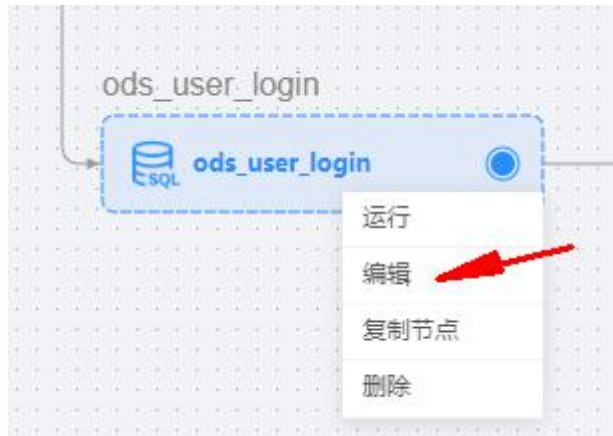
sql语句:


```

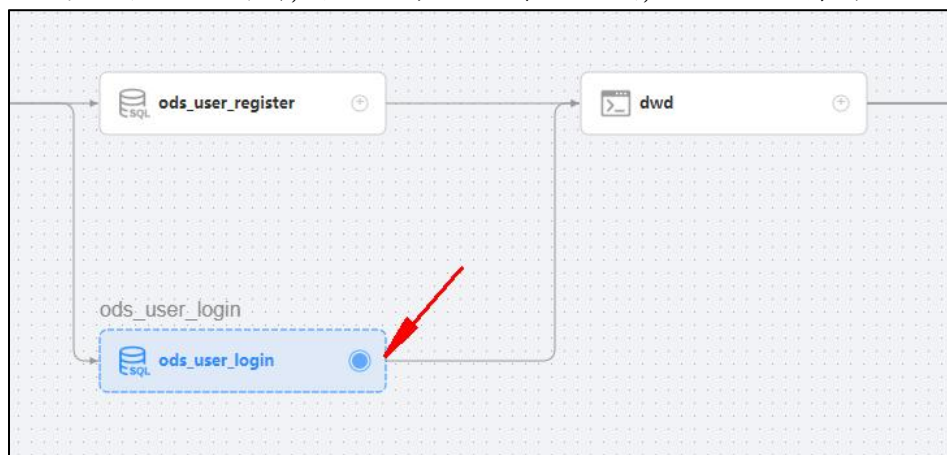
1 select
2   user_id,
3   s.login_name,
4   s.nick_name,
5   s.name,
6   s.phone_num,
7   s.email,
8   s.user_level,
9   s.birthday,
10  s.gender,
11  s.register_ts
    
```

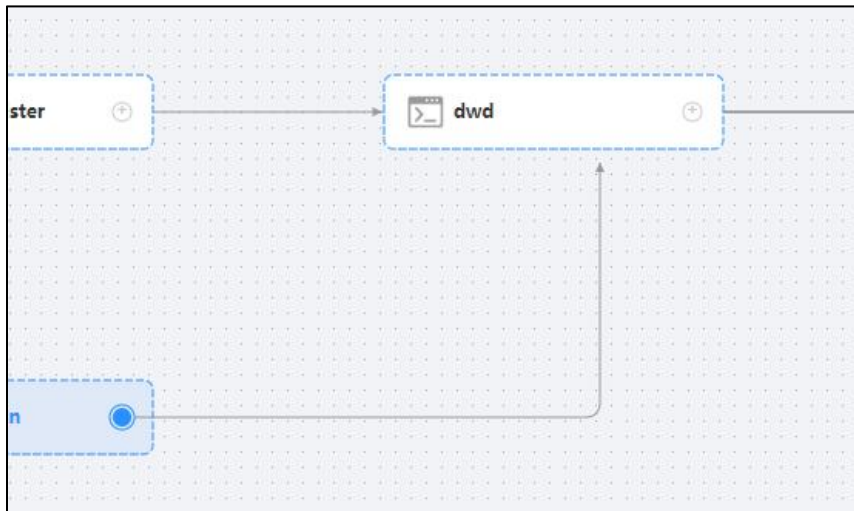
5) 设置依赖

方式 1: 右键点击创建好的 task, 选择编辑, 在编辑框内下滑到最后, 可设置该 task 的前置任务



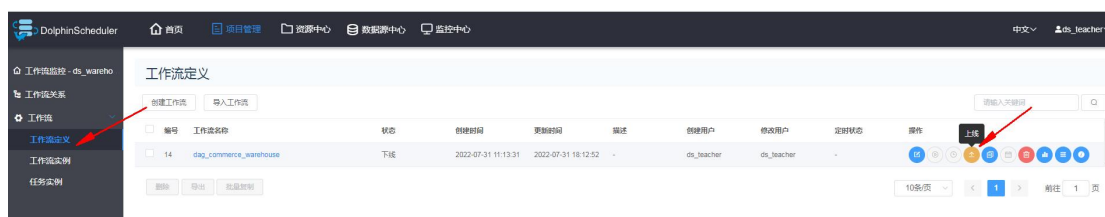
方式2：点击小圆圈，然后会产生一条连接线，直接连接需要依赖的任务即可





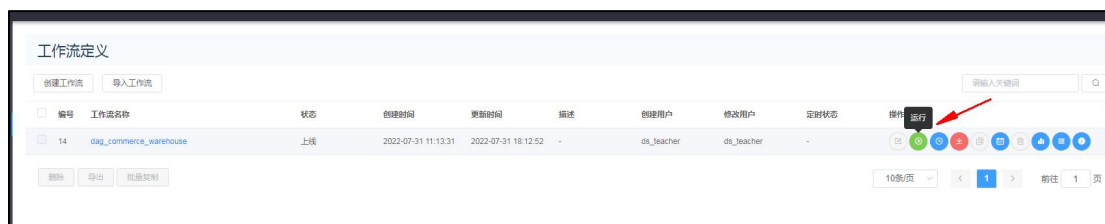
6) 上线

当所有的任务和任务依赖都配置好后，该 DAG 就可以上线了
点击 workflow 定义，找到创建好的 workflow 名称，点击上线

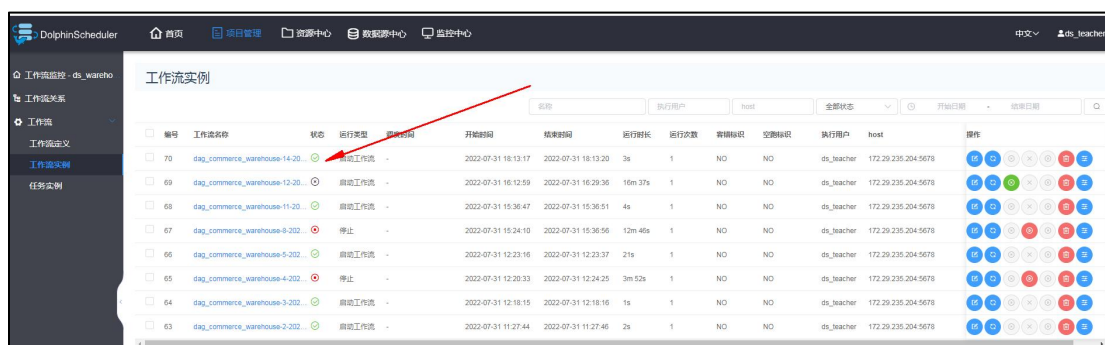


7) 运行和查看日志

上线完成后，会出现运行按钮，点击运行

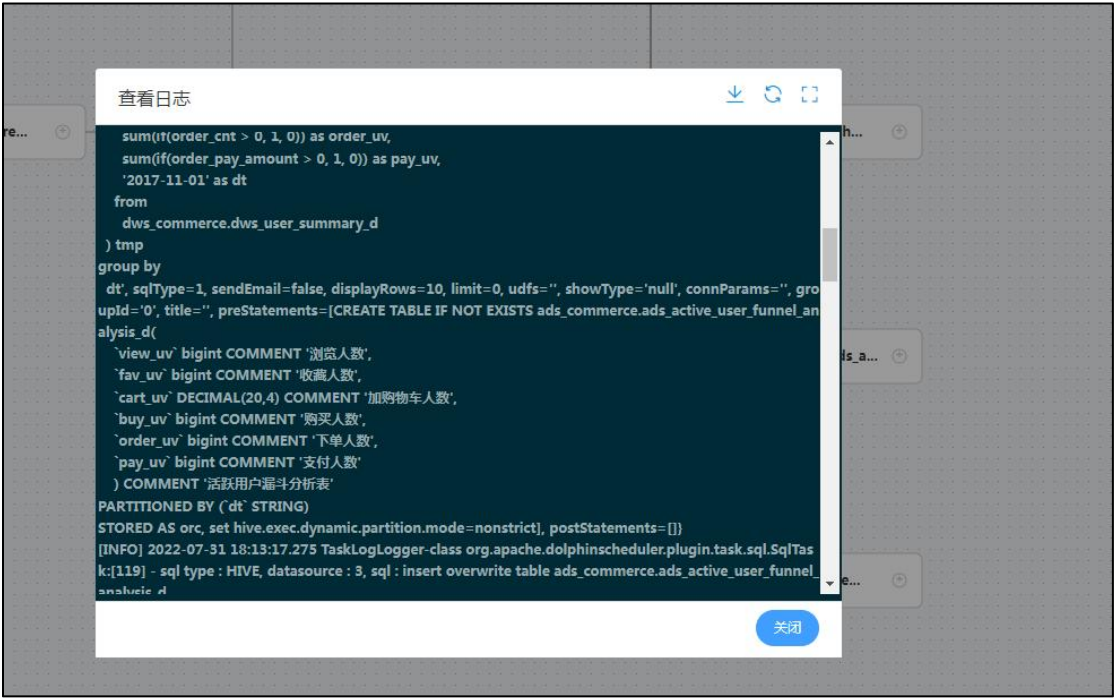


点击 workflow 实例可查看运行中或运行完成的工作流状态



找到对应开始时间的工作流名称，点击进入

找到运行中、运行完成或者运行失败的 task，可查看具体的执行日志



8) 配置定时

一般的数仓任务调度周期会有小时级调度、每天调度、周调度、月调度等等，这时候我们就需要根据需求，设置合理的调度周期。如下图，点击定时：



进入定时编辑界面，设置上海时区，每天早上凌晨 3 点开始执行任务。

定时前请先设置参数

起止时间

2022-07-31 00:00:00 - 2122-07-31 00:00:00

定时

0 0 3 ? * 1/1 *

执行时间

时区

Asia/Shanghai

接下来五次执行时间

2022-08-01 03:00:00
2022-08-02 03:00:00
2022-08-03 03:00:00
2022-08-04 03:00:00
2022-08-05 03:00:00

失败策略

☒ 继续
☐ 结束

通知策略

都不发

流程优先级

MEDIUM

Worker分组

default

环境名称

请选择

告警组

请选择

取消

编辑

定时管理									
编号	作业流名称	开始时间	结束时间	crontab	失败策略	状态	创建时间	更新时间	操作
1	dag_commerce_warehouse	2022-07-31 00:00:00	2122-07-31 00:00:00	0 0 3 ? * 1/1 *	CONTINUE	下线	2022-07-31 18:54:24	2022-07-31 18:54:24	

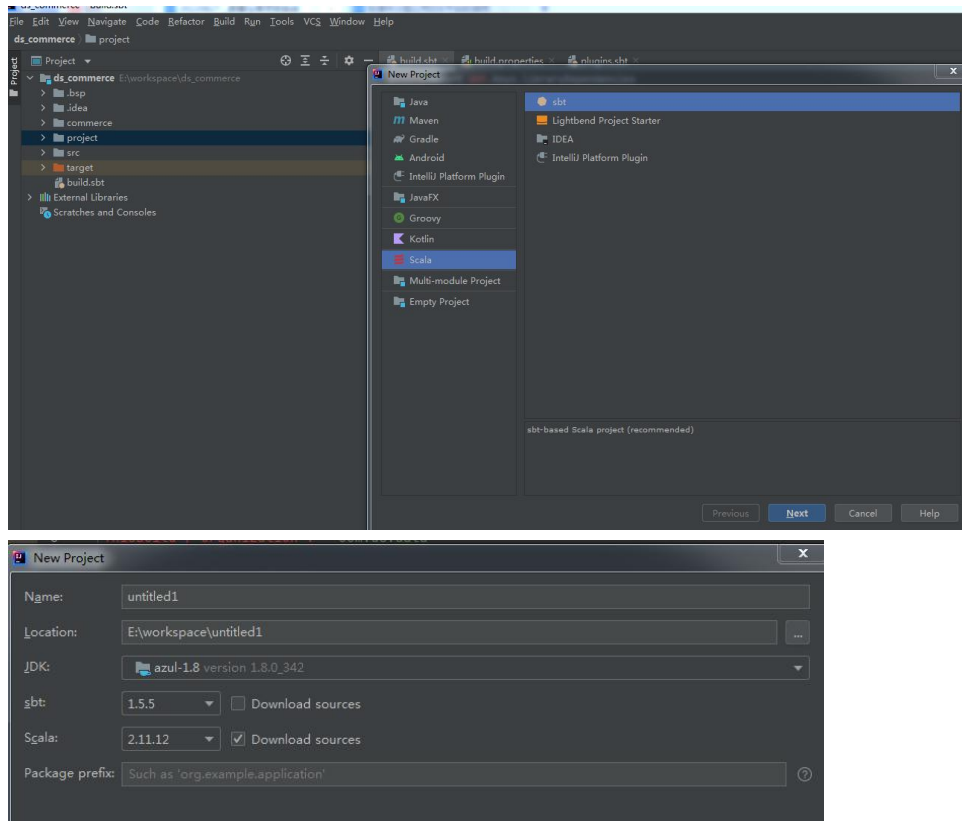
注意：定时周期的设置符合 crontab 语法规则，没了解过的可以了解一下

2 Spark 任务调度部署

2.1 创建 sbt 项目，构建 sbt 依赖

1) 点击 file ->project->选择 Scala sbt->点击 next->选择 java、sbt、scala 版本

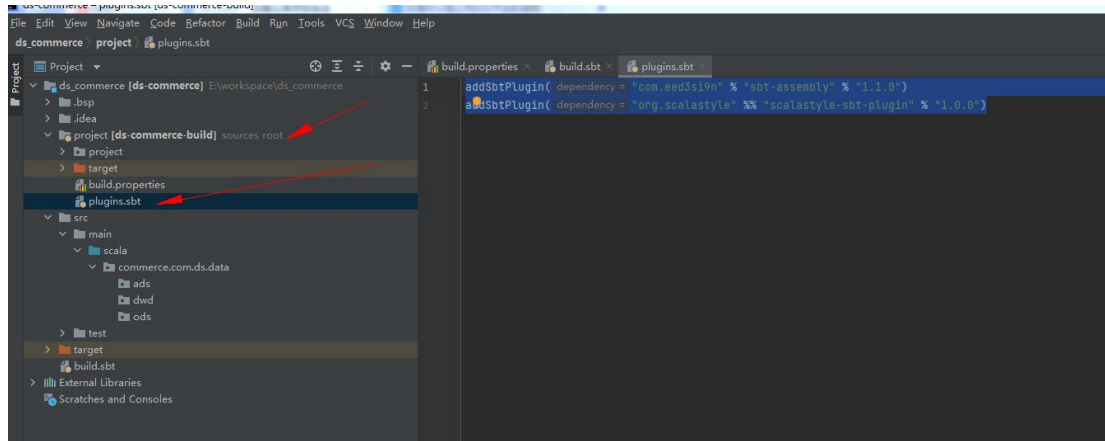
java	jdk1.8
sbt	1.55
scala	2.11.12



2.2 添加 sbt scala 打包依赖

在 project 目录下新建 plugins.sbt 文件，添加如下内容

1. `addSbtPlugin("com.eed3si9n" % "sbt-assembly" % "1.1.0")`
2. `addSbtPlugin("org.scalastyle" %% "scalastyle-sbt-plugin" % "1.0.0")`



2.3 修改 build.sbt 文件，添加依赖

```
import sbtassemblies.AssemblyPlugin.autoImport.assembly
```

```
ThisBuild / version := "0.1"
```

```
ThisBuild / scalaVersion := "2.11.12"
```



```

ThisBuild / organization := "com.ds.data"

val sparkVersion = "2.4.8"

val scalaTest = "org.scalatest" %% "scalatest" % "3.2.9"
val sparkSql = "org.apache.spark" %% "spark-sql" % sparkVersion % "provided"

libraryDependencies += Seq(
  scalaTest,
  sparkSql
)

assembly / assemblyMergeStrategy := {
  case x: String if x.contains("UnusedStubClass") => MergeStrategy.first
  case "module-info.class" => MergeStrategy.discard
  case x =>
    val oldStrategy = (assembly / assemblyMergeStrategy).value
    oldStrategy(x)
}

```

2.4 代码开发

Spark DataFrame 开发示例：

```

package commerce.com.ds.data.user.df

import org.apache.spark.sql.functions.{col, from_unixtime}
import org.apache.spark.sql.{DataFrame, SparkSession}

object DwdUserUserInfo {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder().enableHiveSupport().getOrCreate()
    val df = read(spark)
    val transformDF = transform(df)
    write(transformDF)
  }

  def read(spark: SparkSession): DataFrame = {
    spark.read.table("ods_commerce.ods_user_userinfo")
  }

  def transform(df: DataFrame): DataFrame = {
    df.withColumnRenamed("userid", "user_id")
      .withColumnRenamed("username", "user_name")
      .withColumn("user_money", col("usermoney").cast("decimal(20,4)"))
      .withColumn("frozen_money", col("frozenmoney").cast("decimal(20,4)"))
  }
}

```



```

        .withColumnRenamed("addressid", "address_id")
        .withColumn("reg_time", from_unixtime(col("regtime").cast("bigint"), "yyyy-MM-dd HH:mm:ss"))
        .withColumn("last_login", from_unixtime(col("lastlogin").cast("bigint"), "yyyy-MM-dd HH:mm:ss"))
        .select("user_id", "user_name", "sex", "user_money", "frozen_money", "address_id", "reg_time", "last_login")
    }

def write(df: DataFrame): Unit = {
    df.write.overwrite.saveAsTable("dwd_commerce.dwd_user_userinfo_test")
}
}

```

Spark SQL 开发示例：

```

package commerce.com.ds.data.user.sql

import org.apache.spark.sql.SparkSession

object DwdUserUserInfo {
    def main(args: Array[String]): Unit = {
        val spark = SparkSession.builder().enableHiveSupport().getOrCreate()
        createTable(spark)
        insertTable(spark)
    }

    def createTable(spark: SparkSession): Unit = {
        spark.sql(
            """
            |CREATE TABLE IF NOT EXISTS dwd_commerce.dwd_user_userinfo (
            |  user_id STRING COMMENT '用户 ID'
            |  ,user_name STRING COMMENT '用户名称'
            |  ,sex INT COMMENT '性别'
            |  ,user_money decimal(20,4) COMMENT '钱包'
            |  ,frozen_money decimal(20,4) COMMENT '近一个月的花费的总的金额'
            |  ,address_id STRING COMMENT '用户地址 ID, 0 表示没有获取地址'
            |  ,reg_time STRING COMMENT '注册时间'
            |  ,last_login STRING COMMENT '最后登录时间'
            |) COMMENT '用户表' STORED AS ORC
            |""".stripMargin)
    }

    def insertTable(spark: SparkSession): Unit = {
        spark.sql(
            """

```

```

|insert overwrite table dwd_commerce.dwd_user_userinfo
|select
|  userid as user_id
|  ,username as user_name
|  ,sex
|  ,cast(usermoney as decimal(20,4)) as user_money
|  ,cast(frozenmoney as decimal(20,4)) as frozen_money
|  ,addressid as address_id
|  ,from_unixtime(cast(regtime as bigint),'yyyy-MM-dd HH:mm:ss') as reg_time
|  ,from_unixtime(cast(lastlogin as bigint),'yyyy-MM-dd HH:mm:ss') as last_login
|from ods_commerce.ods_user_userinfo
|"".stripMargin)
}
}

```

2.5 将 scala 代码打成 jar 包

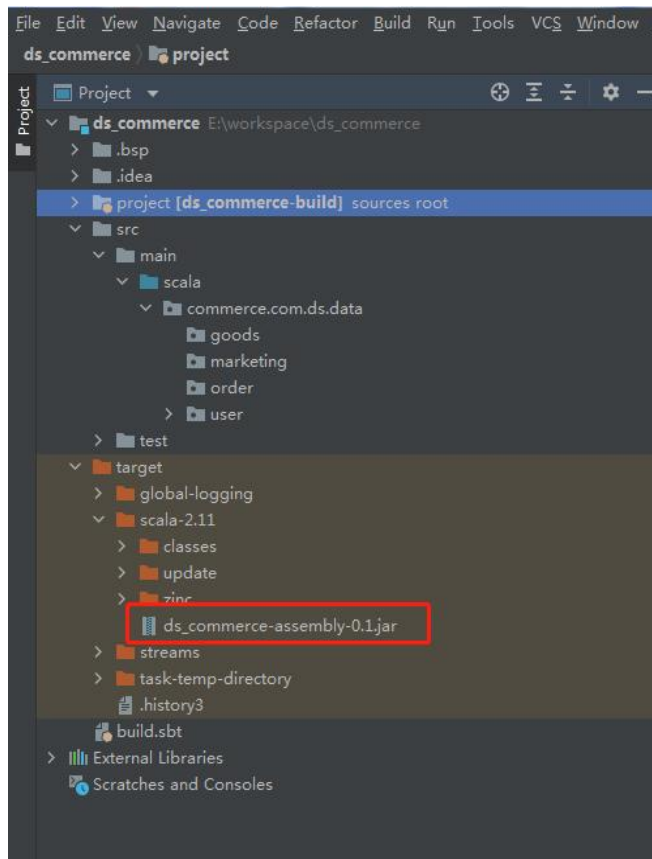
点击 sbt shell,在文本框输入 assembly

```

sbt shell ds_commerce
[info] set current project to ds_commerce (in build file:/E:/workspace/ds_commerce/)
[info] Defining Global / ideaPort
[info] The new value will be used by Compile / compile, Test / compile
[info] Reapplying settings...
[info] set current project to ds_commerce (in build file:/E:/workspace/ds_commerce/)
[IJ]assembly
[info] compiling 1 Scala source to E:\workspace\ds_commerce\target\scala-2.11\classes ...
[info] done compiling
[info] Strategy 'discard' was applied to a file (Run the task at debug level to see details)
[success] Total time: 30 s, completed 2022-7-31 19:28:12
[IJ]assembly
[info] compiling 1 Scala source to E:\workspace\ds_commerce\target\scala-2.11\classes ...
[info] done compiling
[warn] multiple main classes detected: run 'show discoveredMainClasses' to see the list
[info] Strategy 'discard' was applied to a file (Run the task at debug level to see details)
[success] Total time: 17 s, completed 2022-7-31 21:21:15
[IJ]
> assembly

```

jar 包成功打包后，可在此目录下发现 jar 文件：



2.6 上传 jar 包至调度系统

点击资源中->上传文件



上传成功后，jar 包名称会显示出来

文件管理							
创建文件夹		创建文件		上传文件		请输入关键词	
编号	名称	所属用户	是否为文件夹	文件名称	描述	大小	更新时间
1	ds_commerce-assembly-0.1.jar		否	ds_commerce-assembly-0.1.jar	-	22 MB	2022-07-31 21:34:38
							操作

如果需要重新上传，点击重新上传即可

文件管理							
创建文件夹		创建文件		上传文件		请输入关键词	
编号	名称	所属用户	是否为文件夹	文件名称	描述	大小	更新时间
1	ds_commerce-assembly-0.1.jar		否	ds_commerce-assembly-0.1.jar	-	22 MB	2022-07-31 21:34:38
							操作

2.7 配置 spark task

1) spark sql task 案例：

当前节点设置

节点名称

spark_sql_dwd_userinfo

运行标志

☒ 正常 ☐ 禁止执行

描述

请输入描述

任务优先级

MEDIUM

Worker分组

default

环境名称

请选择

失败重试次数

3

(次)

失败重试间隔

5

(分)

延时执行时间

0

(分)

超时告警

☒

超时策略

☒ 超时告警 ☐ 超时失败

超时长

30

分

程序类型

SCALA

Spark版本

SPARK2

主函数的Class

commerce.com.ds.data.user.sql.DwdUserUserInfo

主程序包

ds_commerce-assembly-0.1.jar

部署方式

☐ cluster ☒ client ☐ local

任务名称

commerce.com.ds.data.user.sql.DwdUserUserInfo

Driver核心数

1

Driver内存数

512M

Executor数量

2

Executor内存数

2G

Executor核心数

2

2) spark df task 案例：

当前节点设置

节点名称: spark_df_dwd_user_userinfo

运行标志: ☒ 正常 ☐ 禁止执行

描述: 请输入描述

任务优先级: MEDIUM

Worker分组: default 环境名称: 请选择

失败重试次数: 3 (次) 失败重试间隔: 5 (分)

延时执行时间: 0 (分)

超时告警: ☒ 超时告警 ☐ 超时失败

超时时长: 30 分

程序类型: SCALA

Spark版本: SPARK2

主函数的Class: commerce.com.ds.data.user.df.DwdUserUserInfo

主程序包: ds_commerce-assembly-0.1.jar

部署方式: ☒ cluster ☐ local

任务名称: commerce.com.ds.data.user.sql.DwdUserUserInfo

Driver核心数: 1 Driver内存数: 512M

Executor数量: 2 Executor内存数: 2G

Executor核心数: 2

2.8 运行和测试

接下来就是测试运行和查看运行日志了，大家可以参考上面 SQL 任务调度的内容，来自己动手测试一下。

3 Hive UDF 部署和使用

3.1 上传 UDF Jar 包

点击资源中心->资源管理-上传

DolphinScheduler 首页 项目管理 资源中心 数据源中心 监控中心

文件管理 UDF管理 资源管理 函数管理

UDF资源>udf_test

创建文件夹 上传UDF资源

文件上传

* 文件名称: 请输入名称

描述: 请输入描述

* 上传文件: 上传

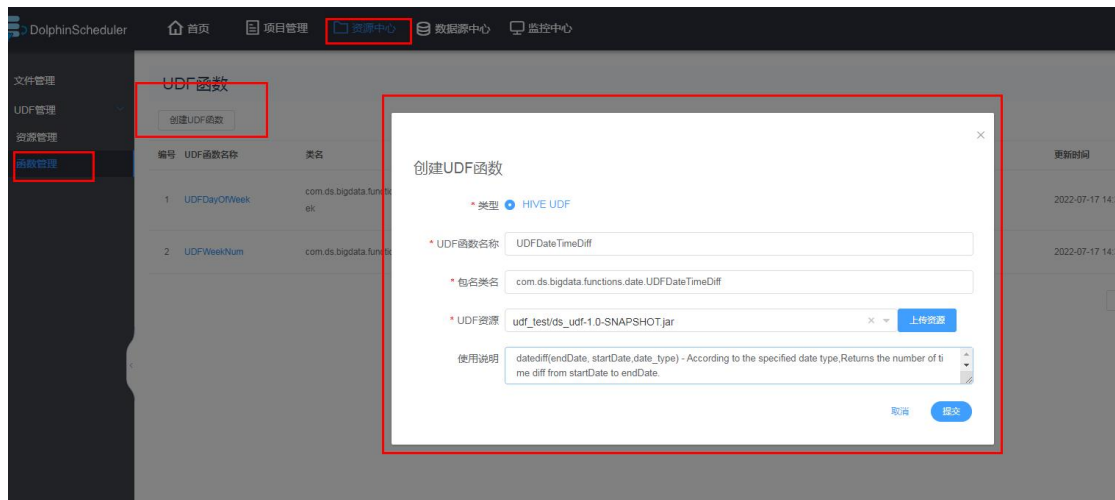
取消 上传

请将文件拖拽到当前上传窗口内！

UDF资源>udf_test							
创建文件夹 上传UDF资源		请输入关键词					
编号	UDF资源名称	是否文件夹	文件名称	文件大小	描述	创建时间	更新时间
1	ds_udf-1.0-SNAPSHOT.jar	否	ds_udf-1.0-SNAPSHOT.jar	49 KB		2022-07-31 21:58:21	2022-07-31 21:58:21

3.2 创建 UDF 函数

点击资源中心->函数管理-创建 UDF 函数->输入相关内容



创建成功的函数就会显示在该列表

UDF函数							
创建UDF函数		请输入关键词					
编号	UDF函数名称	类名	类型	描述	jar包	更新时间	操作
1	UDFDayOfWeek	com.ds.bigdata.functions.date.UDFDayOfWeek	HIVE	输出时间获取，这天是本周的第几天，本周的第一天是从星期一开始；比如2022-07-07，返回是04，表示本周的第4天，周四；	ds_udf-1.0-SNAPSHOT.jar	2022-07-17 14:33:45	 
2	UDFWeekNum	com.ds.bigdata.functions.date.UDFWeekNum	HIVE	输出时间获取，这天是本周的第几天，本周的第一天是从星期一开始；比如2022-07-07，返回是04，表示本周的第4天，周四；	ds_udf-1.0-SNAPSHOT.jar	2022-07-17 14:31:56	 

3.3 创建使用 UDF 函数的 task

节点名称: UDFDayOfWeek

运行标志: ☒ 正常 ☐ 禁止执行

描述: 给出时间获取, 这天是本周的第几天, 本周的第一天是从星期一开始; 比如'2022-07-07', 返回是04, 表示本周的第4天, 周四;

任务优先级: MEDIUM

Worker分组: default 环境名称: 请选择

失败重试次数: 0 (次) 失败重试间隔: 1 (分)

延时执行时间: 0 (分)

超时告警: ☐

数据源:

sql类型: 发送邮件: ☐ 日志显示: 行查询结果

sql参数: 请输入格式为 key1=value1,key2=value2...

sql语句: 1 SELECT UDFDayOfWeek('2022-07-31')

UDF函数: UDFDayOfWeek

3.4 运行和测试

查看日志

```

[INFO] 2022-07-31 21:54:16.023 TaskLogger-class org.apache.dolphinscheduler.plugin.task.sql.SqlTask: sqlType: HIVE, datasource: 2, sql: SELECT UDFDayOfWeek(2022-07-31), localParams: [], udfs: 5, showType: null, connParams: varPool: [], query max result limit: 0
[INFO] 2022-07-31 21:54:16.043 TaskLogger-class org.apache.dolphinscheduler.plugin.task.sql.SqlTask: [418] - after replace sql, preparing: SELECT UDFDayOfWeek(2022-07-31)
[INFO] 2022-07-31 21:54:16.043 TaskLogger-class org.apache.dolphinscheduler.plugin.task.sql.SqlTask: [427] - SqlParams are replaced sql, parameters:
[INFO] 2022-07-31 21:54:17.001 TaskLogger-class org.apache.dolphinscheduler.plugin.task.sql.SqlTask: [335] - hive create function sql: add jar hdfs://ds-bigdata-001:8020/dolphinscheduler/hdfs/udfs/ds_udf-1.0-SNAPSHOT.jar
[INFO] 2022-07-31 21:54:17.055 TaskLogger-class org.apache.dolphinscheduler.plugin.task.sql.SqlTask: [335] - hive create function sql: create temporary function UDFDayOfWeek as 'com.ds.bigdata.function.udate.UDFDayOfWeek'
[INFO] 2022-07-31 21:54:17.099 TaskLogger-class org.apache.dolphinscheduler.plugin.task.sql.SqlTask: [400] - prepare statement replace sql: HikariProxyPreparedStatement@339037537 wrapping org.apache.hive.jdbc.HivePreparedStatement@3037253d
[INFO] 2022-07-31 21:54:17.234 TaskLogger-class org.apache.dolphinscheduler.plugin.task.sql.SqlTask: [256] - display sql result 1 rows as follows
[INFO] 2022-07-31 21:54:17.235 TaskLogger-class org.apache.dolphinscheduler.plugin.task.sql.SqlTask: [259] - row 1: ("04")

```

关闭

ods

spark_sql_dwd_user...

spark_df_dwd_user...

UDFDayOfWeek